

DR. MADARÁSZ LÁSZLÓ
A PIC16C MIKROVEZÉRLŐK



Kecskemét
1996

Dr. Madarász László

A PIC16C mikrovezérlők

Lektorálta:

Hóka László okl. vill. mérnök

Első kiadás

Kecskemét
1996

Felelős kiadó: Dr. Szabó András
Példányszám: 900 Terjedelem: 14,25 A/5 ív
96007. GAMF Sokszorosító, Kecskemét
Felelős vezető: Némethné Falusi Erzsébet

Bevezetés

Nem egyszerű feladat eldönteni, hogy a XX. század utolsó éveiben a mikroelektronika oktatásában mivel célszerű foglalkozni, melyik áramkörökkel, milyen mélységben.

A mikroelektronika csúcspontját képviselő mikroprocesszorokról természetesen mindig szükséges egy átfogó kép ismerete, de nem tételvezetjük fel, hogy a GAMF-ról kikerülő mérnökök, informatikusok a mai mikroprocesszorokkal készülékeket, mikrogépeket fognak tervezni. A mikrovezérlőkkel azonban már más a helyzet. Ezek az egy-csipesz mikroszámítógépek olcsók, rendkívül nagy a változatosságuk, szinte minden beépített vezérlő, irányító egységhez lehet olyan változatot találni, amelyik optimálisan illeszkedik az adott feladathoz. Azt, hogy mikrovezérlőkkel terveznek majd egységeket a jövő évtizedben a végzett hallgatóink, nyugodtan feltételezhetjük.

Annak érdekében, hogy a konkrét feladathoz az optimális típust ki tudják választani, célszerű sok elemcsaládot megismerni. A GAMF jegyzetkészletében már vannak olyanok, melyekből az Intel 8048 és a 8051 családokat megismerhetik. Ez a jegyzet egy másik, fiatalabb cég, a Microchip Technology Inc. (USA, Chandler) nagy népszerűségnek örvendő, sajátos mikrovezérlő családját ismerteti, a PIC16C elemeket - ezeken belül a korábbi fejlesztésű PIC16C5x áramköröket és a korszerűbb PIC16Cxx csipeket. A Microchip már megjelent a harmadik fejlesztési szinttel is, de a PIC17C áramköröket már nem lehetett beszorítani a terjedelmi korlátok közé.

A PIC mikrovezérlők ár/hatékonyság mutatója igen kedvező. A központi egységük (CPU) RISC jellegű, a szűk de rendkívül hatékony utasításkészlet gyorsan megtanulható. A processzorcsalád jól kihasználja a CMOS technológia előnyeit, a fogyasztás üzemi állapotban is alacsony értékű, s még jobban lecsökken a stand-by állapotban.

A PIC mikrovezérlők alapvetően EPROM csipek. Egy konkrét típus egyféle csip. Ez az egyféle csip azonban különféle tokozással készülhet, így tud a gyártó sokszor felhasználható, átprogramozható változatot is kínálni (kvarcablakos, drága tokozásban), a felhasználó által egyszer programozható kivittelt ajánlani (olcsó műanyag tokozással), s a gyártó maga is vállalja a tömeges beprogramozást, a felhasználó által már nem módosítható megoldásnál. (Más mikrovezérlő családok esetében a fenti sokoldalúságot pl. csak úgy tudja elérni a gyártó, hogy EPROM-os és ROM-os csipeket is gyárt, az EPROM-osat a fejlesztésre, a ROM-osat a végleges tömeggyártásra.)

A PIC mikrovezérlő család további sajátossága, hogy csak a belső programtárból képes utasításokat végrehajtani s csak a belső adattárat tudja kezelni - nem lehet külső memória áramköröket hozzákapcsolni. Ez első hallásra hiányosságnak tűnhet, hiszen más mikrovezérlő családok esetében azok előnyös képességeként tüntetik fel, hogy külső memóriákkal is működtethető. A külső memóriákat azonban a mikrovezérlők kizárólag Port pontjaikon keresztül képesek kezelni, az ilyen működtetés a mikrovezérlő szabad jelcsatlakozásainak számát drasztikusan csökkenti. A PIC mikrovezérlők esetében tehát szóba sem jöhetnek olyan kényszermegoldások, ahol a tervező a Port pontok jelentős részét a memóriák kezelésére használja fel.

A Microchip további lényeges előnyt is kovácsolt ebből a korlátozásból. Mivel a PIC mikrovezérlők csak belső tárolóból hajtanak végre programot, lehetségessé vált, hogy az adathosszúság és az utasítás szó hosszúsága eltérjen egymástól. A PIC mikrovezérlők 8 bites áramkörök, a CPU és a belső adatBUSZ is 8 bites - de az utasítások a 16C5x sorozatnál 12 bitesek, a 16Cxx elemeknél 14 bitesek. Ez tette lehetővé, hogy az utasításkészlet minden utasítása egyetlen szavas, ami a működési sebesség egyik fontos tényezője.

A PIC mikrovezérlők használatához az univerzális EPROM, PLD és mikrovezérlő programozó készülékek közül nem mindegyik alkalmas, de azért található azok között is olyan, amelyekkel megoldható a programozás. Mégis célszerűbb komplett PIC fejlesztő rendszert beszerezni a PIC mikrovezérlőkkel történő fejlesztéshez, olyan rendszert, mely a programfejlesztést, a szimulációt, a program letöltését egyaránt lehetővé teszi. A PARALLAX cég szakosodott a PIC mikrovezérlőkre, számos fejlesztő készletük van forgalomban. Mivel a mikrovezérlők felhasználói közül sokan már megszokták az ipari szabványnak tekinthető 8051-es utasításkészletet, a PARALLAX olyan programfejlesztő rendszert is ajánl, mely a 8051 jellegű utasításokat is fogadja és PIC utasításokká alakítva beépíti a PIC programjába.

A beprogramozott PIC belső programtárolójából a fejlesztő rendszer képes kiolvasni a programot, de egy titkosító "biztosíték" kiégetése után már többé nem lehet a beégetett programot kiolvasni. A kvarc-ablakos változatoknál az EPROM törlése feloldja ezt a titkosítást.

A sok vonzó tulajdonság magyarázza, hogy a PARALLAX olyan különleges alkalmazást is kidolgozott a PIC elemekkel, mint a BASIC STAMP, a bélyeg méretű komplett, BASIC nyelven programozható vezérlő számítógép.

A jegyzet alkalmazási oldalról tárgyalja a mikrovezérlőket, a képességeiket foglalja össze. Az egyes változatok elektromos és idő-paramétereit a gyártó katalógusaiból lehet megismerni, azok a terjedelmi korlátok miatt már nem kerülhettek be a jegyzetbe.

Abban a reményben bocsátjuk útjára a Mikroelektronika témakör új jegyzetét, hogy egy hatékony, sokoldalúan hasznosítható áramkörcsaláddal ismerteti meg olvasóit.

Kecskemét, 1995. december

A Szerző és a Lektor

1. A PIC16C mikrovezérlő család felépítése, általános tulajdonságai

A PIC mikrovezérlő sorozat a Microchip Technology Inc. fejlesztése. A legolcsóbb, legkisebb elemek a 16C5x áramkörök; a 16C6x, 16C7x és 16C8x elemek már komolyabb lehetőségeket biztosítanak (utóbbiak közös elnevezése a PIC16Cxx). A PIC áramkörök többsége EPROM csip (néhány kivételes elem EEPROM), s a program tárolásán kívül a konfiguráló részletek is az EPROM (vagy EEPROM) egyes bitjeiként jelennek meg. A PIC mikrovezérlők 8 bitesek, a 16C5x család utasításkészlete 33 utasításból áll, a 16Cxx áramkörök esetében az utasításkészlet kicsit bővült, 35 utasítást tartalmaz. Mindenesetre ezek igen kis számok, jogos tehát a gyártónak az a kitétele, hogy a PIC mikrovezérlők RISC (redukált utasításkészlet) jellegűek. Minden utasítás egy szavas, s a végrehajtásuk általában egyetlen gépi ciklust igényel (kivéve az ugró utasításokat).

Az utasítások nem 8 bitesek, hanem a 16C5x mikrovezérlőknél 12 bit hosszúságúak, a 16Cxx elemeknél 14 bitesek. A címzési lehetőségek sokoldalúak (direkt, indirekt, relatív címzés).

Mint a mikrovezérlők többsége, a PIC áramkörök is a Harvard architektúrát követik, azaz különálló programtárolót és adattárolót tartalmaznak. Így lehetséges, hogy a programtároló 12 illetve 14 bites, míg az adattároló 8 bites.

A PIC mikrovezérlők csak a belső programtárolójukból képesek utasításokat végrehajtani, de ennek a kapacitása széles tartományból választható (0,5k ... 4k szó), a szintén csak belső adattár kapacitása is változó (25 ... 192 bájt). Mivel külső memória áramkörök nélkül dolgozik, a PIC veremtárolója is belső elem, így a mérete korlátozott. A PIC16C5x elemek megszakítást még nem fogadnak, de a 16Cxx áramkörök már sokoldalú megszakítási rendszerrel is rendelkeznek.

A különböző PIC típusokban 1 ... 4 db párhuzamos Port található, s minden típusban megtalálható egy időzítő/számláló egység, mely a valós idő mérésére vagy események számlálására alkalmas, valamint a mikrovezérlő működési biztonságát megnövelő watch-dog timer. A további egységek opcionálisak, egyes változatokban szerepelnek, másokban nem (további számlálók, soros Port aszinkron ill. szinkron átvitelre, A/D konverter analóg multiplexerrel, EEPROM adattároló).

A CMOS technológiának köszönhetően a PIC mikrovezérlők kis fogyasztásúak és széles tápfeszültség tartományban használhatók. A névleges tápáramuk 5V tápfeszültségen, 4MHz órafrekvencia mellett 2 mA, ha azonban a tápfeszültség 3V és az órafrekvencia csak 32kHz, a fogyasztás mindössze 15µA. A stand-by állapot tápáram igénye kisebb mint 1µA (a részletes villamos- és idő-paraméterek az egyes típusok gyári katalógusaiból olvashatók ki).

A PIC mikrovezérlők a RESET áramköri megoldások, az oszcillátor kapcsolási lehetőségek és az utasításkészlet szempontjából egységesek, ezeket itt a jegyzet elején bemutatjuk. A továbbiakban a 16C5x áramköröket lehet együtt tárgyalni, majd külön fejezetben szerepelnek a 16Cxx elemek.

Ellenőrző kérdések

1. Mennyiben sajátos a PIC mikrovezérlők család-kialakítása?
2. Milyen előnyökkel jár, hogy a PIC elemek CMOS áramkörök?
3. Milyen előnyei vannak a 12/14 bites utasítás szó hosszúságnak?
4. Használható-e a PIC mikrovezérlő mellett külső programtár?
5. Használható-e a PIC mikrovezérlő mellett külső adattár?

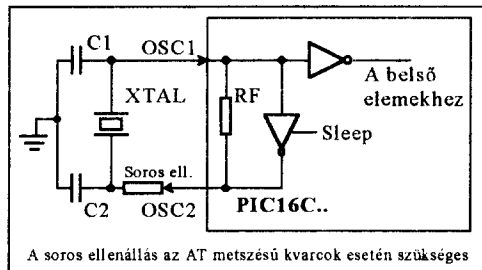
1.1. A PIC mikrovezérlők oszcillátor típusai

A PIC mikrovezérlők minden típusa többféle oszcillátorral is működhet - az oszcillátor típust az EPROM-ban (esetenként EEPROM-ban) kialakított konfigurációs biztosítékokkal lehet kiválasztani. A kvarcablakkal tokozott változatok és az EEPROM alapú PIC-ek esetén a felhasználó a beprogramozással egyidejűleg, a fejlesztő egység segítségével állítja be az oszcillátor típusát. Az egyszer programozható OTP (One Time Programmable) mikrovezérlőket zárt műanyag tokozással gyártják, ezeknél az oszcillátor típusát a gyártó már beállította (az oszcillátor típusnak megfelelően kell rendelni az áramköröket). A PIC mikrovezérlő típusjele ilyenkor az oszcillátor típusára utaló betűkkel végződik, a kvarcablakos változatok típusjelének a végén JW áll.

A négy oszcillátor lehetőség a következő:

- LP: kisteljesítményű kvarcoszcillátor
- XT: kvarcoszcillátor
- HS: nagy sebességű kvarcoszcillátor
- RC: ellenállás/kondenzátor hangolású oszcillátor.

A PIC mikrovezérlőkre közvetlenül ráköthető egy rezgőkvarc vagy egy kerámia rezonátor (1.1. ábra). Ezt a megoldást az XT, LP vagy HS üzemmódokban lehet alkalmazni, párhuzamos metszésű rezgőkvarcot kell



A soros ellenállás az AT metszésű kvarcok esetén szükséges

1.1. ábra Kvarcoszcillátor

felhasználni (a soros metszésű kvarc tényleges rezgési frekvenciája ilyen bekötés

1.1. táblázat Kondenzátorértékek rezonátorhoz

Oscillátor típus	Rezonátor frekvencia	Kondenzátor érték (pF) C1 = C2
XT	455 kHz	150 ... 300
	2,0 Mhz	20 ... 330
	4,0 Mhz	20 ... 330
HS	8,0 Mhz	20 ... 200

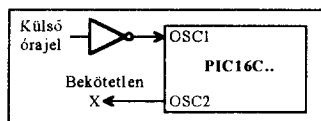
A nagyobb kondenzátor növeli az oszcillátor stabilitását, de szintén megnöveli a start-up időt. A táblázatban szereplő értékek csak ajánlások. Mivel minden rezonátornak sajátos karakterisztikái vannak, célszerű a rezonátor gyártójával konzultálni a külső komponensek értékéről.

1.2. táblázat Kondenzátorok kvarcoszcillátorhoz

Oscillátor típus	Frekvencia	C1 (pF)	C2 (pF)
LP	32 kHz*	15	15
	100 kHz	15	15
	200 kHz	0 ... 15	0 ... 15
XT	100 kHz	15 ... 30	200 ... 300
	200 kHz	15 ... 30	100 ... 200
	455 kHz	15 ... 30	15 ... 100
	1 Mhz	15 ... 30	15 ... 30
	2 Mhz	15	15
	4 Mhz	15	15
HS	4 Mhz	15	15
	8 Mhz	15	5
	20 Mhz	15	15

A nagyobb kondenzátor növeli az oszcillátor stabilitását, de szintén megnöveli a start-up időt. A táblázatban szereplő értékek csak ajánlások. Az Rs csak HS és XT módban szükséges, a kis meghajtó szintű kvarcoknál, a túlterhelés megelőzésére. Mivel minden kvarcnak sajátos karakterisztikái vannak, célszerű a gyártóval konzultálni a külső alkatrészekkel kapcsolatban.

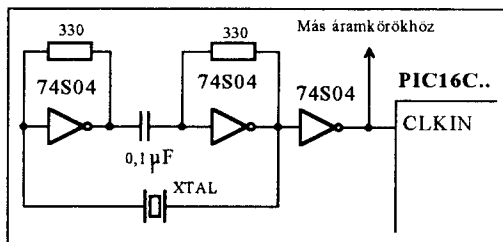
*: ha VDD 4,5V feletti, az ajánlott értékek: C1=C2=30 pF



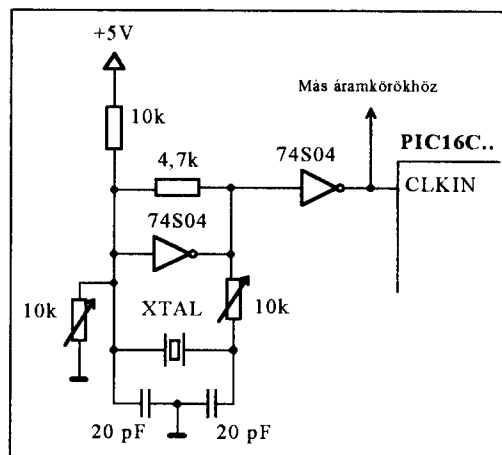
1.2. ábra Külső órajel

re is ad ajánlást (1.1. és 1.2. táblázat).

A kvarcoszcillátor a PIC mikrovezérlő belső elemeinek használata nélkül, önálló külső áramkörként is megépíthető, s a PIC mint külső órajelet, tudja ekkor fogadni az oszcillátor impulzusait (1.2. ábra). A külső kvarc oszcillátor megépíthető TTL vagy CMOS kapukból is. Párhuzamos



1.4. ábra Soros kvarc alkalmazása



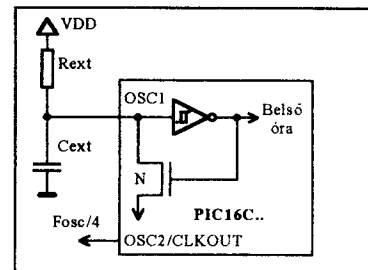
1.3. ábra Oszcillátor párhuzamos kvarccal

mos rezonanciájú rezgőkvarc esetén az oszcillátor áramkör egyszerűen kialakítható (1.3. ábra). A soros rezonanciájú rezgőkvarc az 1.4. ábra szerinti oszcillátor áramkörrel használható fel. A külső oszcillátor alkalmazása akkor célszerű, ha az oszcillátor jelét a mikrovezérlőn kívül más áramkörökhöz is el kell vezetni.

Az RC oszcillátor kapcsolási elrendezése az 1.5. ábrán látható. Ez a legolcsóbb megoldás, a működési frekvencia pontosságára igénytelen áramköri feladatoknál lehet alkalmazni. Az oszcillátor frekvenciája az alkatrész értékeken kívül bizonyos mértékig függ a tápfeszültségtől, a környezeti hőmérséklettől is, sőt, az egyes mikrovezérlő példányok esetén is eltérő lehet, egyébként azonos feltételek mellett is. Az R_{ext} érték 3 kOhm

1.3. táblázat RC oszcillátor adatok

Cext	Rext	Fosc @ 5,0V, 25 °C
20 pF	5,1 k	3,33 MHz $\pm 22\%$
	10 k	1,85 MHz $\pm 22\%$
	100 k	189 kHz $\pm 38\%$
100 pF	5,1 k	1,18 MHz $\pm 15\%$
	10 k	668 kHz $\pm 15\%$
	100 k	67,8 kHz $\pm 26\%$
300 pF	5,1 k	460 kHz $\pm 10\%$
	10k	254 kHz $\pm 13\%$
	100 k	25,1 kHz $\pm 21\%$



1.5. ábra RC oszcillátor

... 100 kOhm közötti lehet. 5V-os tápfeszültség és 25°C környezeti hőmérséklet esetén néhány kombinációra az 1.3. táblázat adja meg a működési frekvencia értékét.

A kvarcablakkal tokozott PIC mikrovezérlők esetén az oszcillátor típusát a felhasználó állítja be, a zárt műanyag tokozású csipeknél a gyártó már rögzítette az oszcillátor változatot, s a típusjelzésben is jelzi. Az 1.4. táblázat a 16C54 áramkör különböző típusjelzésű, zárt tokozású (OTP) változatait foglalja össze példaként.

1.4. táblázat A PIC16C54 OTP változatai

Típus	EPROM	RAM	I/O	Tápfesz.	Oszc. típus	Oszc. fr.	Lábszám
PIC16C54RC				4,0-5,5V	RC	DC-4MHz	
PIC16C54XT	512x12	32x8	13	4,0-5,5V	Q, külső	0,1-4MHz	18
PIC16C54HS				4,5-5,5V	Q, külső	4-20MHz	
PIC16C54LP				3,0-5,5V	Q, külső	DC-40kHz	

Ellenőrző kérdések

1. Hogyan választható oszcillátor típus a kvarcablakos PIC-ek esetében?
2. Hogyan választható oszcillátor típus az OTP PIC-ek esetében?
3. Melyik oszcillátor típusban használható kerámia rezonátor?
4. Melyik oszcillátor típus a legolcsóbb?

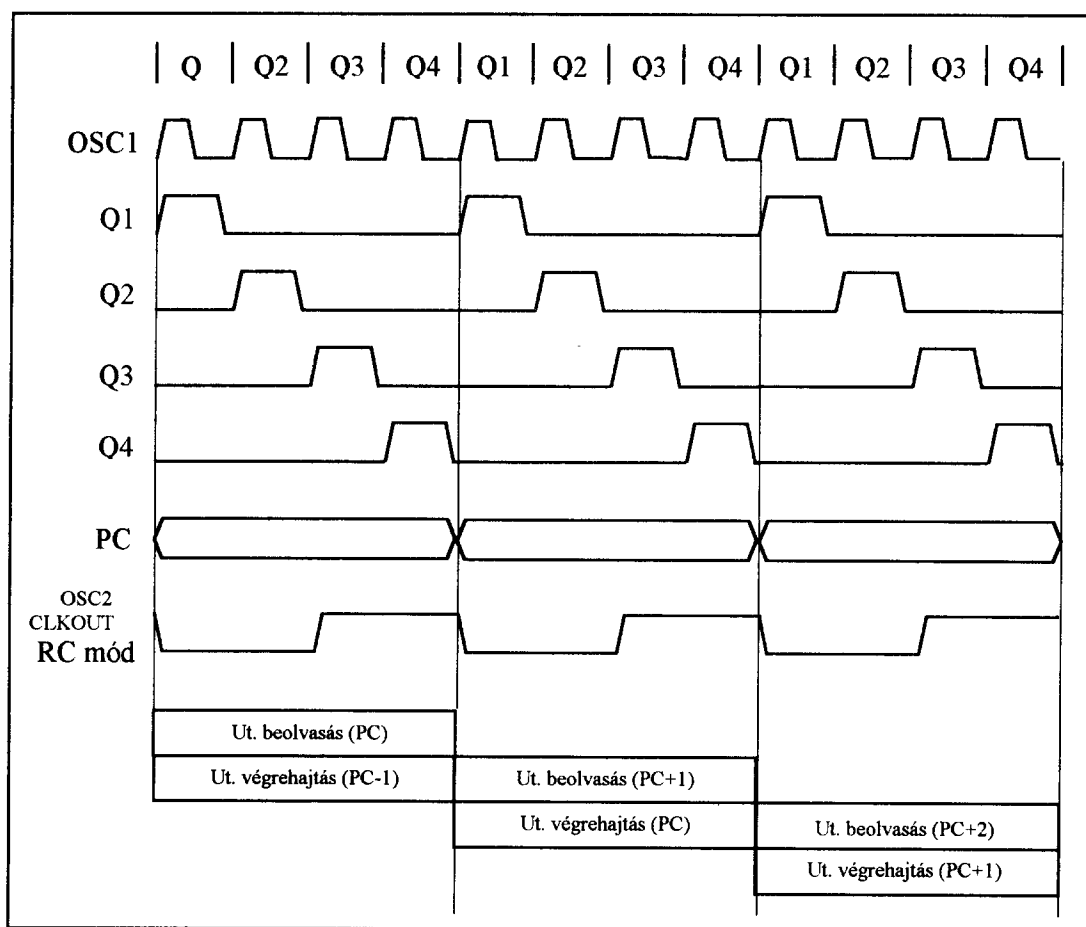
1.2. A PIC mikrovezérlők átfedéses utasítás-végrehajtása

A PIC mikrovezérlőkben külön vezérlő egység kezeli a programmemória kiolvasását és az utasítás végrehajtást. Így lehetséges, hogy az egyik utasítás végrehajtása alatt a másikat a CPU előre kiolvashatja a program memóriából. A PIC mikrovezérlők esetében ez az utasítás elő-olvasás nem esetleges folyamat, hanem az alapműködés eleme.

Az alaposzcillátor (OSC1) impulzusaiból a PIC belső időzítő-vezérlő egysége négy, egymást át nem fedő impulzus sorozatot állít elő, melyek frekvenciája az OSC1 frekvenciájának negyedrésze (Q1 ... Q4). A négy leosztott órajel egy-egy impulzusa alkot egy gépi ciklust. A belső események a leosztott órajelekhez kötötten, szigorú rendben zajlanak.

- Q1 alatt inkrementálódik a PC
- Q2-Q3 alatt olvassa be a megcímzett rekeszből az utasítás szót a CPU
- Q4 alatt a most beolvasott utasítás szó átmásolódik az utasítás regiszterbe
- a következő gépi ciklusban (Q1 ... Q4) az utasítás dekódolódik és végrehajtódik, miközben a fenti rendben egy következő utasítás szót olvas be a CPU a programtárból.

Minden utasítás értelmezése és végrehajtása egyetlen gépi ciklus alatt megtörténik, az ugró utasítások esetén az időközben beolvasott utasítás szó helyett másikat kell a programtárból kiolvasni, ezért az ugró utasításoknál egy további gépi ciklus is szükséges. A PIC mikrovezérlők átfedéses utasítás végrehajtása tehát végeredményében gépi ciklusonként megvalósítja egy-egy utasítás végrehajtását, pedig tulajdonképpen minden ilyen végrehajtás időigénye két gépi ciklus. A működés idődiagramja az 1.6. ábrán látható.



1.6. ábra A PIC mikrovezérlők belső időzítése

Ellenőrző kérdések

1. Milyen a PIC mikrovezérlők belső órajel rendszere?
2. Mi a gépi ciklus, mennyi ideig tart?
3. Milyen lépésekből áll egy utasítás végrehajtása?
4. Miért szükséges egy extra gépi ciklus ugró utasítások esetén?

1.3. A PIC mikrovezérlők utasításkészlete

A PIC mikrovezérlők utasításkészletéről már többször is szó esett. Ebben a fejezetben először táblázatosan összefoglaljuk az utasításokat, majd részletesen bemutatjuk minden egyes utasítás működését. A PIC16C5x áramkörök utasításai 12 bit hosszúságúak, az utasításkészlet 33 utasításból áll. A fájl regiszter referenciás utasításokban a fájl regiszterek címe, a fájl regisztereket kijelölő részlet (f ffff) öt bites, a programmemória címek 8 illetve 9 bit hosszúak. A PIC16C5x utasításkészletet az 1.5. táblázat foglalja össze.

A PIC16Cxx mikrovezérlők (tehát a 16C6x, 16C7x, 16C8x típusok) utasításai 14 bit hosszúságúak. A fájl regiszter címzések itt hét bitesek (fff ffff), a programmemória címek 11 bit hosszúságúak. Az operációs kódok jelentős része megegyezik a 16C5x opkódokkal, de vannak eltérések is. A 16Cxx utasításkészletet, amelyik 35 utasításból áll, az 1.6. táblázat mutatja be.

A PIC mikrovezérlők utasításkészletében 33 utasítás mnemonikja megegyezik, akár 16c5x, akár 16Cxx áramkörrel legyen szó. A fejezet következő részében a mnemonikok ABC sorrendjének megfelelően mutatjuk be az utasításokat. Mivel a mnemonikok megegyeznek (még akkor is, ha esetleg az opkódok eltérnek), a PIC szoftver fejlesztő rendszerek minden típushoz használhatóak. Tulajdonképpen az átlag felhasználó számára a mnemonikoknak a PIC mikrovezérlők esetében nincs is jelentősége, mivel a program csak a mikrovezérlőn belüli programmemóriában tárolható, onnan hajtható végre - oda pedig a fejlesztő rendszer tölti be. A program írása assembly nyelven, mnemonikokkal történik.

1.5. táblázat A PIC16C5x utasításkészlet

UTASÍTÁSKÉSZLET						
BÁJT KEZELŐ, FÁJL REGISZTER UTASÍTÁSOK				(11-6) OPKÓD	(5) d	(4-0) f(FÁJL #)
				d = 0, ha a W a célregiszter d = 1, ha f a célregiszter		
Bináris kód	Hex	Az utasítás neve	Mnemonik, operand.	Működés	Flag-ek	
0000 0000 0000	000	No Operation	NOP		-	
0000 001f ffff	02f	Move W to f	MOVWF f	W → f	-	1 4
0000 0100 0000	040	Clear W	CLRWF -	0 → W	Z	
0000 011f ffff	06f	Clear f	CLRF f	0 → f	Z	4
0000 10df ffff	08f	Subtract W from f	SUBWF f,d	f - W → d [f + $\overline{W}$ + 1 → d]	C,DC,Z	12 4
0000 11df ffff	0Cf	Decrement f	DECF f,d	f - 1 → d	Z	24
0001 00df ffff	10f	Inclusive OR W and f	IORWF f,d	W ∨ f → d	Z	24
0001 01df ffff	14f	AND W and f	ANDWF f,d	W & f → d	Z	24
0001 10df ffff	18f	Exclusive OR W and f	XORWF f,d	W ⊕ f → d	Z	24
0001 11df ffff	1Cf	Add W and f	ADDWF f,d	W + f → d	C,DC,Z	124
0010 00df ffff	20f	Move f	MOVF f,d	f → d	Z	24
0010 01df ffff	24f	Complement f	COMF f,d	$\overline{f}$ → d0	Z	2,4
0010 10df ffff	28f	Increment f	INCF f,d	f + 1 → d	Z	2,4
0010 11df ffff	2Cf	Decrement f, Skip if zero	DECFSZ f,d	f - 1 → d, skip if zero	-	24
0011 00df ffff	30f	Rotate right f	RRF f,d	f(n) → d(n-1), C → d(7), f(0) → C	C	24
0011 01df ffff	34f	Rotate left f	RLF f,d	f(n) → d(n+1), C → d(0), f(7) → C	C	24
0011 10df ffff	38f	Swap halves f	SWAPF f,d	f(0-3) ↔ f(4-7) → d	-	2,4
0011 11df ffff	3Cf	Increment f, Skip if zero	INCFSZ f,d	f + 1 → d, skip if zero	-	2,4
BIT KEZELŐ, FÁJL REGISZTER UTASÍTÁSOK				(11-8) OPKÓD	(7-5) b(bit#)	(4-0) f(FÁJL)
Bináris kód	Hex	Az utasítás neve	Mnemonik, operand.	Működés	Flag-ek	
0100 bbbf ffff	4bf	Bit Clear f	BCF f,b	0 → f(b)	-	24
0101 bbbf ffff	5bf	Bit Set f	BSF f,b	1 → f(b)	-	24
0110 bbbf ffff	6bf	Bit Test f, Skip if Clear	BTFSC f,b	Test bit (b) in file (f), Skip if clear	-	
0111 bbbf ffff	7bf	Bit Test f, Skip if Set	BTFSS f,b	Test bit (b) in file (f), Skip if set	-	
LITERAL (KONSTANS) KEZELŐ ÉS VEZÉRLŐ UTASÍTÁSOK				(11-8) OPKÓD	(7-0) k (LITERAL)	
Bináris kód	Hex	Az utasítás neve	Mnemonik, operand.	Működés	Flag-ek	
0000 0000 0010	002	Load OPTION Register	OPTION -	W → OPTION register	-	
0000 0000 0011	003	Go into standby mode	SLEEP -	0 → WDT, stop oscillator	TO,PD	
0000 0000 0100	004	Clear Watchdog Timer	CLRWDT -	0 → WDT (and prescaler, if assigned)	TO,PD	
0000 0000 0fff	00f	Tristate Port f	TRIS f	W → I/O control register f	-	3
1000 kkkk kkkk	8kk	Return, Place Literal in W	RETLW k	k → W, Stack → PC	-	
1001 kkkk kkkk	9kk	Call subroutine	CALL k	PC + 1 → Stack, k → PC	-	1
101k kkkk kkkk	Akk	Go To address (k is 9 bit)	GOTO k	k → PC (9 bits)	-	
1100 kkkk kkkk	Ckk	Move Literal to W	MOVLW k	k → W	-	
1101 kkkk kkkk	Dkk	Incl. OR Literal and W	IORLW k	k ∨ W → W	Z	
1110 kkkk kkkk	Ekk	AND Literal and W	ANDLW k	k & W → W	Z	
1111 kkkk kkkk	Fkk	Excl. OR Literal and W	XORLW k	k ⊕ W → W	Z	

Megjegyzések:

1. A PC kilencedik bite (PC8) a PC-t (f2) író utasítások hatására nullázódik, kivéve a GOTO utasítást.
2. Ha beolvasunk egy I/O regisztert, a PIC csatlakozópontjain lévő logikai értékeket kapjuk meg.
3. A TRIS f (ahol f = 5, 6 vagy 7) a W tartalmát tölti át a vezérlő regiszterek egyikébe (1: bemenő pont, 0: kimenő pont).
4. Ezen utasítások, ha az f1 fájl regiszterrel kapcsolatosan hajtódnak végre, (és, ahol meg kell jelölni, célként d=1 áll), törlik az előosztót, ha az az RTCC-hez van rendelve.

Érdeemes megkeresni az egyes utasításokat a 16Cxx utasításkészlet táblázatában s összehasonlítani az opkódjukat, illetve megvizsgálni, mennyiben azonosak, mennyiben térnek el egymástól. A mnemonikok egyezősége is ellenőrizhető, amiből a két családrész programjainak azonos formátuma is következik, azaz az egyik családrészre írt program probléma nélkül alkalmazható a másik családrész mikrovezérlőinél is.

1.6. táblázat A PIC16Cxx utasításkészlet

Mnemonik, operandus	Leírás	Ciklus	14 bites opkód		módosuló	megj.
			MSb	LSb	flag	
BÁJT ORIENTÁLT, FÁJL REGISZTERES UTASÍTÁSOK						
ADDWF	f,d	Add W and f	1	00 0111 dfff ffff	C,DC,Z	1,2
ANDWF	f,d	AND W and f	1	00 0101 dfff ffff	Z	1,2
CLRF	f	Clear f	1	00 0001 1fff ffff	Z	2
CLRWF	-	Clear W	1	00 0001 0xxx xxxx	Z	
COMF	f,d	Complement f	1	00 1001 dfff ffff	Z	1,2
DECF	f,d	Decrement f	1	00 0011 dfff ffff	Z	1,2
DECFSZ	f,d	Decrement f, Skip if 0	1 (2)	00 1011 dfff ffff		1,2,3
INCF	f,d	Increment f	1	00 1010 dfff ffff	Z	1,2
INCFSZ	f,d	Increment f, Skip if 0	1 (2)	00 1111 dfff ffff		1,2,3
IORWF	f,d	Inclusive OR W and f	1	00 0100 dfff ffff	Z	1,2
MOVF	f,d	Move f	1	00 1000 dfff ffff	Z	1,2
MOVWF	f	Move W to f	1	00 0000 1fff ffff		
NOP	-	No Operation	1	00 0000 0xx0 0000		
RLF	f,d	Rotate left f through carry	1	00 1101 dfff ffff	C	1,2
RRF	f,d	Rotate right f through carry	1	00 1100 dfff ffff	C	1,2
SUBWF	f,d	Subtract W from f	1	00 0010 dfff ffff	C,DC,Z	1,2
SWAPF	f,d	Swap halves f	1	00 1110 dfff ffff		1,2
XORWF	f,d	Exclusive OR W and f	1	00 0110 dfff ffff	Z	1,2
BIT ORIENTÁLT, FÁJL REGISZTERES UTASÍTÁSOK						
BCF	f,b	Bit Clear f	1	01 00bb bfff ffff		1,2
BSF	f,b	Bit Set f	1	01 01bb bfff ffff		1,2
BTFSC	f,b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff		3
BTFSS	f,b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff		3
LITERAL (KONSTANSOS) ÉS VEZÉRLŐ UTASÍTÁSOK						
ADDLW	k	Add literal to W	1	11 111x kkkk kkkk	C,DC,Z	
ANDLW	k	AND literal to W	1	11 1001 kkkk kkkk	Z	
CALL	k	Call subroutine	2	10 0kkk kkkk kkkk		
CLRWDI	-	Clear watchdog timer	1	00 0000 0110 0100	$\overline{TO}$, $\overline{PD}$	
GOTO	k	Go to address	2	10 1kkk kkkk kkkk		
IORLW	k	Inclusive OR literal to W	1	11 1000 kkkk kkkk	Z	
MOVLW	k	Move literal to W	1	11 00xx kkkk kkkk		
RETFIE	-	Return from interrupt	2	00 0000 0000 1001		
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk		
RETURN	-	Return from subriutine	2	00 0000 0000 1000		
SLEEP	-	Go to standby mode	1	00 0000 0110 0011	$\overline{TO}$, $\overline{PD}$	
SUBLW	k	Subtract W from literal	1	11 110x kkkk kkkk	C,DC,Z	
XORLW	k	Exclusive OR literal to W	1	11 1010 kkkk kkkk	Z	

- 1, Ha egy I/O regiszternél a saját tartalma szerepel az utasításban operandusként, minden esetben a csatlakozó lábon ténylegesen meglévő logikai szint lesz a mérvadó (nem a kimeneti regiszter tartalma).
- 2, Ha ezt az utasítást a TMR0 regiszterrel kapcsolatban hajtjuk végre (s, ahol szerepel, d = 1), az előosztó törlődik - amennyiben a TMR0-hoz van rendelve.
- 3, Amikor a PC módosul vagy ha egy teszt feltétel teljesül, az utasítás két ciklust igényel. A második ciklus úgy zajlik le, mint egy NOP.

Az utasítások ismertetésénél a mnemonic után a szintaxis szerepel, az utasítás szabályos formátuma. Az utasítás működésének jelölése után az utasítás által befolyásolt flag vagy flag-ek, majd az opkódok következnek. A 14 bites kód a 16Cxx elemekhez tartozik, a zárójelben szereplő 12 bites változat a 16C5x mikrovezérlők kódja. A következő egység részletesen bemutatja az utasítás működését, ezt az utasítás száma, a végrehajtás ciklusszáma, majd egy alkalmazási példa követi.

Ha az operandusnál zárójeles érték is található, az a 16C5x elemekre vonatkozó érték, ekkor a zárójel nélküli adat csak a 16Cxx áramkörökre érvényes. A ciklusszámnál előforduló zárójeles érték jelentése más - ez arra utal, ha a feltétel teljesül és ugrást kell végrehajtani.

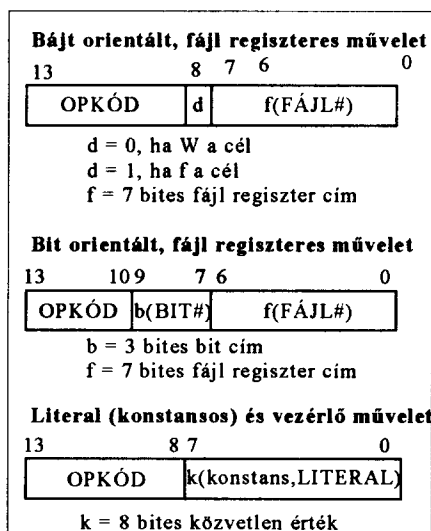
Ahol nem szerepel 12 bites zárójeles érték, az azt jelenti, hogy az illető utasítás hiányzik a 16C5x utasításkészletből.

Az utasítások ismertetésénél alkalmazott jelöléseket az 1.7. táblázat foglalja össze.

1.7. táblázat *Jelölések az utasítások ismertetésénél*

Leírási elem	Magyarázat
f	Regiszter fájl cím
W	Munkaregiszter (akkumulátor)
b	Bit cím egy 8 bites fájl regiszteren belül
k	Konstans (literal) illetve cím adat
x	Közömbös érték, ajánlott a 0 alkalmazása
d	Cél kijelölő elem, d=0: az eredmény a W-be kerül; d=1: az eredmény az f fájl regiszterbe kerül (a default érték: d=1)
label	Címke neve
TOS	A verem teteje (Top of stack)
PC	Programszámláló
PCLATH	Programszámláló felső rész latch
GIE	Globális megszakítás engedélyező bit
WDT	Watchdog Timer számláló
$\overline{TO}$	Időtúlfutás (Time-out) bit
$\overline{PD}$	Tápfeszültség lekapcsolás (Power Down) bit
dest	Cél, lehet a W vagy egy specifikált fájl regiszter egység
[]	Opcionális részlet, nem kötelező az alkalmazása
()	Tartalomra utalás
→	Működésre kijelölés
< >	Regiszter bit csoport
€	Elemkészlet megadása
dőlt betűk	Felhasználó által definiált tartalom

Az utasítások lehetséges felépítését az 1.7. ábra szemlélteti.



1.7. ábra *A PIC utasítások felépítése*

Az utasítások ismertetése tehát a mnemonikkal kezdődik, mellette az utasítás teljes neve látható, angolul. A mnemonik tulajdonképpen az utasítás angol nevéből alkotott betűszó. Az utasítások működését bemutató kis példákban gyakran szerepel számérték is, ez általában hexadecimális szám. A PIC assembler igényeinek megfelelő formátummal jelenik meg ilyenkor a hexadecimális érték, azaz 0x vezet be (pl. 0xC2).

ADDLW Add Literal to W

Szintaxis: [*cimke*] ADDLW k
 Operandus: $0 \leq k \leq 255$
 Működés: $(W) + k \rightarrow (W)$
 Változó flag: C, DC, Z
 14 bites kód: 11 111x kkkk kkkk
 (12 bites kód:) -
 Leírás: A W tartalma és a k konstans összege kerül W-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: ADDLW 0x15
 Előtte: W = 0x10
 Utána: W = 0x25

ADDWF ADD W to f

Szintaxis: [*cimke*] ADDWF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(W) + (f) \rightarrow (\text{dest})$
 Változó flag: C, DC, Z
 14 bites kód: 00 0111 dfff ffff
 (12 bites kód:) (0001 11df ffff)
 Leírás: Összeadja a W és f tartalmát. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: ADDWF FSR, 0
 Előtte: W=0x17, FSR=0xC2
 Utána: W=0xD9, FSR=0xC2

ANDLW AND Literal and W

Szintaxis: [*cimke*] ANDLW k
 Operandus: $0 \leq k \leq 255$
 Működés: $(W) .AND. k \rightarrow (W)$
 Változó flag: Z
 14 bites kód: 11 1001 kkkk kkkk
 (12 bites kód:) (1110 kkkk kkkk)
 Leírás: A W és a k konstans ÉS kapcsolata, az eredmény a W-be kerül.
 Szószám: 1
 Ciklusszám: 1
 Példa: ANDLW 0x5F
 Előtte: W=0xA3
 Utána: W=0x03

ANDWF And W with f

Szintaxis: [*cimke*] ANDWF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(W) .AND. (f) \rightarrow (\text{dest})$
 Változó flag: Z
 14 bites kód: 00 0101 dfff ffff
 (12 bites kód:) (0001 01df ffff)
 Leírás: A W és az f regiszterek tartalma ÉS kapcsolata. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: ANDWF FSR, 1
 Előtte: W=0x17, FSR=0xC2
 Utána: W=0x17, FSR=0x02

BCF Bit Clear f

Szintaxis: [*cimke*] BCF f,b
 Operandus: $0 \leq f \leq 127$ (31), $0 \leq b \leq 7$
 Működés: $0 \rightarrow f$
 Változó flag: -
 14 bites kód: 01 00bb bfff ffff
 (12 bites kód:) (0100 bbbf ffff)
 Leírás: Az f regiszterben a b bitet 0-ra törli
 Szószám: 1
 Ciklusszám: 1
 Példa: BCF FLAG_REG, 7
 Előtte: FLAG_REG = 0xC7
 Utána: FLAG_REG = 0x47

BSF Bit Set f

Szintaxis: [*cimke*] BSF f,b
 Operandus: $0 \leq f \leq 127$ (31), $0 \leq b \leq 7$
 Működés: $1 \rightarrow f$
 Változó flag: -
 14 bites kód: 01 01bb bfff ffff
 (12 bites kód:) (0101 bbbf ffff)
 Leírás: Az f regiszterben a b bitet 1-re állítja
 Szószám: 1
 Ciklusszám: 1
 Példa: BSF FLAG_REG, 7
 Előtte: FLAG_REG = 0x0A
 Utána: FLAG_REG = 0x8A

BTFSC Bit Test, skip if Clear

Szintaxis: [*cimke*] BTFSC f,b
 Operandus: $0 \leq f \leq 127$ (31), $0 \leq b \leq 7$
 Működés: skip if $(f) = 0$
 Változó flag: -
 14 bites kód: 01 10bb bfff ffff
 (12 bites kód:) (0110 bbbf ffff)
 Leírás: Ha az f regiszter b bitje 0, a következő utasítást átlépi (az időközben beolvasott utasítás helyett NOP fut le).
 Szószám: 1
 Ciklusszám: 1 (2)
 Példa: HERE BTFSC FLAG,1
 FALSE GOTO PROCESS
 TRUE
 Előtte: PC = HERE cím
 Utána, ha FLAG<1>=0, PC=TRUE
 ha FLAG<1>=1, PC=FALSE

BTFSS Bit Test, skip if Set

Szintaxis: [*cimke*] BTFSS f,b
 Operandus: $0 \leq f \leq 127$ (31), $0 \leq b \leq 7$
 Működés: skip if $(f) = 1$
 Változó flag: -
 14 bites kód: 01 11bb bfff ffff
 (12 bites kód:) (0111 bbbf ffff)
 Leírás: Ha az f regiszter b bitje 1, a következő utasítást átlépi (az időközben beolvasott utasítás helyett NOP fut le).
 Szószám: 1
 Ciklusszám: 1 (2)
 Példa: HERE BTFSS FLAG,1
 FALSE GOTO PROCESS
 TRUE
 Előtte: PC = HERE cím
 Utána, ha FLAG<1>=1, PC=TRUE
 ha FLAG<1>=0, PC=FALSE

CALL Subroutine Call

Szintaxis: [*cimke*] CALL k
 Operandus: $0 \leq k \leq 2047$ (255)
 Működés: $(PC)+1 \rightarrow TOS$; $k \rightarrow PC$
 Változó flag: -
 14 bites kód: 10 0kkk kkkk kkkk
 (12 bites kód:) (1001 kkkk kkkk)
 Leírás: Szubrutinra ugrás. A visszatérési cím a verembe kerül, a közvetlen cím, k a PC-be íródik, a felső PC bitek kezelése-típusonként változik
 Szószám: 1
 Ciklusszám: 2
 Példa: HERE CALL THERE
 Előtte: PC=HERE cím
 Utána: PC=THERE
 TOS=HERE+1

CLRF Clear f

Szintaxis: [*cimke*] CLRF f
 Operandus: $0 \leq f \leq 127$ (31)
 Működés: $00h \rightarrow f$, $1 \rightarrow Z$
 Változó flag: Z
 14 bites kód: 00 0001 1fff ffff
 (12 bites kód:) (0000 011f ffff)
 Leírás: Az f regiszter tartalma törlődik
 Szószám: 1
 Ciklusszám: 1
 Példa: CLRF FLAG_REG
 Előtte: FLAG_REG = 0x5A
 Utána: FLAG_REG = 0x00, Z=1

CLRW Clear W Register

Szintaxis: [*cimke*] CLRW
 Operandus: -
 Működés: $00h \rightarrow (W)$, $1 \rightarrow Z$
 Változó flag: Z
 14 bites kód: 00 0001 0xxx xxxx
 (12 bites kód:) (0000 0100 0000)
 Leírás: A W regiszter tartalma törlődik
 Szószám: 1
 Ciklusszám: 1
 Példa: CLRW
 Előtte: W = 0x5A
 Utána: W = 0x00, Z=1

CLRWD Clear Watchdog Timer

Szintaxis: [*cimke*] CLRWD
 Operandus: -
 Működés: $00h \rightarrow WDT$, $0 \rightarrow WDT$ előosztó,
 $1 \rightarrow \overline{TO}$, $1 \rightarrow \overline{PD}$
 Változó flag: $\overline{TO}$, $\overline{PD}$
 14 bites kód: 00 0000 0110 0100
 (12 bites kód:) (0000 0000 0100)
 Leírás: Törli a WDT-t és az előosztóját
 Szószám: 1
 Ciklusszám: 1
 Példa: CLRWD
 Utána: WDT=0, WDT előosztó=0
 $\overline{TO} = 1$, $\overline{PD} = 1$

COMF Complement f

Szintaxis: [*cimke*] COMF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $NEGf \rightarrow (dest)$
 Változó flag: Z
 14 bites kód: 00 1001 dfff ffff
 (12 bites kód:) (0010 01df ffff)
 Leírás: Az f regiszter tartalma negálódik. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: COMF REG1, 0
 Előtte: REG1 = 0x13
 Utána: REG1 = 0x13
 W = 0xEC

DECF Decrement f

Szintaxis: [*cimke*] DECF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(f)-1 \rightarrow (dest)$
 Változó flag: Z
 14 bites kód: 00 0011 dfff ffff
 (12 bites kód:) (0000 11df ffff)
 Leírás: Az f tartalma dekrementálódik. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: DECF CNT, 1
 Előtte: CNT=0x01, Z=0
 Utána: CNT=0x00, Z=1

DECFSZ Decrement f, skip if 0

Szintaxis: [*cimke*] DECFSZ f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(f)-1 \rightarrow (d)$; ha az eredmény 0: állép
 Változó flag: -
 14 bites kód: 00 1011 dfff ffff
 (12 bites kód:) (0010 11df ffff)
 Leírás: Az f tartalma dekrementálódik. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be. Ha az eredmény 0, a következő utasítást átlépi, NOP-pal helyettesíti.
 Szószám: 1
 Ciklusszám: 1 (2)
 Példa: HERE DECFSZ CNT, 1
 GOTO LOOP
 CONT
 Előtte: PC=HERE cím
 Utána: CNT=CNT-1,
 ha CNT=0, PC=CONT
 egyébként PC=HERE+1

GOTO Unconditional Branch

Szintaxis: [*cimke*] GOTO k
 Operandus: $0 \leq k \leq 2047$ (511)
 Működés: $k \rightarrow PC$
 Változó flag: -
 14 bites kód: 10 1kkk kkkk kkkk
 (12 bites kód:) (101k kkkk kkkk)
 Leírás: A közvetlen k cím betöltődik a PC-be, a PC további bitjei kezelése típusonként változik
 Szószám: 1
 Ciklusszám: 2
 Példa: GOTO THERE
 Utána: PC = THERE cím

INCF Increment f

Szintaxis: [*címke*] INCF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(f)+1 \rightarrow (\text{dest})$
 Változó flag: Z
 14 bites kód: 00 1010 dfff ffff
 (12 bites kód:) (0010 10df ffff)
 Leírás: Az f tartalma inkrementálódik. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: INCF CNT, 1
 Előtte: CNT=0xFF, Z=0
 Utána: CNT=0x00, Z=1

INCFSZ Increment f, skip if 0

Szintaxis: [*címke*] INCFSZ f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(f)+1 \rightarrow (d)$; ha az eredmény 0: átlép
 Változó flag: -
 14 bites kód: 00 1111 dfff ffff
 (12 bites kód:) (0011 11df ffff)
 Leírás: Az f tartalma inkrementálódik. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be. Ha az eredmény 0, a következő utasítást átlépi, NOP-pal helyettesíti.
 Szószám: 1
 Ciklusszám: 1 (2)
 Példa: HERE INCFSZ CNT, 1
 GOTO LOOP
 CONT
 Előtte: PC=HERE cím
 Utána: CNT=CNT+1,
 ha CNT=0, PC=CONT
 egyébként PC=HERE+1

IORLW Inclusive OR Literal with W

Szintaxis: [*címke*] IORLW k
 Operandus: $0 \leq k \leq 255$
 Működés: $(W).OR. k \rightarrow (W)$
 Változó flag: Z
 14 bites kód: 11 1000 kkkk kkkk
 (12 bites kód:) (1101 kkkk kkkk)
 Leírás: A W és a k konstans VAGY kapcsolata, az eredmény a W-be kerül.
 Szószám: 1
 Ciklusszám: 1
 Példa: IORLW 0x35
 Előtte: W=0x9A
 Utána: W=0xBF

IORWF Inclusive OR W with f

Szintaxis: [*címke*] IORWF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(W).OR. (f) \rightarrow (\text{dest})$
 Változó flag: Z
 14 bites kód: 00 0100 dfff ffff
 (12 bites kód:) (0001 00df ffff)
 Leírás: A W és az f regiszterek VAGY kapcsolata. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: IORWF RESULT, 0
 Előtte: RESULT=0x13, W=0x91
 Utána: RESULT=0x13, W=0x91

MOVLW Move Literal to W

Szintaxis: [*címke*] MOVLW k
 Operandus: $0 \leq k \leq 255$
 Működés: $k \rightarrow (W)$
 Változó flag: -
 14 bites kód: 11 00xx kkkk kkkk
 (12 bites kód:) (1100 kkkk kkkk)
 Leírás: A k konstans kerül W-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: MOVLW 0xA5
 Utána: W = 0x5A

MOVF Move f

Szintaxis: [*címke*] MOVF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in [0,1]$
 Működés: $(f) \rightarrow (\text{dest})$
 Változó flag: Z
 14 bites kód: 00 1000 dfff ffff
 (12 bites kód:) (0010 00df ffff)
 Leírás: Az f tartalmát másolja át. Ha $d=0$, az eredmény a W-be kerül, ha $d=1$, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: MOVF FSR, 0
 Utána: W = FSR

MOVWF Move W to f

Szintaxis: [*címke*] MOVWF f
 Operandus: $0 \leq f \leq 127$ (31)
 Működés: $(W) \rightarrow (f)$
 Változó flag: -
 14 bites kód: 00 0000 1fff ffff
 (12 bites kód:) (0000 001f ffff)
 Leírás: Az f regiszterbe kerül a W tartalma
 Szószám: 1
 Ciklusszám: 1
 Példa: MOVF OPTION
 Előtte: OPTION=0xFF, W=0x4F
 Utána: OPTION=0x4F, W=0x4F

NOP No Operation

Szintaxis: [*címke*] NOP
 Operandus: -
 Működés: Nincs működés
 Változó flag: -
 14 bites kód: 00 0000 0xx0 0000
 (12 bites kód:) (0000 0000 0000)
 Leírás: Nincs működés
 Szószám: 1
 Ciklusszám: 1
 Példa: NOP

OPTION Load Option Register

Szintaxis: [*cimke*] OPTION
 Operandus: -
 Működés: (W) → OPTION
 Változó flag: -
 14 bites kód: 00 0000 0110 0010
 (12 bites kód:) (0000 0000 0010)
 Leírás: A W tartalmát betölti az OPTION regiszterbe (a 16C5x elemekkel való szoftver kompatibilitást szolgálja).
 Szószám: 1
 Ciklusszám: 1
 Példa:

RETFIE Return from Interrupt

Szintaxis: [*cimke*] RETFIE
 Operandus: -
 Működés: TOS → PC, 1 → GIE
 Változó flag: -
 14 bites kód: 00 0000 0000 1001
 (12 bites kód:) -
 Leírás: Visszatérés a megszakítást kezelő szubrutinból, a megszakítások engedélyezésével
 Szószám: 1
 Ciklusszám: 2
 Példa: RETFIE
 Utána: PC=TOS, GIE=1

RETLW Return Literal to W

Szintaxis: [*cimke*] RETLW k
 Operandus: $0 \leq k \leq 255$
 Működés: k → W, TOS → PC
 Változó flag: -
 14 bites kód: 11 01xx kkkk kkkk
 (12 bites kód:) (1000 kkkk kkkk)
 Leírás: Szubrutinból visszatérés, konstans betöltéssel. A veremből a visszatérési cím a PC-be kerül, a k a W-be.
 Szószám: 1
 Ciklusszám: 2
 Példa: RETLW 0x5A
 Utána: PC=TOS, W=0x5A

RETURN Return from Subroutine

Szintaxis: [*cimke*] RETURN
 Operandus: -
 Működés: TOS → PC
 Változó flag: -
 14 bites kód: 00 0000 0000 1000
 (12 bites kód:) -
 Leírás: Szubrutinból visszatérés, a visszatérési cím a veremből a PC-be kerül.
 Szószám: 1
 Ciklusszám: 2
 Példa: RETURN
 Utána: PC=TOS

RLF Rotate Left f through Carry

Szintaxis: [*cimke*] RLF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in \{0,1\}$
 Működés: $f < n \rightarrow d < n+1$, $f < 7 \rightarrow C$,
 $C \rightarrow d < 0$
 Változó flag: C
 14 bites kód: 00 1101 dfff ffff
 (12 bites kód:) (0011 01df ffff)
 Leírás: A Carry flag-en keresztül balra léptetődik az f tartalma. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: RLF REG1, 0
 Előtte: REG1=11100110, C=0
 Utána: REG1=11100110,
 W=11001100, C=1

RRF Rotate Right f through Carry

Szintaxis: [*cimke*] RRF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in \{0,1\}$
 Működés: $f < n \rightarrow d < n-1$, $f < 0 \rightarrow C$,
 $C \rightarrow d < 7$
 Változó flag: C
 14 bites kód: 00 11000 dfff ffff
 (12 bites kód:) (0011 01df ffff)
 Leírás: A Carry flag-en keresztül jobbra léptetődik az f tartalma. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: RRF REG1, 0
 Előtte: REG1=11100110, C=0
 Utána: REG1=11100110,
 W=01110011, C=0

SLEEP

Szintaxis: [*cimke*] SLEEP
 Operandus: -
 Működés: 00h → WDT, 0 → WDT előosztó,
 $1 \rightarrow \overline{TO}, 0 \rightarrow \overline{PD}$
 Változó flag: $\overline{TO}, \overline{PD}$
 14 bites kód: 00 0000 0110 0011
 (12 bites kód:) (0000 0000 0011)
 Leírás: Az oszcillátor leáll, SLEEP módba lép a processzor.
 Szószám: 1
 Ciklusszám: 1
 Példa: SLEEP

SUBLW Subtract W from Literal

Szintaxis: [*cimke*] SUBLW k
 Operandus: $0 \leq k \leq 255$
 Működés: k - (W) → W
 Változó flag: C, DC, Z
 14 bites kód: 11 110x kkkk kkkk
 (12 bites kód:) -
 Leírás: A W tartalma kivonódik a k konstansból (a kettes komplementum módszerrel), az eredmény a W-be kerül.
 Szószám: 1
 Ciklusszám: 1
 Példa: SUBLW 0x02
 Előtte: W = 3
 Utána: W = FF, C=0 (az eredmény negatív)

SUBWF Subtract W from f

Szintaxis: [*cimke*] SUBWF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in \{0,1\}$
 Működés: $(f) - (W) \rightarrow (\text{dest})$
 Változó flag: C, DC, Z
 14 bites kód: 00 0010 dfff ffff
 (12 bites kód:) (0000 10df ffff)
 Leírás: Kivonja f-ből W tartalmát. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: SUBWF REG1, 1
 Előtte: REG1=0, W=1
 Utána: REG1=FF, W=1, C=0, az eredmény negatív

SWAPF Swap f

Szintaxis: [*cimke*] SWAPF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in \{0,1\}$
 Működés: $f < 0:3 \rightarrow d < 4:7, f < 4:7 \rightarrow d < 0:3$
 Változó flag: -
 14 bites kód: 00 1110 dfff ffff
 (12 bites kód:) (0011 10df ffff)
 Leírás: Az f alsó és felső négy bitjét (nibble) felcseréli. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: SWAPF REG, 0
 Előtte: REG=0xA5
 Utána: REG=0xA5, W=0, x5A

TRIS Load TRIS Register

Szintaxis: [*cimke*] TRIS f
 Operandus: $5 \leq f \leq 7$
 Működés: $(W) \rightarrow f$ TRIS regiszter
 Változó flag: -
 14 bites kód: 00 0000 0110 0fff
 (12 bites kód:) (0000 0000 0fff)
 Leírás: A W tartalmát betölti az f I/O vezérlő regiszterbe (a 16C5x elemekkel való szoftver kompatibilitást szolgálja).
 Szószám: 1
 Ciklusszám: 1
 Példa:

XORLW Excl. OR literal with W

Szintaxis: [*cimke*] XORLW k
 Operandus: $0 \leq k \leq 255$
 Működés: $(W) .XOR. k \rightarrow (W)$
 Változó flag: Z
 14 bites kód: 11 1010 kkkk kkkk
 (12 bites kód:) (1111 kkkk kkkk)
 Leírás: A W és a k konstans XOR kapcsolata, az eredmény a W-be kerül.
 Szószám: 1
 Ciklusszám: 1
 Példa: XORLW 0xAF
 Előtte: W=0xB5
 Utána: W=0x1A

XORWF Exclusive OR W with f

Szintaxis: [*cimke*] XORWF f,d
 Operandus: $0 \leq f \leq 127$ (31), $d \in \{0,1\}$
 Működés: $(W) .XOR. (f) \rightarrow (\text{dest})$
 Változó flag: Z
 14 bites kód: 00 0110 dfff ffff
 (12 bites kód:) (0001 10df ffff)
 Leírás: A W és az f regiszterek tartalma XOR kapcsolata. Ha d=0, az eredmény a W-be kerül, ha d=1, akkor az f-be.
 Szószám: 1
 Ciklusszám: 1
 Példa: XORWF REG, 1
 Előtte: REG=0xAF, W=0xB5
 Utána: REG=0x1A, W=0xB5

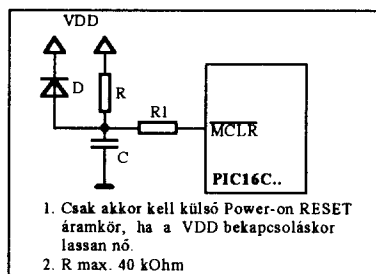
Ellenőrző kérdések

1. Melyek azok az utasítások, melyek a Z flag-et befolyásolják?
2. Melyek azok az utasítások, melyek a DC flag-et befolyásolják?
3. Melyek azok az utasítások, melyek a C flag-et befolyásolják?
4. Adott egy 12 illetve 14 bites bináris érték. Mondja meg, melyik PIC utasításnak felel meg, milyen paraméter értékkel?
5. Keresse meg az eltérő opkódú utasításokat a 16C5x illetve a 16Cxx utasításkészletben!

1.4. Bekapcsolási RESET megoldások

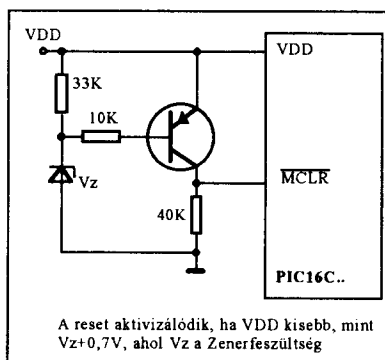
A PIC mikrovezérlők $\overline{\text{MCLR}}$ bemenetén a mikroproceszoroknál, mikrovezérlőknél megszokott, a tápfeszültség bekapcsolásakor automatikusan működő RESET áramkört (Power-Up RESET) alakíthatjuk ki (1. 8. ábra). Ez a megoldás hosszú RESET időtartamot adhat, ami akkor lehet előnyös, ha a bekapcsolt tápfeszültség csak lassan éri el a névleges értékét.

A gyakorlatban más megoldásokkal is lehet találkozni. Sok alkalmazásban az alaphelyzetbe állító bemenetet egyszerűen a tápfeszültség ponthoz kapcsolják. Ha a tápfeszültség gyorsan eléri a névleges értékét, ez az egyszerű (és olcsó) megoldás is megfelelő eredményt ad.



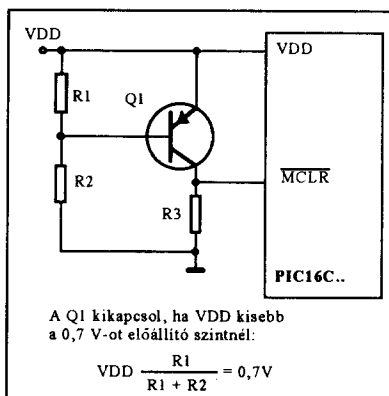
1.8. ábra Egyszerű RESET

Más esetben azonban még az RC tagnál is komolyabb áramköri megoldásokat alkalmaznak. Ha előre nem becsülhető meg a tápfeszültség növekedési üteme a bekapcsolás után, akkor a precíz megoldás az, ha a névleges értéket egy Zener diódás kapcsolás érzékeli, s a küszöbszint felett megszünteti a RESET állapotot. Egy ilyen jellegű áramkört mutat be az 1.9. ábra. Végül az 1.10. ábrán egy kom-



1.9. ábra Zener diódás, precíz RESET áramkör

romisszumos megoldás látható. Ez az áramkör is küszöbfeszültséget érzékel, de nem Zener diódával, csak egy ellenállásosztó segítségével valósítja meg a szint komparálást.



1.10. ábra Zener dióda nélküli küszöbérzékeny RESET

A RESET áramkör talán nem tűnik lényegesnek, mégis figyelmet kell fordítani rá, mert sok alkalmazáskor a rendszer megbízhatósága, üzembiztonsága múlik ezen az áramköri részleten. Ha a RESET kialakítás nem megfelelő, és a belső folyamatok nem mennek végbe a RESET időtartama alatt, a mikrovezérlő egy véletlenszerű állapottal kezdi meg a működését. ebből a helyzetből nem is képes kimenekülni, a teljes működésképtelenség alakul ki. A beépített vezérlő egységeknek általában nincs RESET gombjuk, végső megoldásként a felhasználó csak kikapcsolhatja majd ismét bekapcsolhatja a készüléket - de ekkor ismét ugyanaz a RESET folyamat játszódik csak le. Érdemes tehát a tápegység felépítését, a mikrovezérlő táplálási rendszerét tanulmányozni, s a RESET áramkört azoknak megfelelően úgy kialakítani, hogy minden bekapcsoláskor 100 %-os biztonsággal megvalósuljon a belső RESET folyamat.

A PIC mikrovezérlők belső RESET folyamata mesterségesen nyújtott, az egyes részcsaládoknál majd részletesen megismerjük ezeket az eseménysorozatokat. Ilyen körülmények között még nagyobb figyelmet érdemel a RESET jel precíz kialakítása.

Ellenőrző kérdések

1. A BASIC STAMP mikroszámítógépben a PIC táplálását egy 9V-os rádiótelep oldja meg, ami bekapcsoláskor egy 5V-os feszültség stabilizátoron át közvetlenül a mikrovezérlőre kapcsolódik. Milyen RESET megoldást alkalmazna itt?
2. Egy PIC katalógus adatai ismeretében határozza meg az egyszerű RESET áramkör elemértékeit, adott RESET időtartam eléréséhez.
3. Egy PIC katalógus adatai ismeretében határozza meg a Zener diódás RESET áramkör elemértékeit.
4. Egy PIC katalógus adatai ismeretében határozza meg a tranzisztoros küszöbértékfigyelő RESET áramkör elemértékeit.

2. A PIC16C5x mikrovezérlő csoport

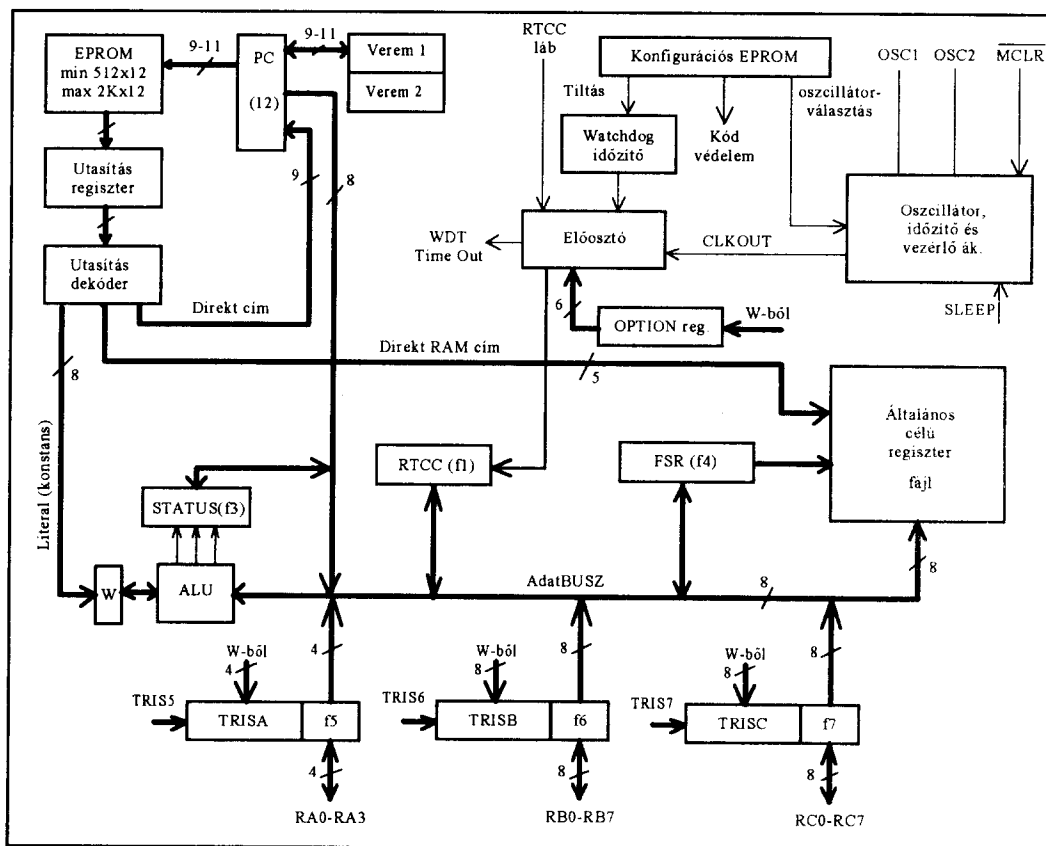
A PIC 16C5x elemsorozat a PIC mikrovezérlő család első csoportja. Ezek az áramkörök még egyszerűbb felépítésűek, kisebb képességűek - és természetesen olcsóbbak is - mint a későbbi 16C6x, 16C7x és 16C8x csipek. Jelentőségüket azonban nem veszítették el, mert sok egyszerűbb feladatra éppen ezek a mikrovezérlők ideálisak.

2.1. A PIC16C5x elemek általános leírása, belső felépítésük

A PIC16C5x áramkörök utasításkészlete, mint láttuk, 33 darab egyszavas utasításból áll, ami rendkívül hatékony, RISC jellegű programozási lehetőséget biztosít. Az utasítások az adatszónál hosszabbak, a 12 bites utasításszó nagyfokú kódkompressziót tett lehetővé - a programok tömörek és hatékonyak.

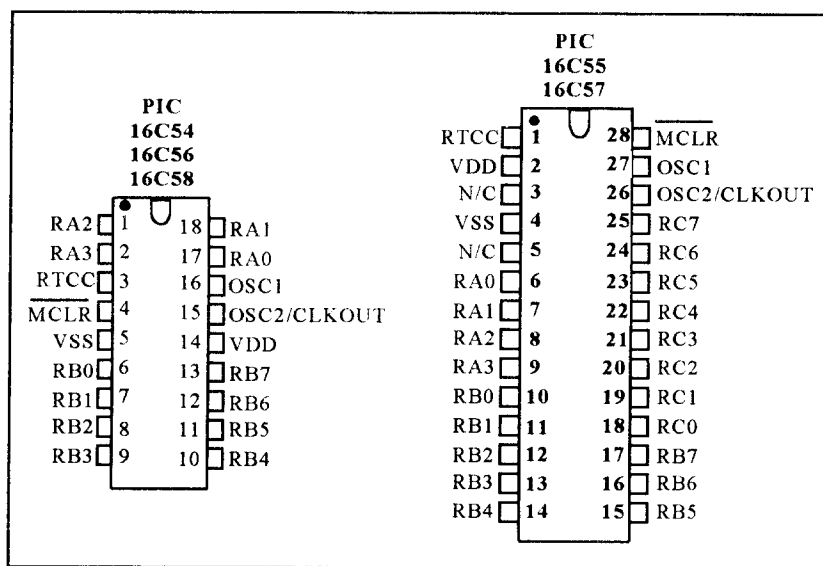
Mint általában a PIC családban, itt is megtalálhatók a programfejlesztéshez használható kvarcablakos áramkörök is és az olcsó, egyszer programozható (OTP) zárt műanyag tokozású változatok is. Az áramkörök négy oszcillátor konfiguráció egyikében működtethetők, a futási biztonságot watch-dog timer növeli, s konkrét tennivaló hiányában az áramkör stand-by állapotba (sleep) vezérelhető.

A PIC16C5x mikrovezérlők belső felépítését a **2.1. ábra** mutatja be.



2.1. ábra A PIC16C5x mikrovezérlők belső felépítése

A mikrovezérlőkre jellemző módon a 16C5x áramkörökben is megtaláljuk a programmemóriát (minden esetben EPROM), az adattárat (SRAM), az I/O elemeket, a CPU-t és egy időzítő/számláló egységet (RTCC). A belső adatBUSZ nyolc bites, az adatmemória és a CPU is, a programmemória rekeszei 12 bitesek. A CPU-t úgy optimalizálták, hogy hatékonyan dolgozik bit-, bájtt- és regiszter műveletek esetén is. A kitüntetett adatregiszter (akkumulátor) neve a PIC mikrovezérlőkben W, ide kerül a műveletek egyik operandusa, s a konstansos (literal) utasításokban lévő számérték. A veremtár (stack) is belső egység, két cím befogadására alkalmas, így kétszeres szubrutin egymásbaágyazás engedhető meg.



2.2. ábra A DIL tokozású PIC16C5x áramkörök lábkiosztása

viszont a 16C5x mikrovezérlőkből a megszakítási lehetőség.

A mikrovezérlők 18 és 28 kivezetéses DIL tokozással is készülnek, ekkor a lábkiosztásuk a 2.2. ábra szerint alakul. A kivezetések szerepe a következő:

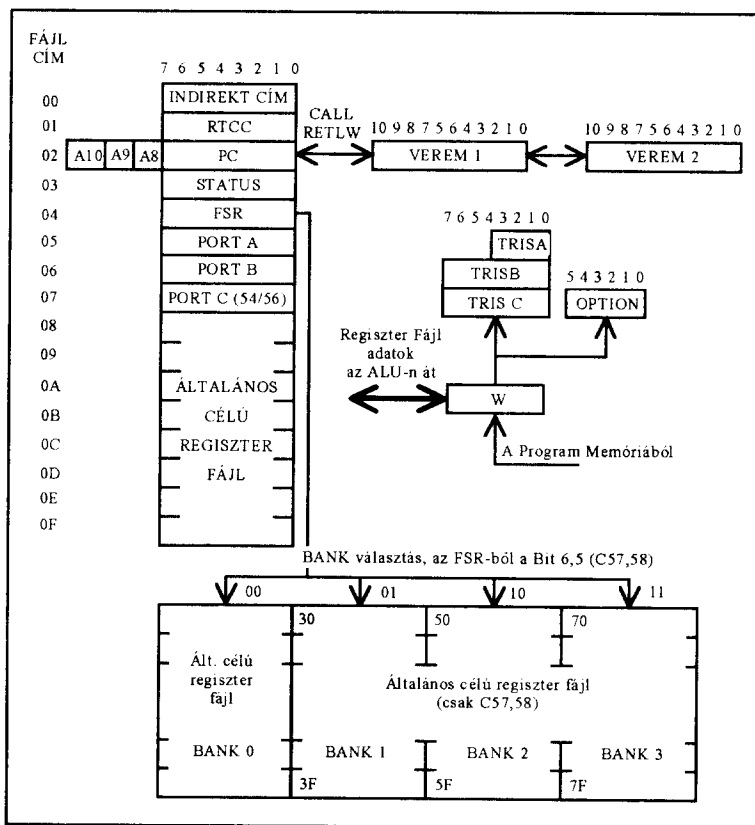
RA0...3	A Port (4 bit)
RB0...7	B Port (8 bit)
RC0...7	C Port (8 bit)
RTCC	időzítő/számláló léptető bemenete
MCLR	alaphelyzetbe állító (Reset) bemenet (Master Clear)
OSC1	oscillátor bemenet
OSC2/CLKOUT	oscillátor kimenet
VDD	tápfeszültség
VSS	0V, GND
N/C	bekötetlen láb.

A mikrovezérlők belső 8

bites adatBUSZ-a két alapvető egységet köt össze egymással, a Regiszter Fájl területet (ebben találhatók az ún. fájl regiszterek is) és a 8 bites ALU-t. A Regiszter Fájl legnagyobb mérete 80 bájt. A Regiszter Fájl egy SRAM, első 32 rekesze közvetlenül címezhető. Ha a csip nagyobb SRAM-ot tartalmaz, a további területek Bank szervezéssel érhetőek el. A SRAM szervezését a 2.3. ábra mutatja be. A SRAM elemei a következők:

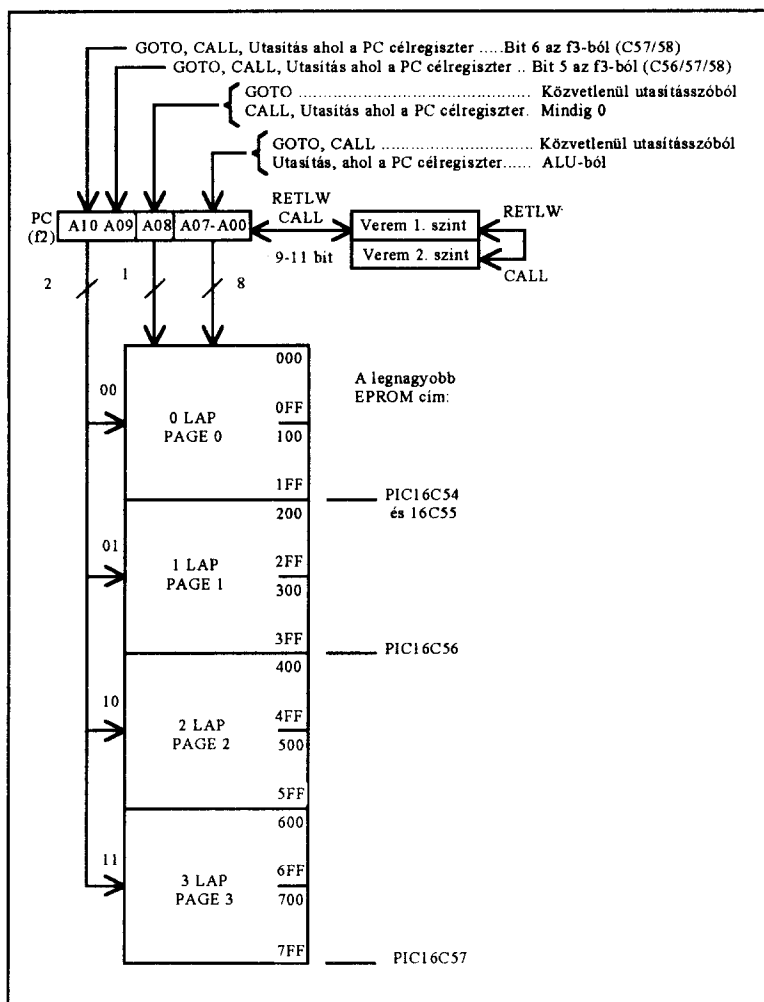
- valós idejű óra/számláló (RTCC, Real Time Clock / Counter)
- programszámláló (PC, Program Counter)
- állapotregiszter (SR, Status register)
- I/O regiszterek (I/O Registers, PORTs)

Az ábrán látható f1 ... f7 jelölések a speciális funkciójú fájl regiszterekre utalnak, ezek és az általános célú regiszter fájl elemek alkotják a belső adattárat (SRAM). A külvilággal a Port pontokon (A, B és C Port) kívül alig van kapcsolata a mikrovezérlőknek - az oszcillátor pontok, az alaphelyzetbe állító MCLR bemenet mellett már csak az RTCC-t léptető külső jelet fogadó pont található meg az áramköröknél. Teljes mértékben hiányzik



2.3. ábra A PIC16C5x elemek SRAM területe

- fájl regiszter választó regiszter (FSR, File Select Register)
- általános célú regiszterek (General Purpose registers).



2.4. ábra A PIC16C5x mikrovezérlők programmemóriája

2.2. A PIC16C5x mikrovezérlő típusok

Az ultraibolya fénnel törölhető, kvarcablakos fémkerámia (CERDIP) tokozású változatokat a 2.1. táblázat foglalja össze. Ezeknél a mikrovezérlőknél az oszcillátor típusát a fejlesztő rendszer segítségével a felhasználó állíthatja be (a default állapot: RC oszcillátor).

Az olcsóbb, egyszer programozható (OTP) változatokat a felhasználó üres EPROM-mal kapja meg, de az oszcillátor típus ezeknél már gyárilag rögzített (a kívánt oszcillátor típus szerinti típusjelet kell a rendeléskor megadni).

A QTP eszközök esetében a felhasználó a kívánt PIC programot küldi be a gyártóhoz (assembly nyelven), s programmal feltöltött, zárt tokozású elemeket kap kézhez. Megfelelő darabszám esetén ez a

Az I/O Port-ok konfigurálására és az előosztó kezelésére további speciális regiszterek szolgálnak, de azok nem elemei a Fájl regiszter területnek.

A 16C5x mikrovezérlők 512 szavas programtárat képesek közvetlenül címezni, amit EPROM-ként alakítottak ki. Egyes változatokban nagyobb kapacitású a programmemória, ilyenkor 512 szavas lapok formájában éri azt el a CPU (2.4. ábra).

A program végrehajtása közben a következő, végrehajtásra kerülő utasítás címe a programszámlálóban (PC) alakul ki, automatikus inkrementálódással. Az ugró utasítások felülírják a PC-ben kialakuló következő címet. A címzési lehetőségek: direkt, indirekt és relatív. A cím-módosító utasítások: bit teszt (Bit Test), átlépés (Skip), szubrutinra ugrás (Call), ugrás Goto), de adatmozgatással is lehet ugrani, hiszen közvetlenül is betölthető egy kívánt érték a PC-be (kiszámított ugrási cím).

2.1. táblázat A kvarcablakos PIC16C5x elemek

Típus	EPROM	RAM	I/O	Tápfesz.	Oszc. fr.	Lábszám
PIC16C54	512x12	32x8	13	4,0-5,5V	DC-20MHz	18
PIC16C55	512x12	32x8	21			28
PIC16C56	1Kx12	32x8	13			18
PIC16C57	2Kx12	80x8	21			28
PIC16C54A	512x12	32x8	13	2,5-6,25V		18
PIC16C57A	2Kx12	80x8	21			28
PIC16C58A	2Kx12	80x8	13			18

legolcsóbb megoldás, a program betöltés esetleges hibái sem fenyegetnek. Az áramkörök teljesen meg-
egyeznek a megfelelő OTP elemekkel, csak a kezelés módjában van eltérés.

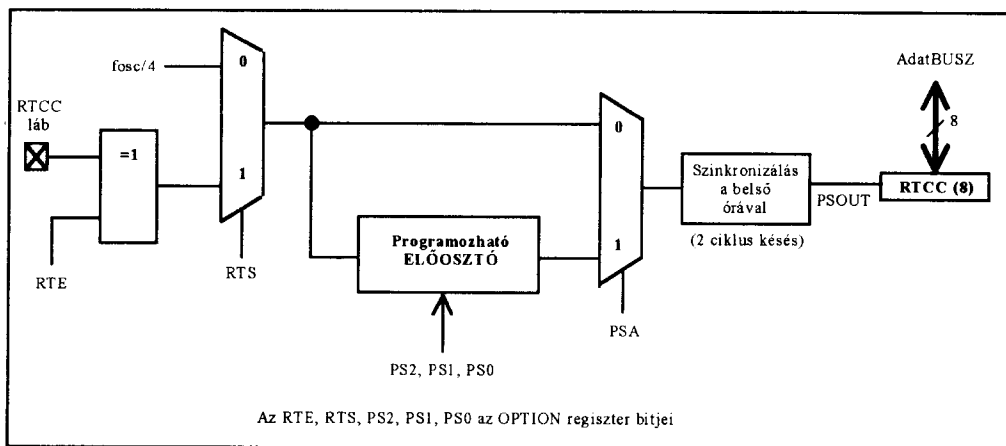
2.3. A belső működések regiszterei

f0 Indirekt adatcímzés (Indirect data Addressing)

Ez a regiszter a valóságban, fizikailag nincs kialakítva a csipen. Ha egy utasításban regiszterként az f0-t adjuk meg, az FSR (fájl regisztert választó regiszter, File Select Register) tartalma jelöli ki, címzi az operandus regisztert. Az f0 így csak egy jelzés az indirekt regiszteres címzésre. Ha indirekt címzés segítségével az f0-t olvassuk, az eredmény 0h lesz; ha írjuk, egy NOP-ot hajt végre a CPU.

f1 Óra/számláló regiszter, RTCC (Real Time Clock / Counter)

Általános, írható/olvasható rekeszként is üzemel, de sajátosan inkrementálható a tartalma, több forrásból is. Léptethető az IC RTCC jelű csatlakozó pontjára érkező impulzusokkal is, vagy a belső, utasításciklus-órajellel (CLKOUT, ami $f_{osc}/4$ frekvenciájú). Az RTCC felépítését a 2.5. ábra mutatja be. (Megjegyezzük, hogy a 16Cxx elemekben is megjelenik majd ez az áramköri részlet, de ott TMR0 lesz a jele.)



2.5. ábra Az RTCC felépítése

Az RTCC működését az OPTION regiszter bitjei vezérlik (RTE, RTS, PSA, PS0...PS2); amelyeket az OPTION utasítással lehet beállítani a kívánt értékre (az OPTION regiszter a 2.6. fejezetben található meg). Az RTCC működésébe beiktatható egy előosztó is (PSA, PS0...PS2). Ha az előosztót a számlálához rendeltük, ezután minden olyan utasítás, amelyik az f1-et írja, az előosztót törölni fogja.

Az OPTION regiszterben az RTS bit (RTCC Signal Source) határozza meg, hogy az f1 belső vagy külső esemény hatására lépjen:

RTS=1 az RTCC jelű IC lábón át, kívülről léptethető (ha RTE=1, lefutó éllel; ha RTE=0, felfutó éllel)

RTS=0 az RTCC vagy az előosztó a belső utasítás-órajellel ($f_{osc}/4$) léptethető.

Ha az RTCC tartalma a léptetések során eléri az FFh értéket, a következő léptetéskor túlszordul. Egyébként minden léptető hatás két utasításciklusnyi késéssel valósítja meg az RTCC tartalom növekedését. (Az RTCC tartalom vizsgálatának célszerű módja az, ha átírjuk a W regiszterbe (MOV f1,W).) A külső léptető jelnek néhány feltételt teljesítenie kell a belső szinkronizáció miatt. Ha nem használjuk az előosztót, a léptető jel H és L szintje időtartamára külön kapunk korlátot:

$$T_{RTH} \geq 2 \cdot t_{osc} + 20ns$$

$$T_{RTL} \geq 2 \cdot t_{osc} + 20ns$$

Ha az előosztón át lépteti külső jel az RTCC-t, a léptető jel H és L szintjei minimális időtartama 10 ns, a periódusidőre az alábbi összefüggésnek kell teljesülnie:

$$T_{RT} \geq \frac{4 \cdot t_{osc} + 40ns}{N},$$

ahol N az előosztó osztási konstansa (2, 4, ... 256 lehet).

f2 programszámláló, PC (Program Counter)

A programszámlálóban alakul ki a következő beolvasásra kerülő program szó címe (ld. 2.4. ábra). A PIC16C5x változatokban a PC (és a veremtár rekeszek) mérete változó:

	PC, veremtár rekesz hossza (bit)
PIC16C54/55	9
PIC16C56	10
PIC16C57/58	11

A RESET folyamat hatására a PC tiszta 1 tartalmat vesz fel. Az utasítások végrehajtása közben automatikusan inkrementálódik, a vezérlésátadó, ugró utasítások közvetlenül módosítják a tartalmát:

- a, GOTO a PC alsó 9 bitjét közvetlenül betöltő utasítás (PC0...8). A PIC16C56/57/58 felső PC bitjeit a STATUS regiszter lap előválasztó bitjei (Page Select, PA0, PA1 azaz STATUS5,6) állítják be
- b, CALL a PC következő utasításra mutató értékét a verembe menti (stack), a PC alsó 8 bitjét közvetlenül betölti (PC0...7); a kilencedik bitet (PC8) nullázza. A PIC16C56/57/58 esetén a felső PC bitek a PA0, PA1 STATUS bitekkel töltődnek fel
- c, RETLW a PC tartalmat a veremből állítja be
- d, ha egy utasításban a PC szerepel célregiszterként (pl. MOVf2, ADDWf2, BSf2,5) - a kiszámított 8 bites érték töltődik a PC alsó 8 bitjére (PC0...7), a kilencedik bit (PC8) törlődik. A PC felső bitek a STATUS regiszter lap előválasztó bitjeiből töltődnek be.

Látható, hogy a PC közvetlen írásakor és a CALL utasításkor a PC8 minden esetben 0 értékre vált. Ezért a kiszámított ugrásokkal elérhető programrészeket és a CALL utasítással elérhető szubrutinokat mindig az 512 rekeszes programmemória lapok első 256 rekeszére, az első féllapra kell helyezni (000...0FF, 200...2FF, 400...4FF, 600...6FF).

A PIC16C56/57/58 mikrovezérlőkben a programtár lapkijelölését a STATUS 5,6 bitek, a PA0, PA1 végzik el (lap előválasztó bitek), de ezek a PC felső bitjeitől fizikailag teljesen függetlenek s csak egyes utasítások hatására töltődnek át a PC-be. A PC egy folyamatos számlánc, így ha egy lap utolsó utasítását hajtja végre a CPU, közben a PC inkrementálódik és a következő lap első utasítására mutat, de ezenközben a STATUS regiszterben a PA bitek nem változtak meg. Egy következő GOTO, CALL stb. utasítás ezért ismét az előző lapra fog hivatkozni, hacsak időközben át nem írtuk a PA biteket is.

A RESET folyamat után a STATUS regiszterben a lap előválasztó bitek 0-ra állnak be (PA0=PA1=0), míg a programszámláló tiszta 1-t tartalmazva az utolsó lap utolsó rekeszére mutat. Ha ott egy GOTO utasítás áll, a CPU máris az első lapon (lapcíme 00) folytatja a program elemek kiolvasását.

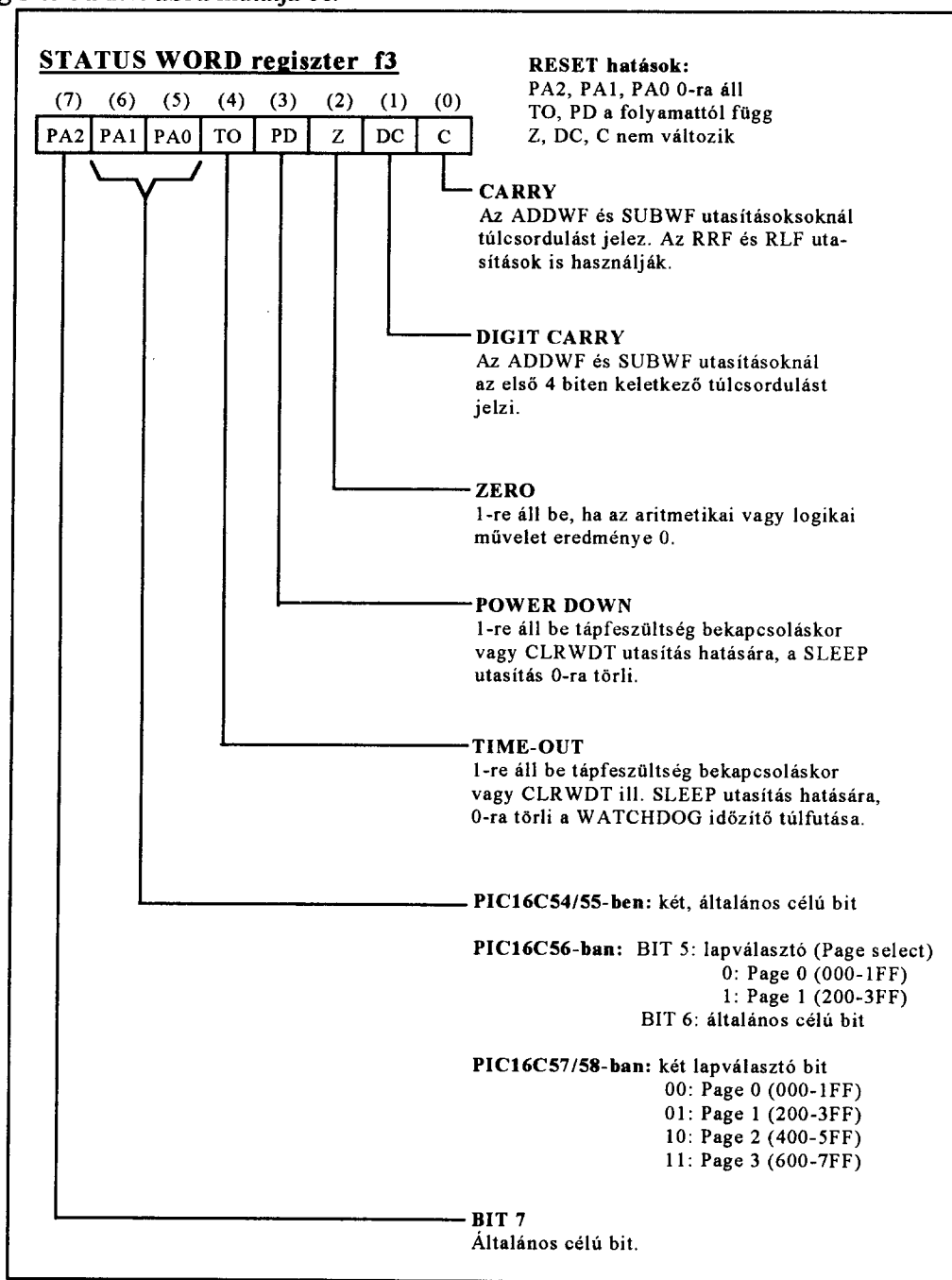
A PIC16C5x elemek kétszintű veremtárat tartalmaznak, ami kizárólag a szubrutinkezelést szolgálja. A CALL utasítás a verem 1. szint tartalmát áttolja a 2. szintre, az 1. szintre betölti a hívó program következő utasítása címét. Ha újabb szubrutinhívás történik mielőtt egy RETLW-t végrehajtana a mikrovezérlő, a most a 2. szinten őrzött cím elvész. Mivel a veremtár rekeszmérete a teljes PC befogadására elegendő, a CALL utasítás a programmemóriában bárhova elhelyezhető. A RETLW utasítás hatására a verem 1. szintjéről a PC feltöltődik, s a 2. szint tartalma átkerül az 1. szintre. A lapválasztó bitek is elmentődnek a verembe s ilyenkor visszaíródnak, a program tehát azon a lapon folytatódik, ahol a CALL utasítás is volt - függetlenül f3 pillanatnyi tartalmától. A RETLW utasításban egy 8 bites konstans is szerepel (literal), amit a W-be tölt be a CPU.

f3 Státusz regiszter, SWR (Status Word register)

Ez a regiszter informál az ALU-ban végbement folyamatról s tartalmaz néhány speciális információs bitet is. A nagyobb méretű programtárolók kezeléséhez itt találhatóak a lap előválasztó bitek is. Az f3 általánosan kezelhető, írható/olvasható regiszter, de nem célszerű ilyen értelemben használni, a státusz bitek állítását célszerű a CPU-ra bízni. Két bit egyébként is csak olvasható (TO, PD). Ha pl. egy törlést írunk elő az f3 regiszterre (CLRF f3), a STATUS regiszter tartalma ez lesz:

000UU100,

ahol U a korábbi érték őrzését jelzi. A Z=1 pedig egy önmagának ellentmondó érték. Ajánlatos az f3-mal kapcsolatban kerülni az olyan utasításokat, melyek megváltoztatják a tartalmát. A STATUS regisztert a 2.6. ábra mutatja be.



2.6. ábra A STATUS regiszter (16C5x)

Az egyes bitek szerepe az ábráról leolvasható. A TO és a PD bitek lehetővé teszik, hogy egy RESET folyamat után a CPU megállapíthassa a RESET-et kiváltó esemény jellegét. Több esemény is kiválthatja ugyanis a RESET folyamatot - így a tápfeszültség bekapcsolás, a Watchdog Timet időtúlfutása, a SLEEP utáni "ébredés", a wake-up. A biteket működtető hatások a következők:

esemény:	TO	PD
Power-Up (bekapcs.)	1	1
WDT időtúlfutás	0	X
SLEEP utasítás	1	0
CLRWDT utasítás	1	1

Egy RESET folyamat után a TO, PD bitek állása alapján a CPU felismeri a megelőző eseményeket:

TO	PD	RESET oka
0	0	WDT időtúlfutás SLEEP-ben
0	1	WDT időtúlfutás, nem SLEEP-ben
1	0	MCLR -ébresztés SLEEP-ből
1	1	Power-Up, bekapcsolás
X	X	LOW impulzus az MCLR bemeneten (a TO, PD biteket nem befolyásolja)

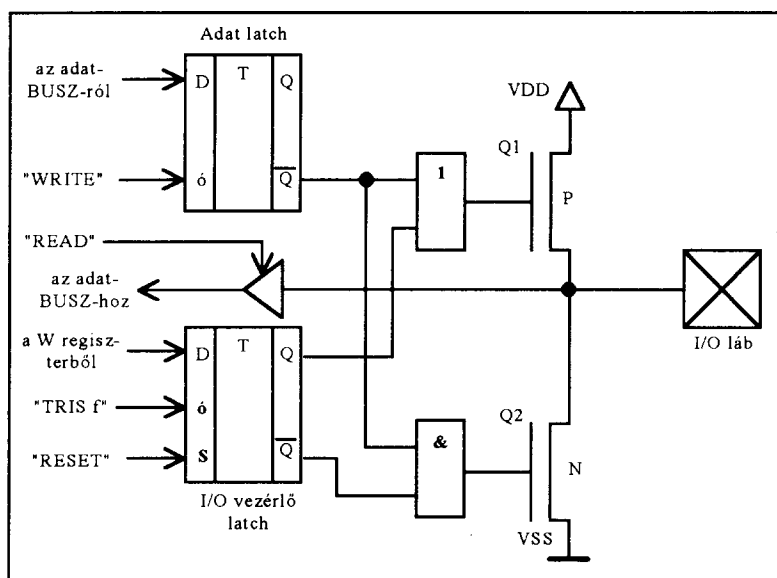
f4 Fájl regisztert választó regiszter, FSR (File select register)

A PIC16C54/55/56-ban a 0...4 bitek 32 lehetséges értéke a rendelkezésre álló 32 fájl regiszter egyikét jelöli ki, indirekt címzési mód esetére (amire az utasításban az f0 utal). Az 5...7 bitek csak olvashatóak, kiolvasva 1-t adnak. Ha nem használjuk az indirekt címzést, az f4 regisztert ötbites általános célú regiszterként lehet működtetni.

A PIC16C57/58-ban az 5...6 bitek a kurrens adatBank számát tartalmazzák (ld. 2.3. ábra). Az összes Bank alsó 16 bájtja fizikailag egyazon 16 bájt, s mindig kiválasztódik, ha a címben a bit4 (pl. indirekt címzéskor az FSR4) 0 értékű. Ha a címértéken belül a Bit4=1, azaz a felső 16 bájtra utal, akkor jut szerephez az FSR5,6 bitpáros, kijelölve az adatmemóriában a 0...3 adatBank egyikét.

A Bit7 mindegyik változatban csak olvasható, kiolvasva 1-t ad.

2.4. Az I/O működések



2.7. ábra I/O Port (16C5x)

Az I/O regisztereket ugyanúgy lehet programból kezelni, mint bármelyik egyéb fájl regisztert. Az olvasás (READ) művelet minden esetben a Port csatlakozó lábai állapotát olvassa be, függetlenül attól, hogy a pont bemenetként vagy kimenetként van-e konfigurálva. A RESET folyamat hatására minden Port pont bemenet (nagy impedanciás) állapotú lesz, azaz az I/O vezérlő regiszterekben (TRISA, TRISB, TRISC) minden bit 1-re áll be. A vezérlő bájtot a W regiszterben kell előkészíteni, s onnan egy TRISf utasítással lehet betölteni a megfelelő TRIS regiszterbe.

f5 Az A Port I/O regisztere (RA0...RA3)

4 bites I/O regiszter, csak a kisebb helyiértékű bitek működnek (a felső bitek olvasáskor 0-t adnak).

f6 A B Port I/O regisztere (RB0...RB7)

8 bites I/O regiszter.

f7 A C Port I/O regisztere (RC0...RC7)

PIC16C55/57: 8 bites I/O regiszter

PIC16C54/56/58: 8 bites általános célú regiszter

Egy Port bit belső áramköri kialakítása látható a 2.7. ábrán. Minden Port bit használható akár bemenetként, akár kimenetként. Bemenetként nincs tárolás, a kimenő adatot tárolja a mikrovezérlő. Az egyes Port bitek egymástól függetlenül állíthatóak a vezérlő regiszter útján bemenetre (vezérlő bit:1) vagy kimenetre (vezérlő bit:0). Ha egy Port pont bemeneti állapotú (nagy impedanciás), külső fel- vagy

lehúzó ellenállás segítségével, huzalozott kapu jelleggel több vezérlő kimenet is ráköthető egyetlen ilyen PIC csatlakozó lábra.

Egy kimenetre író utasítás eredménye a Port ponton a következő ciklus elején jelenik meg. A Port bemenetként történő olvasásakor a ciklus elején történik meg a mintavétel. Közvetlenül egymás után alkalmazott Port írási és olvasási utasítás esetén zavar támadhat, ezért célszerű a két utasítás közé egy NOP beiktatása.

2.5. Általános célú regiszterek

A PIC16C54/55/56 esetén 24 ilyen regiszter van, az f08...f1FH címeken.

A PIC16C57/58 mikrovezérlőkben az adattár regiszter Bank-ekre tagolódik, a teljes regiszter tartomány a következő:

f08...f0Fh	általános célú regiszterek, 8 db. Bank független regiszter
f10...f1Fh	a Bank0 16 általános célú regisztere
f20...f2Fh	fizikailag azonos az f00...f0Fh regiszterekkel
f30...f3Fh	a Bank1 általános célú regiszterei
f40...f4Fh	fizikailag azonos az f00...f0Fh regiszterekkel
f50...f5Fh	a Bank2 általános célú regiszterei
f60...f6Fh	fizikailag azonos az f00...f0Fh regiszterekkel
f70...f7Fh	a Bank3 általános célú regiszterei

Összesen tehát 80 darab fizikailag létező bájt áll rendelkezésre.

2.6. Speciális regiszterek

Munkaregiszter, W (Working Register)

8 bites ideiglenes regiszter, a két operandusos utasítások egyik operandusának előkészítő helye, gyakran az eredmény megjelenési helye.

f5 A Port vezérlő regiszter, TRISA (I/O Control register for Port A)

Az A Port adatirányt vezérlő regisztere. Csak a 0...3 bitek működnek, mivel az A Port 4 bites. (0=kimenet, 1=bemenet).

f6 B Port vezérlő regiszter, TRISB (I/O Control register for Port B)

A B Port adatirányt kijelölő regisztere (0=kimenet, 1=bemenet)

f7 C Port vezérlő regiszter, TRISC (I/O Control register for Port C)

A C Port adatirányt kijelölő regisztere (0=kimenet, 1=bemenet)

A TRIS regiszterek a W regiszterből tölthetők fel, TRISf utasítással. A TRIS-ek csak írhatóak, a RESET folyamat során tiszta 1-re töltődnek fel (minden Port bit bemenet).

Opció regiszter, OPTION (Prescaler/RTCC Option Register)

Ez a regiszter határozza meg, hogy az előosztó az RTCC-hez vagy a WDT (Watchdog Timer) kapcsolódjon, beállítja az előosztó osztási konstansát, kijelöli az RTCC külső léptető jele aktív élet, választ az RTCC külső/belső léptetése között. Mindezeket a funkciókat hat működő bittel látja el az OPTION regiszter (2.8. ábra).

Az egyes bitek szerepe az ábráról leolvasható. A három legkiseb helyiértékű bittel lehet megadni az előosztó osztási értékét, a konkrét érték attól is függ, milyen szerepet programozunk az előosztónak a PSA bittel.

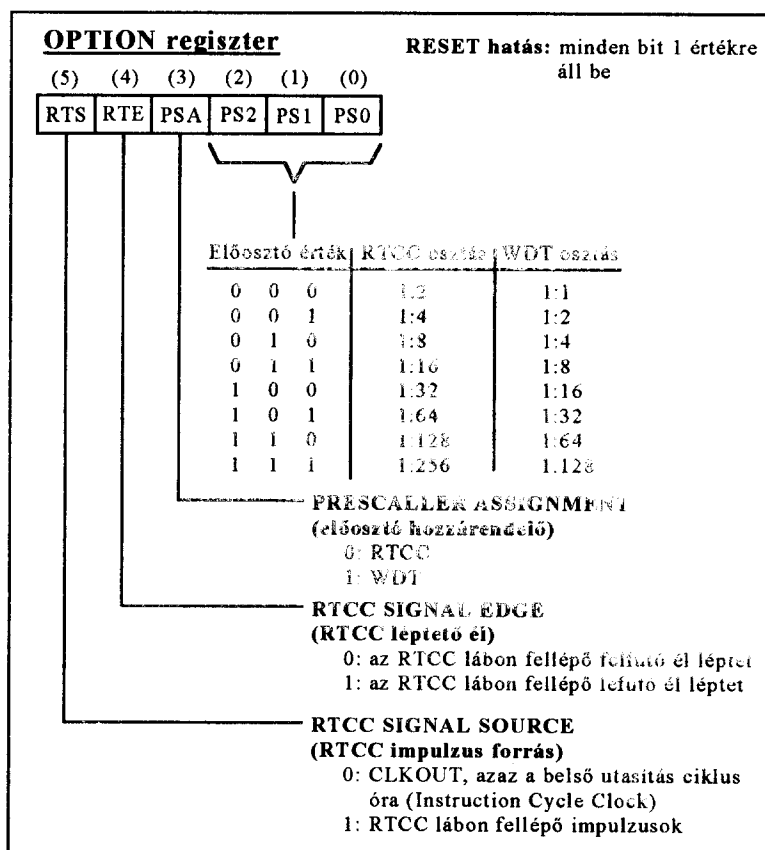
A W regiszterben kell előkészíteni az OPTION regiszter tartalmát, amit egy OPTION utasítás tölt a helyére. A RESET folyamat az OPTION regiszter minden bitjét 1-re állítja.

2.7. A RESET folyamat

A RESET folyamatot három esemény indíthatja el:

- a tápfeszültség bekapcsolás (Power-Up),
- egy LOW impulzus az MCLR bemeneten,
- a Watchdog Timer időtúlfutása (Time-Out).

A mikrovezérlő RESET állapotban marad, míg az MCLR LOW szintű, majd azután is, amíg az oscillátor start-up timer (OST), az oscillátor indulást figyelő időzítő még nem engedélyezi a működést. Az OST azonnal aktivizálódik, amint az MCLR bemeneten HIGH szint alakul ki a LOW után. A Power-Up Reset belső folyamat, maga a mikrovezérlő érzékeli a tápfeszültség bekapcsolását - eközben az MCLR HIGH szinten van, azaz a tápfeszültség megjelenésekor azonnal indul az OST. Ha a WDT időtúlfutás okozta a



2.8. ábra Az OPTION regiszter (16C5x)

RESET folyamatot, a time-out jel végénél közvetlenül indul az OST (az MCLR most is folyamatosan HIGH szinten áll). Az OST működésének névleges időtartama 18 ms, működéséről további részletek a 2.10. fejezetben olvashatók.

A PIC mikrovezérlőkben a RESET hatása a következő:

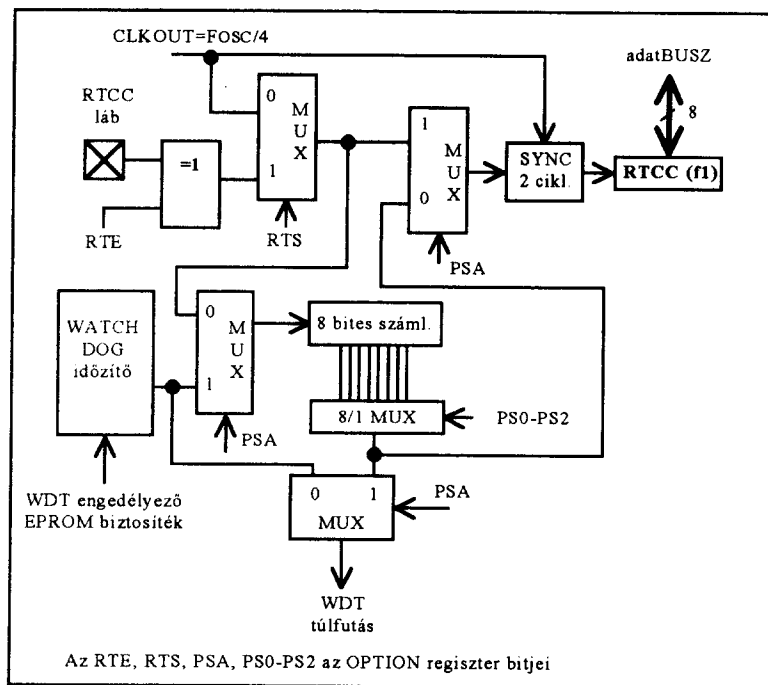
- az oscillátor fut illetve megindul (power-up ill. ébredő, wake-up működés, SLEEP-ből)
- minden I/O láb nagyimpedanciás (bemenet) állapotba kerül, a TRIS bitek mindegyike 1 lesz
- a PC programszámláló minden bite 1 értékű lesz, azaz

PIC16C54/55-ben	PC=1FFh
PIC16C56-ban	PC=3FFh
PIC16C57/58-ban	PC=7FFh
- az OPTION regiszter tiszta 1-re áll be
- a Watchdog-Timer és előosztója törlődik
- a STATUS regiszter (f3) felső 3 bite (lap előválasztó bitek) törlődnek
- RC változatoknál: a CLKOUT jel az OSC2 lábon LOW szintű.

2.8. Az előosztó és működési módjai

A 2.9. ábra mutatja be az előosztó kapcsolódási lehetőségeit. Ez az áramkör vagy az RTCC-nél, vagy a WDT-nél jelenik meg, ahogyan a felhasználó a mikrovezérlőt programozza. Egyidejűleg csak az egyik feladatra használható. Általában előosztónak nevezzük, bár abban az esetben, ha a WDT mellé rendeljük, tulajdonképpen utánosztó feladatot lát el. A felhasználás jellegét, az osztási arányt az OPTION regiszterben lévő PSA és PS0...PS2 bitek kezelik.

Ha az RTCC-hez rendeljük az előosztót, minden olyan utasítás, amelyik az RTCC-t írja, az előosztót is törli. Ha a WDT-hez rendeljük hozzá, egy CLRWDT utasítás használható fel a törlésére, a Watchdog timer törlésével egyidejűleg. Az előosztó az RTCC-ről a WDT-re és vissza folyamatos programfutás közben is átkapcsolható, megfelelő programlépések hatására.



2.9. ábra Az előosztó áramkörei (16C5x)

2.9. A Watchdog-Timer (WDT)

A Watchdog-Timer egy szabadon futó RC oszcillátor a csipen, amely nem igényel külső komponenseket. Ez azt is jelenti, hogy a WDT akkor is fut, ha az órajel az OSC1/OSC2 pontokon szünetel, pl. egy SLEEP utasítást követően. A WDT időtúlfutása (Time-Out) egy RESET feltételt generál. Ha a WDT szolgáltatását igénybe vesszük, az idő túlfutás elérése előtt rendszeresen törölnünk kell. A normál program végrehajtás közben ezek a törölő parancsok meggátolják a RESET kialakulását. Ha valamilyen hatásra a program

normális futása megakad, a WDT nem kap törölő jelet, kiváltja a RESET-et - aminek a hatására a mikrovezérlő a teljes programrendszerét ismét felépíti, s ismét működőképes lesz.

A mikrovezérlők gyakran villamos zajjal szennyezett környezetben dolgoznak, a tápellátó rendszeren át, sőt, az I/O pontjaikon keresztül is érhetik olyan impulzusok őket, melyek hatására pl. a programterületről kiugrik egy véletlen területre a memóriában, vagy olyan adatértéket alakít ki, amelyik miatt végtelen ciklusba kerül. Ezek azok az állapotok, amikor a WDT rendszeres törlése elmarad, kialakul a RESET folyamat - s ismét feláll a helyes szoftver rendszer.

A WDT működését le lehet tiltani egy konfigurációs biztosíték kiégetésével. A konfigurációs biztosítékok valójában egy EPROM rekesz bitjei, és ugyanúgy programozhatóak és törölhetőek, mint a programmemória rekeszei, de a felhasználó számára közvetlenül nem érhetőek el. A fejlesztő rendszer ajánlja fel ezeket a biztosítékokat beállításra, s a mikrovezérlő EPROM törlésekor (az ablakos típusoknál) ezek is alapállapotba kerülnek (további részletek a 2.12. fejezetben).

A WDT névleges futási ideje 18 ms, ha az előosztót nem rendeljük hozzá. A tényleges időtartam függ a tápfeszültségtől, a környezeti hőmérséklettől, de még a mikrovezérlő egyes példányaitól is. Az előosztóval a futási idő megnövelhető, a legnagyobb osztásarány (128) alkalmazásakor az időtartam már 2,5s.

A CLRWDT utasításon kívül a SLEEP utasítás is törli a WDT számlálóját és előosztóját. Így lehetséges a Sleep állapotból a WDT segítségével történő ébresztés (Wake-Up). Amikor letelik a WDT periódusideje, fellép az idő-túlfutás.

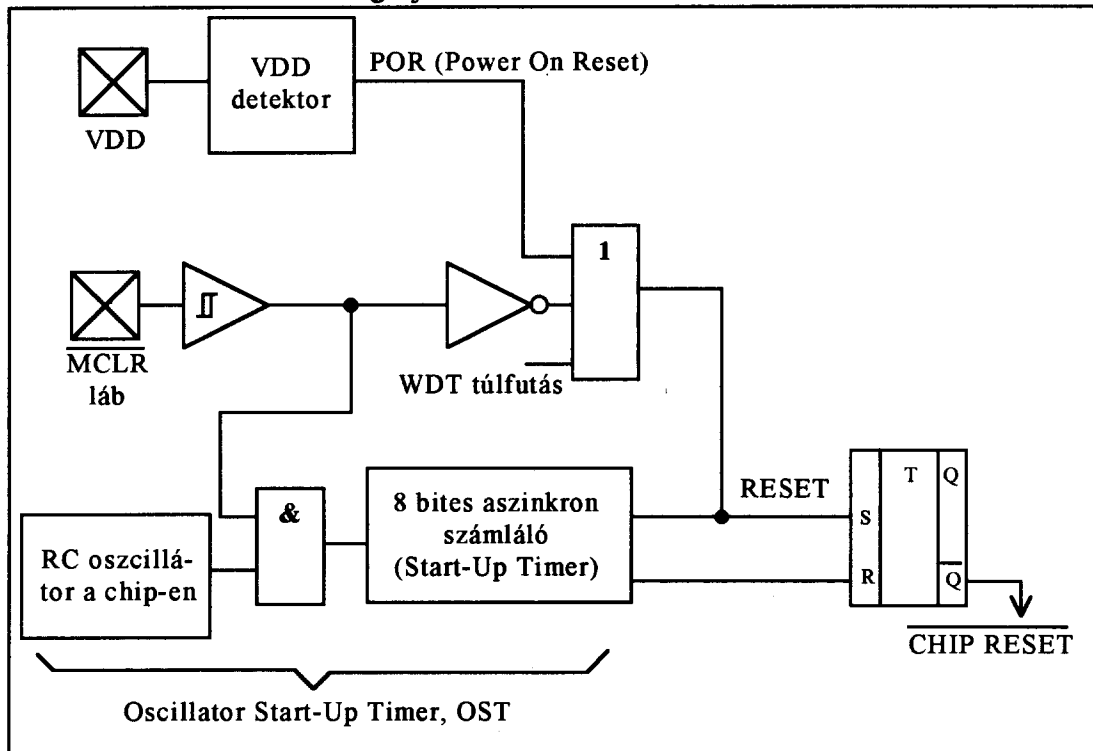
2.10. Az oszcillátor start-up működés

A mikrovezérlő belső oszcillátora, melyet a külső frekvencia meghatározó elemekkel (RC tag, rezgőkvarc vagy kerámia rezonátor) hangolunk, a tápfeszültség bekapcsolásakor nem képes azonnal szabályos alakú és időtartamú impulzusokat előállítani. Ezért integráltak a csipre egy különleges áramkört, az oszcillátor start-up időzítő egységet (Oscillator Start-up Timer, OST). Ez megoldja, hogy abban az időszakban, amikor még nem tud precízen működni az oszcillátor, a mikrovezérlő még ne is kísérelje meg a működést. Amikor az oszcillátor már beállt, a szabályos impulzusokat szolgáltatja, akkor indulhat meg a belső működés.

Amint a RESET folyamat tárgyalásánál már szóba került, az oszcillátor start-up áramkör a külső törölő jel megszűnése után még kb. 18 ms-ig gátolja a belső működést. A WDT időtúlfutás (Time-Out)

funkciója is aktivizálja az OST áramkört, ami akkor hasznos, ha a WDT Sleep állapotból ébreszti a mikrovezérlőt.

A PIC mikrovezérlők, mint már tudjuk, egy belső, tápfeszültség bekapcsolásra aktivizálódó RESET áramkörrel is rendelkeznek (Power-On Reset, POR). Ez aktivizálódik, ha a külső alaphelyzetbe állító csatlakozó pontot, az MCLR pontot a mikrovezérlő tápfeszültség pontjához (VDD) kötjük. A teljes áramkört a 2.10. ábra foglalja össze.



2.10. ábra A POR/OST áramkör (PIC16C5x)

A RESET folyamat az SR flip/flop beírásával kezdődik, ennek hatására a start-up timer is törlődik. Amikor az MCLR ponton HIGH szint jelenik meg, akkor indul a számlálási folyamat. A kb. 18 ms számlálási idő végén a flip/flop törlődik, lezárul a belső RESET folyamat. Mindaddig nincs szükség külső RC RESET hálózatra, amíg a VDD növekedési üteme a bekapcsoláskor nem kisebb 0,005 V/ms-nál.

2.11. Kis fogyasztású üzemmód (Sleep)

A csökkentett tápteljesítményű állapotba (Sleep, alvás) a SLEEP utasítás hatására lép be a mikrovezérlő. Ha engedélyezzük, a WDT törlődik és tovább számlál; törlődik az f3 státusz regiszterben a PD bit; a TO 1-re vált és az oszcillátor meghajtó áramkör kikapcsolódik. Az I/O Portok megőrzik a SLEEP előtti állapotukat.

A legkisebb teljesítmény felvétel úgy érhető el, ha az I/O pontok VDD vagy VSS szintre kerülnek, az RTCC bemenettel együtt; az MCLR pedig V_{IHMC} szintű lesz (min. 0,85 VDD).

A Sleep állapotból az ébredés két úton érhető el:

- a WDT időtúlfutása miatt,
- az MCLR lábon fellépő LOW impulzus hatására.

Bármelyik eset áll elő, a jelek megszűnésekor indul az oszcillátor start-up időzítő és ennek számlálási idejeig a mikrovezérlő még RESET helyzetben marad, majd megindul a normál működés.

A STATUS regiszterben lévő PD bit a SLEEP parancs hatására 1-re váltott, ébredés után jelzi a Sleep állapotot.

2.12. Konfigurációs biztosítékok

A PIC mikrovezérlők néhány alapvető jellemzőjét konfigurációs biztosítékokkal lehet befolyásolni. Ezek is EPROM bitek, de a programtárolásra használt memória területtől függetlenek.

Két biztosíték szolgál az oszcillátor típus beállítására, egy a WDT engedélyezésére, egy a programkódot védő, titkosító biztosíték. A PIC fejlesztő rendszerek speciális parancsok hatására kezelik ezeket a biztosítékokat. Az OTP változatoknál az oszcillátor típust a gyártó állítja be, s a típusjelben ezt fel is tünteti (XT, LP, HS vagy RC).

A PIC16C5x áramkörökben 16 speciális EPROM bit is található, melyek nem részei a normál programmemóriának. Ezek a bitek is a felhasználó rendelkezésére állnak, ide lehet elhelyezni pl. egy azonosítót (ID, Identifier), egy check-sum értéket vagy más értékes információt. Normál programvégrehajtás közben ezek a bitek nem érhetőek el, ezeket is a fejlesztő rendszer parancsaival lehet kezelni.

A kód titkosító biztosíték 0-ra programozásával az EPROM tartalom nem olvasható ki, a tárolt bitminták helyett egy torz adatsor lép ki. A 040h címtől nem is programozható a programtár, de az alsó részét még lehet programozni (0...3Fh). A védett programrekeszt olvasva a következő eredmény adódik:

00000000xxxxb,

ahol az xxxx a tényleges kódtól függő 0,1 sorozat. Összehasonlításra, ellenőrzésre tehát ekkor is van mód.

Ellenőrző kérdések

1. Melyik 16C5x változatban hány darab, hány bites Port található?
2. Melyik 16C5x változatban mekkora a programtár kapacitása?
3. Melyik 16C5x változatban mekkora a SRAM?
4. Hogyan adódik a 80 bájtos adattár méret?
5. Hogyan kezeli a PC-t a GOTO utasítás?
6. Hogyan kezeli a PC-t a CALL utasítás?
7. Hogyan kezelik a PC-t az oda író adatmozgató utasítások?
8. Magyarázza el az indirekt RAM címzés folyamatát.
9. Milyen léptetési lehetőségek vannak az RTCC-nél?
10. Adja meg a vezérlő bitek értékét az RTCC egy meghatározott elrendezéséhez.
11. Adott a STATUS regiszter tartalma. Állapítsa meg az egyes bitek jelentését, hatását.
12. Adott az OPTION regiszter tartalma. Állapítsa meg az egyes bitek jelentését, hatását.
13. Egy kijelölt Port előírt bemeneti/kimeneti viselkedéséhez adja meg a TRIS biteket.
14. Hányféle ébredési lehetősége van a 16C5x mikrovezérlőknek a Sleep állapotból?

3. A PIC16Cxx mikrovezérlők

A Microchip a 16C5x mikrovezérlők sikerét látva rendkívül gyors ütemű fejlesztésbe kezdett. Rövidesen egy új PIC generációval jelent meg, az első ide sorolható áramkörök a PIC16C71 és a PIC16C84 voltak. Mindkettő számos különleges egyedi tulajdonsággal rendelkezik, de az alapfelépítés is jelentősen módosult. Ez a módosított alapfelépítés annyira bevált, hogy egy komplett második generációt alkotott meg vele a gyártó. Az ide tartozó áramkörök, a PIC16C6x, PIC16C7x, PIC16C8x változatok közös megnevezésére használjuk a PIC16Cxx jelölést.

A PIC16C5x mikrovezérlők sok felhasználója hiányolta a megszakítási lehetőségeket - a 16Cxx család már sokoldalú megszakítási rendszerrel készül. Az új generáció is alapvetően CMOS EPROM bázisú, csak a 16C8x áramkörök alapulnak CMOS EEPROM technológiára. A CPU továbbra is RISC jellegű, 8 bites, az utasításkészlet gyakorlatilag nem változott. Az utasítás szóhossza 14 bitre változott, mivel a címek is hosszabbak lettek. A PC két bájtra bővült és a STATUS regiszterben megszűntek a program memória lap előválasztó bitek. A PIC16Cxx elemek is az átfedéses utasítás végrehajtási elv szerint működnek (instruction pipeline), azaz itt is egymást átfedve valósul meg az utasítás beolvasás és a végrehajtás.

A belső elemkészletben sok újdonság fedezhető fel. Megnőtt a programmemória és az adatmemória kapacitása, módosult a belső memóriák kezelése, címzési rendszere is. Minden speciális funkciójú regiszter bekerült a fájl regiszter területre. Újabb, sokoldalúan használható időzítő/számláló áramkörök jelentek meg, impulzus szélesség modulált (PWM) jel kialakítására is van lehetőség. A sorozat egyes tagjaiban soros kommunikációs Port, szinkron soros Port és Slave párhuzamos Port is található. Ha igény van rá, olyan mikrovezérlő is választható a 16Cxx elemek közül, amelyben A/D konverter van, analóg multiplexerrel.

A jegyzet első fejezetében leírtak érvényesek a 16Cxx csoport tagjaira is. A négy oszcillátor lehetőség, a Sleep állapot továbbra is a mikrovezérlők jellemzője, újdonság, hogy a Sleep állapotból megszakítással is kimozdíthatók ezek az áramkörök. A 16Cxx változatokkal való ismerkedést úgy kezdjük el, hogy a 3.1. fejezetben részletesen megismerjük a 16C74 képességeit, működését. Azért esett erre a mikrovezérlőre a választásunk, mert - az EEPROM részletek kivételével - ebben minden egység megtalálható, amit a 16Cxx változatok nyújtani képesek. Ennek igazolására szolgál a 3. táblázat, melyben néhány népszerű 16Cxx áramkör fontosabb adatait lehet összehasonlítani.

A 16Cxx utasításkészlet (a mnemonikok, tehát az assembly szintjén) lényegében megegyezik a 16C5x utasítás készlettel. Ez azért lehetséges, mert a PIC mikrovezérlők a belső elemeiket speciális funkciójú (Microchip elnevezés szerint: fájl) regisztereken keresztül kezelik, azaz az újabb és újabb hardver részletek az utasításkészletben nem igényelnek változást, bővülést. A 16Cxx utasításkészletben 35 utasítás található, ezek közül 31 mnemonikja és formátuma azonos, mint a 16C5x utasításkészletben lévő megfelelő utasításé. Négy új utasítás jelent meg (RETURN, RETFIE, ADDLW és SUBLW), a PIC16C5x elemek két utasítása feleslegessé vált (OPTION, TRIS). A két utóbbinak mégis van 14 bites kódja, a fejlesztő rendszerek elfogadják ezeket - de a Microchip a további mikrovezérlőkkel való szoftver kompatibilitás érdekében nem ajánlja ezen utasítások használatát.

A fejlesztő rendszerek általában minden PIC mikrovezérlő kezelésére alkalmasak, az assembly programokban meg kell jelölni, hogy melyik mikrovezérlőre készüljön a programkód. Az assembly listák is hasonlóak, akár a 16C5x, akár a 16Cxx csoporthoz készülnek. A 16C5x mikrovezérlőkre készült assembly program azonban nem használható fel minden további nélkül a 16Cxx csoportba tartozó mikrovezérlőhöz. Ha egy 16C5x programot 16Cxx áramkörön kívánunk futtatni, előbb el kell végezni a Microchip által ajánlott vizsgálatokat, módosításokat:

1. El kell távolítani a CALL, GOTO utasításokhoz tartozó azon kiegészítő utasításokat, melyek a memória lapozást szolgálják (lap előválasztó bitek beállítása)
2. A kiszámított ugró utasításokat át kell nézni (a PC-t író, módosító utasítások ezek), hogy az új programtár szervezésnek megfeleljenek
3. Az adatmemória lapozási lépéseket meg kell szüntetni, az új adatmemória kialakításhoz kell igazítani az adatkezelést
4. Minden STATUS, OPTION, FSR regisztert befolyásoló utasítást ellenőrizni kell, mert ezek a regiszterek módosultak

5. A RESET vektor változását figyelembe kell venni (a 16Cxx áramkörök esetében a RESET vektor értéke 0000h).

3. táblázat *Néhány PIC16Cxx mikrovezérlő fontosabb jellemzői*

		PIC16C74	PIC16C64	PIC16C71	PIC16C84
Legnagyobb működési frekvencia		20 MHz	20MHz	16MHz	10MHz
Program memória (a szóhosszúsága 14 bit)	EPROM	4k	2k	1k	-
	EEPROM	-	-	-	1k
Adatmemória (bájtban)		192	128	36	36
Adat EEPROM (bájtban)		-	-	-	64
Timer0 (8 bit + 8 bit előosztó, RTCC a 16C5x-nél)		van	van	van	van
Timer1 (16 bit)		van	van	-	-
Timer2 (16 bit)		van	van	-	-
Kiolvasás (Capture)/Komparálás/PWM modul		2	1	-	-
Szinkron soros Port (SPI/IIC)		van	van	-	-
Soros kommunikációs Port (USART)		van	-	-	-
Párhuzamos Slave Port		van	van	-	-
Analog/digitál átalakító (8 bites)		8 csatornára	-	4 csatornára	-
Power On Reset (Tápfeszültség bekapcsolási Reset)		van	van	van	van
Watchdog Timer		van	van	van	van
Külső megszakítási lehetőség		van	van	van	van
Megszakítás források száma		12	8	4	4
Program memória kódvédelem		van	van	van	van
I/O pontok száma		33	33	13	13
Áramterhelhetőség az I/O csatlakozásoknál	Source (forrásként)	25 mA	25 mA	20 mA	20 mA
	Sink (nyelőként)	25 mA	25 mA	25 mA	25 mA
Tokozási megoldások		DIP 40 PLCC 44 PQFP 44	DIP 40 PLCC 44 PQFP 44	DIP 18 SOIC 18	DIP 18 SOIC 18

3.1. A PIC16C74 mikrovezérlő

A 16Cxx csoportba tartozó mikrovezérlők közül a legbonyolultabb, a legnagyobb teljesítményű eszköz a 16C74. Ebben minden olyan egység, modul megtalálható, amit a 16Cxx sorozathoz a Microchip kifejlesztett.

A jegyzet felépítéséből következik, hogy a 16Cxx csoport megismeréséhez, vagy akár a 16C74 tanulmányozásához is először át kell olvasni az 1. fejezetet, hiszen ott mindaz szerepel, ami a PIC mikrovezérlők közös jellemzője. Ezeket az ismereteket ebben a fejezetben nem ismételjük meg. A 2. fejezetet is érdemes azonban átolvasni, mivel a 16C5x áramkörök mintegy bevezetőként szolgálnak az összetettebb 16Cxx elemekhez, s ezért a 16Cxx mikrovezérlők ismertetésekor esetenként a jegyzet is utal a 2. fejezetre is.

3.1.1. A PIC16C74 felépítése, fontosabb jellegzetességei

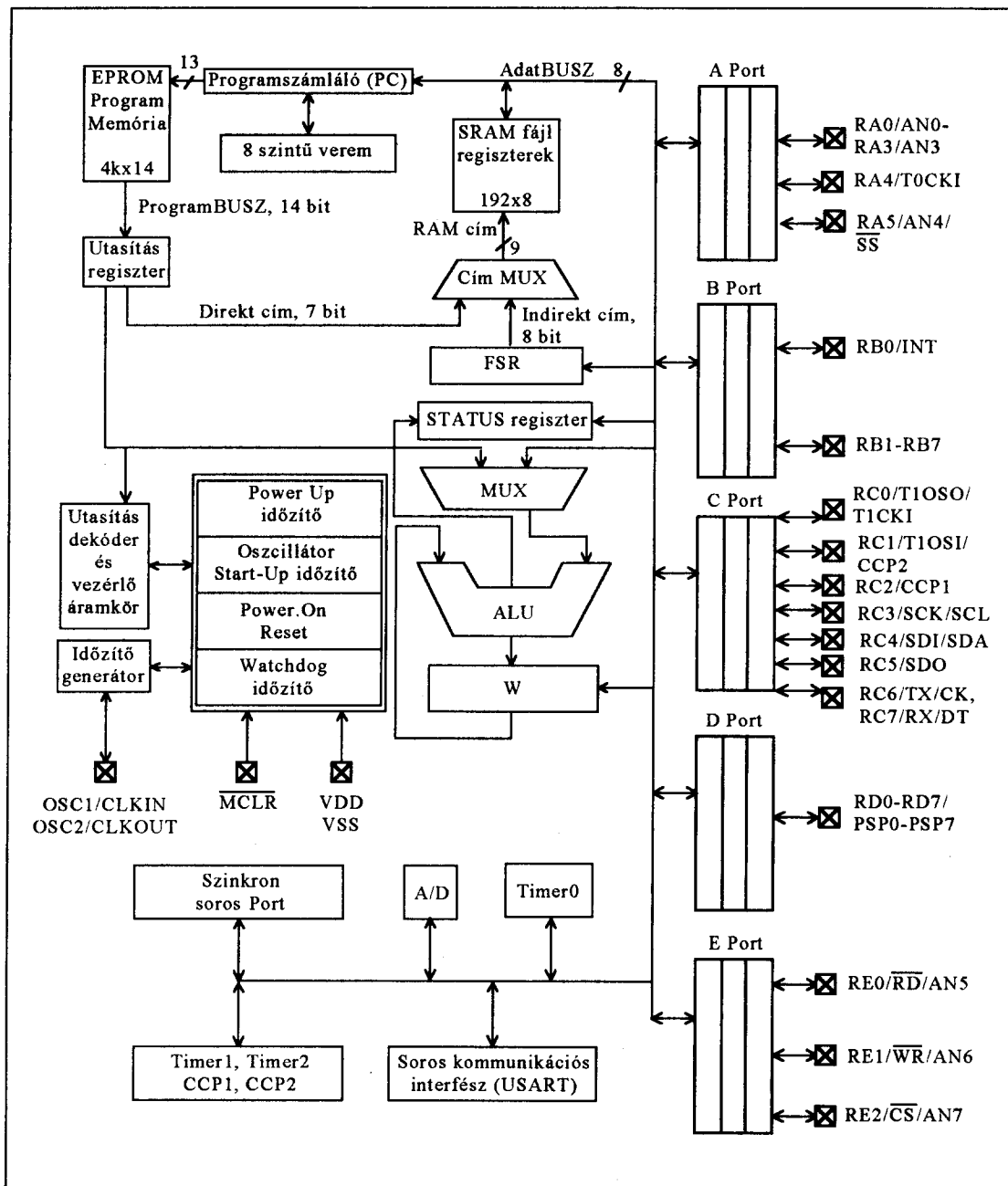
A PIC16C74, mint a PIC család minden eleme, a Harvard architektúrát követi, azaz különálló adatmemóriával (192x8 kapacitású SRAM, fájl regiszter terület) és elkülönített programmemóriával (4kx14 kapacitású EPROM) készül. Minden utasítás egy szavas, az előzőleg beolvasott utasítás végrehajtása és a következő utasítászó beolvasása egyidejűleg, párhuzamosan megy végbe (pipeline működés, ld. 1. fejezet). A veremtár 8 szintesre bővült, a teljes PC tartalmat (13 bit) képes befogadni. Mint minden PIC, a 16C74 is csak belső adattárolójával és programtárolójával képes működni, külső memória elemek nem csatlakoztathatóak hozzá.

A SRAM rekeszeket direkt és indirekt címezéssel is el lehet érni, mind a 45 speciális funkciójú regiszter a fájl regiszter mezőben található. Az egyetlen, a SRAM-tól független belső regiszter a munkaregiszter, a W, mely akkumulátorként szolgál. A W és az ALU 8 bites, a két operandusos utasítások egyik operandusa a W regiszterben van. A másik operandus lehet egy fájl regiszterben, vagy az utasításban mint konstans (literal). A W közvetlenül nem címezhető regiszter.

Az ALU az éppen végrehajtott művelet eredménye alapján állítja egyes utasításoknál a flag biteket (C: Carry, 8 bites átvitel; DC: Digit Carry, átvitel az alsó 4 bitről; Z: Zero, zérus érték jelző; kivonáskor a C és DC negált kölcsön flag, Borrow). A flag biteket a STATUS fájl regiszterben találjuk majd meg. A 16C74 belső felépítésének vázlata a 3.1. ábrán látható. A működésnek nincs alsó sebességhatára, az oszcillátor frekvenciája bármilyen alacsony is lehet - a felső határ 20 MHz.

A 16C74 fontosabb belső egységei és jellemző tulajdonságai a következők:

- 33 I/O csatlakozó láb, egyenkénti adatirány kijelölési lehetőséggel
- nagyteljesítményű kimeneti fokozatok (egy LED közvetlenül működtethető róluk)
- két IC láb az idő kiolvasást (capture) vezérli, vagy a komparáláshoz illetve a PWM működéshez kimenetként szolgál (a kiolvasás - capture - és a komparálás felbontása 16 bit, a PWM felbontása 1 ... 10 bit lehet; 8 bites felbontás esetén a legnagyobb PWM frekvencia 80 kHz, 10 bites esetben csak 20 kHz)
- Timer1, TMR1: 16 bites időzítő/számláló, időalap a kiolvasás (capture) és komparálás működéshez, Sleep állapotban is léptethető külső jellel
- Timer2, TMR2: 8 bites időzítő/számláló, 8 bites periódus regiszterrel, előosztóval és utóosztóval, a PWM működéshez szolgáltat időalapot
- Timer0, TMR0: a PIC16C5x áramkörökben lévő RTCC-vel azonos felépítésű és használatú, 8 bites időzítő/számláló egység, 8 bites programozható előosztóval
- 8 bites A/D, nyolc bemenő csatornával, analóg multiplexerrel, a konverziós idő 16µs
- Soros kommunikációs interfész, SCI (USART), teljes duplex aszinkron vagy félduplex szinkron soros átvitelhez
- szinkron soros Port, két üzemmóddal:
 - = három vezetékes SPI
 - = I²C (IIC)
- párhuzamos Slave Port (PSP), 8 bites, külső $\overline{RD}$, $\overline{WR}$, $\overline{CS}$ vezérlő jelekkel működtethető Port (mikroprocesszor BUSZ interfész)



3.1. ábra A PIC16C74 belső felépítése

- Tápfeszültség bekapcsolási Reset (Power On Reset, POR)
- Tápfeszültség felfutási idő mérő (Power-up Timer, PWRT) és oszcillátor indító (Oscillator Start-up Timer, OST)
- működést felügyelő számláló (Watchdog Timer, WDT), belső oszcillátorral
- kódvédő EPROM biztosíték
- tápteljesítmény felvételt csökkentő Sleep üzemmód
- EPROM biztosítékokkal választható oszcillátor típus
- soros in-system programozási lehetőség (2 IC lábon át).

A CMOS technológiának köszönhetően a működési sebesség nagy; a fogyasztás kis értékű s még tovább csökkenthető a Sleep állapot kiváltásával. A mikrovezérlő bekapcsolási folyamata módosult, a WDT, az OST és PWRT együttműködése révén még nagyobb biztonsággal valósul meg a mikrovezérlő bekapcsolásakor és Sleep állapotból való ébredésekor az, hogy csak teljesen kialakult, tökéletes alakú és időzítésű órajelek mellett kezdődik meg a belső egységek működése.

A PIC16C5x mikrovezérlőkből a felhasználók nagyon hiányolták a megszakítási lehetőségeket. A 16C74 nagyszámú megszakítási lehetőséggel rendelkezik (külső megszakítás, TMR0 túlcsordulás, a B

Port négy pontján fellépő értékváltozás miatti megszakítás, a TMR1 és a TMR2 megszakításai, az SCI aszinkron vételi és adási megszakítása, a szinkron soros Port megszakítása, az SCI megszakítása, az A/D megszakítása és a párhuzamos Slave Port megszakítása). A megszakítások globálisan engedélyezhetők illetve tilthatók, de egyenként is maszkolhatóak (ha globálisan engedélyezettek). Arra is lehetőség van, hogy egy megszakítás a Sleep állapotból felébressze a mikrovezérlőt.

Módosította a Microchip a kódvédelmet is. A 16C5x mikrovezérlők felhasználói esetenként nehezményezték, hogy vagy titkosították a teljes programtárat, vagy egyáltalán nem titkosíthattak - nem voltak fokozatok. A 16C74 esetében programozható, hogy a teljes programtárat kívánjuk-e titkosítani, vagy a felső felét, vagy a felső harmadát.

A PIC 16C74, mint minden PIC mikrovezérlő, beszerezhető kvarcablakos tokozással, ezek a változatok programfejlesztésre használhatóak. A fejlesztő munka során, ha szükséges, a korábbi program kitörölhető, ugyanúgy, mint egy közönséges EPROM-ból.

A kvarcablakos, Cerdip tokozású áramkörök ára meglehetősen magas, sokkal olcsóbbak a zárt, fröccsöntött műanyag tokozású változatok. Igaz, ezeket nem lehet törölni és átprogramozni (OTP, One Time Programmable). A már kifejlesztett, tesztelt programot ezekbe betöltve készülhet el a sorozatgyártott termék. Az OTP áramkörökben az oszcillátor típust a gyártó rögzíti.

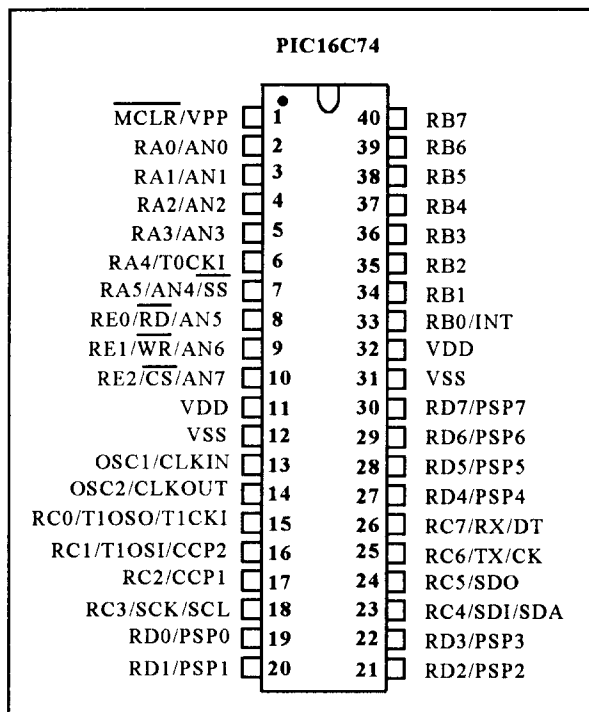
A Microchip további kényelmi fokozatot is biztosít. A Quick-Turnaround-Production (QTP) áram-

körök tulajdonképpen OTP elemek, de a beprogramozást, tesztelést a gyártó vállalja magára, a felhasználó a kész, programozott és ellenőrzött mikrovezérlőket kapja kézhez.

A Serialized Quick-Turnaround-Production (SQTP) megoldás a QTP-hez hasonló, de a sorozat egyes darabjai tartalma kis mértékben eltérhet egymástól. A felhasználó igényének megfelelően néhány rekeszbe a Microchip ilyen rendeléskor az egyedi sorozatszámot égeti be. Ez a sorozatszám lehet véletlen érték, álvéletlen vagy folyamatosan növekvő.

A 16C74 mikrovezérlőt 40 kivezetésű DIL, 44 kivezetésű PLCC és 44 kivezetésű PQFP tokozással forgalmazza a Microchip. A DIP tokozás lábkiosztása látható a 3.2. ábrán.

A nagyszámú belső egység, a sokoldalú kapcsolati rendszer a külső környezettel sokkal több csatlakozási pontot kívánna meg, mint 40 illetve 44. Más gyártók az ilyen bonyolultságú mikrovezérlőket már nem is gyártják DIP tokozással. A 40 kive-



3.2. ábra A 16C74 lábkiosztása

zetés csak úgy elegendő a 16C74 számára, hogy az IC lábak többsége többfunkciós, van, amelyiknek három különféle feladata is lehet. A csatlakozó pontok szerepét, mindhárom tokozás esetére, a 3.1. táblázat foglalja össze.

Ellenőrző kérdések:

1. Hány bites a programmemória rekesze, hány bites az adatmemóriáé?
2. Hány bites a PC? Kihhasználja-e a 16C74?
3. Hány szintű a veremtár?
4. Hány időzítő/számláló egysége van a mikrovezérlőnek, melyik hány bites?
5. Hány analóg bemenetet kezel a 16C74?
6. Mi a célja a párhuzamos Slave Port-nak?
7. Melyik speciális regiszter nincs a fájl regiszter területen belül?
8. Milyen címzés módokkal érhetőek el a fájl regiszterek?
9. Külső programtárat és adattárat kezel-e a 16C74?

3.1. táblázat A PIC16C74 csatlakozó pontjai szerepe

Megnevezés	DIP	PLCC	PQFP	Típus	Puffer	Leírás
OSC1/CLKIN	13	14	30	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	14	15	31	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR /VPP	1	2	18	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0/AN0	2	3	19	I/O	TTL	A PORT, kétirányú I/O Port 0 analóg bemenet 1 analóg bemenet 2 analóg bemenet 3 analóg bemenet/VREF TMR0 órabemenete, kimenetként OC szinkron soros Portnál slave választó/4 analóg bemenet
RA1/AN1	3	4	20	I/O	TTL	
RA2/AN2	4	5	21	I/O	TTL	
RA3/AN3/VREF	5	6	22	I/O	TTL	
RA4/T0CKI	6	7	23	I/O	ST	
RA5/AN4/SS	7	8	24	I/O	TTL	
RB0/INT	33	36	8	I/O	TTL/ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem írható elő minden bemeneten. Külső megszakítás (ekkor ST bemenet) Megszakítás változásra Megszakítás változásra Megszakítás változásra/soros program. óra (ekkor ST) Megszakítás változásra/soros program. adat (ekkor ST)
RB1	34	37	9	I/O	TTL	
RB2	35	38	10	I/O	TTL	
RB3	36	39	11	I/O	TTL	
RB4	37	41	14	I/O	TTL	
RB5	38	42	15	I/O	TTL	
RB6	39	43	16	I/O	TTL/ST	
RB7	40	44	17	I/O	TTL/ST	
RC0/T1OSO/T1CKI	15	16	32	I/O	ST	C Port, kétirányú I/O Port Timer1 oszcillátor kimenet/Timer1 óra bemenet Timer1 oszc. bemenet/Capture2 bemenet, Compare2 kimenet/PWM2 kimenet Capture1 bemenet/Compare1 kimenet/PWM1 kimenet Szinkron soros óra (SPI, IIC üzemmódhoz) Data In (SPI)/adat I/O (IIC) Data Out (SPI) aszinkron soros adás/szinkron óra (SCI) aszinkron soros vevő/szinkron adat (SCI)
RC1/T1OSI/CCP2	16	18	35	I/O	ST	
RC2/CCP1	17	19	36	I/O	ST	
RC3/SCK/SCL	18	20	37	I/O	ST	
RC4/SDI/SDA	23	25	42	I/O	ST	
RC5/SDO	24	26	43	I/O	ST	
RC6/TX/CK	25	27	44	I/O	ST	
RC7/RX/DT	26	29	1	I/O	ST	
RD0/PSP0	19	21	38	I/O	ST/TTL	D Port, kétirányú I/O Port vagy párhuzamos slave Port, mikroprocesszor BUSZ-hoz (ekkor TTL)
RD1/PSP1	20	22	39	I/O	ST/TTL	
RD2/PSP2	21	23	40	I/O	ST/TTL	
RD3/PSP3	22	24	41	I/O	ST/TTL	
RD4/PSP4	27	30	2	I/O	ST/TTL	
RD5/PSP5	28	31	3	I/O	ST/TTL	
RD6/PSP6	29	32	4	I/O	ST/TTL	
RD7/PSP7	30	33	5	I/O	ST/TTL	
RE0/ RD /AN5	8	9	25	I/O	ST/TTL	E Port, kétirányú I/O pontok TTL írásvez. párhuzamos slave Porthoz/5 analóg bemenet TTL olvasásvez. párh. slave Porthoz/6 analóg bemenet TTL kijelölés vez. párh. slave Porthoz/7 analóg bemenet
RE1/ WR /AN6	9	10	26	I/O	ST/TTL	
RE2/ CS /AN7	10	11	27	I/O	ST/TTL	
VSS	12, 31	13,34	6,29	P	-	GND
VDD	11, 32	12,35	7,28	P	-	Pozitív tápfeszültség
NC	-	1,17, 28,40	12,13 33,34	-	-	Nincs belső csatlakozás, használatkor maradjanak bekötetlenek

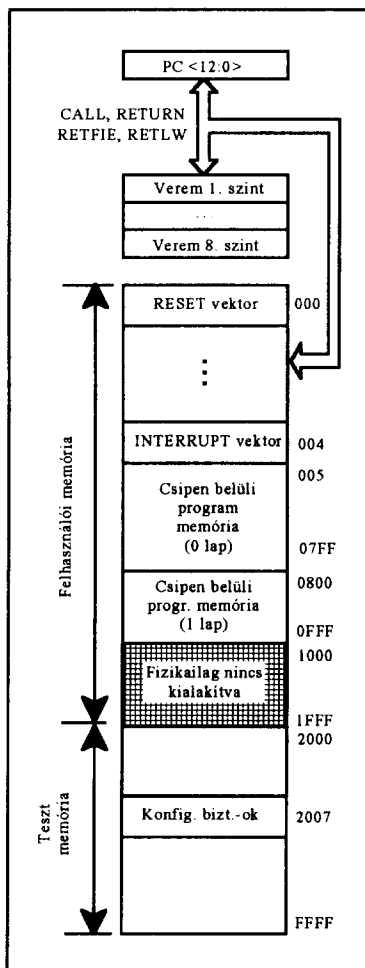
I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás, TTL=TTL bemenet, ST=Schmitt-trigger bemenet

3.1.2. A 16C74 memória szervezése

A programmemóriát (EPROM) a PC tartalmával címzi a mikrovezérlő. A 16C74 programszámlálója 13 bites, de a mikrovezérlő csak 12 bitet használ ebből (0000-0FFFh), hiszen a programtára 4kx14 kapacitású. A RESET vektor (a RESET folyamat után aktivizálódó cím) 0000h, a megszakítási vektor (a megszakításkor aktivizálódó cím) 0004h (3.3. ábra).

Az adatmemória, azaz a SRAM (3.4. ábra) két Bank-re van bontva. Itt találjuk a speciális funkciójú regisztereket (fájl regiszterek) és az általános célra használható rekeszeket is. A Bank0 akkor választódik ki, ha a STATUS regiszterben RP0=0; az RP0=1 a Bank1-et jelöli ki. Mindkét Bank 128 bájtos, mindkettő első 32 bájtja a speciális funkciójú regiszterek számára van fenntartva. A 20h...7Fh (Bank0) és az A0h...FFh (Bank1) regiszterek az általános célra felhasználható SRAM rekeszek.

Az összesen 256 SRAM rekesz mindegyike elérhető közvetlen címzéssel is és közvetett, regiszteres címzéssel is (ekkor az FSR fájl választó regiszter tartalma a cím). A speciális funkciójú regisz-



3.3. ábra Programmemória a 16C74-ben

terek tartalmának változtatásával lehet a mikrovezérlő belső moduljait kezelni, programozni.

Tulajdonképpen a speciális funkciójú regiszterek használata a kulcsa a mikrovezérlő család sokrétűsége ellenére egységes utasításkészletének. A különféle hardver egységek mindegyike a fájl regiszterek írásán és olvasásán keresztül működtethető, az utasításkészletben másfajta utasításra nincs szükség. Ha az egyik mikrovezérlőben (pl. 16C71) csak 4 csatornás A/D és egyetlen Timer van, akkor is ugyanazokkal az utasításokkal lehet programozni, mint a most tárgyalt, rendkívül összetett hardver felépítésű 16C74-et.

A következő oldalon található 3.2. táblázat összefoglalja a 16C74 speciális funkciójú regisztereit. A * -gal jelölt regisztereket mindkét bank tartalmazza. A táblázatban megadtuk a fájl regiszter címét, a nevét, s a regiszter szerepét illetve vezérlő regiszterek esetén az egyes vezérlő bitek nevét. A besötétített biteket a 16C74 nem használja. (A későbbiekben, a 16Cxx sorozat többi tárgyalt mikrovezérlője esetében is ez a táblázat mindig szerepel, így a speciális funkciójú regiszterek készletét, címét azoknál is mindig megadjuk.)

Fájl címek			
00	Indirect addr.	Indirect addr.	80
01	TMR0	OPTION	81
02	PCL	PCL	82
03	STATUS	STATUS	83
04	FSR	FSR	84
05	PORT A	TRIS A	85
06	PORT B	TRIS B	86
07	PORT C	TRIS C	87
08	PORT D	TRIS D	88
09	PORT E	TRIS E	89
0A	PCLATH	PCLATH	8A
0B	INTCON	INTCON	8B
0C	PIR1	PIE1	8C
0D	PIR2	PIE2	8D
0E	TMR1L	PCON	8E
0F	TMR1H		8F
10	T1CON		90
11	TMR2		91
12	T2CON	PR2	92
13	SSPBUF	SSPADD	93
14	SSPCON	SSPSTAT	94
15	CCPR1L		95
16	CCPR1H		96
17	CCP1CON		97
18	RCSTA	TXSTA	98
19	TXREG	SPBRG	99
1A	RCREG		9A
1B	CCPR2L		9B
1C	CCPR2H		9C
1D	CCP2CON		9D
1E	ADRES		9E
1F	ADCON0	ADCON1	9F
20	Általános célú regiszter	Általános célú regiszter	A0
7F			FF

0 Bank 1 Bank

Használaton kívüli rekesz, olvasva 0-kat ad.

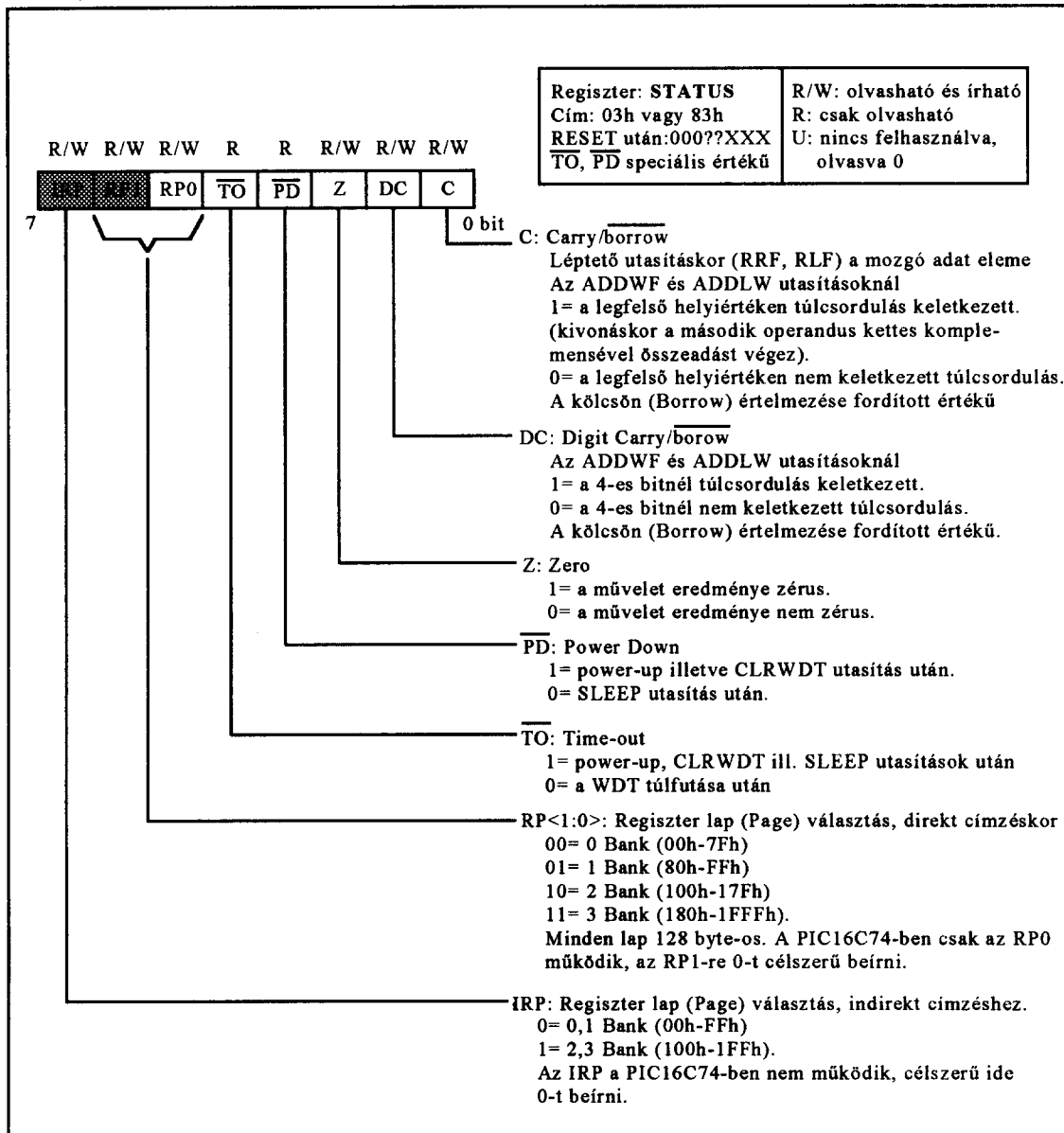
3.4. ábra Adatmemória (16C74)

3.2. táblázat A speciális funkciójú regiszterek (16C74)

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a rekesssel címezve, címként az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó bájtja							
03*	STATUS	IRP	RP1	RP0	T $\bar{O}$	PD	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pontere							
05	PORTA	A Port adat latch (írásakor), az A Port csatlakozó láb jelszintje (olvasáskor)							
06	PORTB	B Port adat latch (írásakor), a B Port csatlakozó láb jelszintje (olvasáskor)							
07	PORTC	C Port adat latch (írásakor), a C Port csatlakozó láb jelszintje (olvasáskor)							
08	PORTD	D Port adat latch (írásakor), a D Port csatlakozó láb jelszintje (olvasáskor)							
09	PORTE	E Port adat latch (írásakor), az E Port csatlakozó láb jelszintje (olvasáskor)							
0A*	PCLATH	A programszámláló (PC) felső öt bite pufferelt regisztere							
0B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
0C	PIR1	PSPIF	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
0D	PIR2	-	-	-	-	-	-	-	CCP2IF
0E	TMR1L	Timer1 alsó bájt							
0F	TMR1H	Timer1 felső bájt							
10	T1CON	-	-	TICKPS1	TICKPS0	T1OSCEN	T1INSYNC	TMR1CS	TMR1ON
11	TMR2	Timer2							
12	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
13	SSPBUF	Szinkron soros Port vevő puffer / adó regiszter							
14	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
15	CCPR1L	Capture/Compare/Duty Cycle Regiszter (LSB)							
16	CCPR1H	Capture/Compare/Duty Cycle Regiszter (MSB)							
17	CCP1CON	-	-	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0
18	RCSTA	SPEN	RC8/9	SREN	CREN	-	FERR	OERR	RCD8
19	TXREG	SCI adó adat regiszter							
1A	RCREG	SCI vevő adat regiszter							
1B	CCPR2L	Capture/Compare/Duty Cycle Regiszter 2 (LSB)							
1C	CCPR2H	Capture/Compare/Duty Cycle Regiszter 2 (MSB)							
1D	CCP2CON	-	-	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0
1E	ADRES	A/D eredmény regiszter							
1F	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO / DONE	-	ADON
1 Bank									
80*	INDF	Ezzel a rekesssel címezve az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION	RBPU	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó bájtja							
83*	STATUS	IRP	RP1	RP0	T $\bar{O}$	PD	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pontere							
85	TRISA	A Port adatirány regisztere							
86	TRISB	B Port adatirány regisztere							
87	TRISC	C Port adatirány regisztere							
88	TRISD	D Port adatirány regisztere							
89	TRISE	IBF	OBF	IBOV	PSPMODE	-	TRISE2	TRISE1	TRISE0
8A*	PCLATH	A programszámláló (PC) felső öt bite pufferelt regisztere							
8B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
8C	PIE1	PSPIE	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
8D	PIE2	-	-	-	-	-	-	-	CCP2IE
8E	PCON	-	-	-	-	-	-	POR	-
8F									
90									
91									
92	PR2	Timer2 Period regiszter							
93	SSPADDD	Szinkron soros Port (IIC üzemmód) cím regiszter							
94	SSPSTAT	-	-	D/A	P	S	R/W	UA	BF
95									
96									
97									
98	TXSTA	CSRC	TX8/9	TZEN	SYNC	-	BRGH	TRMT	TXD8
99	SPBRG	Baud rate generátor							
9A									
9B									
9C									
9D									
9E									
9F	ADCON1	-	-	-	-	-	PCFG2	PCFG1	PCFG0

A következőkben az egyes regiszterekkel részletesen megismerkedünk majd.

A STATUS regiszterben (03h, 83h) találjuk meg a flag biteket és az adatmemória lapozó biteket (3.5. ábra).



3.5. ábra A STATUS regiszter (16C74)

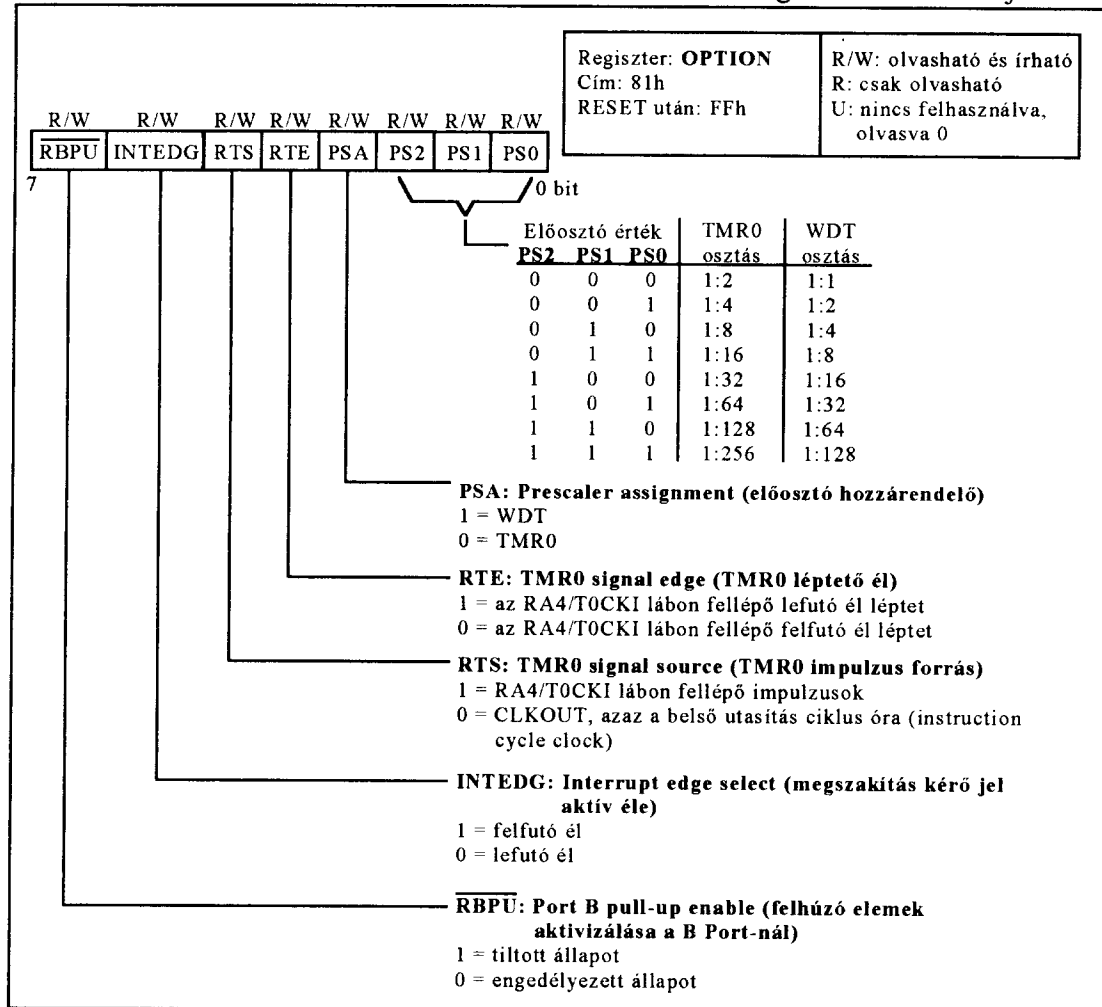
Az egyes bitek szerepe az ábráról leolvasható. A C és a DC túlsordulás jelző flag-ek összeadás utáni túlsordulást jeleznek. Kivonáshoz esetenként kölcsön szükséges, kivonás után ez a két bit a kölcsön tényét jelzi (a teljes 8 biten illetve az alsó 4 biten), de a jelzőbit ekkor negáltan, fordítva működik, azaz 0 értékkel jelzi a kölcsön tényét (Borrow).

A PD, TO bitek lehetővé teszik a 16C74 számára is, hogy RESET folyamat, Sleep-ből való ébredés után kiderítthesse, milyen előzményeket kell figyelembe vennie. Ezek a részletek azonosak a 16C5x elemek megfelelő részleteivel, így a 2. fejezetben részletesen megtalálhatóak (a STATUS regiszter felső három biteje tér el a 16C5x és a 16Cxx csoport esetében, a többi bit azonos szerepű).

A legfelső STATUS bit, IRP indirekt címzésnél a 9. címbitet tartalmazná, de a 16C74-ben erre nincs szükség (itt a teljes fájl cím csak 8 bit hosszúságú). Az RP0-RP1 közül is csak a kisebb helyiértékű bit működik, ezek a direkt címzéskor egészítik ki az utasításban elhelyezett 7 bites fájl regiszter címet (az alfejezet végén még visszatérünk e bitek szerepére). Valószínűleg a Microchip laboratóriumaiban már készül az a 16C7x változat, amelyik nagyobb programmemóriát és SRAM-ot tartalmaz, s abban ezek a bitek is szerephez jutnak majd. Egyelőre a gyártó azt ajánlja, hogy ezeken a biteken 0 álljon.

A STATUS regiszter írható/olvasható regiszter, mégsem ajánlatos a mikrovezérlő használata közben ide különféle értékeket beírni. Először is vannak csak olvasható bitjei is, másodsorban a közvetlenül beírt érték a belső működéseket megzavarhatja, esetenként önmagával is ellentétes tartalmat alakíthat ki a regiszterben.

Az OPTION regiszter (81h) is olvasható/írható regiszter, ebben is különféle vezérlő biteket helyeztek el (3.6. ábra). Ezek a bitek kezelik a TMR0 számlálót (amelyik megfelel a 16C5x változatokban lévő RTCC modulnak), az előosztót, a B Port felhúzó elemeit s a külső megszakítás aktív szintjét.

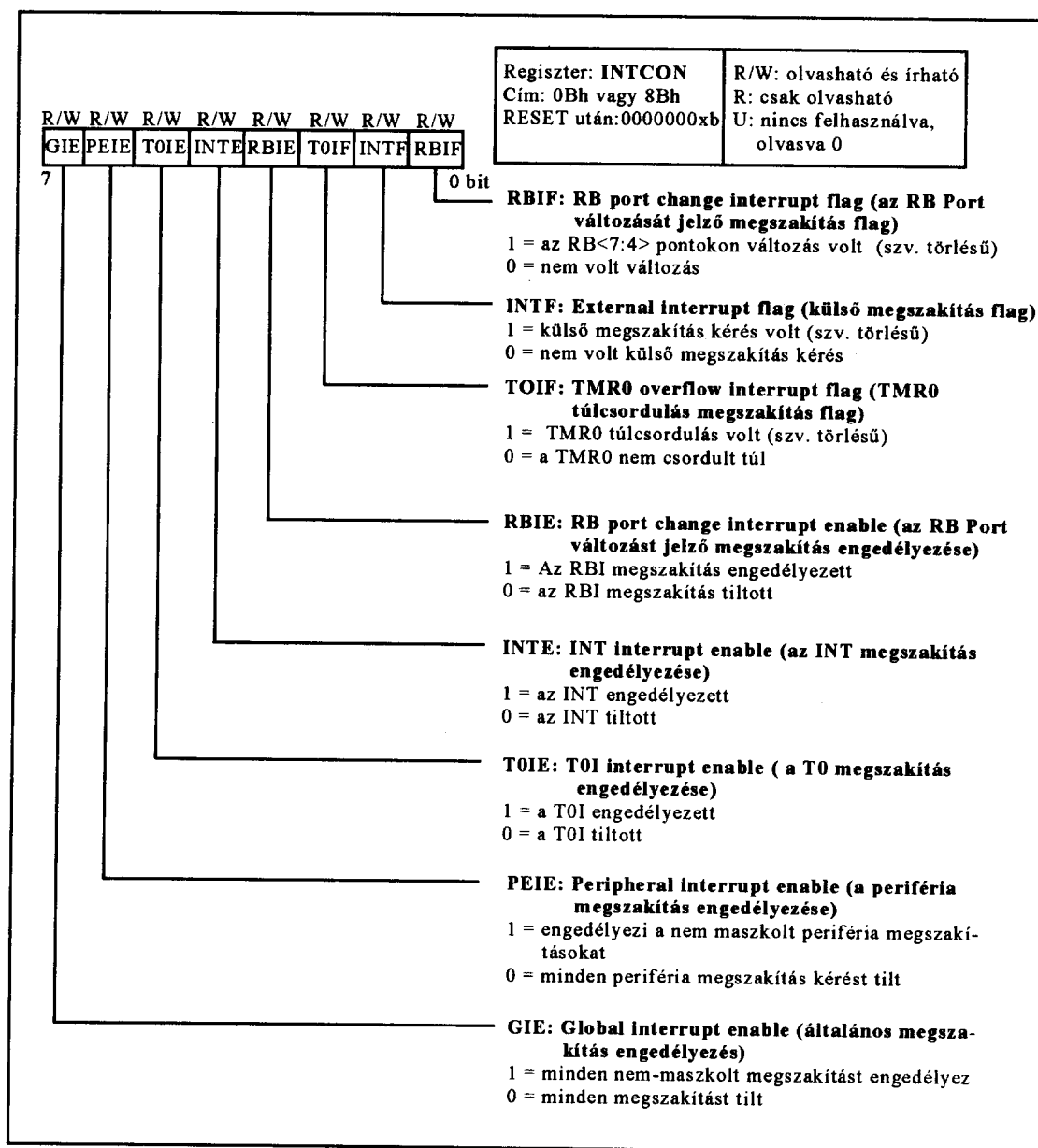


3.6. ábra Az OPTION regiszter (16C74)

Az egyes bitek szerepe az ábráról leolvasható. Az OPTION regiszter alsó 6 bitje, azok szerepe azonos a 16C5x változatok OPTION regiszterénél leírtakkal. A két legmagasabb helyiértékű bit látja el az új feladatokat.

Az INTCON regiszter (INTerrupt CONtrol register, címe: 0Bh, 8Bh) a megszakításokat kezelő egyik regiszter, bitjei elnevezését és szerepét a 3.7. ábrán találjuk meg. Itt található a globális megszakítás engedélyező (GIE) bit és a perifériák megszakítását közösen engedélyező bit (PEIE). A megszakítások pontos hierarchiáját a későbbiekben részletesen megbeszéljük majd. A GIE minden megszakításra hat, engedélyezésével (azaz 1-re állításával) az összes, egyenként nem maszkolt, nem tiltott megszakítást engedélyezi. Ha a GIE tiltott (azaz 0 értékű), minden megszakítási kérést letilt. Minden megszakítási bitnek GIE=1 esetén további, egyedi tiltási, maszkolási lehetősége is van. A RESET folyamat a GIE bitet nullázza, a megszakításokat tiltja.

Ha a mikrovezérlő elfogad egy megszakítást, a GIE tiltott értékre, 0-ra vált; a visszatérési címet (a megszakított program első végre nem hajtott utasítása címét) a verembe menti s a PC-t beállítja a megszakítási vektor címre (0004h). Mint minden mikrovezérlő, a PIC sorozat sem szakít félbe utasítás végrehajtást megszakítás kérés miatt, így a megszakítási eseménynek várakoznia kell, néha 3...4 ciklust is.



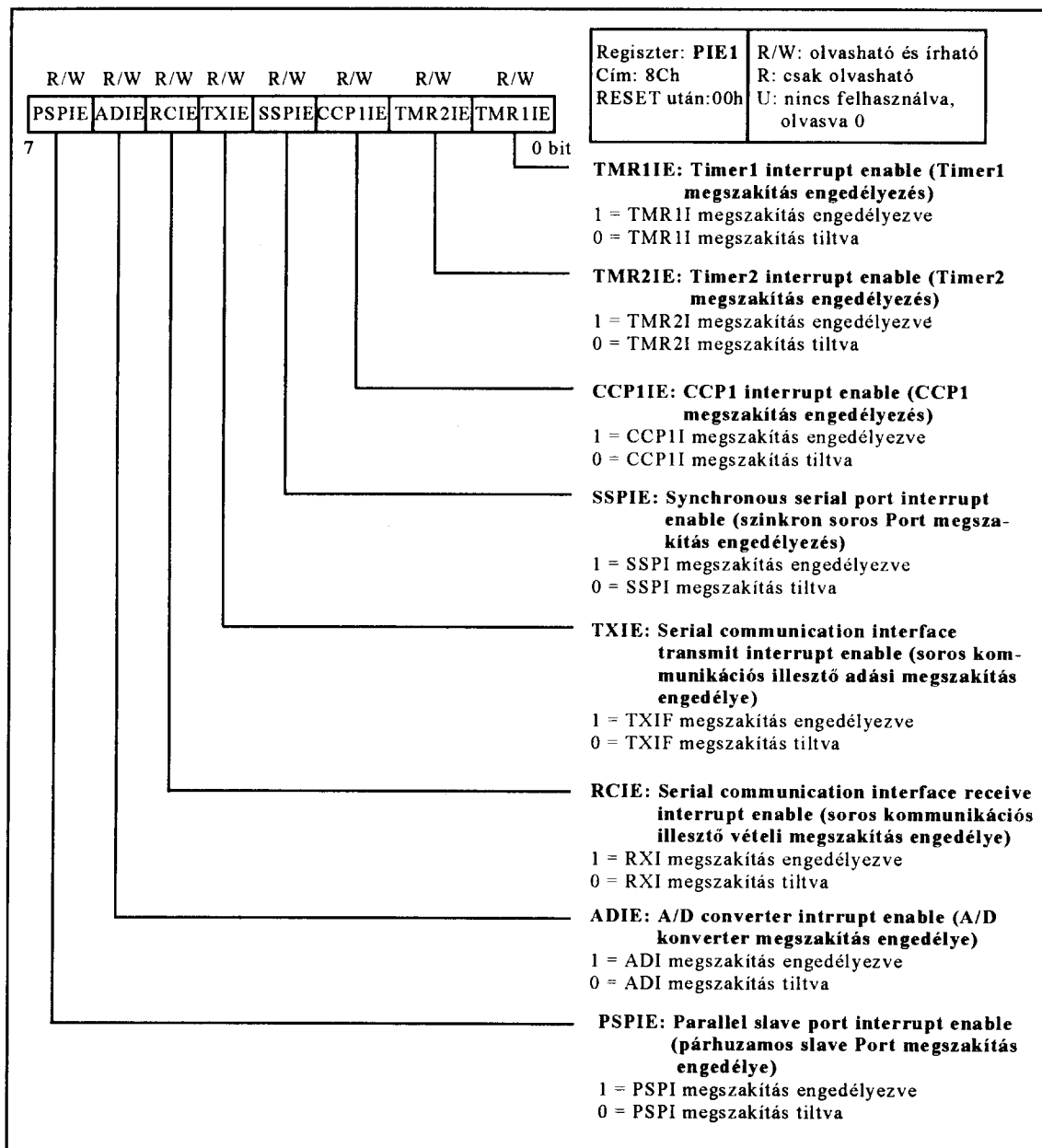
3.7. ábra Az INTCON regiszter (16C74)

A megszakítást kezelő rutin vizsgálja végig a megszakítás flag biteket, így keresi meg a megszakítás forrást - ezután a flag-et törölni is kell. A megszakítási flag bit, amikor egy megszakítási esemény bekövetkezik, mindenképpen 1-re vált, függetlenül a különféle engedélyező bitek értékétől.

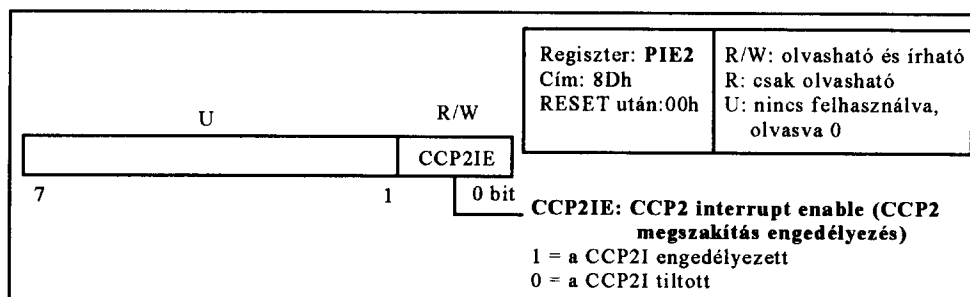
A megszakítási rutinból visszatérítő utasítás, a RETFIE (RETurn From Interrupt, interrupt Enable) utasítás a GIE bitet engedélyezett állapotra váltja.

A 16C74 a beépített periféria egységeket sajátosan kezeli, a megszakításaik is külön csoportot alkotnak. A PIE1 regiszter (8Ch, 3.8. ábra) és a PIE2 regiszter (8Dh, 3.9. ábra) a perifériák által kezdeményezett megszakítások engedélyező bitjeit tartalmazzák. Az egyes bitek szerepe az ábrákról leolvasható, s így az is látható, hogy milyen belső egységek tartoznak a periféria egységek csoportjába.

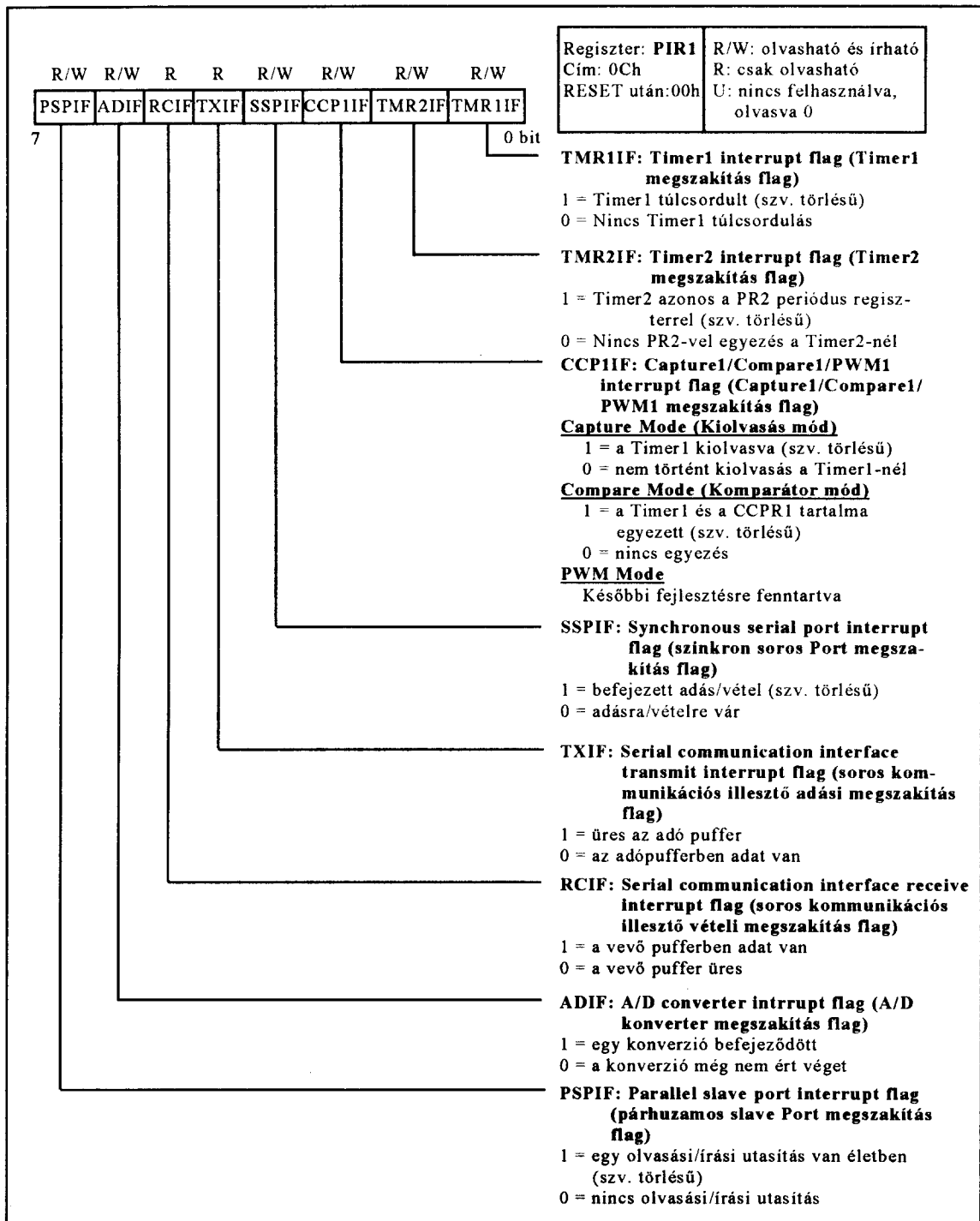
A periféria egységek megszakítási flag bitjei a PIR1 regiszter (0Ch, 3.10. ábra) és a PIR2 regiszter (0Dh, 3.11. ábra) bitjeiként érhetőek el. Az egyes bitek szerepe az ábrákról leolvasható. A periféria egységek működését, az írási vagy olvasási igényt a flag bitek segítségével az egység a CPU tudomására tudja hozni, így a CPU idejét optimálisan ki lehet használni. Nem szabad azonban elfelejteni, hogy a 16C74 mikrovezérlő egyetlen megszakítást ismer, s ha a megszakítás bekövetkezik, a tényleges forrást a megszakítást kezelő szubrutin elején, az összes használt megszakítás flag végigvizsgálatával lehet megállapítani. Ez azt is jelenti, hogy minél több lehetséges megszakítás forrást kezelünk, annál időigényesebb lesz a tényleges megszakítás forrást kereső programrész.



3.8. ábra A PIE1 regiszer (16C74)



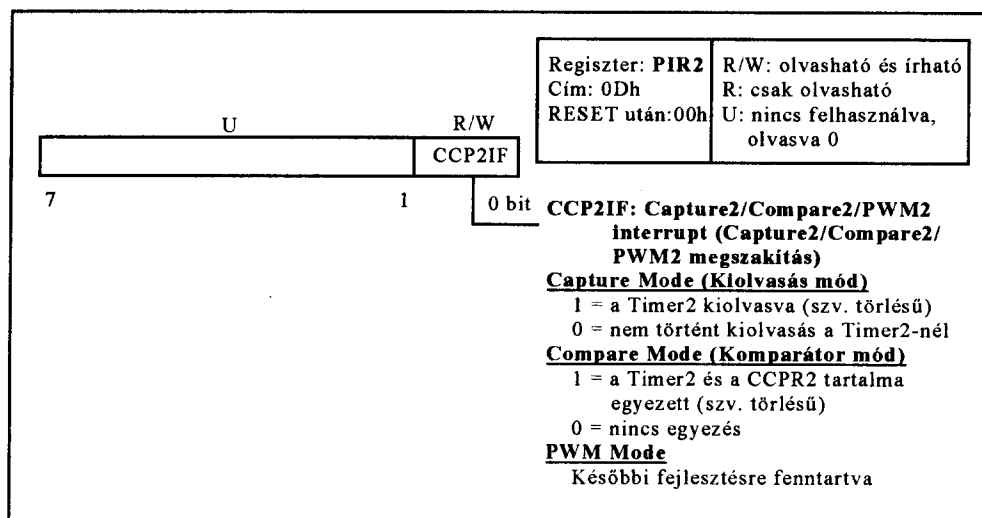
3.9. ábra A PIE2 regiszer (16C74)



3.10. ábra A PIR1 regiszter (16C74)

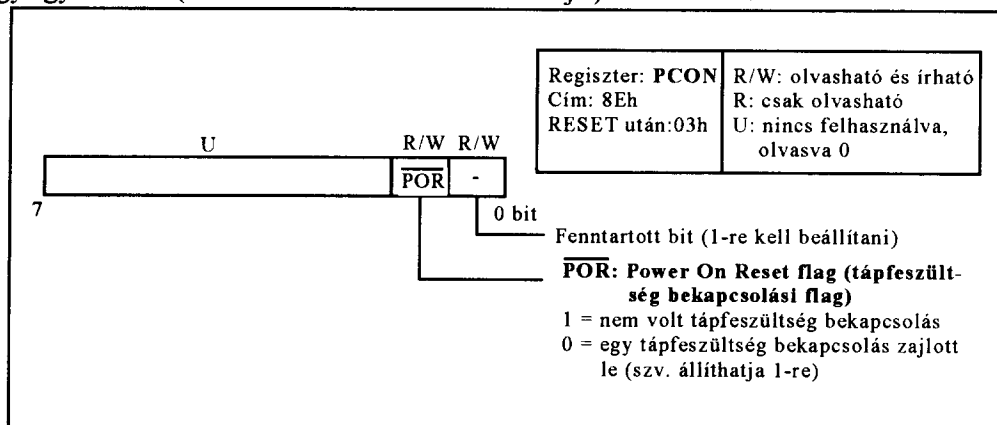
A megszakítási rendszerrel egy későbbi alfejezetben még részletesen foglalkozunk majd, akkor derül majd ki a periféria egységek megszakítási kérései és az egyéb megszakítási kérések közötti hierarchikus kapcsolat. A periférius egységek megszakítás kérései közösen rendelkeznek egy engedélyező bittel (PEIE), de a globális megszakítás engedélyező bit is vonatkozik ezekre a megszakításokra is.

A 16C74-ben a megszakításoknak nincs prioritásuk, amelyik először jelentkezik, azt szolgálja ki a processzor, s a kiszolgálás közben további megszakítás kérésekkel nem foglalkozik.



3.11. ábra A PIR2 regiszter (16C74)

A PCON regiszter (8Eh, 3.12. ábra) egy olyan flag bitet tartalmaz, amely lehetővé teszi, hogy a CPU megállapíthassa egy RESET folyamat után, hogy azt a tápfeszültség bekapcsolása (Power-On) vagy egyéb hatás (WDT időtúlfutás illetve MCLR jel) váltotta-e ki.



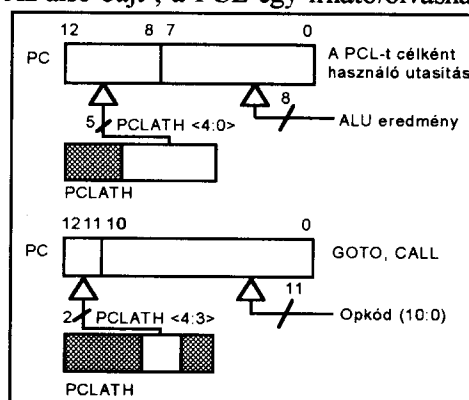
3.12. ábra A PCON regiszter (16C74)

A 16C74 mikrovezérlő programszámlálója (PC) 13 bites. Az alsó bájt, a PCL egy írható/olvasható regiszter (02h vagy 82h címen). A felső bájt, a PCH közvetlenül nem olvasható és nem írható. A PC felső bájtját a PCLATH regiszteren (0Ah vagy 8Ah) keresztül lehet írni. Amikor a PC tartalma átíródik egy CALL, GOTO illetve PCL író utasítás miatt, akkor a PC felső bitejére bemásolódik a PCLATH tartalma (a PCL-t író utasítás 8 bites címrészt tartalmaz, a CALL és a GOTO 11 biteset, ezért a két működés között is van eltérés, amit a 3.13. ábra szemléltet).

A PC tartalmának utasítással történő közvetlen módosítása, azaz a kiszámított GOTO alkalmazása első-sorban táblázatok kezelésekor célszerű.

A veremtár 8 darab 13 bites rekeszből áll, így a teljes PC tartalmat el lehet menteni. Csak a CALL utasítást és a megszakítási rutinokat szolgálja ki a verem, közvetlenül nem kezelhető, a veremmutató (SP) sem érhető el a programozó számára. CALL utasításkor, megszakításkor a PC tartalma letöltődik a verembe (PUSHed), a RETURN, RETLW vagy RETFIE utasítások hatására a verem tetején lévő cím visszahívódik (POPed) a PC-be. Mindezek a verem működések nem befolyásolják a PCLATH tartalmát.

A 16C74-ben 4k szó kapacitású a programmemória, de a CALL és a GOTO utasításokban csak 11 bites cím fér el (ld. a 16Cxx utasításkészlet táblázatát). Így a programmemóriát tulajdonképpen lapozni

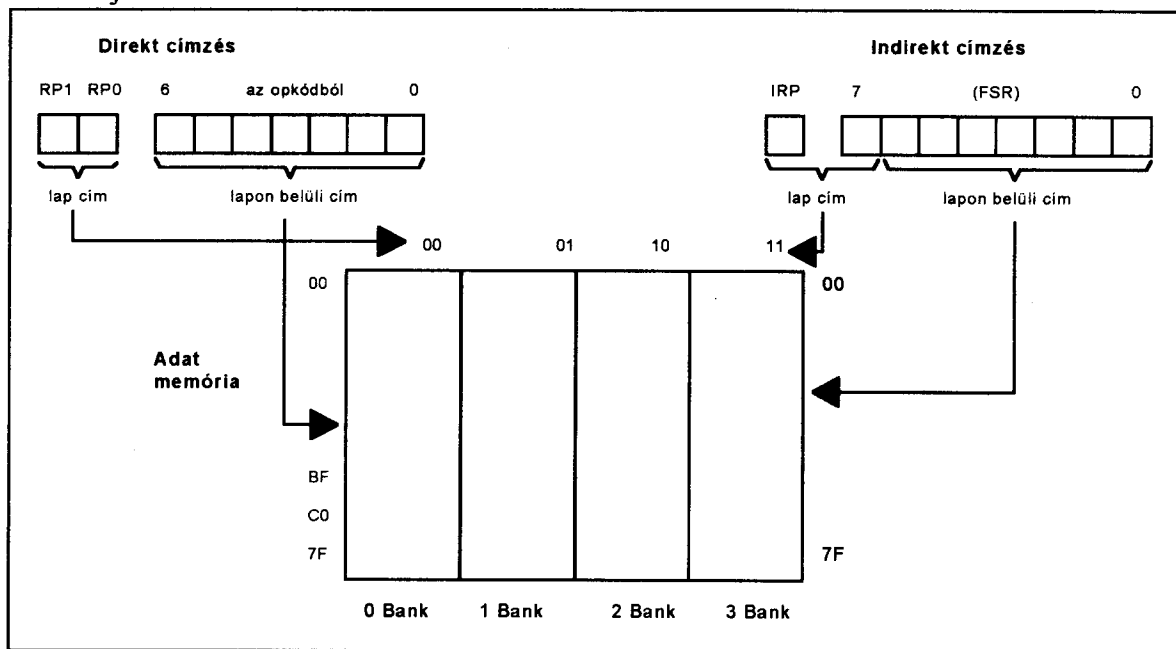


3.13. ábra A PC felső bájt (16C74)

kell, a lapok kapacitása 2k szó. A PCLATH<3> bit szerepel lapozó bitként, s arról a felhasználónak kell gondoskodnia, hogy amikor erre a bitre szükség van, a megfelelő értékű legyen. A verembe a lapozó bit értéke is bekerül, így a RET utasításoknál nincs különösebb teendő - a visszatérési címben a lapcím is megtalálható. A 16C74 PC teljes számlálási kapacitása 13 bit, de a legfelső bitet ez a mikrovezérlő nem használja fel.

Amint már többször említettük, a SRAM rekeszei, azaz a fájl regiszterek (s közöttük a speciális funkciójú regiszterek is) kezelhetők közvetlen címezéssel is (ekkor az utasítás tartalmazza a fájl regiszter címének 7 alsó bitjét, fff ffff); vagy indirekt címezéssel. Az indirekten címző utasításnak azonban nincs külön mnemonikja, ugyanaz az utasítás ez, mint a direkt címző - abból tudja a CPU, hogy most közvetett címezést kell használnia, hogy az utasításban szereplő fájl cím a 0h. A speciális funkciójú regiszterek táblázataiban a 0h címen lévő regisztert INDF jelöli, ez tehát az indirekt címző fájl regiszter. Ilyen regiszter azonban fizikailag nem létezik, a 0h cím csak jelzés: azt fejezi ki, hogy nem az utasításban szereplő címértéket kell használni, hanem a fájl regisztert címző regiszter, az FSR (File Select Register, 04h vagy 84h) tartalmát. Ha az INDF regisztert olvassuk, eredményül 0h érték adódik, s írni sem lehet.

A teljes SRAM kezeléshez, mint tudjuk, 8 bit szükséges a 16C74-ben. Közvetlen címezéskor a legfelső bitet (SRAM Bank választó bit, RP0) a STATUS regiszter tartalmazza (STATUS <5>). Közvetett címezéskor az FSR a teljes 8 bites fájl regiszter címet tartalmazza. A 16C74-be beépített címző rendszer 512 bájtos, azaz négy Bank méretű SRAM kezelését biztosítaná (3.14. ábra), de a 16C74 ennek csak a felét használja ki.



3.14. ábra A SRAM címzése a 16Cxx mikrovezérlőkben

Ellenőrző kérdések:

1. Melyik speciális funkciójú regiszterek érhetőek el két különböző SRAM címen?
2. Hány szabadon felhasználható SRAM rekesze van a 16C74-nek? A speciális funkciójú regiszterek területén hány kihasználatlan rekesz van?
3. Hogyan adható meg a SRAM Bank címe?
4. Miért nem célszerű direkt módon írni a STATUS regisztert?
5. Ha a GIE bit 0, lehet-e egyenként engedélyezni a megszakításokat?
6. Ha a GIE bit 1, lehet-e egyenként tiltani a megszakításokat?
7. Mi a POR bit szerepe (PCON regiszter)?
8. Hogyan áll elő a 13 bites PC tartalom GOTO és CALL utasítások esetén?
9. Hogyan áll elő a 13 bites PC tartalom kiszámított GOTO esetén?
10. Hány megszakítás flag bit van a 16C74-ben?
11. Hogyan lehet meghatározni, hogy az előosztó a WDT-hez vagy a TMR0-hoz tartozzon?

12. Az ebben az alfejezetben részletesen megismert fájl regiszterek között volt-e csak olvasható? Ha a teljes regiszter írható/olvasható, volt-e egy vagy néhány csak olvasható bitjük?
13. A közvetett címzésre az utal, ha a fájl regiszter címet tartalmazó utasításban ez a cím 0h; az aktuális fájl regiszter cím pedig az FSR tartalma. Az FSR címe 04h vagy 84h. Miért nem a 04h vagy a 84h áll jelzésként a közvetett címzésű utasításokban?

3.1.3. Az I/O Portok

A PIC16C74 öt általános célú, párhuzamos Port-ot tartalmaz (A Port ... E Port). A Port pontok többsége azonban egyéb, alternatív funkciókra is szolgálnak, ezért multiplexelt megoldásúak. A Port pontok áramköri megoldása egyébként alapvetően megegyezik a 16C5x elemeknél megismert felépítéssel (2.7. ábra), csak az alternatív funkciók miatt további részletek is csatlakoznak hozzájuk. Alapvetően két regiszter tartozik egy Port-hoz, egy adatregiszter és egy irány kijelölő regiszter (TRIS).

Az A Port kezelésére a PORTA és a TRISA fájl regiszterek szolgálnak (3.3. táblázat).

3.3. táblázat Az A Port regiszterei

Regiszter név	Funkció	Cím	Power-On Reset után
PORTA	A Port lábak jelszintje, olvasáskor A Port latch, íráskor	05h	--XX XXXX
TRISA	A Port adatirány regiszter 0 = kimenet, 1 = bemenet	85h	--11 1111

Az A Port 6 bites. Az RA4 Schmitt trigger bemenet és nyitott kollektoros (OC) kimenet, a többi RA pont kimenetként teljes CMOS ellenütemű meghajtó fokozattal készült s bemenetként TTL szinteket fogad. Mint minden Port esetében, itt is minden bithez tartozik a TRISA regiszterben egy adatirány kijelölő bit (bemenet:1, kimenet:0). Mint minden Portnál, az A Port írása a kimeneti latch-be tölt adatot, az A Port olvasása a Port csatlakozó lábain lévő jelértéket szolgáltatja.

Az RA4 pont a TMR0 óra bemenetével multiplexelt, a többi RA pont alternatív funkcióként analóg bemenetként szolgál, illetve az A/D referencia feszültségét is itt lehet bevezetni. Az analóg funkciókat az ADCON1 regiszterben lehet elrendelni. Ha egy A Port pont analóg bemenet, olvasásakor 0-t kapunk. A bekapcsolási (Power On) RESET hatására az RA pontok analóg bemeneti állapotba kerülnek. Ha az analóg bemeneteket használjuk, a megfelelő TRISA pontokon bemenetet kell előírni. Az A Port pontjainak lehetséges szerepeit a 3.4. táblázat foglalja össze.

3.4. táblázat Az A Port funkciói

Név	Bit	Puffer	Funkció
RA0/AN0	0	TTL	I/O vagy analóg bemenet
RA1/AN1	1	TTL	I/O vagy analóg bemenet
RA2/AN2	2	TTL	I/O vagy analóg bemenet
RA3/AN3	3	TTL	I/O vagy analóg bemenet
RA4/T0CKI	4	ST	I/O vagy TMR0-hoz óra bemenet (OC kimenet)
RA5/AN4/SS	5	TTL	I/O vagy analóg bemenet vagy slave select bemenet szinkron soros Port-hoz

TTL=TTL bemenet, ST=Schmitt-trigger bemenet, OC kimenet=nyitott kollektoros kimenet

A B Port kezelésére szolgáló regisztereket a 3.5. táblázat mutatja be.

3.5. táblázat A B Port regiszterei

Regiszter név	Funkció	Cím	Power-On Reset után
PORTB	B Port lábak jelszintje, olvasáskor B Port latch, íráskor	06h	XXXX XXXX
TRISB	B Port adatirány regiszter 0 = kimenet, 1 = bemenet	86h	1111 1111
OPTION	Gyenge felhúzó vezérlő bitje ($\overline{\text{RBPU}}$)	81h	1111 1111

A B Port nyolcbites kétirányú Port, adatirány regisztere a TRISB. Továbbra is 1 jelenti a TRISB egy bitjén azt, hogy a megfelelő B Port bit bemenet. Minden B Port csatlakozó ponthoz a csipre telepítettek egy gyenge felhúzó elemet, a tipikus áramérték 100 μA . Az összes felhúzó elemet egy vezérlő bit kezeli (OPTION <7>). A felhúzó ellenállás automatikusan kikapcsolódik, ha egy B Port pontot kimenetként konfigurálunk. A bekapcsolási RESET folyamat is kikapcsolja a felhúzó elemeket.

Négy B Port pont, az RB<7:4> pontcsoport megszakítás forrásként is működhet, ehhez bemenetként konfiguráljuk a pontokat. Ha a megadott pontok valamelyikét kimenetként jelöljük meg, azt kivonjuk a megszakítási működés alól. Az RB4...7 pontok pillanatnyi jelértékét a CPU összehasonlítja a korábbi

értékkel, s ha változás történt, megszakítás lép fel (RBIF megszakítás, $INTCON<0>$). Ez a megszakítás Sleep állapotból is felébresztheti a mikrovezérlőt.

Ha a különbözőség miatt fellép a megszakítás, a megszakítást kezelő rutinnak meg kell szüntetnie a megszakítási helyzetet - ennek két módja is van:

a, letiltjuk ezt a megszakítást ($INTCON<3>$ bit = 0)

b, a B Port olvasásával megszüntetjük a különbözőséget, majd töröljük az RBIF bitet.

Ezt a megszakítási lehetőséget jól fel lehet használni pl. billentyűzet kezelésére, a billentyű leütésére váró mikrovezérlő akár Sleep állapotban is lehet. A B Port pontjainak szerepeit a 3.6. táblázat foglalja össze.

3.6. táblázat A B Port pontjai

Név	Bit	Puffer	Funkció
RB0/INT	0	TTL/ST*	I/O vagy külső megszakítás
RB1	1	TTL	I/O
RB2	2	TTL	I/O
RB3	3	TTL	I/O
RB4	4	TTL	I/O
RB5	5	TTL	I/O
RB6	6	TTL/ST**	I/O
RB7	7	TTL/ST**	I/O
TTL=TTL bemenet, ST=Schmitt-trigger bemenet, OC kimenet=nyitott kollektoros kimenet *=külső megszakításként konfigurálva Schmitt-trigger bemenet **=Schmitt-trigger bemenet soros programozó üzemmódban Minden lábnál szoftverrel aktivizálható gyenge felhúzó elem van beépítve			

A C Port regisztereit a 3.7. táblázatban láthatjuk.

3.7. táblázat A C Port regiszterei

Regiszter név	Funkció	Cím	Power-On Reset után
PORTC	C Port lábak jelszintje, olvasáskor C Port latch, íráskor	07h	xxxx xxxx
TRISC	C Port adatirány regiszter 0 = kimenet, 1 = bemenet	87h	1111 1111

A C Port nyolcbites, kétirányú Port, az egyes bitek adatirányát a TRISC jelöli ki a szokott módon (1: bemenet). A C Port pontjai különféle periféria funkciókkal multiplexeltek. Minden C Port pont Schmitt trigger bemenetű. a C Port pontjainak feladatait a 3.8. táblázat mutatja be.

3.8. táblázat A C Port bitjei

Név	Bit	Puffer	Funkció
RC0/T1OSO/T1CKI	0	ST	I/O vagy Timer1 oszc. kimenet vagy Timer1 óra bemenet
RC1/T1OSI/CCP2	1	ST	I/O vagy Timer1 oszc. bemenet vagy Capture2 bemenet, Compare2 kimenet, PWM2 kimenet
RC2/CCP1	2	ST	I/O vagy Capture1 bemenet, Compare1 kimenet, PWM1 kimenet
RC3/SCK/SCL	3	ST	I/O vagy szinkron soros óra (SPI, IIC)
RC4/SDI/SDA	4	ST	I/O vagy SPI adatbemenet (SPI) vagy adat I/O (IIC)
RC5/SDO	5	ST	I/O vagy szinkron soros Port adat kimenet
RC6/TX/CK	6	ST	I/O vagy aszinkron soros adás vagy SCI szinkron óra
RC7/RX/DT	7	ST	I/O vagy aszinkron soros vétel vagy SCI szinkron adat
ST=Schmitt-trigger bemenet			

A D Port nyolcbites, a bemenetein Schmitt triggert tartalmazó Port, bitjei adatirányát a TRISD regiszterben lehet előírni. A D Port regisztereit a 3.9. táblázatban találjuk meg.

3.9. táblázat A D Port regiszterei

Regiszter név	Funkció	Cím	Power-On Reset után
PORTD	D Port lábak jelszintje, olvasáskor D Port latch, íráskor	08h	xxxx xxxx
TRISD	D Port adatirány regiszter 0 = kimenet, 1 = bemenet	88h	1111 1111

A D Port különleges szolgáltatása, hogy nyolcbites, párhuzamos Slave Port-ként is működhet (mint párhuzamos alárendelt Port, Parallel Slave Port egy mikrovezérlő-mikroprocesszor kapcsolatban). A párhuzamos Slave Port aktivizálása érdekében a PSPMODE vezérlő bitet kell 1 értékre beállítani ($TRISE<4>$). Ebben az üzemmódban a bemeneti pufferek nem Schmitt triggerek, hanem TTL jellegű bemenetek. A párhuzamos Slave Port üzemmódot a D és az E Port együtt szolgálják ki, a D Port az adatmozgást végzi, az E port bitjei a vezérlő jelek.

A D Port bitjeinek szerepeit a 3.10. táblázatban foglaltuk össze.

3.10. táblázat *A D Port bitjei*

Név	Bit	Puffer	Funkció
RD0/PSP0	0	ST/TTL	I/O vagy párhuzamos slave port bit
RD1/PSP1	1	ST/TTL	I/O vagy párhuzamos slave port bit
RD2/PSP2	2	ST/TTL	I/O vagy párhuzamos slave port bit
RD3/PSP3	3	ST/TTL	I/O vagy párhuzamos slave port bit
RD4/PSP4	4	ST/TTL	I/O vagy párhuzamos slave port bit
RD5/PSP5	5	ST/TTL	I/O vagy párhuzamos slave port bit
RD6/PSP6	6	ST/TTL	I/O vagy párhuzamos slave port bit
RD7/PSP7	7	ST/TTL	I/O vagy párhuzamos slave port bit
TTL=TTL bemenet, ST=Schmitt-trigger bemenet			

Az E Port mindössze három bites, regisztereit a 3.11. táblázat mutatja be.

3.11. táblázat *Az E Port regiszterei*

Regiszter név	Funkció	Cím	Power-On Reset után
PORTE	E Port lábak jelszintje, olvasáskor E Port latch, íráskor	09h	---- -xxx
TRISE	E Port adatirány regiszter 0 = kimenet, 1 = bemenet	89h	0000 -111

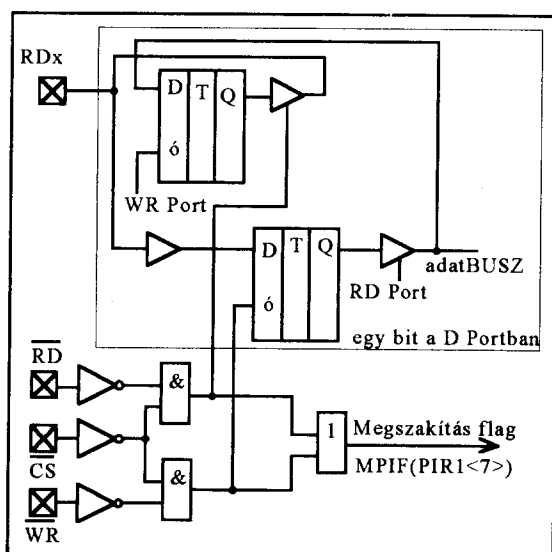
A három Port bit Schmitt triggeres, puffertelt bemenet. A TRISE regiszternek csak a három legkisebb helyiértékű bitje szolgál adatirány kijelölésre. Ha a PSPMODE bit 1 értékű, a D Port mikroprocesszor párhuzamos Slave Port-ként működik, az E Port bitjei lesznek ekkor a vezérlő jelek. Eközben a TRISE 0...2 biteken bemenetet kell előírni (1 érték), az E Port bitek ebben az üzemmódban TTL szintekkel működnek.

Az E Port bitjei lehetséges szerepeit a 3.12. táblázat tartalmazza.

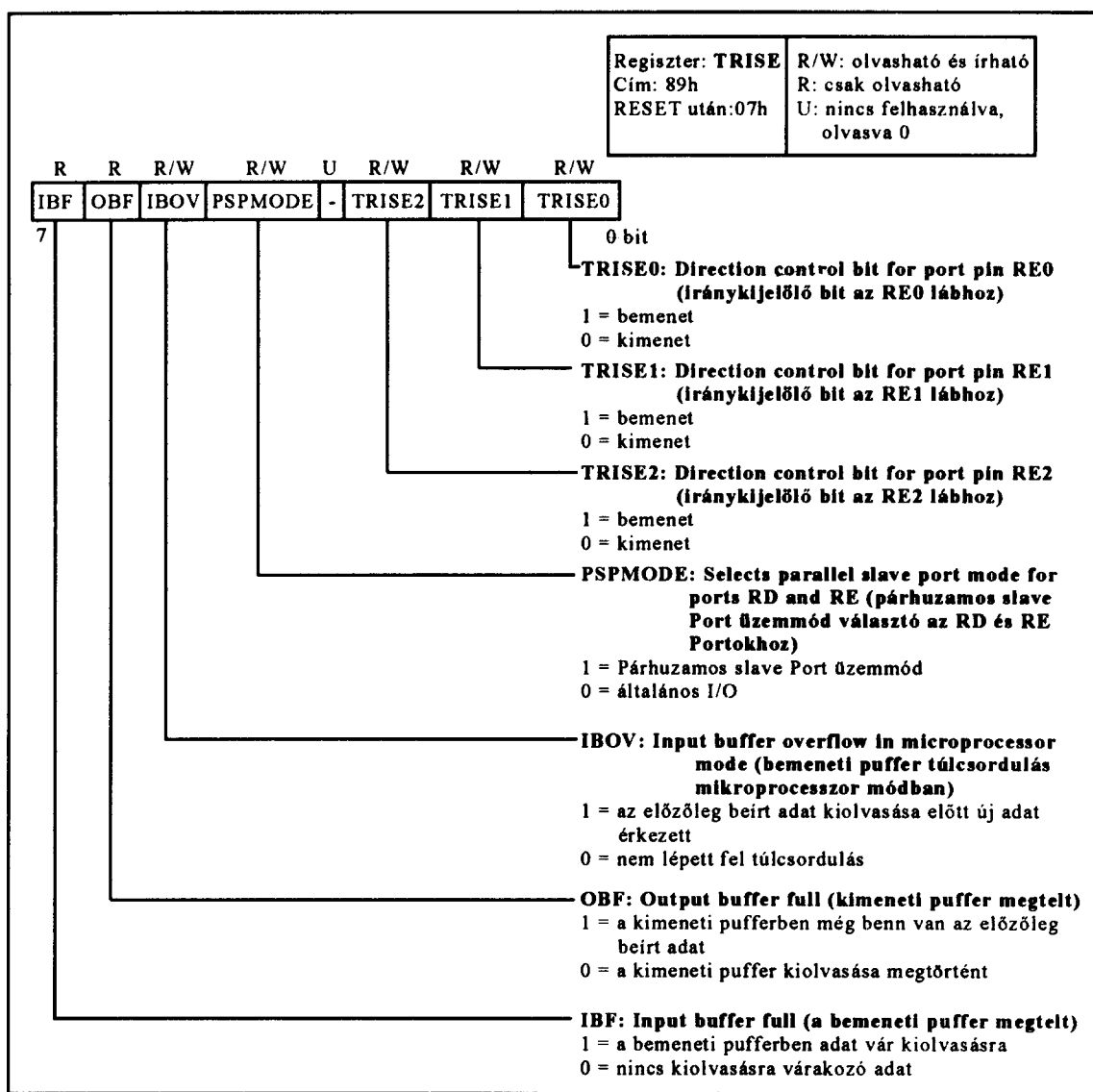
3.12. táblázat *Az E Port bitjei*

Név	Bit	Puffer	Funkció
RE0/ $\overline{\text{RD}}$ /AN5	0	ST/TTL	I/O, párhuzamos slave Port módban olvasás vezérlő (a D Port regiszter olvasható), analóg bemenet
RE1/ $\overline{\text{WR}}$ /AN6	1	ST/TTL	I/O, párhuzamos slave Port módban írás vezérlő (a D Port regiszter írható), analóg bemenet
RE2/ $\overline{\text{CS}}$ /AN7	2	ST/TTL	I/O, párhuzamos slave Port módban chip select vezérlő, analóg bemenet
TTL=TTL bemenet, ST=Schmitt-trigger bemenet			

Amikor a mikrovezérlő párhuzamos Slave Port üzemmódban működik, tulajdonképpen úgy viselkedik, mint egy mikroprocesszor programozható kiegészítő áramköre. Ha a PSPMODE (TRISE<4>) értéket 1-re állítjuk, ezzel kiváltottuk a Slave Port működést. Az RE0 lesz az olvasás vezérlő jel, az RE1 az írás vezérlő jel, az RE2 pedig a csip kijelölő jel, a 3.12. táblázatnak megfelelően. A párhuzamos Slave Port tárolt bemenetű és tárolt kimenetű Port. A mikrovezérlő CPU a 8 bites kimeneti latch-be tud írni, s a szintén 8 bites bemeneti tárolót tudja olvasni (ezek azonos címen érhetőek el, de fizikailag elkülönültek). A külvilág és a latch áramkörök között az adatforgalmat a három vezérlő jel (tehát a külső processzor) irányítja. A párhuzamos Slave Port áramköri kialakítását a 3.15. ábra mutatja be.

**3.15. ábra** *A Párhuzamos Slave Port*

flag bitek tájékoztatják a mikrovezérlő központi egységét, ezek a jelző bitek is a TRISE regiszterben találhatóak. A TRISE bitjeinek elnevezését, szerepét a 3.16. ábra mutatja be. Amint már említettük, itt csak három bit szolgál adatirány kijelölésre, a többi a párhuzamos Slave Port üzemmódot támogatja (PSPMODE, IBOV, OBF, IBF).



3.16. ábra A TRISE regiszter (16C74)

A TRISE<7> IBF (Input Buffer Full, a bemeneti puffer feltöltve) állapotbit akkor áll be 1 értékre, ha a mikroprocesszor által beírt szó várja, hogy a mikrovezérlő átvegye, kiolvassa. Amikor a 16C74 CPU végrehajt egy D Port olvasó utasítást, az IBF törlődik. Az IBF csak olvasható állapotbit.

A TRISE<6> OBF (Output Buffer Full, a kimeneti puffer feltöltve) állapotbit akkor vált 1-re, ha a D Port kimeneti tárolójában egy, a mikrovezérlő CPU által beírt szó várja, hogy a külső processzor kiolvassa. Amikor a külső mikroprocesszor elvégzi a kiolvasást, az OBF 0-ra vált. Az IBOV állapotbit a TRISE<5> (Input Buffer Overflow, bemeneti puffer felülíródott) bit akkor vált 1-re, ha a külső mikroprocesszor úgy írt be egy adatot a bemeneti tárolóba, hogy onnan az előzőt a mikrovezérlő még nem olvasta be. Ez írható/olvasható bit, a mikrovezérlő CPU-nak kell törölnie.

Amikor egy párhuzamos Slave Port írási vagy olvasási művelet teljesen végbement, a PIR1<7> biten lévő PSPIF (MPIF) megszakítás flag bit 1-re vált. Ezt a bitet szoftverrel lehet csak törölni, a megszakítást a PSPIE (PIE<7>) engedélyező bittel lehet engedélyezni vagy tiltani. A párhuzamos Slave Port üzemmódban szerepet játszó regisztereket a 3.13. táblázat foglalja össze.

3.13. táblázat A párhuzamos Slave Port regiszterei

Regiszter név	Funkció	Cím	Power-On Reset után
PORTD	Párhuzamos slave Port, olvasott/írt adatok	08h	xxxx xxxx
TRISD	D Port adatirány regiszter	88h	1111 1111
PORTE	Párhuzamos slave Port, olvasás/írás/chip select vezérlő jelek	09h	--- -xxx
TRISE	Vezérlő bitek a D Port perifériához	89h	0000 -111
PIR1	Megszakítás regiszter, PSPIF bit	0Ch	0000 0000
PIE1	Megszakítás engedély regiszter, PSPIE bit	8Ch	0000 0000

Ha a D,E Portok nem párhuzamos Slave Port üzemmódban dolgoznak, az IBF, OBF bitek 0 értéken maradnak. Ha előzőleg az IBOV flag 1 értéken maradt, szoftver úton törölni kell azt is.

Ellenőrző kérdések:

1. Hogyan lehet kijelölni a Port pontok adatirányát?
2. Mik az A Port pontok alternatív funkciói?
3. Mik a B Port pontok alternatív funkciói?
4. Mik a C Port pontok alternatív funkciói?
5. Mik a D Port pontok alternatív funkciói?
6. Mik az E Port pontok alternatív funkciói?
7. Mi a párhuzamos Slave Port működésének lényege?

3.1.4. Az időzítő modulok

A PIC 16C74-ben három időzítő modult találunk, mindegyik generálhat megszakítást, ha túlsordul:

- Timer0 (TMR0) modul (megfelel a 16C5x elemekben lévő RTCC-nek)
- Timer1 (TMR1) modul
- Timer2 (TMR2) modul.

Két további modul is kapcsolódhat az időzítő egységekhez, ezek a következők:

- Kiolvasó (Capture)/komparáló (Compare)/PWM1 modul (CCP1)
- Kiolvasó (Capture)/komparáló (Compare)/PWM2 modul (CCP2).

A Timer0 (TMR0) 8 bites számláló, működtethető belső órajellel (OSC/4) vagy külső impulzusokkal, ekkor a külső jelnél programozható az aktív él. Programozható előosztó rendelhető a TMR0-hoz, de az előosztó tartozhat a WDT modulhoz is (a PSA bittől függően, OPTION<3>). Az előosztó osztási arányát a PS2...0 bitekkel lehet beállítani (OPTION<2:0>). Ha előosztóként a TMR0-hoz rendeljük, a külső órajel frekvenciája meghaladhatja a mikrovezérlő órajel frekvenciáját, a legnagyobb érték 50 MHz lehet.

A Timer1 (TMR1) 16 bites időzítő/számláló egység. Léptethető belső órajellel (OSC/4) vagy külső impulzusokkal. A külső léptetés esetén előírható vagy kiiktatható a belső szinkronizáció. Külső léptető jellel, szinkronizálatlan működési módban akár SLEEP állapotban is számlálhat. A TMR1-nek is van programozható előosztója, az osztásarány 1:1, 1:4 vagy 1:8 lehet. A TMR1 összerendelhető a CCP modullal, ekkor a CCP1 vagy a CCP2 16 bites időbázisaként működik, kiolvasás (Compare) illetve komparálás (Compare) üzemmódokban. A CCP modullal csak szinkronizált működésben dolgozhat együtt az időzítő.

A Timer2 (TMR2) 8 bites időzítő. A TMR2-nek programozható előosztója és utóosztója is van, valamint egy nyolcbites periódus regisztere (PR2). A TMR2-t a CCP modulhoz is hozzárendelhetjük (PWM működéshez), továbbá a szinkron soros Port-nál (SSP) is felhasználható, mint Baud Rate generátor. Az előosztó lehetséges osztási arányai: 1:1, 1:4 vagy 1:16. Az utóosztó lehetővé teszi, hogy a TMR2 tartalmát összehasonlítsa a CPU a periódus regiszterével, még mielőtt megszakítás keletkezne. Az utóosztó 1:1 - től 1:16 - ig programozható.

A CCP1 és a CCP2 modulok lehetséges üzemmódjai:

- 16 bites kiolvasás (Capture)
- 16 bites komparálás (Compare)
- 8 vagy 10 bites impulzusszélesség moduláció (PWM).

A kiolvasás (Capture) üzemmód külső jel hatására a futó TMR1 számláló tartalmát átmásolja a CCPRxH:CCPRxL regiszterpárba. A kiolvasást kiváltó esemény programozható, lehet a CCPx IC láb felépő felfutó él, lefutó él, minden 4. felfutó él vagy minden 16. felfutó él.

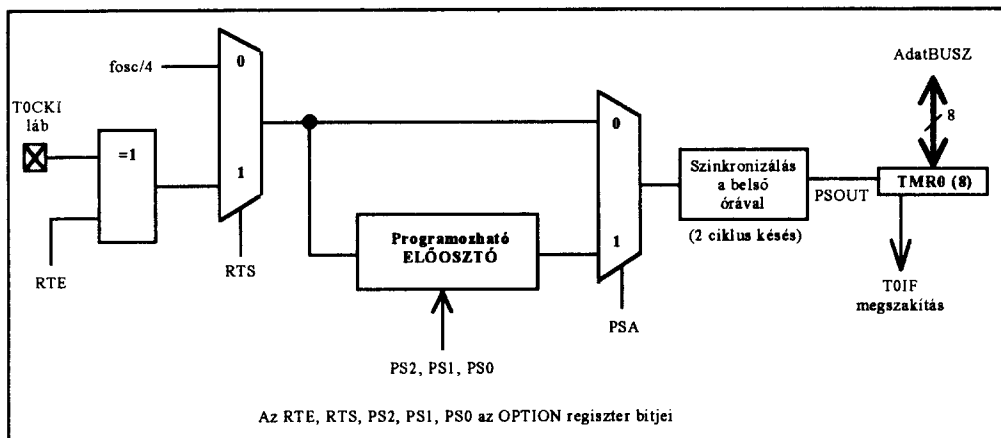
A komparálás módban a TMR1 16 bites pillanatnyi tartalma folyamatosan összehasonlításra kerül a CCPRxH:CCPRxL regiszterpár tartalmával. Ha egyezés lép fel, megszakítás generálódik és a CCP1 kimeneti IC láb előírt szintre vált (L vagy H, programozhatóan). Előírható (TMR1 esetén) hogy az egyezéskor a TMR1 törlődjön, a TMR2 esetében a törlődésen kívül az A/D konverzió is indulhat egyezéskor. A CCP működéseket a CCPxM3...CCPxM0 bitek állítják be (CCPxCON regiszterek).

A PWM üzemmód a TMR2 tartalmát veti össze egy 10 bites értékkel (Duty Cycle, jelidő a periódus időn belül) és a 8 bites periódus regiszterrel (PR2). Amikor TMR2=PR2, a TMR2 00h tartalomra vált,

egy megszakítás generálódik és a CCPx IC lábán, ha az kimenet, HIGH szint jelenik meg. Amikor TMR2=Duty Cycle regiszter, a CCPx láb LOW szintre vált.

3.1.4.1. A TMR0 modul

A TMR0 és a 16C5x mikrovezérlőkben lévő RTCC gyakorlatilag azonos működésű modul, mindkét csoportban a 01h című fájl regiszterként alakították ki. Az egyetlen újdonság, hogy a 16Cxx elemekben ez az időzítő is képes megszakítást generálni, ha túlszordul (3.17. ábra).

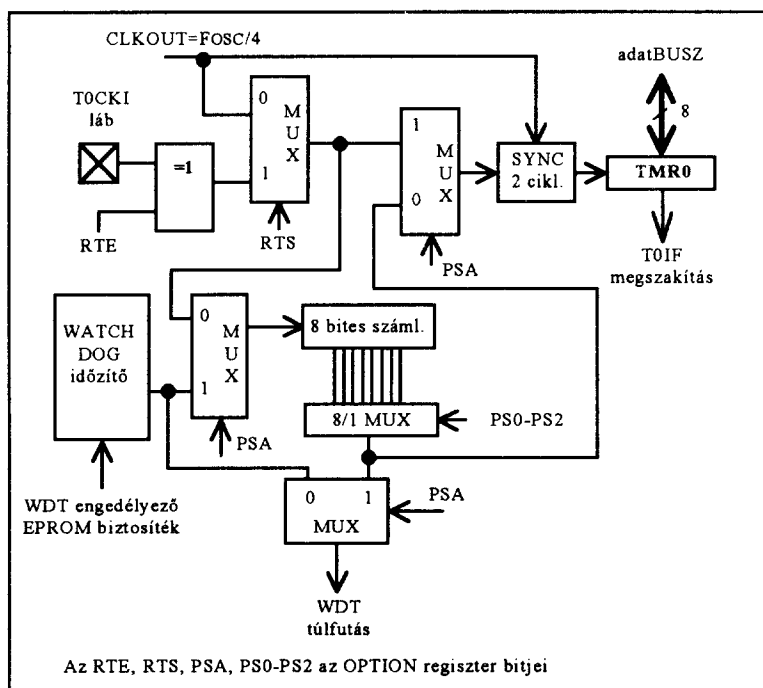


3.17. ábra A TMR0 modul (16C74)

A TMR0 elemeit és az előosztót az OPTION regiszter bitjei kezelik, amint azt a 3.17. ábra is feltünteti (ezek a részletek is azonosak az RTCC-vel).

A TMR0 megszakítás jele akkor keletkezik, ha a számláló FFh-ról 00h-ra túlszordul, ekkor vált 1-re a TOIF bit (INTCON<2>). Ezt a megszakítást maszkolhatjuk a TOIE bittel (INTCON<5>). A TOIF csak szoftverrel törölhető. Ez a megszakítás nem ébresztheti fel a SLEEP állapotból a csipet, mivel a TMR0 nem működik a SLEEP-ben.

Ha a TMR0 külső órájellel működik, az időzítésekre hasonló korlátok érvényesek, mint a 16C5x változatok esetében, az előosztó használata is meg-
egyezik (3.18. ábra). A Timer0 programozására, vezérlésére vonatkozó tudnivalókat a 16C5x áramkörök leírásánál, az RTCC-nél találja meg.



3.18. ábra Az előosztó alkalmazása (16C74)

3.1.4.2. A Timer1 modul

A Timer1 (TMR1) modul egy 16 bites időzítő/számláló, mely két nyolcbites regiszterből áll (TMR1H és TMR1L), ezek írhatóak és olvashatóak. A TMR1 folyamatosan léptethető 0000h-tól FFFFh értékig, a következő léptetés hatására túlszordul, a tartalma ismét 0000h lesz. Ekkor (ha engedé-

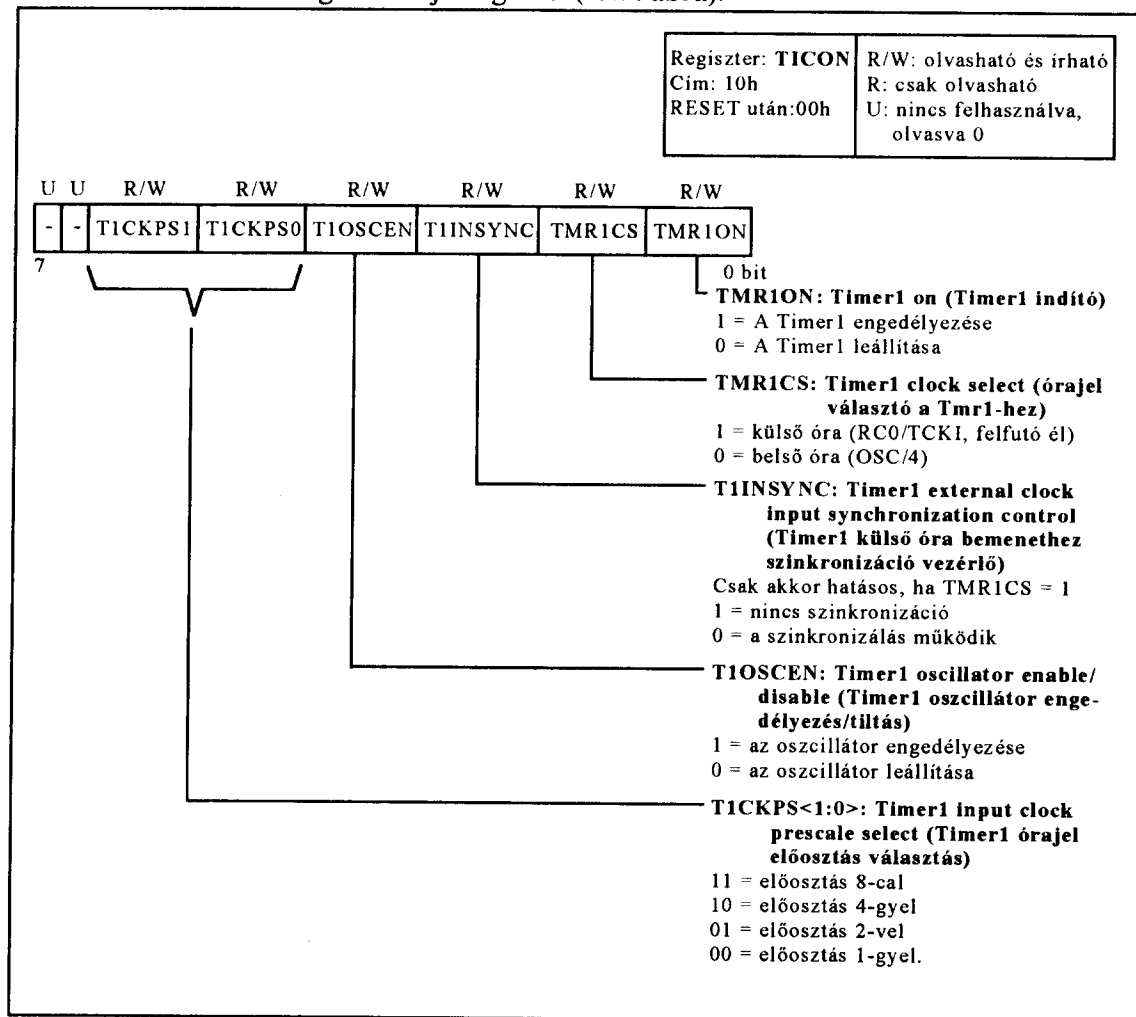
lyezett), megszakítás lép fel, a megszakítás flag a TMR1IF (PIR1<0>). A Timer1 két lehetséges üzemmódban működtethető:

- időzítőként,
- számlálóként,

e két üzemmód közül a TMR1CS (TICON<1>) bit választ.

Időzítő módban a TMR1-et minden gépi ciklus lépteti, számláló módban külső jellel lehet inkrementálni - utóbbi az RC0/T1OSO/T1CKI IC lábán keresztül érkezik (T1CKI).

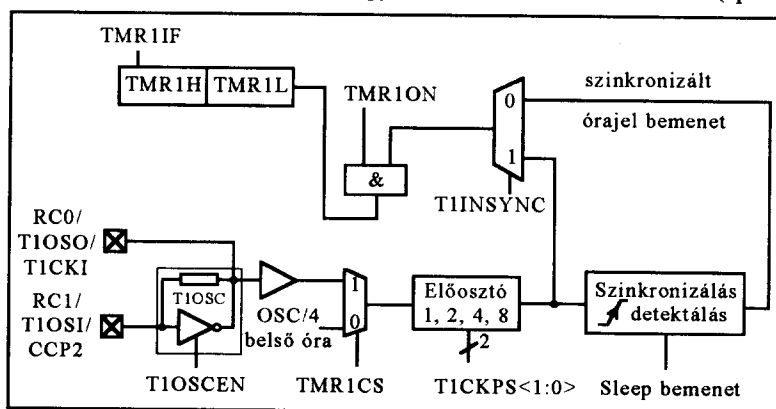
A Timer1 futása közben bármikor ki- illetve bekapcsolható a TMR1ON bittel (TICON<0>). A TMR1 kezelését a TICON regiszter bitjei végzik el (3.19. ábra).



3.19. ábra A TICON regiszter (16C74)

A Timer1 egy belső reset jellel is nullázható, amit a CCP1 vagy a CCP2 modul állíthat elő (speciális triggerelési esemény).

Ha a Timer1 saját oszcillátorának működését engedélyezzük (azaz a T1OSCEN=1), az RC1/T1OSI/CCP2 IC láb bemenetként működik (T1OSI), ekkor a TRISC<1> értékét figyelmen kívül hagyja a CPU. A belső órajel külső elemek számára is hozzáférhető az RC0/T1OSO/T1CKI lábán.



3.20. ábra A Timer1 blokkvázlata (16C74)

Az **időzítő (Timer) módban**, amikor a $TMR1CS=0$ ($T1CON<1>$), a számlálót az $OSC/4$ belső jel lépteti. A szinkronizációt vezérlő bit ($T1INSYNC$ a $T1CON<2>$ biten) nem határos, mivel a belső órajellel léptetés mindenképpen szinkronizált működést jelent.

A **számláló (Counter) módot** a $TMR1CS$ bit 1-re váltásával lehet elérni. Ebben az üzemmódban külső léptető jel felfutó élére inkrementálódik a számláló, ami érkezik az $RC0/T1OSO/T1CKI$ lábón, vagy (ha $T1OSCEN=1$) az $RC1/T1OSI/CCP2$ lábón. A lehetséges léptető jelek a **3.20. ábrán** tanulmányozhatók.

Ha a $T1INSYNC$ bit 0-ra törölt, a külső órajel úgy éri el a számlálót, hogy előzőleg a belső működési fázisokkal szinkronizálódik. A szinkronizáció az előosztó után valósul meg, az előosztó tehát mindenképpen aszinkron számlálóként dolgozik. A szinkronizáló részletek nem működnek **SLEEP** állapotban, így szinkron számláló üzemmóddal nem lehet a csipet ébreszteni.

A **szinkronizált számláló módban** az előosztó kimenetéről minden utasítás ciklusban kétszer mintát vesz a CPU. A külső léptető jel frekvenciája 0...200 kHz közötti lehet. A periódusidőn belül a H és L tartomány időtartama minimális értéke is kötött. Ha az előosztó nem végez osztást (azaz a beállított osztásarány 1:1), a $T1CKI$ H és L részletei minimális időtartamára érvényes összefüggések a következők:

$$TCKI\ H\ időtartam \geq 2\ t_{osc} + 20\ ns,$$

$$TCKI\ L\ időtartam \geq 2\ t_{osc} + 20\ ns.$$

Ha az előosztó N értékre van beállítva, a kimenetén négyszöghullám jelenik meg - ezért ekkor csak a periódusidőre kell időkorlátot megadni:

$$T_{TICKI\ per} \geq \frac{4t_{osc} + 20ns}{N},$$

ahol N az előosztó osztási száma (2, 4 vagy 8).

Ha a $T1INSYNC$ bitet 1-re állítjuk, a külső órajel szinkronizációja elmarad - ez az **aszinkron számláló üzemmód**. Ebben az üzemmódban **SLEEP** állapotban is működik a számláló. A külső jel L illetve H szintje minimális időtartama 8 t_{osc} . Aszinkron számláló üzemmódban a **Timer1** nem lehet **CCP** működés időbázisa.

A futó számláló tartalmát bármikor közvetlenül is ki lehet olvasni, de a két 8 bites kiolvasás közben a számláló tartalma változhat, az eredmény félrevezető lehet. Ha kezdőértéket írunk a számlálóba, ahhoz le kell állítani a számlálást. A kezdőérték az első inkrementáló élnél kerül be a számlálóba.

A **3.20. ábrán** látható a **Timer1** bemenetén elhelyezett áramkörü részlet, amit kvarcoszcillátorként vagy RC oszcillátorként is lehet működtetni, hasonló kapcsolási megoldásokkal, mint a 16C család mikrovezérlői alaposzcillátort (ld. 1.1. illetve 1.5. ábra). Ehhez a $T1OSCEN$ bitet ($T1CON<3>$) 1-re kell állítani. 200 kHz-ig az oszcillátor LP (Low Power) jellegű, s **SLEEP** alatt is üzemel. A Microchip által a kvarcoszcillátorhoz ajánlott kondenzátor értékek a **3.14. táblázatban** láthatók.

3.14. táblázat A **Timer1** kvarcoszcillátor kondenzátor értékei

Osc. típus	Frekvencia	C1	C2
LP	32 kHz*	15 pF	15 pF
	100 kHz	15 pF	15 pF
	200 kHz	0-15 pF	0-15 pF
A nagyobb kapacitás növeli az oszcillátor stabilitását, de megnöveli a start-up időt is. A táblázat adatai csak ajánlások.			
* = ha a VDD nagyobb, mint 4,5V; $C1=C2=30$ pF ajánlott.			

Ha a **CCP1** vagy **CCP2** modulhoz rendelve működik a **Timer1**, speciális triggerelő eseményt kezelnek a **CCP** modulok - egy belső törlési lehetőségük van, ha perogramozzuk ($CCP1M<3:0>=1011$). Ha ez a speciális belső törlés egy léptető hatással egyszerre jelentkezik, a léptetés jut érvényre. A **CCP** működéshez a **Timer1**-et időzítő vagy szinkron számláló módban kell használni, aszinkron számláló módban nem szolgálhat **CCP** időalapként. (Az időzítő működés mindenképpen szinkronizált a belső órajellel).

A **Timer1**-et csak a **CCP1** speciális reset jele törli, a **POR** és más **RESET** események nem. A **T1CON** regisztert azonban törli a **Power On Reset (POR)**, ezt a regisztert pedig az egyéb **RESET** események nem tudják törölni. Az előosztót minden **TMR1** írási utasítás törli.

A **Timer1** működését kezelő regiszterek összefoglalása a **3.15. táblázatban** látható.

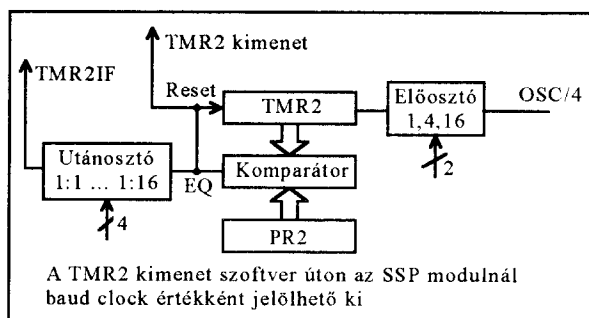
3.15. táblázat A Timer1-et kezelő regiszterek

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8Bh	INTCON	GIE	PEIE						
0Ch	PIR1		-	-	-				TMR1IF
8Ch	PIE1		-	-	-				TMR1IE
0Eh	TMR1L	Timer1 alsó helyiértékű bájt							
0Fh	TMR1H	Timer1 felső helyiértékű bájt							
10h	T1CON	-	-	T1CKPS1	T1CKPS0	T1OSCEN	TIINSYNC	TMR1CS	TMR1ON

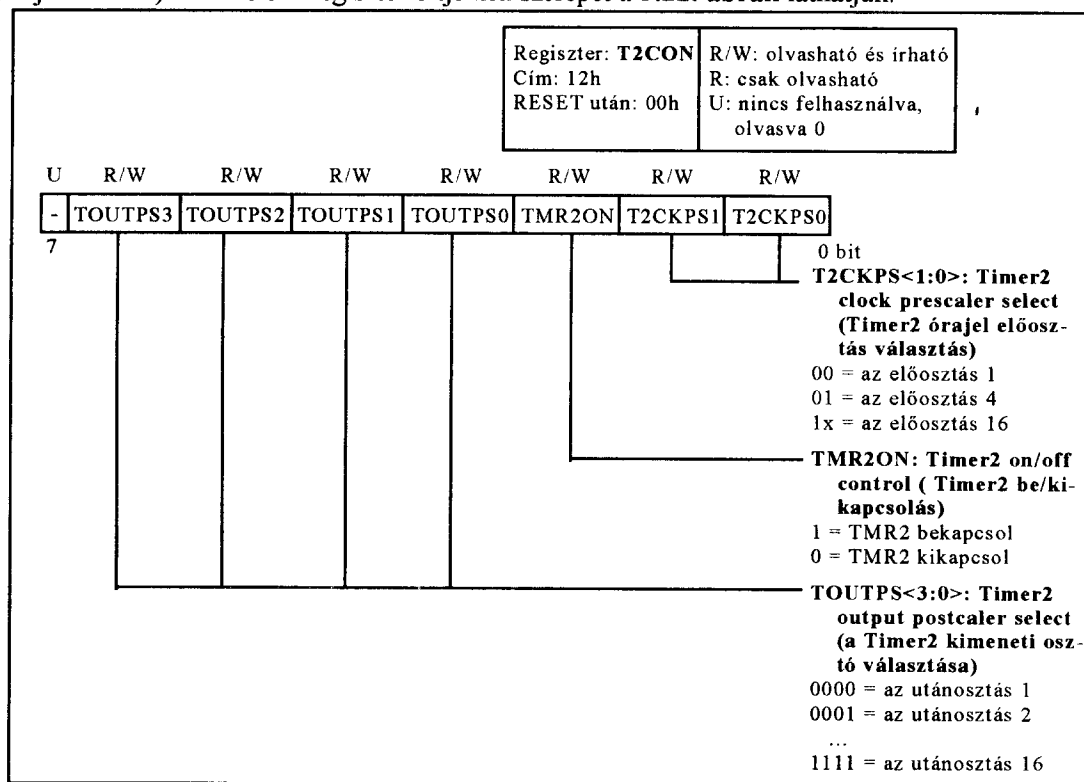
A sötétített biteket nem használja a Timer1 modul

3.1.4.3. A Timer2 modul

A **Timer2 (TMR2)** modul 8 bites szám-láló, melyhez előosztó és utóosztó áramkör is csatlakozik. Írható/olvasható regiszter, elsődlegesen a PWM működés időalapjaként működtethető (a CCP modulok PWM üzemmódjában). A Timer2 felépítését a 3.21. ábrán lehet szemügyre venni. A bemenő órajelet (OSC/4) az előosztó fogadja (az osztási arány: 1, 4 vagy 16, amit a T2CON regiszterben lévő TCKPS1, TCKPS0 vezérlő bitek jelölnek ki). A T2CON regiszter bitejének szerepét a 3.22. ábrán láthatjuk.



3.21. ábra A Timer2 felépítése (16C74)



3.22. ábra A T2CON regiszter (16C74)

A Timer2 modulban egy komparátor és egy periódus regiszter is található (PR2). A Timer2 00h-tól inkrementálódik, míg a PR2-vel azonos nem lesz a tartalma - ekkor ismét 00h-ra vált. A PR2 egy írható/olvasható regiszter. Amikor a tartalma a PR2-vel azonosává válik s emiatt nullázódik, a TMR2 egy léptető jelet küld az utóosztóba (amelyik 1:1 ... 1:16 közötti osztás arányokra állítható be, a T2CON regiszterben lévő TOUTPS0 ... 3 bitekkel). Az utóosztóból kilépő, leosztott impulzusok megszakítást generálnak (TMR2IF, a PIR1<1> biten). Ha nincs rá szükség, a TMR2 leállítható a TMR2ON bit 0 értékével (T2CON<2>), ezáltal csökkenthető a mikrovezérlő által felvett tápteljesítmény.

Az előosztó és az utóosztó néhány esetben automatikusan törlődnek:

- ha a TMR2 regisztert írjuk
- ha a T2CON regisztert írjuk
- ha valamilyen RESET folyamat megy végbe (Power On Reset, WDT túlfutási Reset, MCLR)

A TMR2 kimenő jeleit, amiket az utóosztó fogad, a mikrovezérlő más célra is fel tudja használni: a szinkron soros Port számára Baud Rate léptető jelként.

A Timer2 kezelésében résztvevő regisztereket a **3.16. táblázatban** foglaltuk össze.

3.16. táblázat A Timer2 regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Bh	INTCON	GIE	PEIE						
0Ch	PIR1		-	-	-			TMR2IF	
8Ch	PIE1		-	-	-			TMR2IE	
11h	TMR2	Timer2							
12h	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
92h	PR2	Timer2 periódus regiszter							

A sötétített biteket nem használja a Timer2 modul

Ellenőrző kérdések:

1. Melyek a PIC16C74 időzítő/számláló moduljai?
2. Melyik PIC16C74 időzítő modul felel meg a PIC16C5x mikrovezérlőkben lévő RTCC-nek? Mi a különbség a két majdnem azonos modul között?
3. Melyik időzítő vesz részt a kiolvasás (Capture) működésben?
4. Melyik időzítő vesz részt a komparálás (Compare) működésben?
5. Melyik időzítő vesz részt a PWM működésben?
6. Ébresztheti-e a csipet SLEEP állapotból a Timer0?
7. Ébresztheti-e a csipet SLEEP állapotból a Timer1?
8. Ébresztheti-e a csipet SLEEP állapotból a Timer2?
9. Hány bites a Timer0? Van-e előosztója, utóosztója, segédregisztere?
10. Hány bites a Timer1? Van-e előosztója, utóosztója, segédregisztere?
11. Hány bites a Timer2? Van-e előosztója, utóosztója, segédregisztere?
12. Milyen szinkronozott üzemmódjai vannak a Timer1-nek?
13. Milyen szinkronozott üzemmódjai vannak a Timer2-nek?
14. Mi a különbség az időzítő és a számláló üzemmódok között?
15. Mi lépteti a számlálókat időzítő üzemmódban?
16. Lehet-e a Timer2-vel 00h-tól FFh-ig számláltatni?
17. A Timer2 milyen belső jelet állít elő a mikrovezérlő számára?
18. Melyik időzítő modulokat lehet a CCP modulból származó belső törlő jellel nullázni?

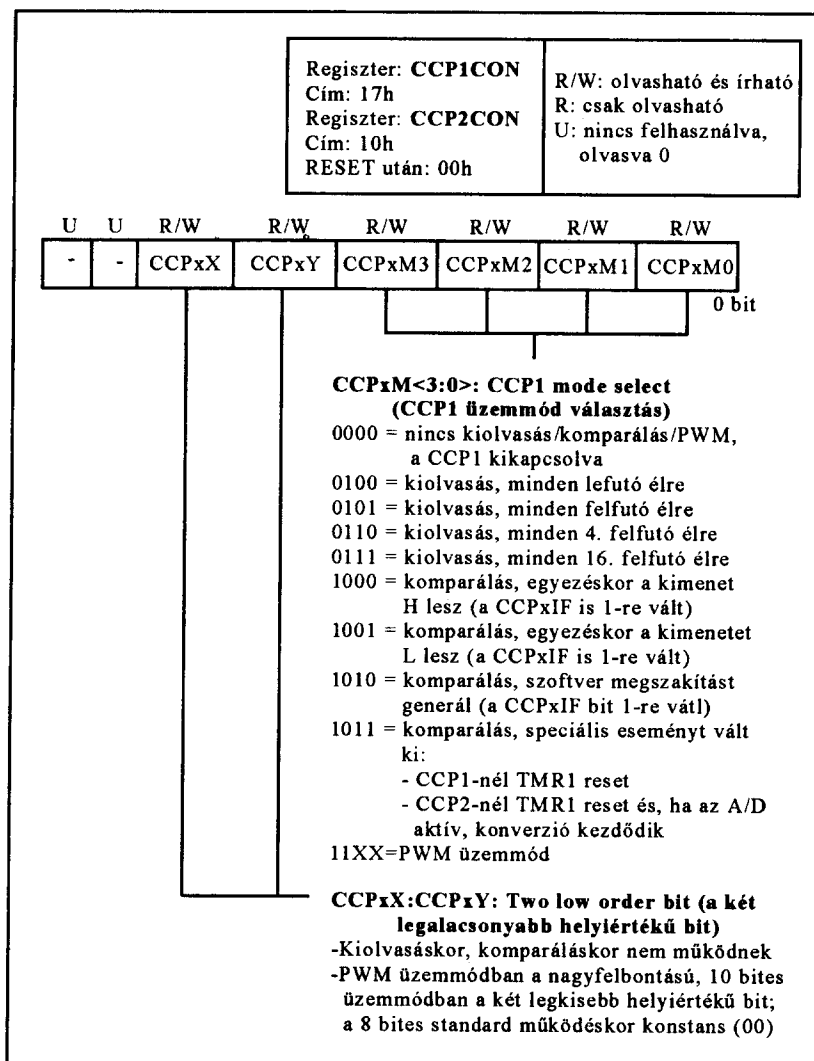
3.1.5. A CCP modulok

A CCP modulok (Capture/Compare/PWM; kiolvasás/komparálás/impulzusszéleség moduláció modulok) fő eleme egy-egy 16 bites számláló, amit Capture regiszterként, Compare regiszterként illetve PWM regiszterként működtet a CPU. A CCP1 és a CCP2 felépítése, működése teljesen megegyezik, az egyetlen kivétel az, hogy a speciális triggerelési esemény eredménye különbözik, ezt majd részletesen leírjuk. A továbbiakban a CCP1-et tárgyaljuk, de minden részlet igaz a CCP2-re is - az említett triggerelési esemény kivételével.

A CCP1 16 bites regisztere tulajdonképpen két darab egy bájtos regiszterből épül fel, a CCPR1L (alsó bájt) és a CCPR1H (felső bájt) regiszterekből. (A CCP2 modul 16 bites regiszterének alsó bájtja a CCP2RL, a felső bájtja a CCP2H). Minden CCPRxX bájt közvetlenül írható és olvasható. A CCP modulok kezelését a CCP1CON illetve a CCP2CON regiszterek biteivel lehet megoldani.

Mindkét CCP regiszter alkalmas mindhárom üzemmódra, de a Capture és a Compare működésbe csak a Timer1 időzítő modult lehet bevonni, a PWM működés alapja pedig csak a Timer2 időzítő lehet. A CCP regiszterek egyéb működésre, pl. önálló számlálásra vagy időmérésre nem is alkalmazhatóak.

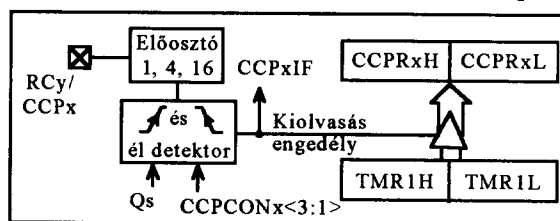
Mivel a két vezérlő regiszter felépítése is megegyezik, egyetlen ábrán mindkettőt lehet szemléltetni (3.23. ábra).



3.23. ábra A CCPxCON regiszterek (16C74)

1-re (PIR1<2>), ezt később szoftverrel kell törölni. Ha egy kiolvasást úgy követ egy második Capture esemény, hogy addigra a CCPR1-et a CPU még nem olvasta ki, az előző kiolvasás eredménye elvesz. Ha Capture működéskor az RC2/CCP1 láb kimenetnek van konfigurálva, minden kimenetre írás Capture eseményként hat. Amikor a Capture beállításokat megváltoztatjuk, egy hamis Capture megszakítás igény keletkezhet, amit törölni kell.

A Capture események előosztója (1, 4 vagy 16-os osztásértékkel) törölődik, ha a CCP modult kikapcsoljuk, vagy nem Capture módban üzemeltetjük, s törli a RESET folyamat is. Az előosztási arány megváltoztatása azonban nem törli. A CCP1 modul Capture üzeme esetén a Timer1 számlálót időzítő módban vagy szinkron számlálóként kell használnunk. A Capture mód logikai kapcsolatait a 3.24. ábra mutatja be.



3.24. ábra A Capture üzemmód (16C74)

3.1.5.1. A Capture (kiolvasás) működés

A kiolvasás (Capture) mód lehetővé teszi, hogy a TMR1 16 bites számláló tartalmát egy külső esemény fellépésének pillanatában betöltsük a CCP egyik regiszterébe. Ez a külső esemény a CCP1 esetében (amikor is a TMR1 tartalmát a CCPR1H:CCPR1L 16 bites regiszterpár fogadja be) az RC2/CCP1 IC lábón lép fel (a megfelelő TRIS bittel bemenetre kell állítani), s a következők egyike lehet:

- lefutó él
- felfutó él
- minden negyedik lefutó él
- minden tizenhatodik felfutó él.

A Capture esemény kiválasztására is a CCP1CON regiszter legalsó 4 bitjét használjuk. A capture esemény megszakítást is kiválthat, a CCP1IF flag bitet állítja

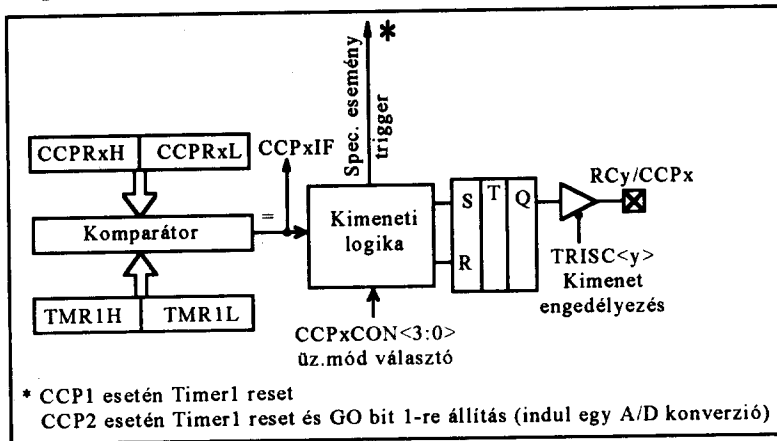
3.1.5.2. A komparálás (Compare) működés

A Compare (komparálás) üzemmódban a 16 bites CCPR regiszter tartalmát egy 16 bites komparátor folyamatosan összehasonlítja a TMR1 tartalmával. Amikor egyezést (match) észlel a komparátor, ezt jelezheti a mikrovezérlő RC2/CCP1 csatlakozó pontján is, de mindenképpen jelzi a CCP1IF flag bit 1-re váltásával. A komparálás bekövetkezésekor történő esemény lehetséges változatai:

- a kimenet H szintre vált

- a kimenet L szintre vált
- a kimenet nem változik (csak a belső megszakítással jelez a modul).

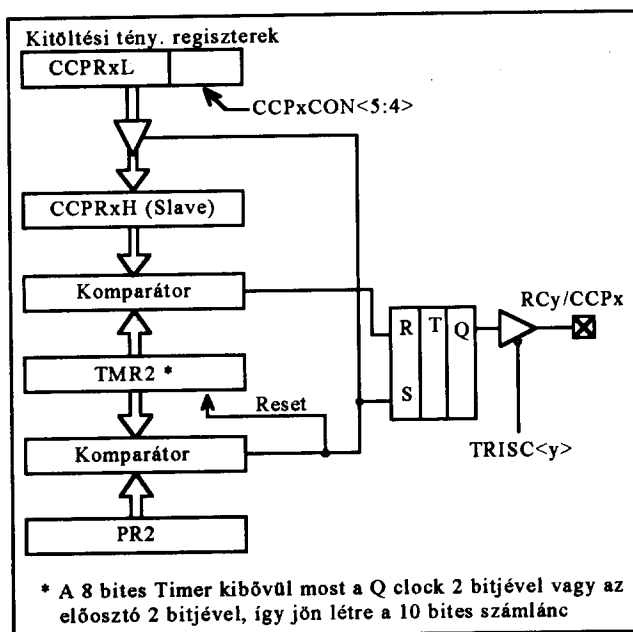
Ezeket a működési lehetőségeket a CCP1CON alsó 4 bitjén lehet beállítani. A komparáláskor működő megszakítás flag bit, a CCP1IF a PIR1 regiszterben található. Az RC2/CCP1 IC láb adatarányát a



3.25. ábra A komparálás üzemmód (16C74)

is vannak, ezekkel bizonyos belső működéseket lehet kezdeményezni. A CCP1 speciális triggerelési lehetősége az, hogy a komparátor match jelzésekor reseteli a Timer1-et (ha engedélyezzük ezt a működést). Így a CCPR1 regiszter a TMR1 periódus regisztereként használható. A CCP2 speciális triggerelő kimenete is törli a TMR1-et, de emellett megindít egy A/D konverziót is. Ez lehetővé teszi, hogy periódikus A/D konverzió történjen, a CPU közvetlen felügyelete és irányítása nélkül. A komparálás üzemmód logikai kapcsolatait a 3.25. ábrán láthatjuk.

3.1.5.3. A PWM működés



3.26. ábra A PWM üzemmód (16C74)

A PWM periódusidő hossza:

$$[(PR2) + 1] \cdot 4t_{osc} \cdot (TMR2 \text{ előosztó érték})$$

a PWM kimenet H szintje időtartama pedig

$$(DC1) \cdot t_{osc} \cdot (TMR2 \text{ előosztó érték}),$$

Compare működéshez kimenetre kell beállítani. A CCP1CON regiszter törlésekor, pl. RESET eseményénél az RC2/CCP1 kimenet L szintre áll be, ez a default állapota, ezt az értéket nem a C Port kimeneti adat latch tárolja.

A Timer1-nek a komparálási működéshez időzítő vagy szinkron számlálási üzemmódban kell lennie. A komparálási módban speciális triggerelési lehetőségek

Az impulzusszélesség moduláció (Pulse Width Modulation, PWM) módban az RC2/CCP1 kivezetésen egy legfeljebb 10 bites felbontású PWM jel lép ki (ehhez ezt az IC lábát kimenetként kell konfigurálni). A CCPR1L alsó regiszter bájtba (master) kerül a 8 bites kitöltési tényező, duty cycle (a H szint időtartam) érték. A CCPR1H slave puffer regiszterként üzemel. Amikor a PWM kimenet H szintre vált, a 8 bites adat a master-ből áttöltődik a slave-be, ez a kettős pufferezés a precíz PWM működés biztosítéka. A PWM módban a CCPR1H olvasható de nem írható.

A periódus hosszát a Timer2 periódus regisztere szabja meg (a logikai elrendezést a 3.26. ábra mutatja be). A periódus időt és a H szintet mérő regisztereket a mikrovezérlő különböző frekvenciájú jelekkel lépteti.

ahol DC1 a CCPR1L és a CCPxCON<5:4> alkotta 10 bites érték (így lesz a PWM legnagyobb felbontása 10 bites).

A CCP1CON regiszter törlése az RC2/CCP1 PWM kimenetet L szintre állítja, ez a default kimeneti érték (nem a C Port kimeneti latch tárolja).

3.17. táblázat PWM frekvenciák, 20 MHz-hez

legnagyobb felbontás	frekvencia		
	TMR2 előosztó=1	TMR2 előosztó=4	TMR2 előosztó=16
10 bit	19,53 kHz	4,88 kHz	1,22 kHz
9 bit	39,06 kHz	9,77 kHz	2,44 kHz
8 bit	78,13 kHz	19,53 kHz	4,88 kHz

A 3.17. táblázat bemutat néhány PWM frekvencia értéket, 20 MHz órajel frekvencia esetére. A 3.18. táblázatból a PWM frekvencia alapján lehet a beállítási értékeket kiolvasni.

3.18. táblázat PWM beállítási adatok

PWM frekvencia	1,22 kHz	4,88 kHz	19,53 kHz	78,13 kHz	157,5 kHz	210,53 kHz
Előosztás	16	4	1	1	1	1
PR2 érték	0xFF	0xFF	0xFF	0xFF	0x7F	0x5F
Felbontás (nagy felbontás mód)	10 bit	10 bit	10 bit	8 bit	7 bit	6,5 bit
Felbontás (standard felb. mód)	8 bit	8 bit	8 bit	6 bit	5 bit	4,5 bit

A következő táblázatok három esetre foglalják össze a regisztereket:

3.19. táblázat: Timer1, Kiolvasás (Capture) üzemmód

3.20. táblázat: Timer1, Komparálás (Compare) üzemmód

3.21. táblázat: Timer2, PWM üzemmód.

3.19. táblázat Timer1, Kiolvasás (Capture) üzemmód regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8Bh	INTCON	GIE	PEIE						
0Ch	PIR1		-	-	-		CCP1IF		TMR1IF2
8Ch	PIE1		-	-	-		CCP1IE		TMR1IE
0E	TMR1L	Timer1 alsó helyiértékű bájtt							
0F	TMR1H	Timer1 felső helyiértékű bájtt							
10	TICON	-	-	TICKPS1	TICKPS0	TIOSCEN	TIINSYNC	TMR1CS	TMR1ON
15h	CCPR1L	Timer1 Capture (kiolvasás) regiszter (LSB)							
16h	CCPR1H	Timer1 Capture (kiolvasás) regiszter (MSB)							
17h	CCP1CON	-	-			CCP1M3	CCP1M2	CCP1M1	CCP1M0
1Bh	CCPR2L	Timer1 Capture (kiolvasás) regiszter 2 (LSB)							
1Ch	CCPR2H	Timer1 Capture (kiolvasás) regiszter 2 (MSB)							
1Dh	CCP2CON	-	-			CCP2M3	CCP2M2	CCP2M1	CCP2M0

A sötétített biteket ez az üzemmód nem használja

3.20. táblázat Timer1, Komparálás (Compare) üzemmód regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8Bh	INTCON	GIE	PEIE						
0Ch	PIR1		-	-	-		CCP1IF		TMR1IF2
8Ch	PIE1		-	-	-		CCP1IE		TMR1IE
0E	TMR1L	Timer1 alsó helyiértékű bájtt							
0F	TMR1H	Timer1 felső helyiértékű bájtt							
10	TICON	-	-	TICKPS1	TICKPS0	TIOSCEN	TIINSYNC	TMR1CS	TMR1ON
15h	CCPR1L	Timer1 Compare (komparátor) regiszter (LSB)							
16h	CCPR1H	Timer1 Compare (komparátor) regiszter (MSB)							
17h	CCP1CON	-	-			CCP1M3	CCP1M2	CCP1M1	CCP1M0
1Bh	CCPR2L	Timer1 Compare (komparátor) regiszter 2 (LSB)							
1Ch	CCPR2H	Timer1 Compare (komparátor) regiszter 2 (MSB)							
1Dh	CCP2CON	-	-			CCP2M3	CCP2M2	CCP2M1	CCP2M0

A sötétített biteket ez az üzemmód nem használja

3.21. táblázat *Timer2, PWM üzemmód*

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8Bh	INTCON	GIE	PEIE						
0Ch	PIR1		-	-	-			TMR2IF	
8Ch	PIE1		-	-	-			TMR2IE	
11h	TMR2	Timer2							
92h	PR2	Timer2 periódus regiszter							
12h	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
15h	CCPR1L	Timer2 Duty Cycle Regiszter (kitöltési tényező regiszter)							
16h	CCPR1H	Timer2 Duty Cycle register (kitöltési tényező regiszter) (slave)							
17h	CCP1CON	-	-	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0
1Bh	CCPR2L	Timer2 Duty Cycle Regiszter (kitöltési tényező regiszter) 2							
1Ch	CCPR2H	Timer2 Duty Cycle register (kitöltési tényező regiszter) (slave)2							
1Dh	CCP2CON	-	-	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0

A sötétített biteket ez az üzemmód nem használja

Ellenőrző kérdések:

1. Mi a kiolvasás (Capture) működés lényege?
2. A Capture üzemmód melyik számlálót, milyen üzemmódban használja?
3. Mi a komparálás (Compare) működés lényege?
4. A Compare üzemmód melyik számlálót, milyen üzemmódban használja?
5. Mi a PWM működés lényege?
6. A PWM üzemmód melyik számlálót, milyen üzemmódban használja?

3.1.6. A szinkron soros Port

A szinkron soros Port (Synchronous Serial Port, SSP) a mikrovezérlő és a külső egység, periféria közötti szinkronizált kommunikációt biztosítja újszerű, szabványos soros jelátvitellel. A különféle szabványos soros kommunikációs interfészek ma már sok áramkörnek beépített egységei, így ez a modul lehetővé teszi pl. soros adatkezelésű EEPROM, A/D konverter, léptető regiszter csatlakoztatását a mikrovezérlőhöz (vagy két mikrovezérlő között az információcserét). A PIC mikrovezérlőkbe beépített SSP két szabványos működésmódot tesz lehetővé, ezek:

- Serial Peripheral Interface (SPI),
- Inter-Integrated Circuit (IIC, I²C).

Az SSP-t kiszolgáló két regiszter az állapotregiszter (SSPSTAT, 3.27. ábra) és a vezérlő regiszter (SSPCON, 3.28. ábra). Az SSPSTAT regiszterben az IIC átvitel Start feltétel állapotát Start bitnek, a Stop feltétel állapotát Stop bitnek nevezi a Microchip.

3.1.6.1. Az SPI üzemmód

Az SPI üzemmódot 8 bites adatok szinkronizált kiküldésére (adására) és fogadására (vételére) lehet használni - akár egyidejűleg is. Három pontos kapcsolatot igényel ez az átviteli megoldás:

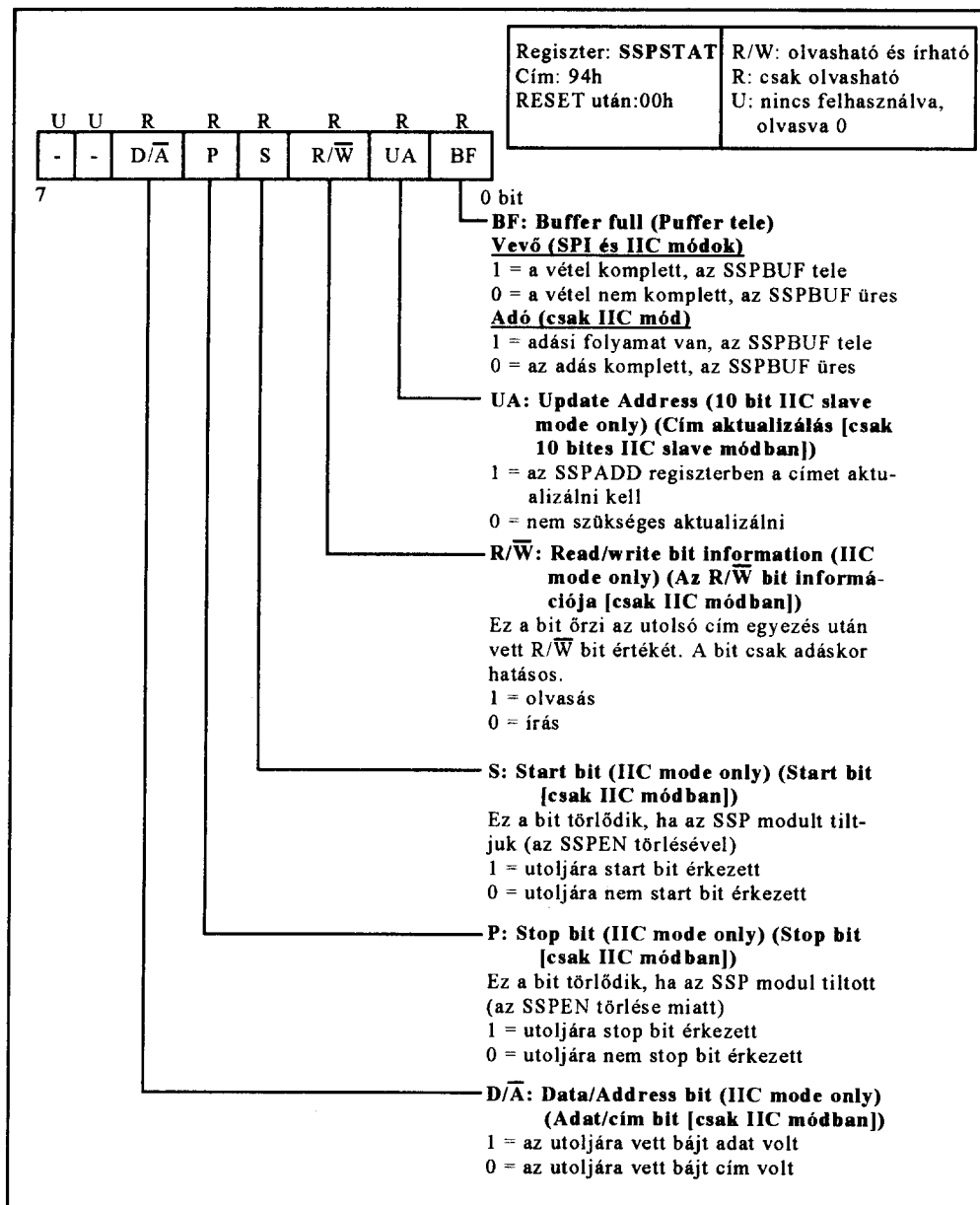
- Serial Data Out (soros adat kimenet, SDO)
- Serial Data In (soros adat bemenet, SDI)
- Serial Clock (soros órajel, SCK).

Ha egy áramkör a rendszerben alárendelt, (slave) módon üzemel, egy negyedik vezetéken át lehet vezérelni:

- Slave Select (slave kiválasztó, $\overline{SS}$).

Az SPI működést inicializálni kell, a lehetséges opciók közül ki kell választani a szükséges beállításokat. Erre szolgál az SSPCON első hat bitje. A kijelölhető opciók:

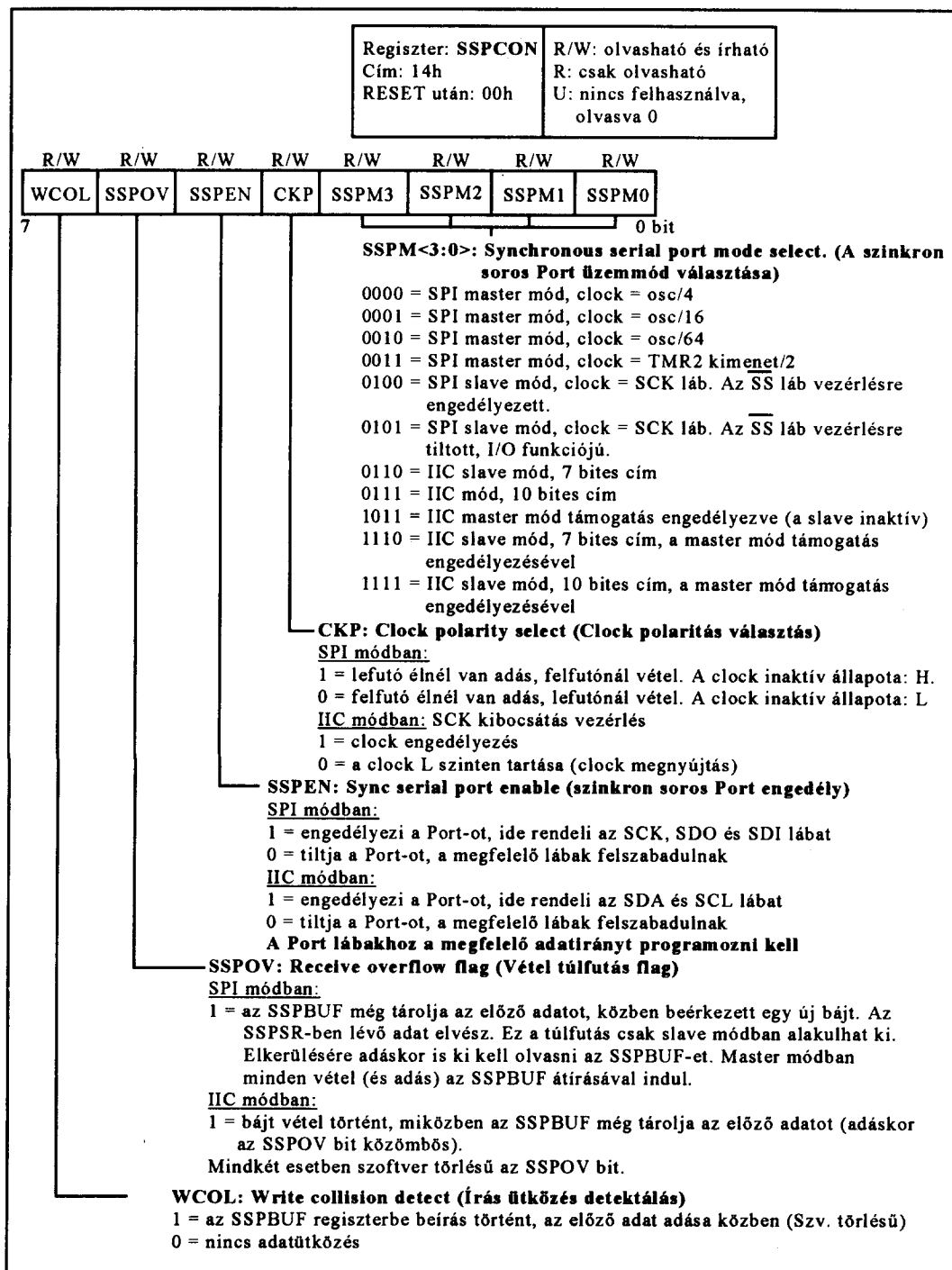
- Master mód (az SCK óra kimenet)
- Slave mód (az SCK óra bemenet)
- Clock Polarity (órajel aktív él kijelölés, a be- illetve kimenő adat az órajel felfutó vagy lefutó élénél érvényes-e)
- Clock Rate (órajel sebesség, csak Master módban)
- Slave Select Mode (Slave kijelölés mód, csak Slave módban).



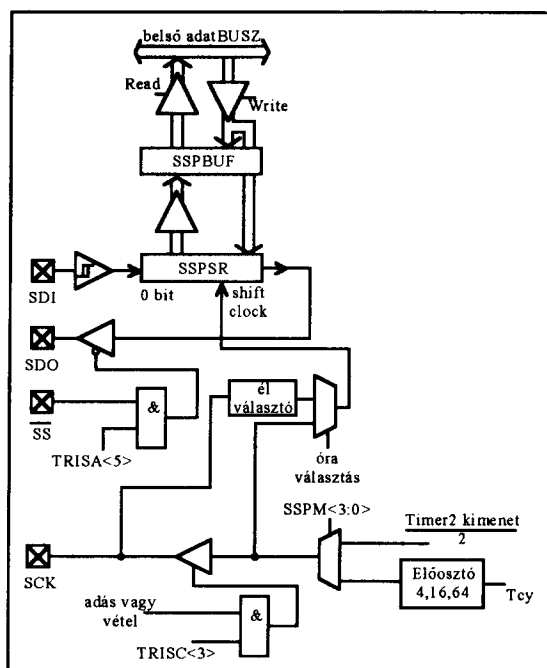
3.27. ábra Az SSPSTAT regiszter (16C74)

Az SSP legfontosabb eleme egy léptető regiszter, ez valósítja meg a párhuzamos/soros illetve a soros/párhuzamos átalakításokat. Az adatvesztés elkerülése érdekében puffereltek az adatirányok. A különféle áramköri egységeket a beprogramozott működési módnak megfelelően kapcsolja össze egymással a CPU.

A 3.29. ábra az SSP modul blokkvázlatát mutatja akkor, amikor SPI átvitelre programoztuk be.



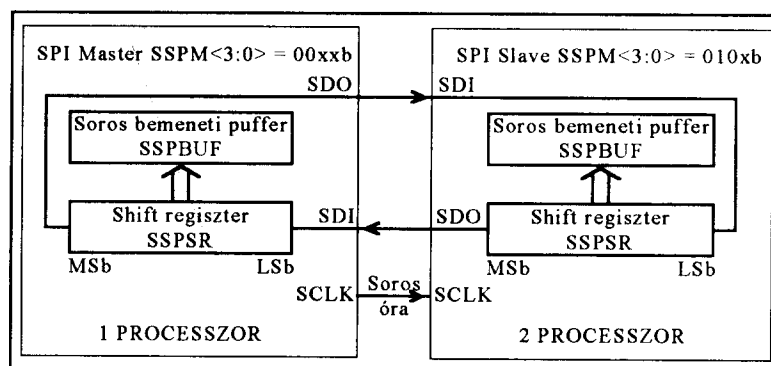
3.28. ábra Az SSPCON regiszter (16C74)



3.29. ábra Az SSP, SPI módban

ss alternatív funkciók aktivizálódnak. A TRISC adatirány regiszterben a megfelelő adatirányokat ki kell jelölni, az SCK pont adatirányát annak megfelelően, hogy Master vagy Slave módban fog majd üzemelni a mikrovezérlő.

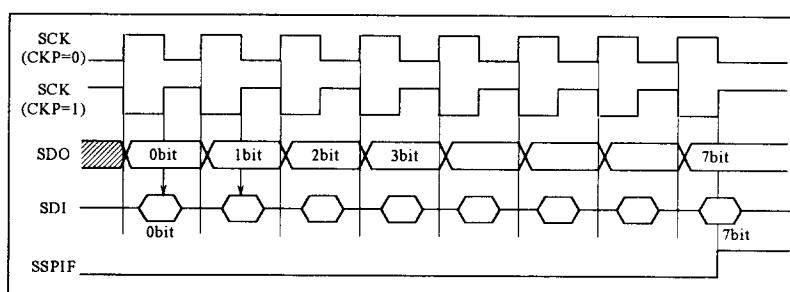
A 3.30. ábrán két SPI egység kapcsolata látható. A Master vezérlő processzor (1) indítja az átvitelt, az SCK kiküldésével. A két léptető regiszterben egyszerre léptetődik az adat a megfelelő órajel éleknél. Minden adatátvitelkor mindkét processzor egyidejűleg ad és vesz adatot - ha nincs kiküldendő adat, dummy (közömbös) adatot léptet ki az egyik.



3.30. ábra SPI kapcsolat

A Master bármikor elkezdhet adni, hiszen az SCK jellel ez kezdeményezi a kapcsolatot. Azt szoftver protokollal kell megoldani, hogy a Master tudomására jusson, ha a Slave kíván adni. Slave módban az SCK ponton fellépő külső órajelnél történik az adás és a vétel is. Az utolsó bit kezelésekor a megszakítási flag 1-re vált (SSPIF).

A jelformákat a Master módra és az $\overline{SS}$ nélküli Slave módra a 3.31. ábra mutatja be.



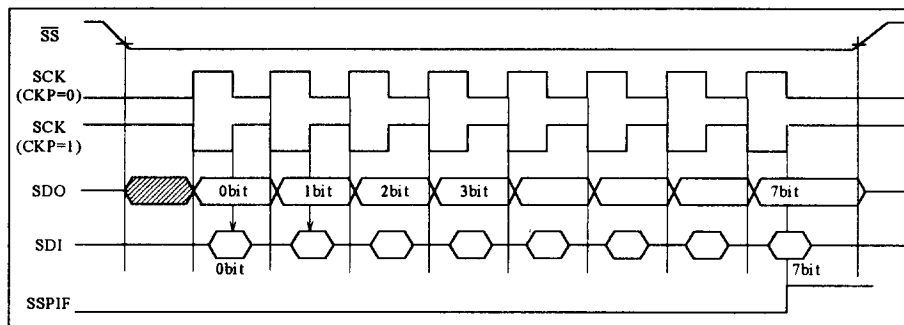
3.31. ábra SPI jelalakok, Master és $\overline{SS}$ nélküli Slave esetben

Az SSPSR a léptető regiszter, az SSPBUF a puffér - ez tárolja tehát az SSPSR-hez a CPU által küldött illetve vétel után az SSPSR-ből származó adatot. Ha egy 8 bites adat vétele megtörtént és az információ bekerült az SSPBUF-be, a BF (Buffer Full, puffér betöltve bit az SSPSTAT<0> biten) és az SSPIF megszakítás kérő flag bit 1-re váltanak. A léptető regiszter így máris fogadhatja a következő adatot, a beléptetés alatt van idő arra, hogy a CPU az előzőt kiolvassa a pufférból. Ha adat adás/vétel közben a CPU a pufférbe akar írni, a művelet nem hajtható végre és írás ütközési hiba (write collision) detektálódik (WCOL bit, SSPCON<7>). Ezt csak szoftverrel lehet törölni.

A vett adatot, amit a BF és az SSPIF jelez, a CPU kiolvassa a pufferból - ekkor a BF automatikusan törlődik.

A szinkron soros Port működés engedélyezéséhez az SSP engedélyező bitet (SSPEN) 1-re kell állítani - ennek hatására az SDI, SDO, SCK és az

A 3.32. ábrán az $\overline{SS}$ jellel vezérelt Slave jeleit láthatjuk.



3.32. ábra SPI működés, $\overline{SS}$ jellel vezérelve (Slave)

A programozható órajel változatok a következők:

- OSC/4 (Tcy)
- OSC/16 (4Tcy)
- OSC/64 (16Tcy)
- Timer2 kimenet / 2.

Eszerint pl. 20 MHz-es alaposzcillátor esetén a legnagyobb órajel frekvencia 5 MHz lehet. Az SPI működés regiszter készletét a 3.22. táblázat foglalja össze.

3.22. táblázat Az SPI működés regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8B	INTCON	GIE	PEIE						
0C	PIR1		-	-	-	SSPIF			
8C	PIE1		-	-	-	SSPIE			
13	SSPBUF	Szinkron soros Port vevő puffer / adó regiszter							
14	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
94	SSPSTAT	-	-	D/A			R/W	LA	BF

A sötétített bitek nincsenek felhasználva ebben az üzemmódban

Az SPI használatakor kétvezetékes kapcsolat is létesíthető - ehhez közösiíteni kell az SDO és az SDI pontokat. Amikor az SPI vevőként kíván működni, az SDO-t át kell konfigurálni bemenet módra.

3.1.6.2. Az I²C üzemmód

Az IIC (I²C) a Phillips/Signetics által kifejlesztett szinkron soros kommunikációs BUSZ rendszer, kétvezetékes kapcsolattal. Az IIC a protokollt is részletesen előírja. Adáskor egy eszköz Master (amelyik az órajeleket adja), a többi Slave. A 3.23. táblázat felsorolja az IIC BUSZ-rendszer leírásában használt kifejezéseket.

3.23. táblázat Az IIC BUSZ kifejezései

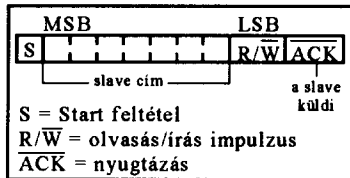
Kifejezés	Magyarítás
Adó (Transmitter)	Az az eszköz, amelyik adatot küld a BUSZ-ra
Vevő (Receiver)	Az az eszköz, amelyik adatokat fogad a BUSZ-ról
Master	Az az eszköz, amelyik kezdeményezi és lezárja az adatmozgásokat, s az órajeleket a BUSZ-ra küldi
Slave	A master által megcímzett eszköz
Multi-master	Egynél több master-t tartalmazó rendszer. A mestereknek úgy kell kezelni a BUSZ-t, hogy üzeneteik ne ütközzenek
Arbitráció, elrendelés (Arbitration)	Az az eljárás, mely biztosítja, hogy egy adott időben csak egy master vezérelje a BUSZ-t, az adatok ne sérüljenek.
Szinkronizáció	Az az eljárás, melynek eredményeképpen két vagy több eszköz órajelei szinkronizálódnak.

Az IIC protokoll szerint minden eszköznek címe van. A Master a kapcsolat érdekében kiküldi a Slave címét (amelyikkel kapcsolatba kíván kerülni). A címmel együtt kiküldött vezérlő bit jelzi, hogy írni vagy olvasni kíván a Master. A két vezeték, az SCL (óra) és az SDA (adat) a csipen nyitott kollektoros fokozatú, ezért külső felhúzó ellenállást kell alkalmazni.

Az adatátvitel indítása és leállítása tehát a Master hatásköre. Az átvitel szünetében (idle, inaktív szakasz) mindkét vezeték H szintű, a külső ellenállás felhúzó hatása miatt. Az átvitelt indító esemény a Start Condition (Start feltétel, Start bitnek is szokták írni), a lezáró esemény a Stop Condition (Stop

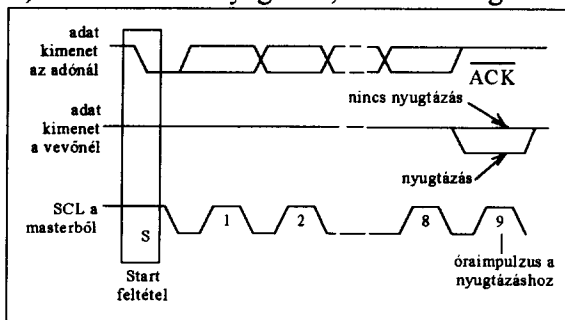
feltétel, Stop bit). A Start Condition: SCL H szintje mellett az SDA-n lefutó él alakul ki, a Stop Condition: SCL H szintnél az SDA-n felfutó él (3.33. ábra).

Az IIC rendszerben két címzési megoldás használható. Az egyszerűbb a 7 bites címzés, amit a 3.34. ábrán láthatunk (a címmel együtt érkezik az olvasás/írás bit is).



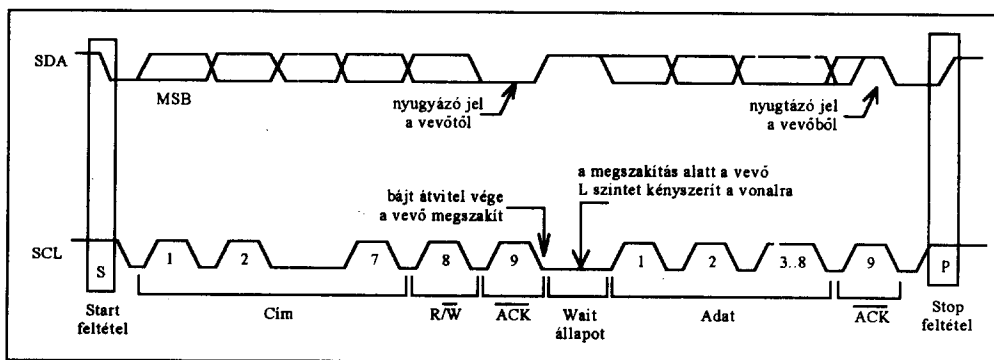
3.34. ábra 7 bites cím

Minden átvitel bájt-méretben történik, az egyetlen átviteli kapcsolatban (a Start és Stop feltételek között) mozgatott bájtok száma nincs korlátozva. A Slave vevő minden bájt végén egy LOW aktív szintű nyugtázó jelet generál: Acknowledge, $\overline{ACK}$ (3.36. ábra). Ha elmarad a nyugtázás, a Master megszakítja az adást, a folyamatot lezárja egy Stop feltétellel.



3.36. ábra Az $\overline{ACK}$ jel

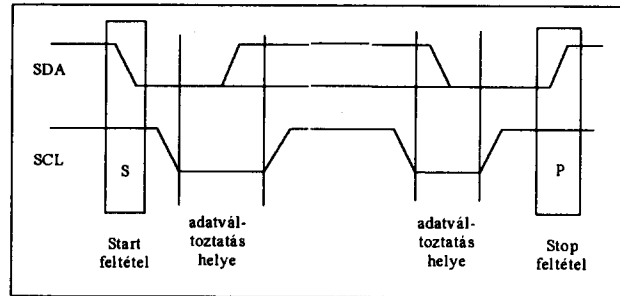
Az adattranszfer akkor folytatódik, ha a Slave felszabadítja az SCL vezetéket (3.37. ábra). Ez a várakoztatás bit szinten is használható.



3.37. ábra Adattranszfer várakoztatással

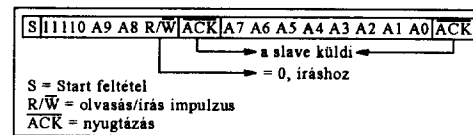
Az I²C BUSZ működési módjait néhány ábra segítségével célszerű összefoglalni. A 3.38. ábrán a Master adás, a 3.39. ábrán a Master vétel lefolyását lehet követni. A Master úgy is kiadhat egy Start feltételt (isméltelt, Repeated Start Condition), hogy előtte nem volt Stop feltétel. Erre akkor van szükség, ha pl. új Slave címet akar kiküldeni (3.40. ábra).

Az IIC protokoll lehetőséget ad arra is, hogy egyszerre egynél több Master legyen jelen egyazon BUSZ-on - ez a Multi-Master állapot. Ha 2 vagy több Master működik egyidejűleg, az egyértelmű működéshez az arbitrációt (hozzárendelés) és a szinkronizációt nyújtja az IIC BUSZ rendszer.



3.33. ábra Start/Stop feltétel

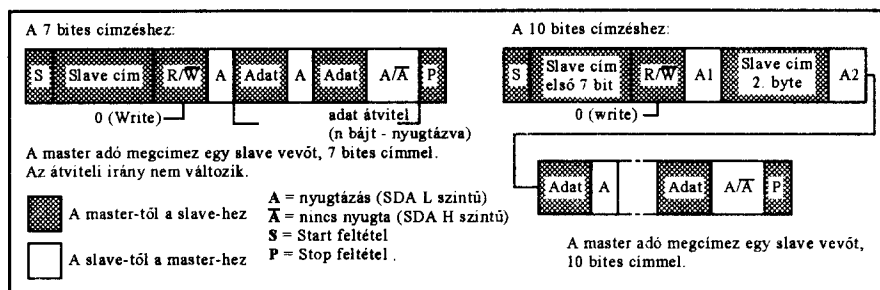
A 10 bites címzéshez két bájtot kell kiküldeni és fogadni, erre az első bájt első öt bitje tartalma utal (3.35. ábra).



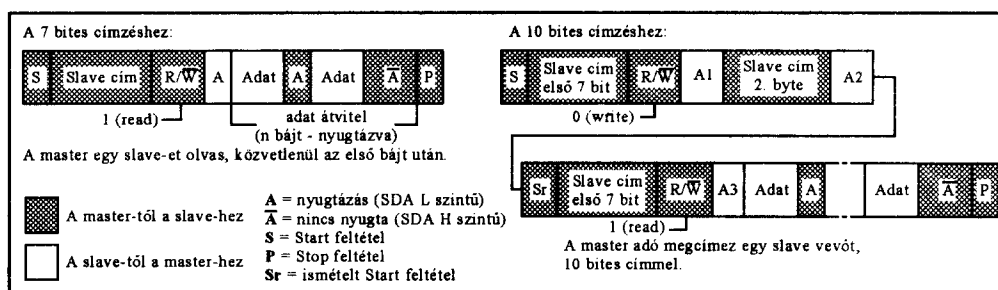
3.35. ábra 10 bites cím

Amikor a Master adatot fogad (Master Receiver), ez fog minden vett adat végén nyugtázó jelet küldeni a Slave számára - kivéve az utolsó bájt nál, amikor a nyugtázó jel elmaradása jelzi a Slave számára, hogy a Master befejezte az adatfogadást. A Slave ekkor felszabadítja az adatvonalat, a Master pedig létrehozza a Stop feltételt.

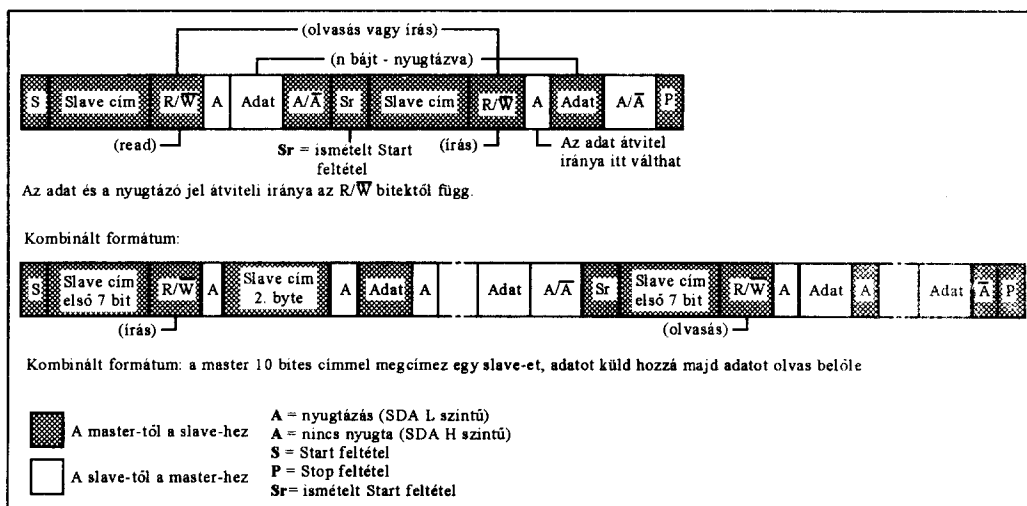
Előfordulhat, hogy az adó Slave-nek időre van szüksége a következő bájt kiküldése előtt. Ekkor a Slave az SCL vonalat LOW szintre kényszeríti s ott tartja, a Master ezt érzékeli és várakozni fog.



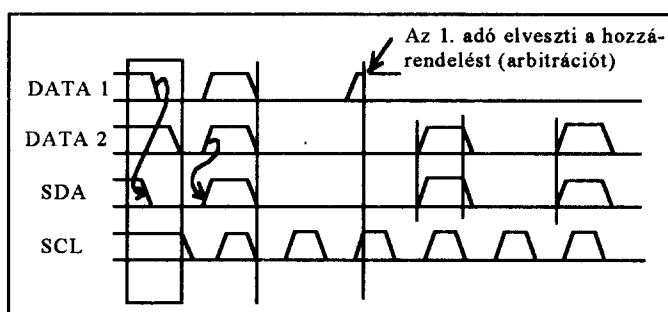
3.38. ábra Master adás



3.39. ábra Master vétel



3.40. ábra Ismételt Start feltétel

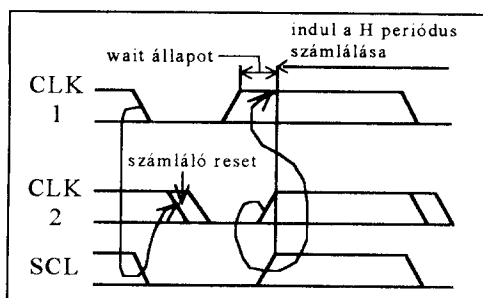


3.41. ábra Arbitráció

Az arbitrációs esemény alapja az SDA vonal, amikor az SCL magas szintű. Az a master, amelyik H szintet ad, amikor egy másik L szintet küld ki, elveszíti a hozzárendelést (3.41. ábra), kikapcsolja a kimenetét - átadja a BUSZ irányítást.

A Master, amelyik éppen elvesztette az arbitrációt, a bájttá végéig még kiküldi az órajeleket, majd átvált Slave vételi módra, hogy az új, arbitrációt elnyert Master meg tudja

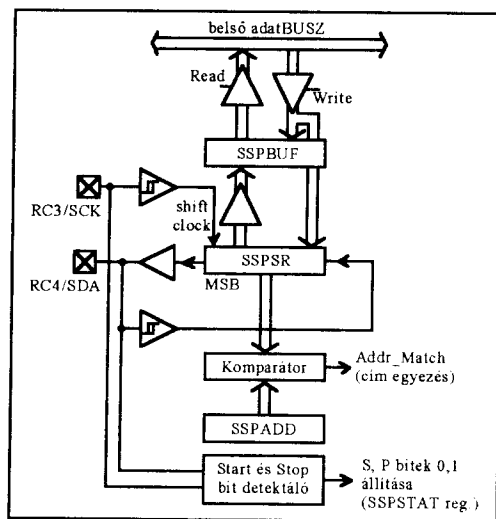
címezni. Az arbitrációs versenyt nem szabad kiváltani ismételt Start feltételkor, Stop feltétel és adat versenyként.



3.42. ábra Szinkronizáció

Az arbitrációt követően az órajelek szinkronizációja is bekövetkezik, amit az SCL vonal huzalozott ES jellege tesz lehetővé. Egy H-L átmenet az SCL vonalon minden kapcsolódó, órajelet kiküldő eszközben indítja az L időtartam számlálást. Addig nem vált H szintre az SCL, míg az összes egységben véget nem ér az L fázis. Amikor az SCL vonal H szintre vált, akkor kezd minden egység a H szint számlálását, amelyik már hamarabb H szintre állt volna, az várakozik idáig (3.42. ábra).

Természetesen itt csak néhány alapelvet tudunk bemutatni az I²C BUSZ rendszerrel kapcsolatban, a részletes leírás maga is egy teljes kötetet vesz igénybe.



3.43. ábra Az SSP IIC működése

Az SSP modul I²C működése minden Slave funkciót teljesen megvalósít. A Master funkciókat szoftverrel kell kialakítani, de az azokhoz szükséges minden hardver elemet megtaláljuk a modulban. Alkalmasság az SSP a standard (100 kbps) és a gyors (400 kbps) működésre is, képes a 8 bites címzésen kívül a 10 bites címzésre is. Az I²C BUSZ csatlakozó pontjai:

RC3/SCK/SCL (SCL, óra)
RC4/SDI/SDA (SDA, adat).

Az IC csatlakozó pontok be- illetve kimeneti jellegét precízen be kell állítani a TRISC<4:3> bitekkel. Az SSP modul IIC működését a 3.43. ábra mutatja be.

Az IIC üzemmódot kiszolgáló regiszterek:

SSP Control register (SSPCON)
SSP Status register (SSPSTAT)
Serial Receiver/Transmitter Buffer (SSPBUF)
SSP Shift register (SSPSR)
Address register (címregiszter, SSPADD).

A lehetséges IIC beállítások (SSPCON<3:0>):

- I²C Slave mód, (7 bites címmel)
- I²C Slave mód, (10 bites címmel)
- I²C Slave mód, (7 bites címmel), Master mód támogatással
- I²C Slave mód, (10 bites címmel), Master mód támogatással,
- I²C Master mód, a Slave: inaktív.

Miután valamelyik üzemmódot kijelöltük, az SSPEN bit 1-re állításával aktivizálható a modul (ekkor az SCL és az SDA pontok nyitott kollektoros jellegűekké válnak).

Az SSPSTAT regiszter tartalma utal az adattanszfer állapotára. Itt látható a Start vagy Stop feltétel detektálása, itt tudható meg, hogy egy vett bájt cím volt-e vagy adat, s a címzés 7 vagy 10 bites, a működés írás vagy olvasás. Az SSPSTAT regiszter csak olvasható.

Az SSPBUF fogadja a vett vagy adásra váró adatot, az SSPSR regiszter végzi el a soros/párhuzamos átalakítást. Vételkor az SSPSR és az SSPBUF kettős puffer rendszert alkotnak, a következő bájt vétele megkezdődhet, mielőtt a CPU az előzőt kiolvasná. Amikor egy teljes bájt vétele megtörténik, az átkerül az SSPSR-ből az SSPBUF-be, s ekkor vált 1-re az SSPIF bit. Ha az SSPBUF kiolvasása előtt egy újabb bájt vétele is befejeződik, ezt az SSPOV hibajelző bit 1-re váltása jelzi.

Az SSP Slave módjában az SCL és az SDA pontokat bemenetként kell konfigurálni (amikor a működés megkívánja, az adatirányt a CPU ideiglenesen felülbírája). Cím egyezéskor, adat vétele után a hardver automatikusan generálja a nyugtázó jelet. Két esetben marad el az $\overline{\text{ACK}}$ LOW impulzus kiküldése:

- ha a BF (puffer betöltve) jelzőbit már vétel közben is 1 értékű volt,

- ha az Overflow (SSPOV, felülírás) jelző bit vétel közben 1 értékű volt.

Ezekben az esetekben az SSPSR tartalma nem kerül át az SSPBUF-be, de az SSPIF bit mégis 1-re vált. A 3.24. táblázat foglalja össze azokat az állapotokat, melyek egy bájt vétele után a státusz biteknél kialakulhatnak.

3.24. táblázat Az IIC státusz bitek vétel után

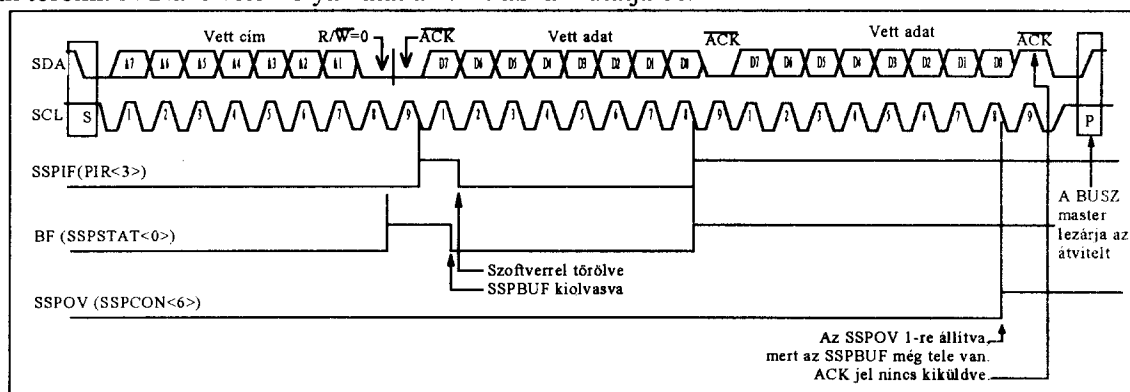
A státuszbitek beérkezésekor	egy adat		Generálódik-e ACK pulzus	Az SSPIF bit beáll 1-re SSP megszakítás
BF	SSPOV	SSPSR→SPBUF		
0	0	Igen	Igen	Igen
1	0	Nem	Nem	Igen
1	1	Nem	Nem	Igen

A sötétítéssel jelzett állapot esetén a felhasználói szoftver nem végezte el helyesen az SSPOV bit törlését.

Amikor az SSP modul IIC működését engedélyezzük, az várakozni kezd a Start feltételre. Azután 8 bitet beléptet a shift regiszterbe, minden bitből az SCL vonalon érkező órajel felfutó élénél vesz mintát. Az SSPSR tartalmát a CPU összeveti az SSPADD regiszter tartalmával (az SCL nyolcadik impulzusa lefutó élekor). Ha egyezik a cím, a BF és az SSPOV bitek értéke 0, a következő események mennek végbe:

- az SSPSR tartalma áttöltődik az SSPBUF-be
- a BF flag bit 1-re változik
- kilép az ACK impulzus
- az SSP megszakítás flag (SSPIF) a kilencedik SCL impulzus lefutó élénél 1-re vált.

Ha a címzésnél az R/w bit 0 és a cím egyezik, az SSPSTAT regiszterben lévő R/w bit is 0-ra vált (**Slave Reception, Slave vétel mód**). A vett cím áttöltődik az SSPBUF-be, a mikrovezérlő kiküldi az ACK jelet. A vétel során, a bájtok beérkezését jelző megszakítás kérés flag-et (SSPIF) szoftverrel lehet csak törölni. A Slave vétel folyamatát a 3.44. ábra mutatja be.



3.44. ábra IIC Slave vétel, 7 bites címzéssel

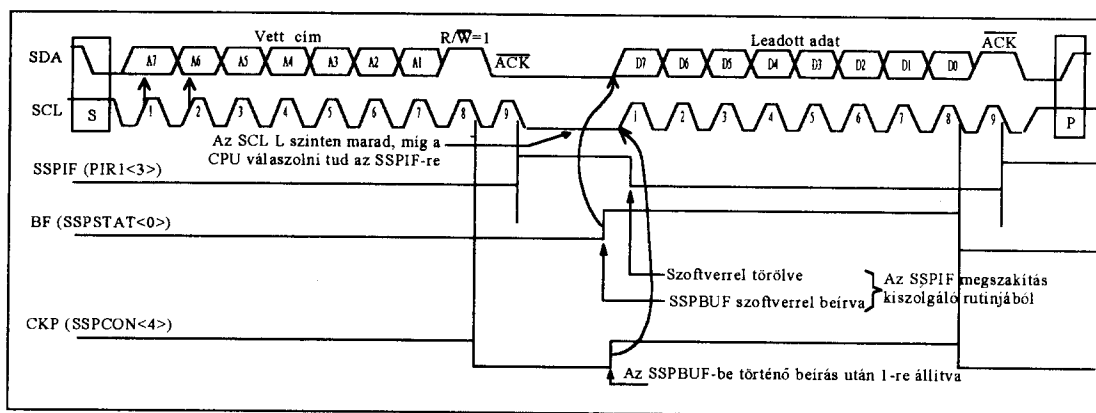
Ha a cím vételekor a cím egyezik és az R/w bit 1 értékű, az SSPSTAT regiszterben lévő R/w bit is 1-re vált (**Slave Transmission, Slave adás mód**). A vett cím átkerül az SSPBUF regiszterbe, a mikrovezérlő kiküldi az ACK impulzust és az SCL vonalat LOW szintre állítja (Master várakoztatás).

A kiküldendő adatot a CPU behelyezi az SSPBUF-be, onnan kerül majd az SSPSR léptető regiszterbe. Ezután engedélyezhető az SCL működése (CKP bit az SSPCON<4> regiszterben). Az órajel lefutó élénél a 8 adatbitet kilépteti az áramkör (3.45. ábra).

Minden leadott bájt után egy megszakítás kérés keletkezik (SSPIF), a kilencedik órajel impulzus lefutó élénél. A megszakítás kérés bitet csak szoftverrel lehet törölni.

Mint Slave adó, az ACK impulzust a Mastertől várja a mikrovezérlő. A nyugtázó impulzus fogadása érdekében a kilencedik SCL óraimpulzus felfutó élénél az SDA már felszabadul, s a kívülről oda küldött jelet vizsgálja a CPU. Ha ekkor az SDA HIGH szintű, tehát nem érkezett nyugtázó jel, az adattranszfert lezárja a mikrovezérlő és Stop feltételre vár.

Ha a nyugtázó jel beérkezett (SDA=L, azaz van ACK impulzus), új adat kerül az SSPBUF regiszterbe, onnan az SSPSR léptető regiszterbe és végbemegy a következő adatkivitel.



3.45. ábra IIC Slave adás, 7 bites címmel

Master módban szoftverrel kell megoldani az SDA, SCL pontok kezelését. A TRISC regiszter megfelelő bitjeivel kell kezelni a pontokat, kimenetként az állapotuk független a C Port regiszter tartalmától. Kimeneti 0-t úgy érünk el, ha a TRIS bit 0, s a TRIS bit 1 értéke eredményez kimeneti 1-t. A Master móddal párhuzamosan a Slave mód is aktivizálható - a két üzemmódot szoftverrel kell követni.

A 3.25. táblázat foglalja össze az IIC működés regiszter készletét.

3.25. táblázat Az IIC működés regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8B	INTCON	GIE	PEIE						
0C	PIR1		-	-	-	SSPIF			
8C	PIE1		-	-	-	SSPIE			
13	SSPBUF	Szinkron soros Port vevő puffer / adó regiszter							
93	SSPADD	Szinkron soros Port cím regiszter (IIC üzemmód)							
14	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
94	SSPSTAT	-	-	D/A	P	S	R/W	UA	BF

A sötétített bitek nincsenek felhasználva ebben az üzemmódban

Ellenőrző kérdések:

1. Milyen üzemmódjai vannak a szinkron soros Port-nak?
2. Hány vezetékből áll az SPI BUSZ? Melyek ezek?
3. Milyen az SPI üzemmód blokkvázlata?
4. Az SPI működéskor van-e szerepe a TRIS regiszternek?
5. Mi a különbség az SPI Master és Slave működés között?
6. Milyen I²C üzemmódokra alkalmas a 16C74?
7. Mi a különbség az I²C Master és Slave működés között?
8. Hogyan alakul az adattranszfer indítása és lezárása az I²C BUSZ-on?
9. Mi a címzés szerepe az I²C rendszerben?
10. Mi a 7 bites és mi a 10 bites címzés lefolyása?
11. Mi a nyugtázó jel szerepe, mikor melyik egység küldi ki?

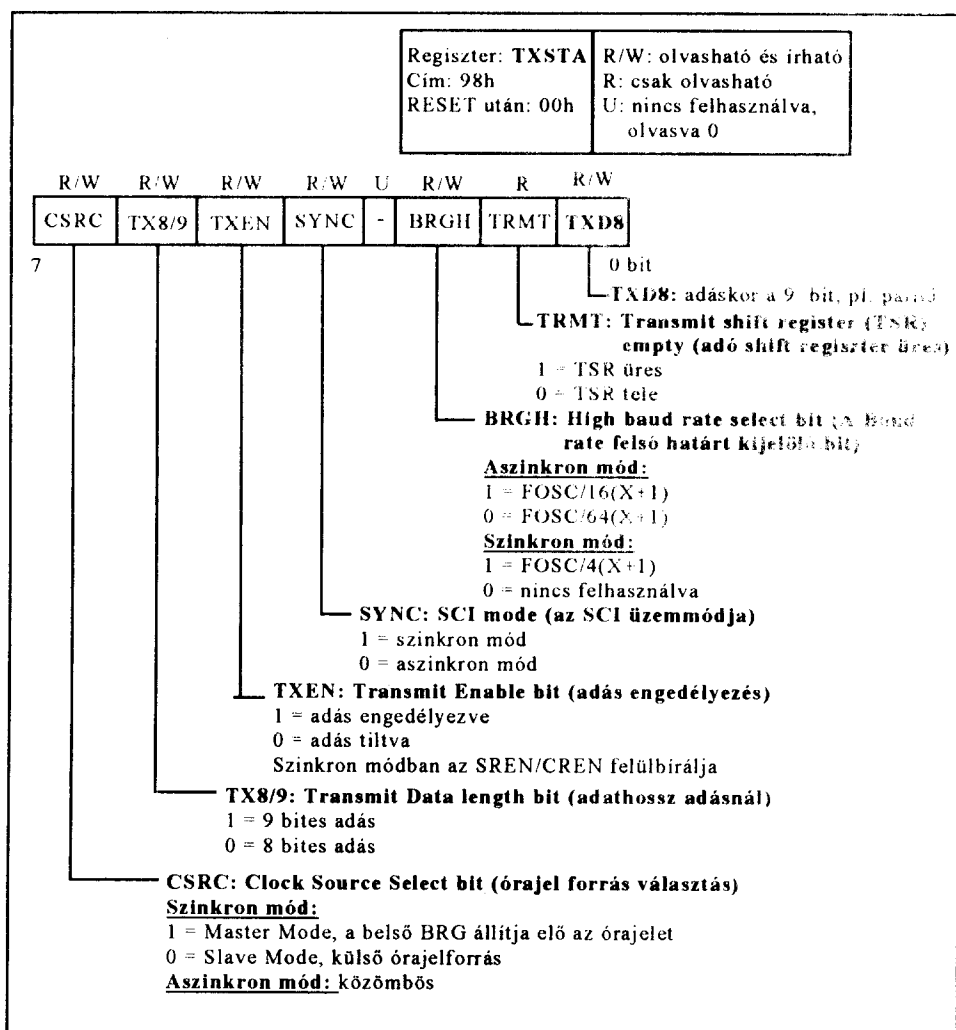
3.1.7. A soros kommunikációs interfész (SCI) modul

A soros kommunikációs interfész modul (Serial Communication Interface, SCI) egy soros Port egység, teljes duplex aszinkron vagy félduplex szinkron soros átvitelre alkalmas. a konfigurálási lehetőségek:

- aszinkron (teljes duplex)
- szinkron - Master (félduplex)
- szinkron - Slave (félduplex).

(Az SCI szinkron soros átvitel nem a szinkronszavakkal szinkronizált, egy vezetékes, órajelet nélküli soros átvitel jelenti, hanem az SCI-hez, IIC-hez hasonló, órajelet is átküldő soros kapcsolatot.)

Az adás működés állapot- és vezérlő regisztere a TXSTA (Transmission Status and Control Register, 3.46. ábra). A vétel üzemmód állapotbitjei és vezérlő bitjei az RCSTA regiszterben találhatók (Receive Status and Control Register, 3.47. ábra).



3.46. ábra A TXSTA regiszter (16C74)

Az SCI működést az RCSTA<7> bitjén lévő SPEN vezérlő bit 1-re állításával lehet engedélyezni. A helyes működéshez az szükséges, hogy az adóban és a vevőben megegyező értékű legyen az átvevő órajel sebessége, a Baud Rate. A 16C74-ben Baud Rate generátor (BRG) állítja elő a kívánt értékű órajel frekvenciát. Egy szabadonfutó 8 bites számlálóra épül ez a működés, azt vezérli az SPBRG regiszter, de a Baud Rate értékét még a TXSTA<2> bit is befolyásolja. A 3.26. táblázat mutatja be a számítási képleteket, attól függően, hogy milyen értékű a SYNC bit (TXSTA<4>) és a BRGH bit (TXSTA<2>).

3.26. táblázat Baud Rate értékek kiszámítása

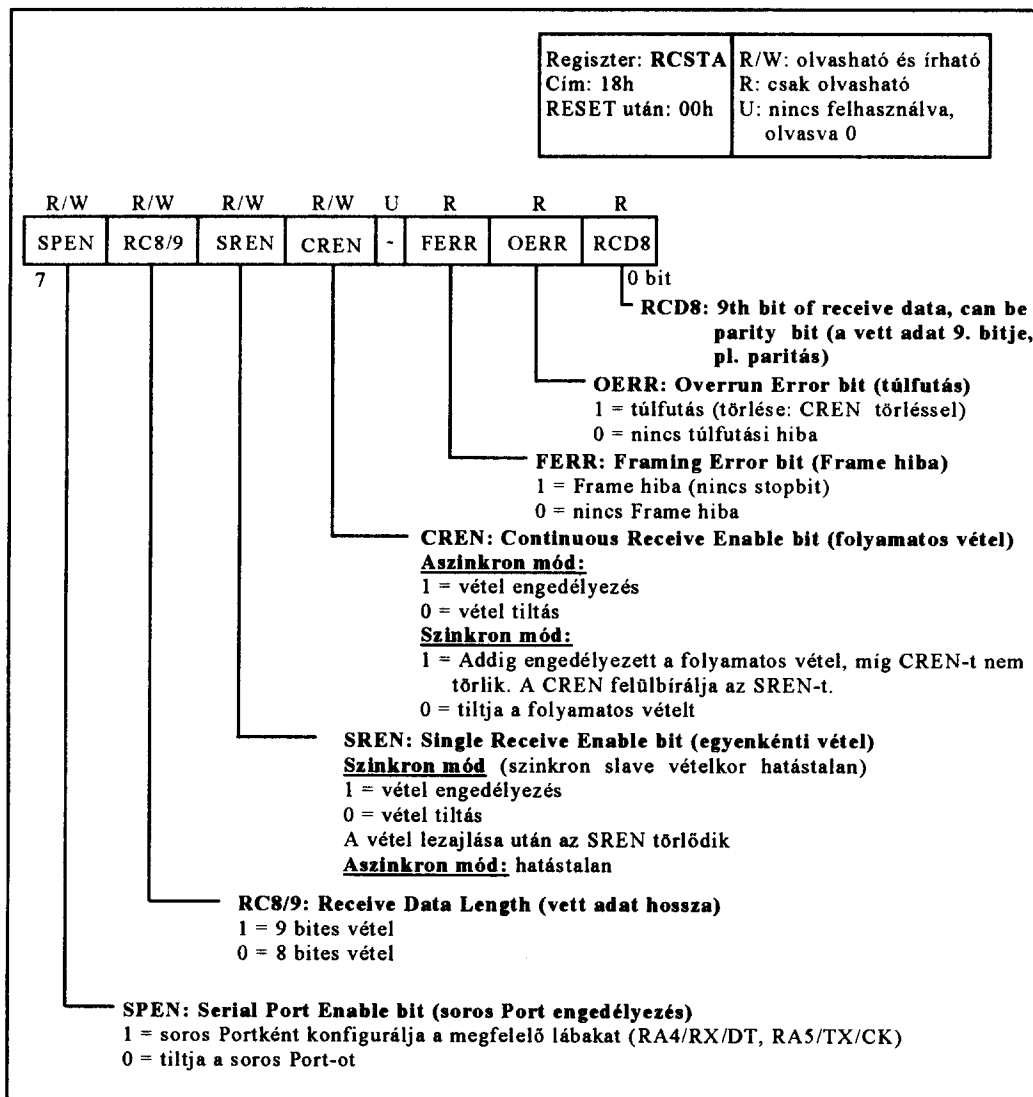
SYNC	BRGH=0	BRGH=1
0	Baud rate = Fosc/64(X+1)	Baud Rate = Fosc/16(X+1)
1	Baud rate = Fosc/4(X+1)	-

X az SPBRG tartalma (0 ... 255)

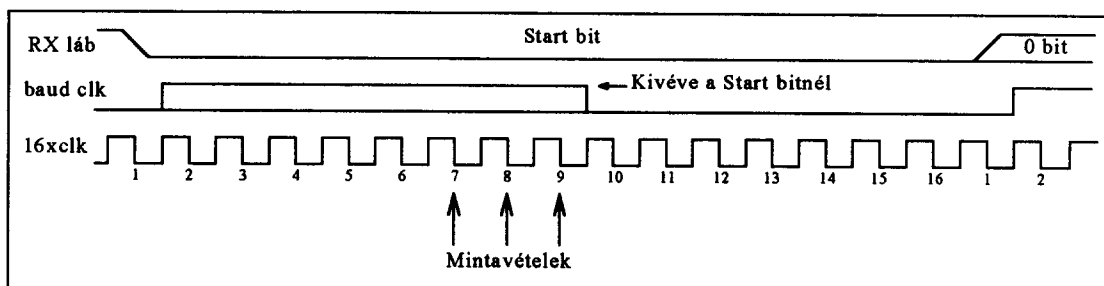
A szabványos Baud Rate értékeket 100 %-os pontossággal nem lehet beállítani, de a szükséges pontossággal igen. Ha pl. 9600 baud a kívánt érték és FOSC = 16 MHz, a BRGH=0, a SYNC=0, az osztási arány X=25-re adódik (az osztási arány természetesen csak egész szám lehet); ekkor a Baud Rate pontos értéke 9615 lesz, a hiba értéke: 0,16%.

Az RX csatlakozó ponton érkező jelet a mikrovezérlő minden bitérték megállapításához a bitidő beosztásában, háromszor mintavételezi. Ha BRGH=0 (azaz a Baud Rate alacsony értékű), a mintavételezés a 7., 8. és a 9. órajel lefutó élénél történik meg (a 16xclk órajel éleinél); ez látható a 3.48. ábrán.

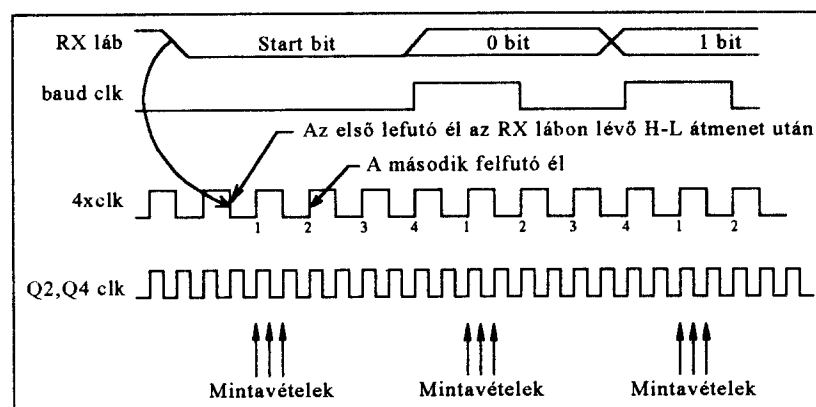
Ha a BRGH=1 (nagyobb Baud Rate értékeknél), ismét három egymást követő órajelnél történik a mintavételezés (Q2, Q4 clk), amikor az RX LOW szintre vált, a 4xclk ezután első lefutó élét követő második felfutó élét éppen megelőzve (3.49. ábra).



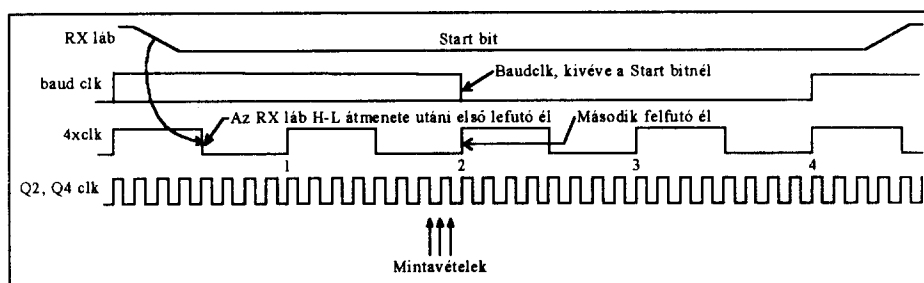
3.47. ábra Az RCSTA regiszter (16C74)



3.48. ábra RX mintavételezés (BRGH=0)

3.49. ábra RX mintavétel ($BRGH=1$)

A nagyobb Baud Rate esetén működő mintavételezési megoldást részletesebben mutatja be a **3.50. ábra**.

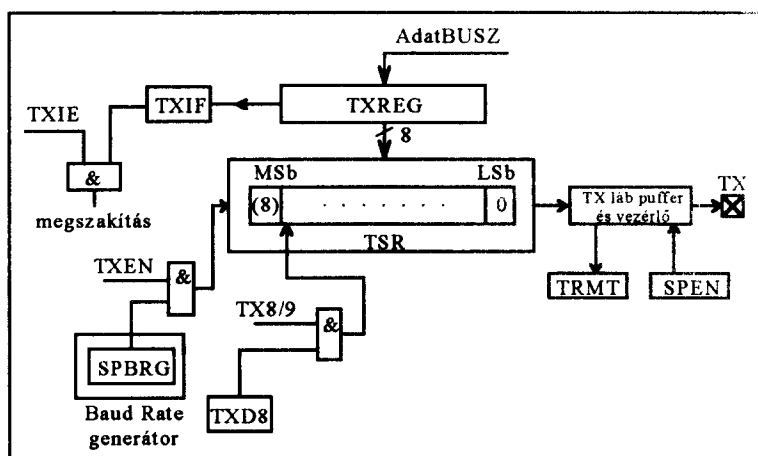


3.50. ábra RX mintavétel ($BRGH=1$), nagyítva

3.1.7.1. Az SCI aszinkron működése

Az **SCI aszinkron mód** (SYNC=0 a TXSTA<4> biten) a szabványos, 0-ra vissza nem térő (NRZ) formátumot használja (egy START bit, 8 vagy 9 adatbit, egy STOP bit). A szokásos adathosszúság 8 bit. Az adó és a vevő funkcionálisan függetlenek egymástól, de közös az adathosszúságuk és a Baud Rate értékük. Paritásbitet a hardver nem állít elő, de szoftverrel, kilencedik bitként megoldható a kezelése. SLEEP módban az aszinkron kommunikáció szünetel..

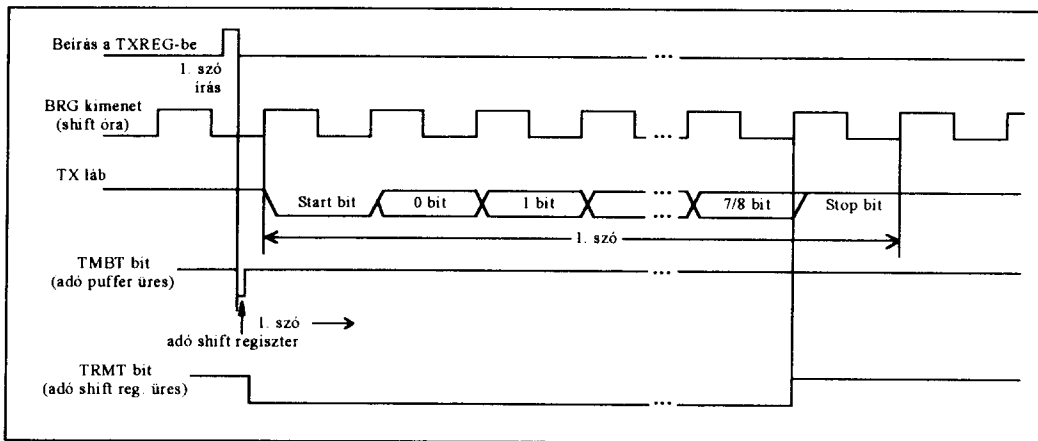
Az SCI aszinkron adó blokkvázlata a 3.51. ábrán látható. Az adó központi eleme a TSR léptető regiszter, mely nem eleme a speciális funkciójú regiszter mezőnek. A párhuzamos adatot az írható/olvasható TXREG fogadja be. Amikor az előző adat, a végén a STOP bittel kilépett a léptető regiszterből, a TXREG tartalma áttöltődik a TSR-be. ekkor a TXREG ürességét megszakítás bit is jelzi (TXIF, a PIR1-ben). A megszakítás bitet szoftverrel kell törölni. A TRMT bit (a



3.51. ábra Az SCI aszinkron adó

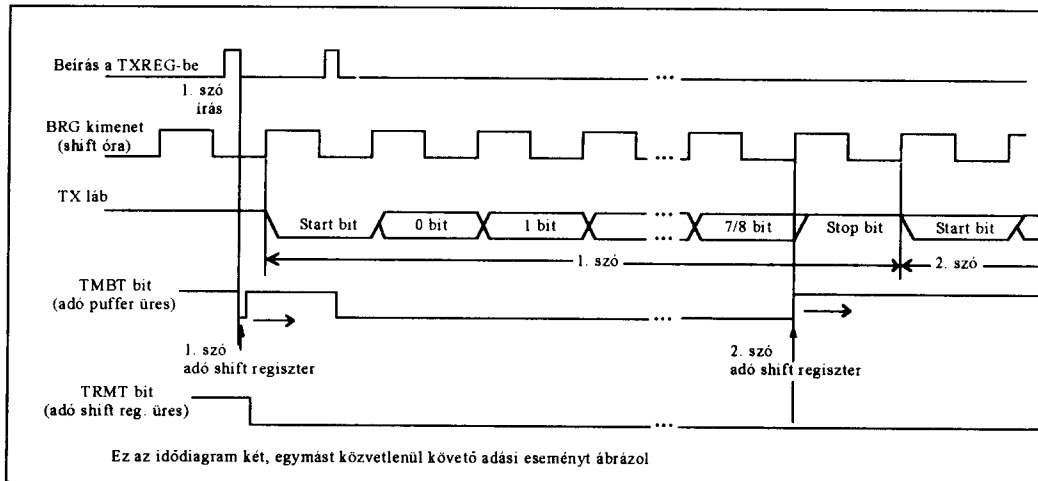
TXSTA-ban) az STA állapotát jelzi, ha üres a léptető regiszter, a jelzőbit 1-re vált. Az adást a TXEN bit engedélyezi (TXSTA regiszter). Az adás feltétele még, hogy a TXREG legyen betöltve és a BRG küldje az órajeleket. Az aszinkron adás jelfolyamatát a **3.52. ábrán** láthatjuk.

Ha adás közben töröljük a TXEN bitet, az adás azonnal félbeszakad, a TX IC láb nagyimpedanciás állapotba kerül (lebeg).



3.52. ábra Aszinkron adás

A soros adás megindítható úgy, ha az adatot betöltjük a TXREG-be, majd a TXEN bitet 1-re váltjuk. Az adat áttöltődik a TSR s a helyére máris a második adatot lehet beírni a TXREG-be. A két adat így egymás nyomában küldhető ki (Back-To-Back átvitel, 3.53. ábra)



3.53. ábra Két, egymást követő bájt-adás

A 9 bites átvitelt a TX8/9 bit (TXSTA<6>) 1-re állításával lehet elérni, a 9. bitet a TXD8 bitbe (TXSTA<0>) kell behelyezni, mégpedig azelőtt, hogy a megfelelő 8 bites rész a TXREG-be kerül. Az aszinkron adás teljes előkészítése tehát az alábbi lépésekből áll:

- a megfelelő Baud Rate inicializálása (BRGH bit, SPBRG regiszter)
- aszinkron adás engedélyezése (SYNC=0, SPEN=1)
- ellenőrizni: CREN=SREN=0
- ha megszakítással dolgozunk, TXIE=1
- 9 bites adáshoz: TX8/9=1
- adás engedélyezése: TXEN=1
- 9 bites esetben a 9. bit beírása a TXD8-ba
- adat betöltése a TXREG-be (az adás azonnal indul).

Az aszinkron adás regisztereit a 3.27. táblázat foglalja össze.

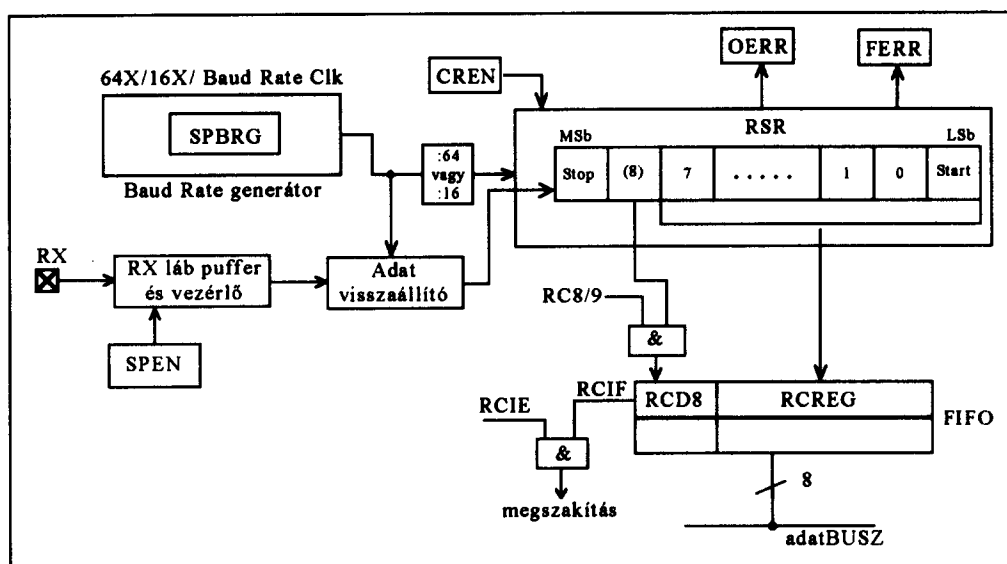
3.27. táblázat Az aszinkron adás regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
98h	TXSTA				SYNC	-	BRGH		
99h	SPBRG	Baud rate regiszter							

A sötétített biteket ez a beállítás nem használja

Az aszinkron vevő blokkvázlata a 3.54. ábrán található. Az adat az RX IC lábán érkezik, és az adat regeneráló blokk fogadja (Data Recovery). Az adat regeneráló egység egy nagysebességű léptető regiszter, az aktuális Baud Rate 16x-osával működik.

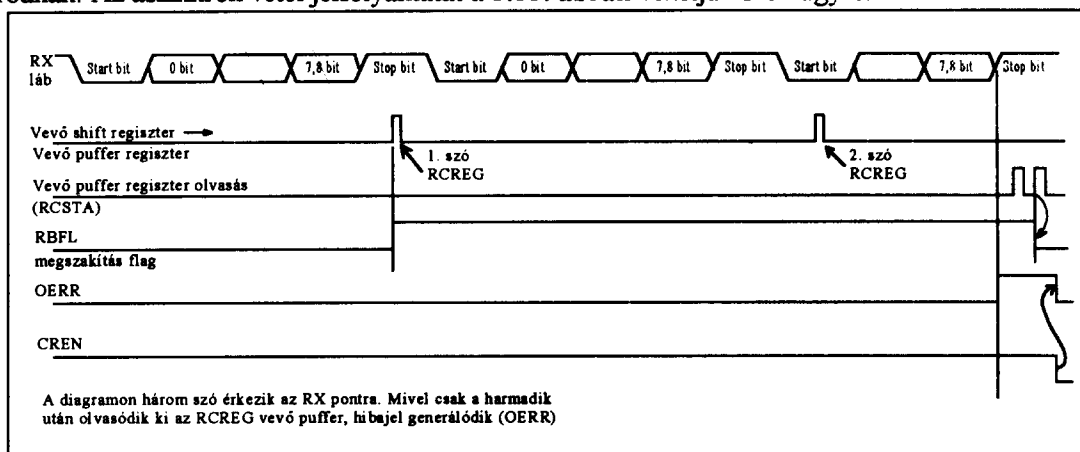
Ha az aszinkron mód már ki van jelölve, a vétel engedélyezéséhez a CREN bit (RCSTA<4>) 1-re állítása szükséges.



3.54. ábra Az SCI aszinkron adó

A vevő modul központi eleme a vevő shift regiszter (RSR). A STOP bit észlelése után az RSR-ből a vett adat átíródik az RCREG-be (ha az üres volt), s az RCIF megszakítás flag is 1-re vált. Ez a flag bit csak olvasható s automatikusan törlődik, amikor a CPU kiolvassa az adatot az RCREG-ből. Az RCREG kétszeresen puffertelt regiszter, pontosabban egy kettős mélységű FIFO. Így két bájtot lehet behelyezni s a harmadik shiftelés is megkezdődhet, adatvesztés nélkül. Ha a harmadik bájt utáni STOP bit is beérkezik és a CPU még nem olvasott ki adatot, akkor lép fel hibajelzés (OERR=1 az RCSTA<1> biten), az RSR-ben most kialakult adat pedig elvész. (Az OERR bitet szoftverrel kell törölni).

A formátum hiba bit (FERR az RCSTA<2> biten) akkor vált 1-re, ha STOP bit helyett LOW szintet érzékel a mikrovezérlő. A FERR és a 9. bit ugyanúgy tárolódik, mint a vett adat. Az RCREG olvasása előtt kell ezeket megvizsgálni, mert az RCREG olvasásakor az új adattal együtt ezek a bitek is átíródnak. Az aszinkron vétel jelfolyamatát a **3.55. ábrán** vehetjük szemügyre.



3.55. ábra Aszinkron vétel

Az aszinkron vétel előkészítésének lépései tehát a következők:

- a Baud Rate érték inicializálása (BRGH bit, SPBRG regiszter)
- az aszinkron soros Port engedélyezése (SYNC=0, SPEN=1)
- megszakításos működéshez RXIE=1
- 9 bites átvitelhez: RX8/9: 1
- vétel engedélyezése: CREN=1

A vétel lezajlása után az RCIF 1-re vált és megszakítást kezdeményez. Először az RCSTA regisztrert kell kiolvasni, onnan kapható meg a 9. adatbit és az esetleges hibajelzés. Ezután kell kiolvasni az RCREG tartalmát (a 8 bites adatrészt). Ha bármilyen hibajelzés fellépett, a hibajelző bitet törölni kell.

Az aszinkron vételkor működő regiszter csoportot a 3.28. táblázat foglalja össze.

3.28. táblázat Az aszinkron vétel regiszterei

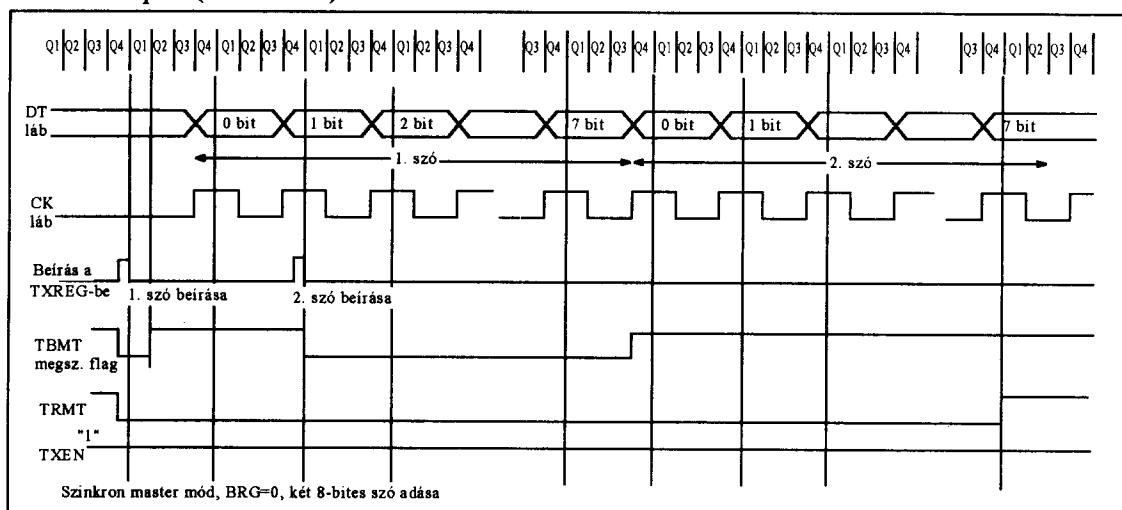
Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Ch	PIR1			RCIF					
18h	RCSTA	SPEN	RC8/9	SREN	CREN	-	FERR	OERR	RCD8
1Ah	RCREG	RX7	RX6	RX5	RX4	RX3	RX2	RX1	RX0
8Ch	PIE1			RCIE					
98h	TXSTA				SYNC	-	BRGH		
99h	SPBRG	Baud rate regiszter							

A sötétített bitek ebben a beállításban nincsenek felhasználva

3.1.7.2. Az SCI szinkron működése

Az **SCI szinkron Master módban** félduplex adattranszfer zajlik, azaz egyidejűleg nincs adás és vétel - egyik a másikat letiltja. A szinkron módot a SYNC bit 1-re állítása váltja ki. A SPEN=1 hatására az RC6 és az RC7 IC láb felveszi a CK (óra) és a DT (adat) vonal szerepét. A Master mód jellemzője, hogy ez az IC küldi ki az órajeleket. A Master módba akkor lépünk be, amikor a CSRC bitet 1-re állítjuk.

A **szinkron Master adáskor** az adó modul hasonlóan épül fel, mint aszinkron esetben (3.48. ábra). Az adás működés lefolyása is hasonló, de a shift regiszterből az adat első bitje most a CK órajel első felfutó élénél lép ki (3.56. ábra).

**3.56. ábra** Szinkron adás (SCI, Master mód)

Ha adás közben a TXEN bitet töröljük, vagy a CREN, SREN valamelyike 1-re vált, az adás félbeszakad. 9 bites adatokat ugyanúgy lehet kezelni, mint az aszinkron esetben. A szinkron Master mód beállítási lépései adáshoz a következők:

- a Baud Rate érték beállítása
- szinkron Master soros Port konfigurálása (SYNC=1, SPEN=1, CSRC=1)
- igény esetén a megszakítás engedélyezése (TXIE=1)
- 9 bites működéshez: TX8/9=1
- adás engedélyezése (TXEN=1)
- 9 bites adás esetén először a 9. bitet kell beírni a TXD8-ba
- a TXREG betöltésekor az adás azonnal megkezdődik.

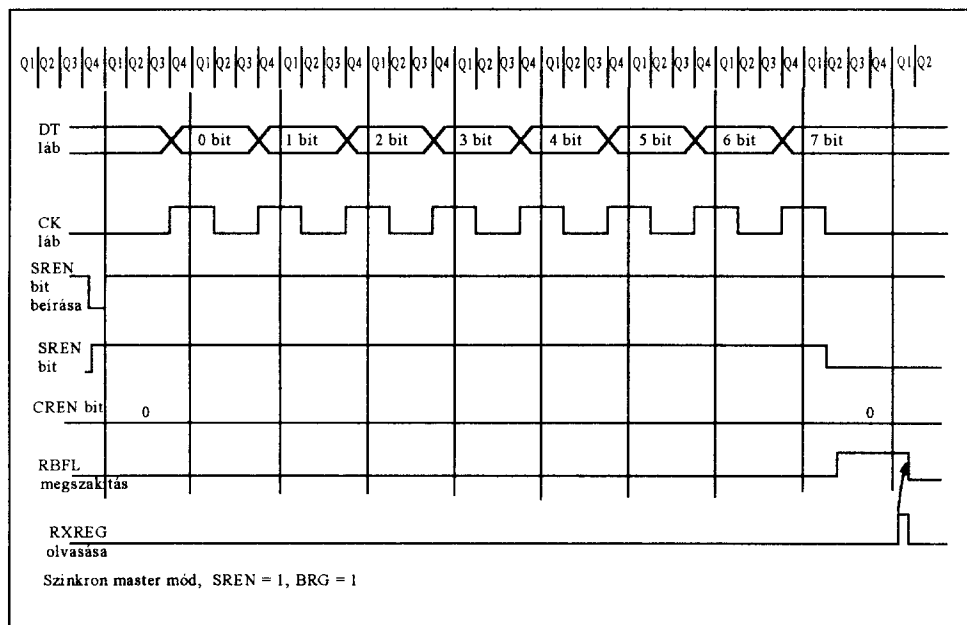
A szinkron Master adás regisztereit a 3.29. táblázat tartalmazza.

3.29. táblázat a szinkron Master adás regiszterei (SCI)

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Ch	PIR1				TXIF				
18h	RCSTA	SPEN	RC8/9	SREN	CREN	-			
19h	TXREG	TX7	TX6	TX5	TX4	TX3	TX2	TX1	TX0
8Ch	PIE1				TXIE				
98h	TXSTA	CSRC	TX8/9	TXEN	SYNC	-		TRMT	TXD8
99h	SPBRG	Baud rate regiszter							

A sötétített bitek ebben a beállításban nincsenek felhasználva

Az **SCI szinkron Master vétel** a szinkron mód kiválasztása után úgy érhető el, ha az SREN vagy a CREN bitet 1-re állítjuk. A DT IC lábon fellépő jel mintavételezése először az órajel következő lefutó élénél történik meg. Ha SREN=1, csak egyetlen szó vétele történik meg, a CREN=1 folyamatos vételt jelent, míg ezt a bitet nem töröljük (ha mindkét bit 1, a CREN jut érvényre). Az utolsó bit vétele után az adat átiródik az RCREG-be. A vétel végén az RCIF megszakítás flag 1-re vált (automatikusan törlődik, ha a CPU kiolvassa az RCREG regisztert). Az RCREG itt is kettős mélységű FIFO, így két teljes szót képes tárolni átmenetileg. Ha a harmadik szó vétele is befejeződik miközben a FIFO-ban már két korábbi adat van - akkor lép fel az OERR hibajelzés. Az OERR és a 9. bit kezelése megegyezik az aszinkron esettel (3.57. ábra).



3.57. ábra Szinkron vétel (SCI, Master mód)

A szinkron Master vétel előkészítésének lépései:

- a Baud Rate beállítása
- a szinkron Master soros Port engedélyezése (SYNC=1, SPEN=1, SCRC=1)
- megszakítás kezeléshez RXIE=1
- 9 bites működéshez: RX8/9=1
- egyszavas működéshez SREN=1, folyamatos vételhez CREN=1

A vétel megtörténtekor az RCIF 1 értékre vált, az RCSTA regiszterből kell kiolvasni a 9. bitet és az esetleges hibajelzést, majd kiolvasható az RCREG tartalma is. Ha hiba lépett fel, a CREN törlésével a hibajelzés is törlődik. A szinkron Master vétel regiszterkészletét a 3.30. táblázatban találjuk meg.

3. 30. táblázat A szinkron Master vétel regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Ch	PIR1			RCIF					
18h	RCSTA	SPEN	RC8/9	SREN	CREN	-		OERR	RCD8
1Ah	RCREG	RX7	RX6	RX5	RX4	RX3	RX2	RX1	RX0
8Ch	PIE1			RCIE					
98h	TXSTA	CSRC			SYNC	-			
99h	SPBRG	Baud rate regiszter							

A sötétített bitek ebben a beállításban nincsenek felhasználva

Az **SCI szinkron Slave üzemmódja** annyiban tér el a szinkron Master módtól, hogy az órajelet most kívülről kapja a mikrovezérlő. Ezt az üzemmódot SLEEP állapotban is képes teljesíteni az áramkör. A Slave üzemmódba a CSRC bit törlésével lehet belépni.

Az adáskor ugyanúgy működik az áramkör, mint Master módban, kivéve, hogy külső órajel végzi a léptetést. A TXREG-be SLEEP előtt két szót lehet beírni, majd kiváltható a SLEEP állapot. Az egyik szó azonnal átkerül a TSR-be és kilép az IC-ből. A második szó addig a TXREG-ben marad, s a TXIF nem vált 1-re. Amint az első szó kilépett, a második is átkerül a TSR-be, a TXIF flag ekkor válik 1 értékűvé. Ha a TXIE engedélyezett, a SLEEP-ből a megszakítás felébreszti a CPU-t.

A szinkron Slave adás beállítási feladatai:

- a szinkron soros Port konfigurálása (SYNC=1, SPEN=1, CSRC=0)
- CREN=0
- megszakításos működéshez TXIE=1
- 9 bites működéshez: TX8/9=1
- az adás engedélyezése: TXEN=1
- 9 bites esetben a 9. bit betöltése a TXD8-ba
- amikor az adatot betöltjük a TXREG-be, az adás megindul.

A szinkron Slave soros adás regisztereit a 3.31. táblázat tartalmazza.

3.31. táblázat A szinkron Slave adás regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Ch	PIR1	TXPIF	ADIF	RCIF	TXIF	TXPFE	CCP1IF	TMR2IF	TMR1IF
18h	RCSTA	SPEN	RC8/9	SREN	CREN	-	PEEP	OERR	RCD8
19h	TXREG	TX7	TX6	TX5	TX4	TX3	TX2	TX1	TX0
8Ch	PIE1	TXPFE	ADIE	RCIE	TXIE	TXPFE	CCP1IE	TMR2IE	TMR1IE
98h	TXSTA	CSRC	TX8/9	TXEN	SYNC	-	BRGH	TRMT	TXD8
99h	SPBRG	Baud rate regiszter							

A sötétített bitek ebben a beállításban nincsenek felhasználva

A szinkron Slave soros vétel áramköri elrendezése, működése alapvetően megegyezik a Master adással, az a különbség, hogy most az órajel kívülről érkezik. A SREN bit most hatástalan. SLEEP állapotban is megtörténik a vétel, ha a SLEEP előtt engedélyezzük. A vétel végén az adat betöltődik az RCREG-be, s ekkor - ha RCIE=1 - megszakítás generálódik s a CPU felébred a SLEEP-ből.

A szinkron Slave beállítási tennivalói:

- a szinkron Slave soros Port konfigurálása (SYNC=1, SPEN=1, CSRC=0)
- megszakításos működéshez: RCIE=1
- 9 bites működéshez: RX8/9=1
- a vétel engedélyezése: CREN=1.

Ha a vétel befejeződik, az RCIF 1-re vált, megszakítás keletkezik (ha engedélyeztük). Az RCSTA olvasásával megkapható a 9. bit és az esetleges hibajelzés, majd az RCREG-ből olvassuk ki a 8 bites adatot. A hibajelzés a CREN törlésével törölhető.

Az aszinkron Slave soros vétel regisztereit a 3.32. táblázat foglalja össze.

3.32. táblázat A szinkron Slave vétel regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0Ch	PIR1	TXPIF	ADIF	RCIF	TXIF	TXPFE	CCP1IF	TMR2IF	TMR1IF
18h	RCSTA	SPEN	RC8/9	SREN	CREN	-	PEEP	OERR	RCD8
1Ah	RCREG	RX7	RX6	RX5	RX4	RX3	RX2	RX1	RX0
8Ch	PIE1	TXPFE	ADIE	RCIE	TXIE	TXPFE	CCP1IE	TMR2IE	TMR1IE
98h	TXSTA	CSRC	TX8/9	TXEN	SYNC	-	BRGH	TRMT	TXD8
99h	SPBRG	Baud rate regiszter							

A sötétített bitek ebben a beállításban nincsenek felhasználva

Ellenőrző kérdések:

1. Milyen üzemmódokra képes az SCI?
2. Az SCI beállításoknál van-e szerepe a megfelelő TRIS biteknek?
3. Milyen az aszinkron soros jel adatformátuma?
4. Hogyan állítható be a Baud Rate?
5. Hogyan kezelhető a paritásbit az SCI-ben?
6. Mi a formátum hiba?
7. Hány bites adatokat kezel az SCI?
8. Mi a kétszeresen puffertelt regiszterek (FIFO) szerepe? Hol található ilyen az SCI-ben?
9. SLEEP állapotban melyik üzemmódok működhetnek?

3.1.8. Az analóg/digitál konverziós modul

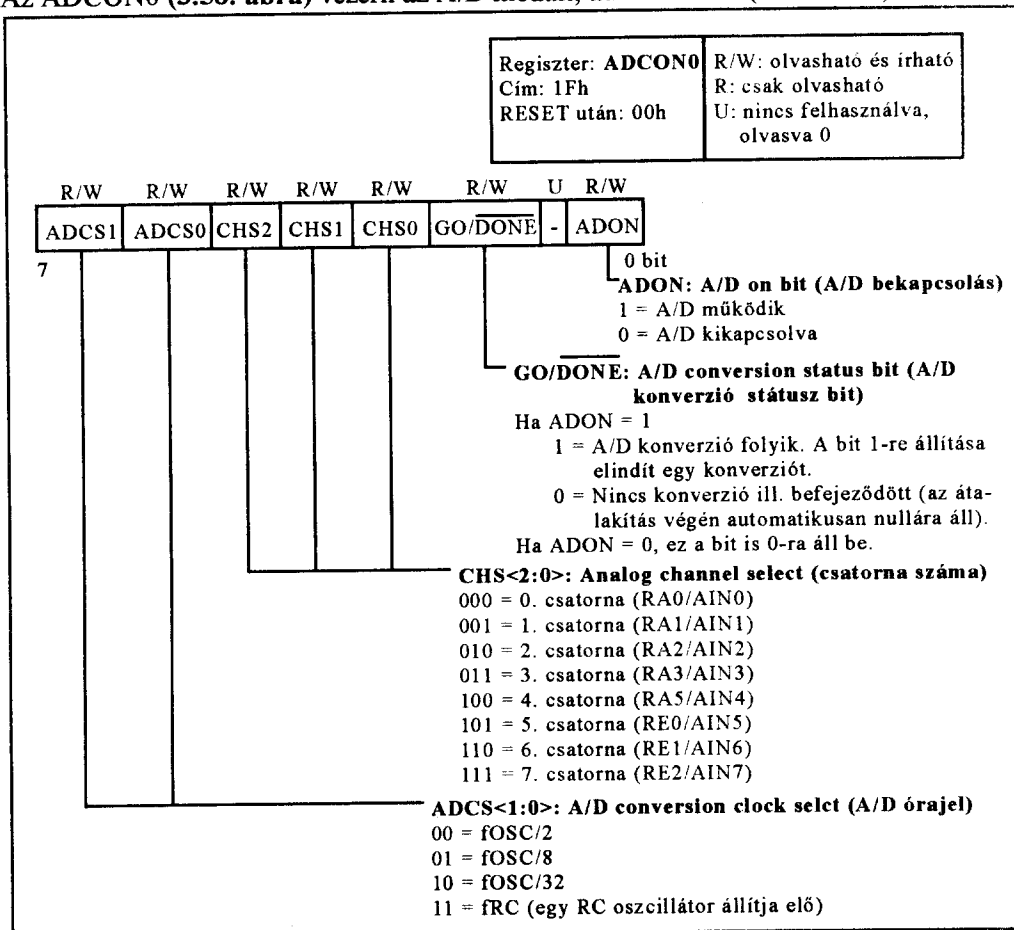
Az A/D konverter 8 bites digitális eredményt szolgáltat. A 16C74-ben 8 analóg bemenet kezelhető, ezeket egy analóg multiplexer vezeti a mintavevő/tartó (MvT) áramkörre. Az MvT kimenete csatlakozik az A/D átalakítóra; amelyik fokozatos megközelítéssel (successive approximation) működik. Az analóg

referencia feszültségforrás szoftverrel választható, lehet a csip pozitív tápfeszültsége (VDD), vagy az RA3/AN3/VREF IC lábón át kívülről bevezetett érték. SLEEP állapotban az A/D konverzió működhet.

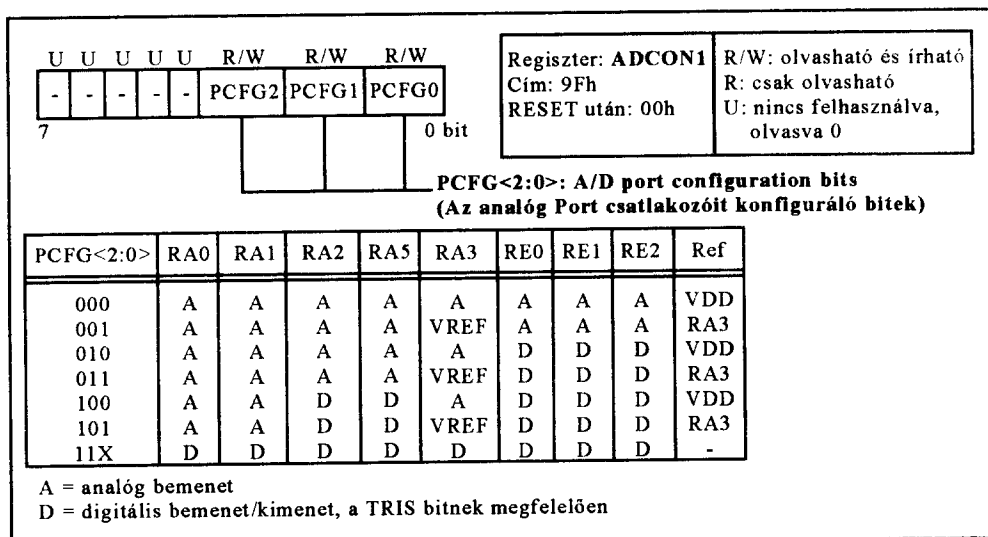
Az A/D modult kiszolgáló regiszterek:

- A/D eredmény regiszter (ADRES)
- A/D 0. vezérlő regiszter (ADCON0)
- A/D 1. vezérlő regiszter (ADCON1)

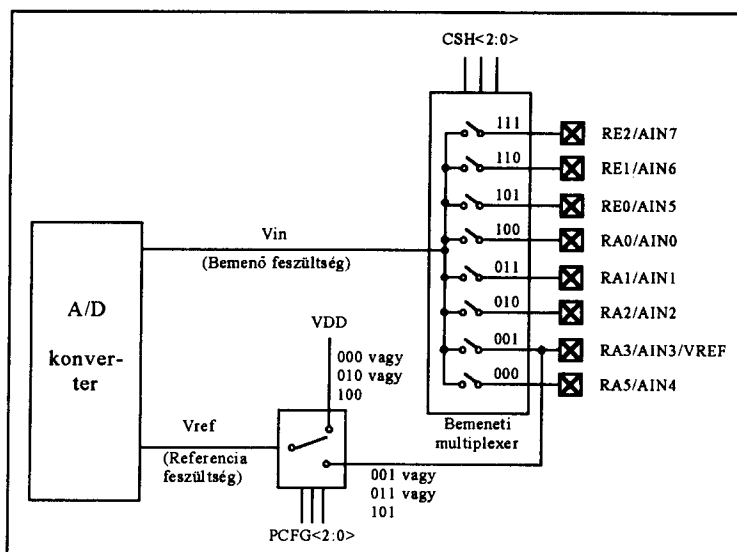
Az ADCON0 (3.58. ábra) vezérli az A/D modult, az ADCON1 (3.59. ábra) az IC lábakat.



3.58. ábra Az ADCON0 regiszter (16C74)



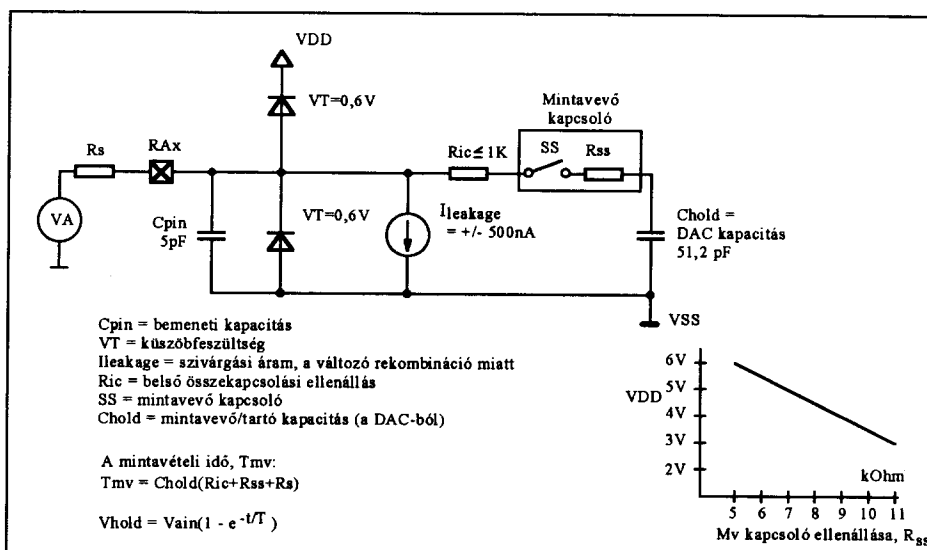
3.59. ábra Az ADCON1 regiszter (16C74)



3.60. ábra Az A/D modul felépítése (16C74)

Az A/D konverzió végrehajtásának lépései a következők:

- az A/D modul konfigurálása
 - = az analóg bemenetek, referencia feszültség beállítása (ADCON1<2:0>)
 - = az A/D bemeneti csatorna kiválasztása (ADCON0<5:3>)
 - = az A/D órajel, T_{ad} kiválasztása (ADCON0<7:6>)
 - = az A/D modul bekapcsolása (ADCON0<0>)
- az A/D megszakítás konfigurálása (ha szükséges)
 - = az ADIF bit törlése (PIR1<6>)
 - = az ADIE bit 1-re állítása (PIE1<6>)
 - = a GIE bit 1-re állítása (INTCON<7>)
- gondoskodni kell a mintavételi idő kivárásáról
- konverzió megindítása
 - = a $\overline{GO/DONE}$ bitet 1-re kell változtatni (ADCON0<2>)
- meg kell várni, míg az A/D konverzió befejeződik
 - = vagy a $\overline{GO/DONE}$ bit rendszeres vizsgálatával,
 - = vagy az A/D megszakítás segítségével.



3.61. ábra Az analóg bemenet, a mintavétel időzítése

- az ADRES regiszterből kiolvasható a konverzió eredménye
- törölhető az ADIF bit

Amikor az A/D folyamat befejeződik, az eredmény az ADRES regiszterbe kerül, a $\overline{GO/DONE}$ bit 0-ra vált, az A/D megszakítási flag bit (ADIF) pedig 1-re. Az A/D modul blokkvázlatát a 3.60. ábra mutatja be.

Ha az A/D modult konfiguráltuk, a kiválasztott analóg csatornán mintát kell venni, mielőtt a konverzió megindul. (Az A/D alap órajele periódus ideje T_{ad} , ennek beállítási lehetőségéről a későbbiekben lesz szó, ettől függ a működés időtartama is.) Az analóg csatornák belépő pontjait a megfelelő TRIS bitekkel bemenetként kell konfigurálni.

- a következő konverzióhoz tartozó mintavétel előtt minimum $2T_{ad}$ időt ki kell várni.

Az analóg bemenet áramköri rajza látható a 3.61. ábrán.

A precíz konverzió feltétele, hogy a Chold tartó kondenzátor teljesen feltöltődjön az analóg feszültség értékére. A feltöltődési idő függ az analóg jelforrás belső ellenállásától (legfeljebb 10 k Ω lehet), a mintavető kapcsoló elem ellenállásától (R_{ss}). A 3.61. ábrán megtalálható a számítási képlet. A mintavételi idő ennél hosszabb, mivel az erősítő elemek beállási idejét (tipikusan 5 μ s) és a csip hőmérsékletét is figyelembe kell venni. 25°C-on, 10 k Ω R_s érték és névleges csip paraméterek esetén a mintavételi idő 11,974 μ s.

Az A/D konverzió időigénye 10 T_{ad} . A T_{ad} értékét az ADCON0 regiszterben az ADSC1,0 bittel kell beállítani (3.33. táblázat).

A 3.34. táblázatban különféle órajel frekvenciákhoz, az ADSC bitek beállítási kombinációi alapján szerepel néhány konkrét T_{ad} érték - látható, hogy nem minden érték felel meg a követelményeknek.

3.33. táblázat $A T_{ad}$ értéke

ADCS1,0	T_{ad}	feltétel
00	2 t_{osc}	$t_{osc} \geq 1 \mu s$
01	8 t_{osc}	$t_{osc} \geq 0,25 \mu s$
10	32 t_{osc}	$t_{osc} \geq 50 ns$
11	PIC16LC74: 6 μs PIC16C74: 4 μs	belső RC oszcillátor

3.34. táblázat $A T_{ad}$ függése az órajelfrekvenciától

AD órajel forrás (T_{ad})		A PIC eszköz frekvenciája (Hz)			
Működés	ADCS1:ADCS0	20 M	5 M	1,25 M	333,33 k
2 T _{osc}	00	100 ns*	400 ns*	1,6 μs	6 μs
8 T _{osc}	01	400 ns*	1,6 μs	6,4 μs	24 μs **
32 T _{osc}	10	1,6 μs	6,4 μs	25,6 μs **	96 μs **
RC	11	2-6 μs °	2-6 μs °	2-6 μs °	2-6 μs °

° = Az RC forrás alkalmazásakor a T_{ad} idő tipikusan 4 μs

* = Ezek az értékek nem felelnek meg a minimális T_{ad} idő előírásának

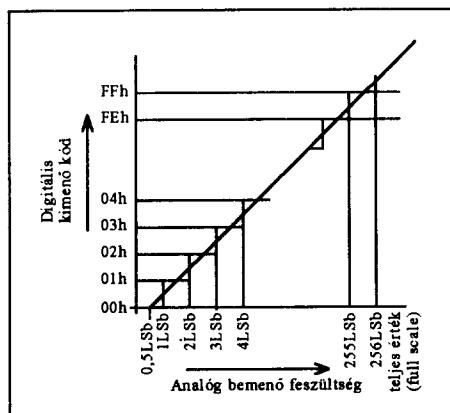
** = A gyorsabb konverzió érdekében más forrás választása ajánlott

Az analóg bemeneti pontokat a TRIS bitekkel bemenetté kell konfigurálni. Ha egy IC pont nincs analóg bemenetté konfigurálva, a rávezetett analóg jel a mikrovezérlőt tönkretetheti.

Előfordulhat, hogy egy alkalmazásban nincs szükség az A/D 8 bites felbontására, viszont gyorsabb konverzió lenne célszerű. Ekkor megtehető, hogy a kívánt bitszám (pl. 4 bit) konverziós ideje letelte után olyan T_{ad} értéket állítunk be, ami érvénytelen. Ekkor az utolsó bitek helytelenül, de gyorsan konvertálódnak. Ha N bites átalakításra van csak szükség, a T_{ad} átkapcsolás időpontja így határozható meg:

$$\text{átkapcsolás időpontja} = 2 T_{ad} + N T_{ad} + (8-N)(2 T_{osc}).$$

A kiszámolt időtartam pl. szoftver hurokkal vagy egy időzítő egységgel mérhető. Példaként tegyük fel, hogy 4 bites felbontással megelégszünk, a mikrovezérlő órajele 20 MHz-es, az A/D óra legyen 32 T_{osc} értékre programozva. A konverzió megkezdése után 6 T_{ad} elteltével programozzuk át az A/D órajelet 2 T_{ad} értékre. A 2 T_{ad} kívül esik a megengedett tartományon (1,6 μs), ezért az utolsó négy bit hibás lesz, viszont a konverzió kb. 10 μs alatt megtörténik.



3.62. ábra Az A/D transzfer karakterisztikája

Az analóg bemeneti pontokon is van a csipen belül védődioda a VDD és a VSS pontok felé, ezért az analóg jeltartomány is a VSS - VDD sáv.

Az A/D konverzió indítható a CCP2 modul különleges kimenő jelével, a speciális trigger eseménnyel is. Ehhez a CCP2M<3:0> biteket a CCP2CON regiszterben 1011 kombinációra állítjuk, az A/D modult engedélyezzük (ADON=1). Amikor a trigger jel kialakul, a GO/DONE bit 1-re vált, a konverzió megkezdődik, a Timer1 számláló nulázódik. Periódikus A/D konverzió végezhető így, de minden eredményt ki kell olvasni az ADRES regiszterből. A periódusidő legyen elegendő a mintavételhez.

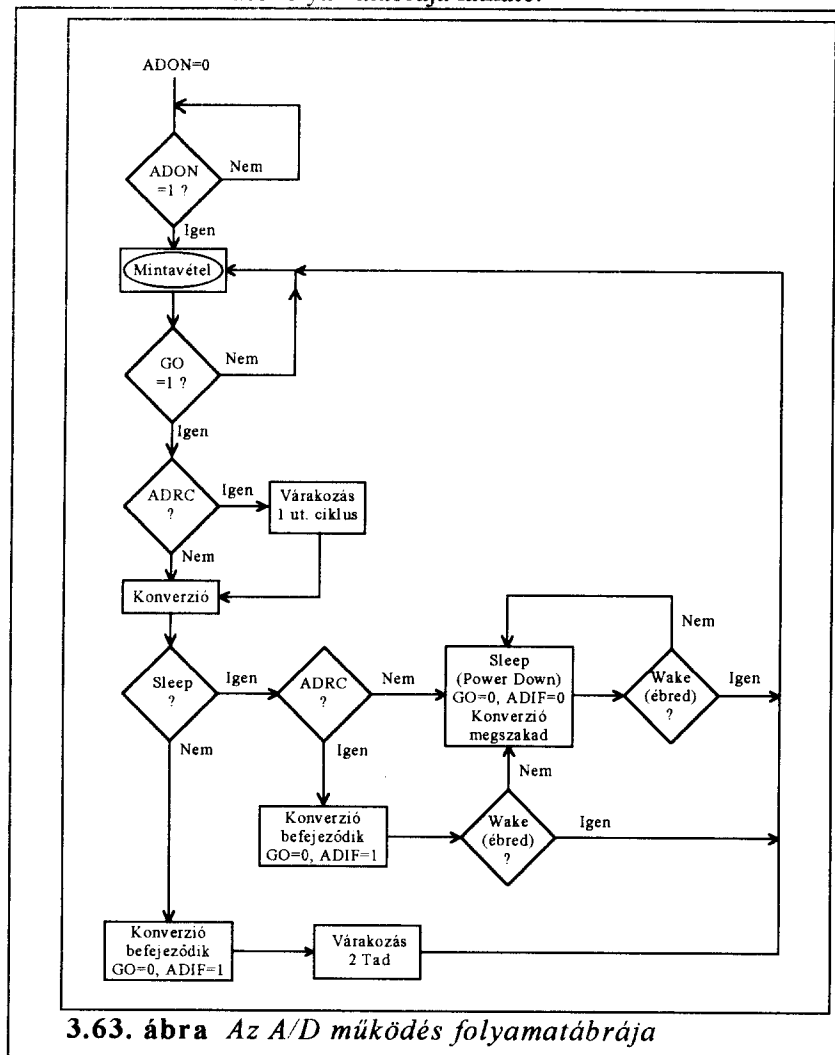
SLEEP állapotban is működhet az A/D (ekkor az eredmény pontosabb lesz, nincsenek belső digitális zajok), de ehhez RC oszcillátoros üzemmódot kell beállítani. A konverzió végén megszakítás alakul ki, a CPU felébreszthető.

A RESET folyamat az A/D modult kikapcsolja, a folyó konverziót megszakítja. Az ADRES tartalmát nem befolyásolja a RESET.

Ha RC szűrőn át vezetjük be a mikrovezérlőbe az analóg jelet, annak ellenállás értékét is figyelembe kell venni (a megengedett külső ellenállás legfeljebb 10 k Ω). Az A/D konverter ideális transzfer karakteristikáját a 3.62. ábra mutatja be. A legkisebb digitális érték (01h) akkor áll elő, ha az analóg jel éppen meghaladja az 1LSB értéket ($V_{ref}/256$).

A belső órajelek alkalmazása kisebb digitális zajt jelent az A/D környezetében, mint az RC oszcillátor, mivel a belső órajelek és az A/D működése teljes szinkronban van; ezért működő mikrovezérlő esetén nem ajánlott az RC oszcillátoros A/D konverzió.

A 3.63. ábrán az A/D működés folyamatábrája látható.



3.63. ábra Az A/D működés folyamatábrája

A 3.35. táblázat pedig összefoglalja az A/D modul működését támogató regisztereket.

3.35. táblázat Az A/D működés regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0B/8B	INTCON	GIE	PEIE	RTIE	INTF	RBIF	RTIF	INTF	RBIF
0C	PIR1	PSPIF	ADIF	RXIF	TNIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
8C	PIE1	PSPIE	ADIE	RXIE	TNIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
0D	PIR2	-	-	-	-	-	-	-	CCP2IF
8D	PIE2	-	-	-	-	-	-	-	CCP2IE
1E	ADRES	A/D eredmény regiszter							
1F	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	-	ADON
9F	ADCON1	-	-	-	-	-	PCFG2	PCFG1	PCFG0
05	PORTA	A Port adat latch íráskor, A Port olvasáskor							
85	TRISA	A Port adatirány regiszter							
09	PORTE	E Port adat latch íráskor, E Port olvasáskor							
89	TRISE	E Port adatirány regiszter							

Ellenőrző kérdések:

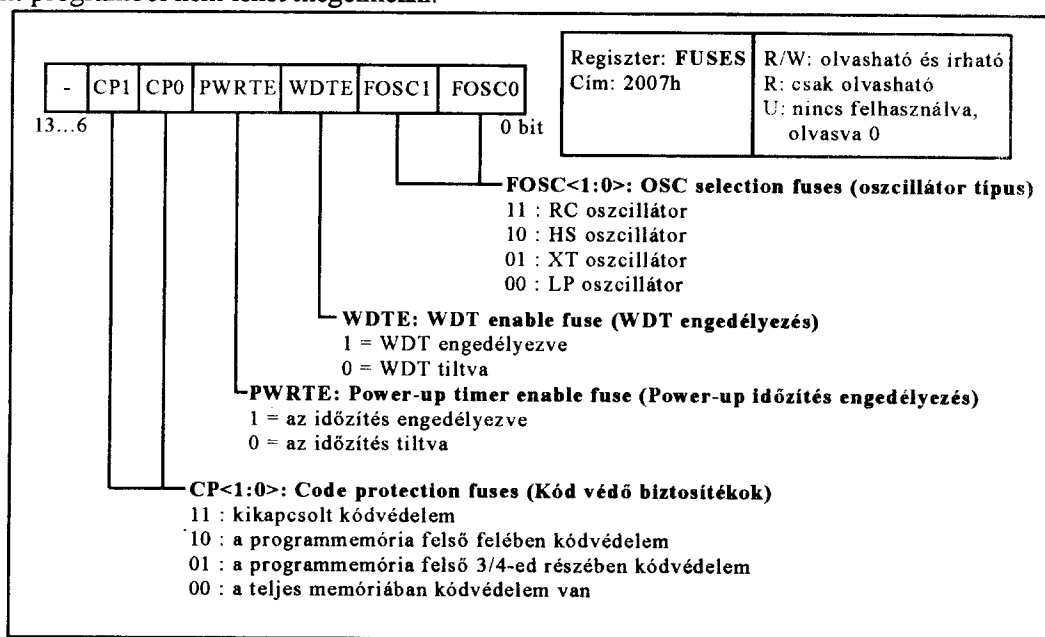
1. Milyen elven működik a 16C74 A/D konvertere?
2. SLEEP állapotban működhet-e az A/D átalakító? Mi ennek a feltétele?
3. Ébreszthet-e az A/D konverter SLEEP-ből? Mi ennek a feltétele?
4. Ha egy kimenetként konfigurált IC láb az A/D konverzióval feldolgozásra kerül, milyen értéket kapunk eredményül?
5. Mekkora lehet legfeljebb az analóg jelforrás belső ellenállása?
6. Mitől függ a mintavételezési idő?
7. Mit kell figyelembe venni a Tad beállításakor?
8. Hogyan lehet növelni a konverziós sebességet, ha kisebb felbontás is elegendő?
9. A CCP2 modul hogyan indíthat A/D konverziót?
10. Az RC vagy a leosztott belső óra kedvezőbb pontosság szempontjából?
11. Mi a legpontosabb A/D konverzió feltétele?
12. Milyen megkötés érvényes az analóg bemenetre csatlakozó RC szűrőre?
13. Mi az abnalog jel megengedett tartománya?

3.1.9. A 16C74 speciális tulajdonságai

A 16C74 speciális tulajdonságai jelentős része megegyezik a 16C mikrovezérlő család többi elemével. Itt is konfigurációs biztosítékokkal lehet az alapfunkciókat kijelölni. Megtaláljuk ebben a mikrovezérlőben is a saját belső RC oszcillátorral működő Watchdog Timer-t, amit csak konfigurációs biztosítékkal lehet kikapcsolni. A tápfeszültség bekapcsolásakor az oszcillátor még kialakulatlan impulzusait fedi át az oszcillátor start-stop timer (OST), tovább növeli a bekapcsolási működés biztonságát a tápfeszültség növekedését átfedő power-up timer (PWRT), a maga 72 ms-os késleltetésével.

A kisfogyasztású SLEEP állapotból a mikrovezérlő felébreszthető külső RESET jellel, a Watchdog timer időtúlfutás jelzésével és megszakításokkal. A 16C74-nél is léteznek az oszcillátor opciók, a legolcsóbb az RC, a legkisebb fogyasztású az LP kvarcoszcillátor.

A 16C74-ben, mint általában a 16Cxx mikrovezérlőkben, hat **konfigurációs biztosíték** található, mind EPROM kialakítású (3.64. ábra). Ezek a bitek programozatlanul 1 értékűek, programozva (kiegítve) az értékük 0. A konfigurációs szó a programmemória 2007h című rekesze. Ezt a rekeszt a felhasználó nem érheti el, mivel a 2000h - 3FFFh speciális teszt/konfigurációs memória területen van, amit programból nem lehet megcímezni.

**3.64. ábra** A konfiguráló szó (16C74)

Az **oszillátorok kapcsolási megoldásai**, a külső alkatrészek ajánlott értékei azonosak a 16Cxx és a 16C5x elemeknél. A **RESET** folyamat azonban összetettebb a 16Cxx mikrovezérlők esetében; a lehetséges RESET esetek a következők:

- Power-on Reset (POR, tápfeszültség bekapcsolási RESET, belső áramkör érzékeli a tápfeszültség megjelenését)
- MCLR RESET, normál működés közben
- MCLR RESET, SLEEP állapot közben
- WDT időtúlfutás RESET, normál működésben
- WDT időtúlfutás RESET, SLEEP állapot közben.

Sok olyan rekesz van a mikrovezérlőben, melyeket a RESET folyamat nem érint. Ezek POR esetén véletlenszerű tartalmat kapnak, más RESET folyamat során a tartalmuk nem változik meg. Azok a regiszterek, melyeknek van határozott, RESET folyamat utáni értékük, ezt az értéket veszik fel

MCLR, POR és normál működés közben fellépő WDT RESET hatására. Ha azonban SLEEP-ből ébreszt a WDT, ezek a regiszterek nem módosulnak korábbi állapotukhoz képest, mivel ez a folyamat normál működésnek minősül, nem RESET-nek. A STATUS regiszter jelzőbitjei adnak információt arról, hogy egy RESET folyamatot milyen esemény váltott ki (**3.36. táblázat**).

A regiszterek RESET utáni állapotát a **3.37. táblázat** foglalja össze (83. oldal), de csak azok a regiszterek szerepelnek, melyek bitjeinek határozott RESET utáni állapota van.

Három regiszter esetében mind a hat RESET formához megadjuk a kialakuló állapotokat (**3.38. táblázat**).

3.36. táblázat A RESET lehetséges okai

POR	TO	PD	
0	1	1	Bekapcsolási (Power-On) reset
0	0	X	Illegális, TO érvényes POR-nál
0	X	0	Illegális, PD érvényes POR-nál
1	0	1	WDT reset, normál működés közben
1	0	0	WDT reset, Sleep-ből ébredéssel
1	1	1	MCLR reset, normál működésnél
1	1	0	MCLR reset, Sleep-ből ébredéssel

3.38. táblázat Néhány regiszter RESET utáni tartalma

	PCL 02h	STATUS 03h	PCON 8Eh
Power-On (bekapcsolási) reset	000h	0001 1xxx	---- --0-
MCLR reset, normál működéskor	000h	0001 1uuu	---- --u-
MCLR reset, Sleep közben	000h	0001 0uuu	---- --u-
WDT reset, normál működéskor	000h	0000 1uuu	---- --u-
WDT reset, Sleep közben	PC + 1	uuu0 0uuu	---- --u-
Megszakítás ébresztés Sleep-ből	PC + 1*	uuu1 0uuu	---- --u-

u = változatlan, x = határozatlan, - = használaton kívüli bit, olvasva 0

* = Ha a sleep-ből megszakítás ébreszt és a GIE = 1, a PC-be a megszakítás vektor (0004h) töltődik be.

Az induló oszcillátor torz, kialakulatlan impulzusai idejére a belső működést késlelteti az OST (oszcillátor start-up timer), a tápfeszültség növekedési idejére védi a belső részleteket a hamis működéstől a PWRT (power-up timer). A tápfeszültség ponton lévő feszültséget figyelve alakítja ki a belső RESET folyamatot a POR (power-on reset) áramköri egység. A POR akkor lép működésbe, ha a VDD lábán, növekedés közben, 1,6 ... 1,8 V közötti a feszültség. Ha a POR áramkörre kívánjuk bízni a mikrovezérlő bekapcsolási RESET-elését, az MCLR pontot egyszerűen össze kell kötni a VDD tápfeszültség ponttal. Csökkenő tápfeszültségnél a POR nem állít elő RESET jelet.

A PWRT 72 ms-os rögzített működési idejű késleltető egység, a POR jele hozza működésbe. A PWRT-t egy független belső RC oszcillátor működteti, s a mikrovezérlő belső elemeit mindaddig RESET állapotban tartja, míg a késleltetési ideje le nem telik. A PWRT konfigurációs biztosítékkal a PWRT működés engedélyezhető (alaphelyzet, 1 érték) vagy tiltható (0 érték, programozott állapot).

Az OST áramkör 1024 oszcillátor ciklusig (az oszcillátor jeleket az OSC1 bemeneten érzékeli) késlelteti a belső elemek indulását. Az OST akkor indul, amikor a PWRT "lejár". Csak XT, LP és HS oszcillátorok esetén működik és csak tápfeszültség bekapcsoláskor valamint SLEEP-ből ébredéskor.

A tápfeszültség bekapcsolásakor először a PWRT késleltet, amikor letelik a 72 ms, akkor kezd működni az OST áramkör. Ez az oszcillátor első impulzusait számlálja, így működési ideje az oszcillátor frekvencia függvénye. Mire ez a késleltetés is letelik, a belső részletek biztonsággal indulhatnak, hiszen a tápfeszültség már beállt a névleges értékre s az oszcillátor is tökéletes formájú, névleges időzítésű impulzusokat állít elő.

3.37. táblázat Regiszterek állapota RESET folyamat után (16C74)

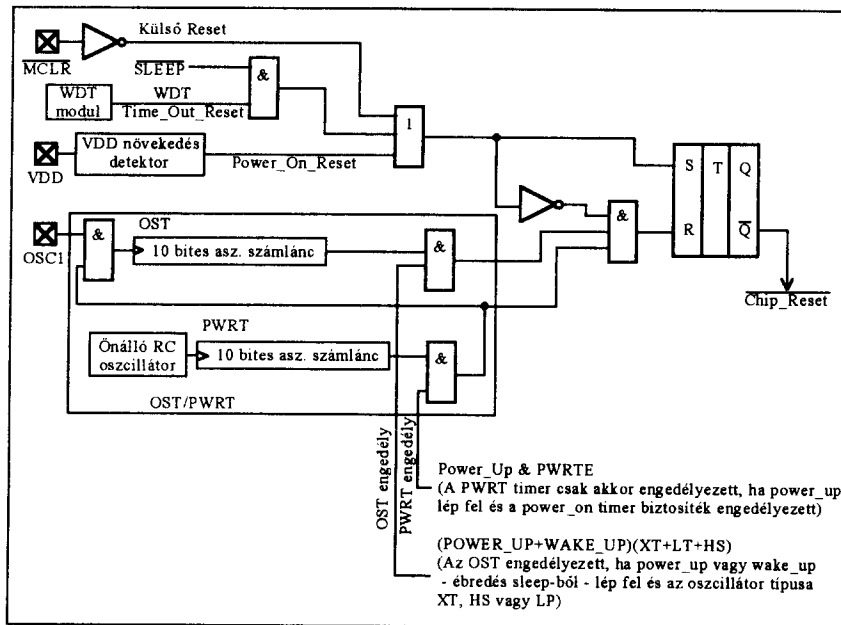
Regiszter	Cím	Bekapcsolási (Power-Up) Reset	MCLR Reset - normál működéskor - Sleep-ből WDT időtúllépés normál működéskor	Megszakításos ébredés Sleep-ből WDT időtúllépés - ébredés Sleep-ből
W	-	xxxx xxxx	uuuu uuuu	uuuu uuuu
INDF	00h	-	-	-
RTCC	01h	xxxx xxxx	uuuu uuuu	uuuu uuuu
PC	02h	0000h	0000h	PC+1 (2)
STATUS	03h	0001 1xxx	000? ?uuu (3)	uuu? ?uuu (3)
FSR	04h	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTA	05h	-xx xxxx	-uu uuuu	--uu uuuu
PORTB	06h	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTC	07h	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTD	08h	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTE	09h	--- -xxx	--- -uuu	--- -uuu
PCLATH	0Ah	--0 0000	--0 0000	--u uuuu
INTCON	0Bh	0000 000x	0000 000u	uuuu uuuu (1)
PIR1	0Ch	0000 0000	0000 0000	uuuu uuuu (1)
PIR2	0Dh	--- --0	--- --0	--- --u (2)
TMR1L	0Eh	xxxx xxxx	uuuu uuuu	uuuu uuuu
TMR1H	0Fh	xxxx xxxx	uuuu uuuu	uuuu uuuu
T1CON	10h	--00 0000	--uu uuuu	--uu uuuu
TMR2	11h	xxxx xxxx	uuuu uuuu	uuuu uuuu
T2CON	12h	--00 0000	--00 0000	--uu uuuu
SSPBUF	13h	xxxx xxxx	uuuu uuuu	uuuu uuuu
SSPCON	14h	0000 0000	0000 0000	uuuu uuuu
CCPR1L	15h	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCPR1H	16h	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCP1CON	17h	--00 0000	--00 0000	--uu uuuu
RCSTA	18h	0000 0000	0000 0000	uuuu uuuu
TXREG	19h	xxxx xxxx	uuuu uuuu	uuuu uuuu
RCREG	1Ah	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCPR2L	1Bh	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCPR2H	1Ch	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCP2CON	1Dh	0000 0000	0000 0000	uuuu uuuu
ADRES	1Eh	xxxx xxxx	uuuu uuuu	uuuu uuuu
ADCON0	1Fh	0000 0000	0000 0000	uuuu uuuu
INDF	80h	-	-	-
OPTION	81h	1111 1111	1111 1111	uuuu uuuu
PC	82h	0000h	0000h	PC+1
STATUS	83h	0001 1xxx	000? ?uuu (3)	uuu? ?uuu (3)
FSR	84h	xxxx xxxx	uuuu uuuu	uuuu uuuu
TRISA	85h	--11 1111	--11 1111	--uu uuuu
TRISB	86h	1111 1111	1111 1111	uuuu uuuu
TRISC	87h	1111 1111	1111 1111	uuuu uuuu
TRISD	88h	1111 1111	1111 1111	uuuu uuuu
TRISE	89h	0000 -111	0000 -111	uuuu -uuu
PCLATH	8Ah	--0 0000	--0 0000	--u uuuu
INTCON	8Bh	0000 000x	0000 000u	uuuu uuuu (1)
PIE1	8Ch	0000 0000	0000 0000	uuuu uuuu
PIE2	8Dh	--- --0	--- --0	--- --u
PCON	8Eh	--- --0-	--- --u-	--- --u-
PR2	92h	xxxx xxxx	uuuu uuuu	uuuu uuuu
SSPADD	93h	0000 0000	0000 0000	uuuu uuuu
SSPSTAT	94h	--00 0000	--00 0000	--uu uuuu
TXSTA	98h	0000 0000	0000 0000	uuuu uuuu
SPBRG	99h	xxxx xxxx	uuuu uuuu	uuuu uuuu
ADCON1	9Fh	--- -000	--- -000	--- -uuu

Néhány esetre a 3.39. táblázat mutatja be a bekapcsolási késleltetési idő alakulását (ld. a 84. oldalon).

3.39. táblázat Bekapcsolási késleltetési idők

Oscillátor konfiguráció	Bekapcsolás (Power Up)		Ébredés (Wake Up) Sleep-ből
	PWRT=1	PWRT=0	
XT, HS, LP	72 ms + 1024 tosc	1024 tosc	1024 tosc
RC	72 ms	-	-

Ha több mikrovezérlőt szinkronban kell indítani, ez is elérhető; ehhez közös MCLR jelet kell rájuk



3.65. ábra A RESET hardver (16C74)

vezetni s a tápfeszültség bekapcsolásakor ide L szintet kell előállítani. Ezt az L szintet nem szüntetjük meg, míg a belső késleltetések letelnek. Ha a $t_{PWRT} + t_{OST}$ időtartam letelt, az MCLR ponton H szintet alakítunk ki - ekkor egyszerre indulnak a mikrovezérlők.

A PCON regiszterben lévő POR (Power On Reset) bit informál a bekapcsolási helyzetről. A tápfeszültség bekapcsolásakor ez a bit 0-ra törlődik, később szoftverrel lehet csak alapál-

lapotába (1) visszaállítani. A teljes RESET hardvert a 3.65. ábrán mutatjuk be.

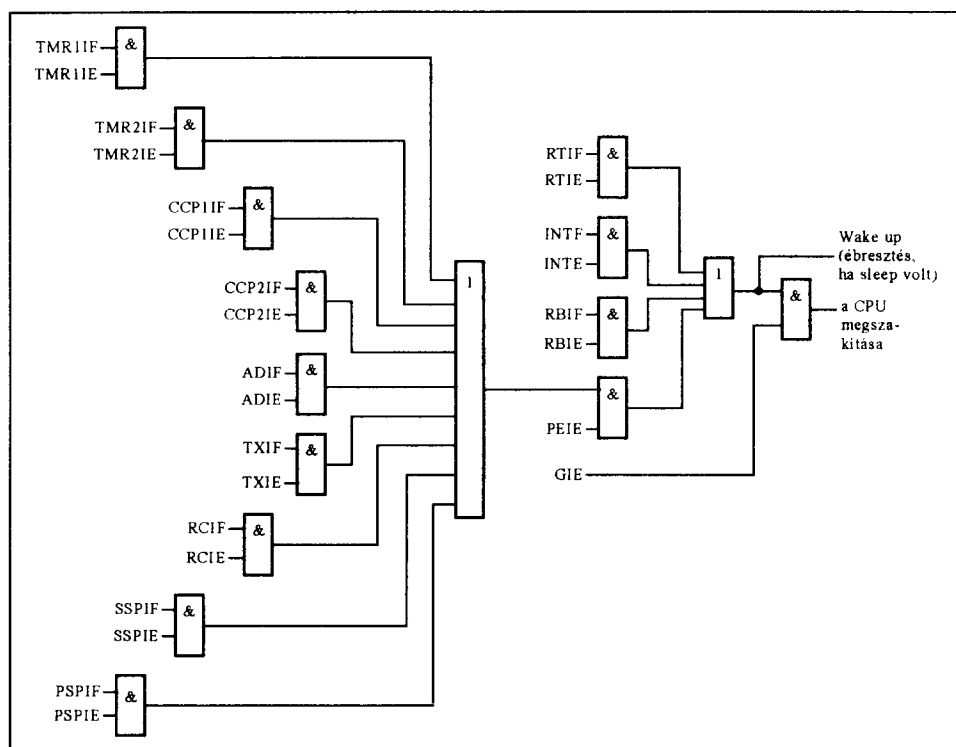
A 16C74 összesen 12 megszakítás forrást kezel:

- külső megszakítás (RB0/INT)
- Timer0 (TMR0, RTCC) túlszordulás
- B Port változás megszakítás (RB<7:4>)
- Timer1 túlszordulás megszakítás
- Timer2 túlszordulás megszakítás
- CCP1 megszakítás
- CCP2 megszakítás
- SCI aszinkron adás illetve vétel megszakítás
- szinkron soros Port megszakítás
- SCI megszakítás
- A/D megszakítás
- mikroprocesszor Port megszakítás.

A megszakítás vezérlő regiszter (INTCON) tárolja a megszakítás flag bitek egy részét. A flag bitek akkor is 1-re váltanak egy esemény kialakulásakor, ha a megfelelő megszakítás nincs engedélyezve. Minden megszakítás tiltható egyedileg és globálisan. Ha a globális megszakítás engedélyező bit (GIE, az INTCON<7> biten) 1 értékű, az összes, egyedileg nem maszkolt megszakítás érvényesülhet; ha tiltott értékű (azaz 0), akkor minden megszakítást letilt (az egyedi engedélyező bitektől függetlenül). A RESET folyamat a GIE bitet törli. A megszakítást kezelő rutint lezáró RETFIE utasítás egyetlen lépésben biztosítja a szubrutinból való visszatérést és a GIE engedélyezését.

A perifériák megszakítás flag bitjei a PIR regiszterekben találhatók, az engedélyező bitek a PIE regiszterekben. Elfogadott megszakításkor a GIE törlődik, így a megszakítási rutin közben újabb megszakítással nem foglalkozik a CPU - mindaddig, amíg nem engedélyezzük ismét a megszakítás elfogadást. Egy megszakítás elfogadásakor az első, végre nem hajtott utasítás címét a verembe menti a CPU majd a 0004h címre ugrik. Ha nem egyértelmű a megszakítás eredete, ekkor kell végigvizsgálni a megszakítás flag biteket - így lehet megtudni, melyik megszakítás forrás kezdeményezte az adott megszakí-

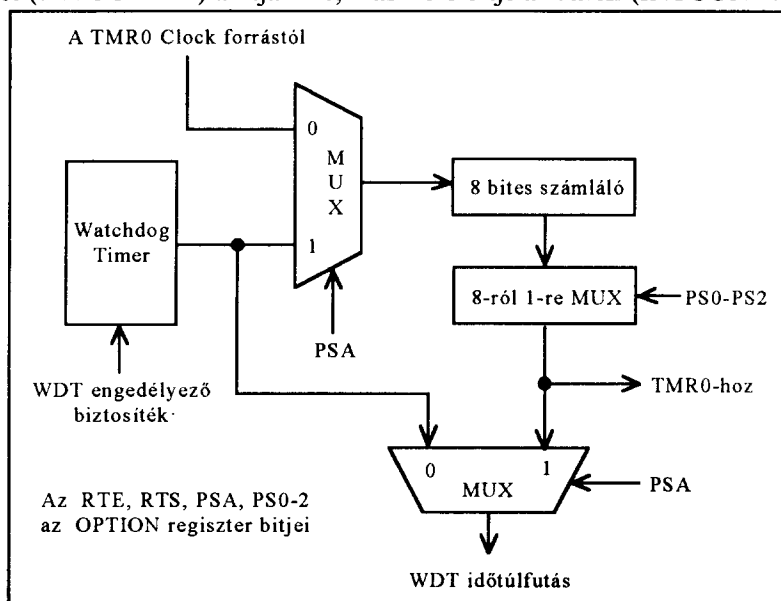
tást. Célszerű ilyenkor a flag bitet azonnal törölni is. A megszakítások hierarchiáját a 3.66. ábra mutatja be.



3.66. ábra A megszakítások hierarchiája (16C74)

Külső megszakítás kérés az RB0/INT IC lábra érkezik, ez él-aktív kérés, az aktív él programozható (felfutó élhez INTEDG=1, lefutó élhez INTEDG=0, az OPTION<6> biten). Amikor az INT lábon a kijelölt él fellép, az INTF flag bit (INTCON<1>) 1-re vált. Ezt a megszakítást az INTE bit (INTCON<4>) maszkolhatja, törölni szoftverrel kell. Ha SLEEP előtt engedélyezzük, a külső megszakítás ébresztheti a CPU-t. A GIE bit értéke az ébredést nem befolyásolja, de az induló működést igen (ld. később, a SLEEP leírásánál).

A TMR0 (RTCC) az FFh → 00h túlcsoforduláskor állítja 1-re a T0IF bitet (INTCON<2>). Ez a megszakítás maszkolható a T0IE bittel (INTCON<5>). A B Port <7:4> lábain fellépő változás az RBIF flag bitet (INTCON<0>) állítja 1-re, maszkoló bitje az RBIE (INTCON<4>).



3.67. ábra A WDT felépítése (16C74)

Az általános programozói gyakorlat az, hogy a szubrutinokban, így a megszakítás kezelőkben is a verembe mentik azokat a regisztertartalmakat, melyekre a főprogram folytatásakor szükség lesz. A PIC veremtára igen kicsiny, ezért a programozó nem is fér hozzá, csak a szubrutinra ugró utasítások kezelik. A szubrutinban szükséges adatmentéshez ezért a RAM rekeszeit kell felhasználni.

A Watchdog Timer (WDT) egy szabadonfutó RC oszcillátorra épül, nem igényel külső jelet vagy külső alkatrészt. Akkor is futhat, ha az

OSC1/OSC2 pontokon nincs jel, pl. SLEEP állapotban is. Normál működés során a WDT időtúlfutás RESET folyamatot gerjeszt. SLEEP módban, ha a WDT időtúlfutás fellép, feléleszti a CPU-t, de a normál működés folytatódik (a SLEEP utasítást a programban követő utasítással). A WDT felépítése a 3.67. ábrán látható (85. oldal).

A WDT névleges futási ideje 18 ms (utóosztó nélkül). Ez az időtartam kismértékben függ a környezeti hőmérséklettől, a VDD értékétől, a mikrovezérlő egyedi áramkörü paramétereitől. Hosszabb időtúlfutási periódus érhető el, ha az előosztó áramkört utóosztóként a WDT-hez rendeljük. Ha az osztási arány 128:1, a teljes futási idő már 2,3 s lesz. A WDT időtúlfutás kialakulását a STATUS regiszterben lévő TO bit jelzi.

A WDT-t a programban elhelyezett CLRWDT utasítások törlik a normál működés közben - ezzel ellenőrizhető a normál utasításvégrehajtó tevékenység. A SLEEP utasítás is törli, így a teljes futási idő rendelkezésre áll, mielőtt felébreszti a CPU-t.

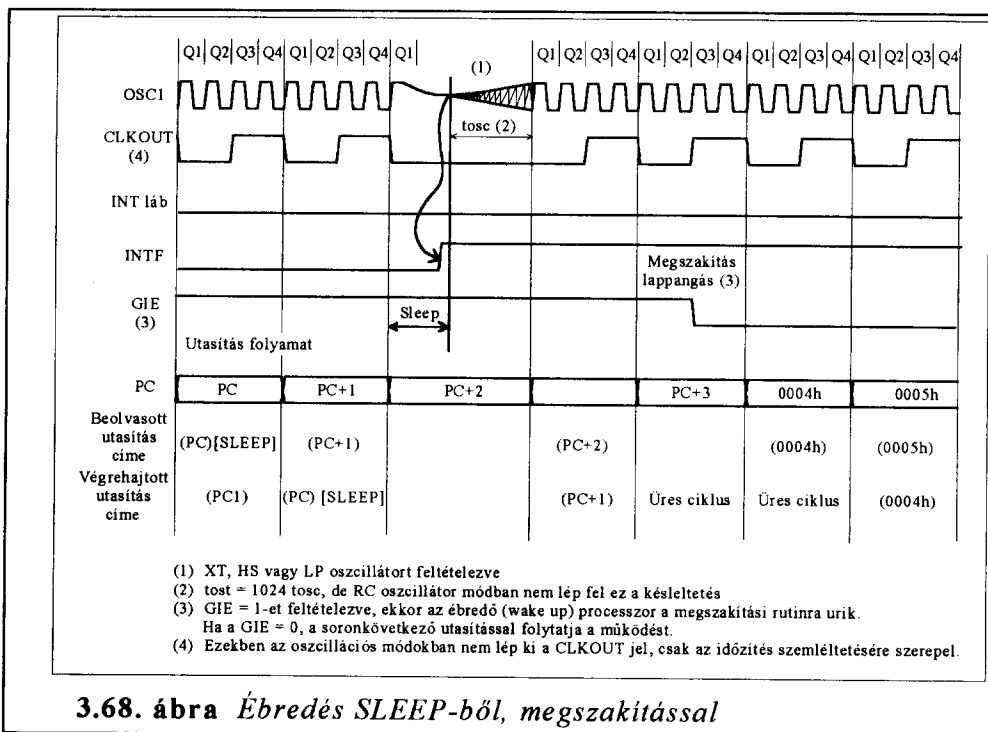
A WDT-vel kapcsolatos regisztereket a 3.40. táblázat mutatja be.

3.40. táblázat A WDT regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
2007h	Konfig.bizt.	-	-	CP1	CP0	PWRTE	WDTE	FOSC1	FOSC0
81h	OPTION	RBP	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0

A sötétített biteket nem használja a Watchdog regiszter

A SLEEP (csökkentett teljesítményfelvételű üzemmód, Power Down Mode) a SLEEP utasítás végrehajtásakor áll elő. A WDT (ha működése engedélyezett) törlődik, de tovább számol, a STATUS regiszterben a PD bit 0 lesz, a TO pedig 1. A mikrovezérlőben az alap órajel oszcillátoránál a meghajtó fokozat kikapcsol, az oszcillátor leáll. Az I/O lábak utolsó állapotukban maradnak. A legkisebb fogyasztás úgy érhető el, ha az I/O lábak a SLEEP alatt VSS vagy VDD feszültségen vannak, s nem kell külső terhelést meghajtaniuk. A TMR0 (RTCC) bemenete is legyen VSS vagy VDD szinten (hacsak nem használjuk eseményszámlálásra a SLEEP közben), az MCLR bemeneten is legyen H szint.



A SLEEP állapotból három módon ébreszthető a mikrovezérlő:

- külső RESET jel az MCLR lábon (L szint)
- a WDT időtúlfutása (ha a WDT engedélyezett)
- megszakítás az INT lábról, az RB pontokról, a TMR0 túlcsordulása miatt vagy perifériáról.

Az ébresztést a következő perifériák válthatják ki, megszakítás kérésükkel:

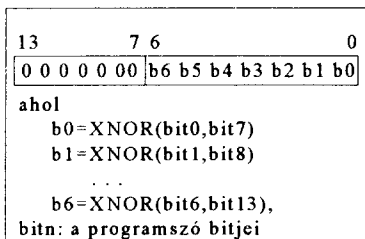
- TMR1
- SSP Start/Stop bit detektálása
- CCP kiolvasás (Capture) mód
- Slave Port írása/olvasása.

A lehetséges ébresztő hatású események közül az első, amelyik bekövetkezik, kiváltja a SLEEP lezárását. Amikor a SLEEP utasítás végrehajtása megkezdődik, a következő utasítást már beolvasta a CPU (ld. pipeline működés), így ébredéskor az már az utasítás regiszterben lesz. Ha megszakítással kívánjuk ébreszteni a mikrovezérlőt, a megfelelő maszk bitet 1-re kell állítani (engedélyező állapot). Az ébresztés ténye ezután a GIE értékétől független. Ha a CPU megszakítással ébred és a GIE=0 (tiltott), a programban a SLEEP-et követő (és már korábban beolvasott) utasítással folytatódik a működés. Ha megszakítással ébred a mikrovezérlő és a GIE=1, a 0004h címről indul a működés (megszakítást kiszolgáló rutin). A WDT minden esetben törlődik, amikor a mikrovezérlő a SLEEP állapotból ébred. A megszakításra történő ébredés menetét a **3.68. ábra** mutatja be (86. oldal).

A **kódvédelmet** a konfiguráló szó két bitje vezérli, a CP<1:0>. Ha a PIC programtárolójában lévő programot védetté tesszük, azt nem lehet másolás, értelmezés érdekében kiolvasni. A kódvédelem elrendelése ellenére a konfiguráló szó tartalmát a fejlesztő rendszer továbbra is képes kiolvasni. A 16Cxx mikrovezérlőkben a kódvédelem rugalmasabb lehetőség, mint a 16C5x elemeknél. A következő lehetőségek programozhatóak be:

CP<1:0>

- | | |
|----|--|
| 11 | nincs kódvédelem |
| 10 | a programmemória felső fele védett (16C74-nél: 07FF-0FFFh) |
| 01 | a programmemória felső 3/4-e védett (itt: 03FF-0FFFh) |
| 00 | a teljes programmemória védett. |

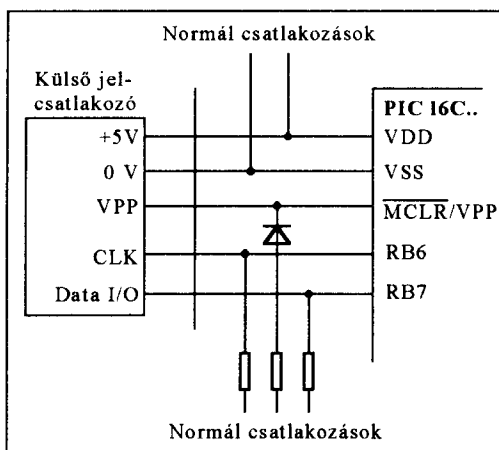


3.69. ábra Védett rekesz olvasása

Ha egy védett területen lévő rekesz tartalmát kiolvassuk, a kapott adat a **3.69. ábra** szerint alakul. A Microchip azért választotta ezt a megoldást, mert így a programok összehasonlítása, valamint a verifikálás megoldható még a védett programmemóriák esetén is. Azonos programmemória tartalmak esetén ugyanis a kiolvasható adatoknak meg kell egyezniük.

A védetté tett memóriaterület a fejlesztő rendszer számára sem hozzáférhető, ezért nemcsak a kiolvasása, de a továbbprogramozása sem lehetséges.

A 16Cxx áramköröknél - így a 16C74-nél is - megoldható, hogy a beépítés, a felhasználás helyén, a felhasználói áramköri környezetben lehessen programozni. Erre a célra soros programozási csatlakozási lehetőséget építettek be a mikrovezérlőkbe (In-Circuit Serial Programming), melyet két vezetéken át lehet kezelni (órjel, adatjel). Ezekon kívül a programozáshoz szükség van a tápfeszültség csatlakozásokra (VDD, VSS), és a programozó feszültségre. A soros programozáshoz szükséges jelkapcsolatokat a **3.70. ábra** mutatja be.



3.70. ábra Soros programozás

A soros programozási módba úgy lép be a mikrovezérlő, ha az MCLR lábán kialakuló felfutó él közben RB6=RB7=LOW. Ezután az RB6 lesz a programozási óra, az RB7 a programozási adat. A csipen ezt a két pontot ekkor Schmitt trigger fogadja.

A programozási módba belépve, a PC tartalma 0000h. Egy hat bites parancs küldhető be (adat olvasás, adat beírás, cím inkrementálás), majd az adatmozgások következnek. A soros programozás részleteivel a Microchip #ZDS30228 azonosítószámú alkalmazási segédlete foglalkozik.

Ellenőrző kérdések:

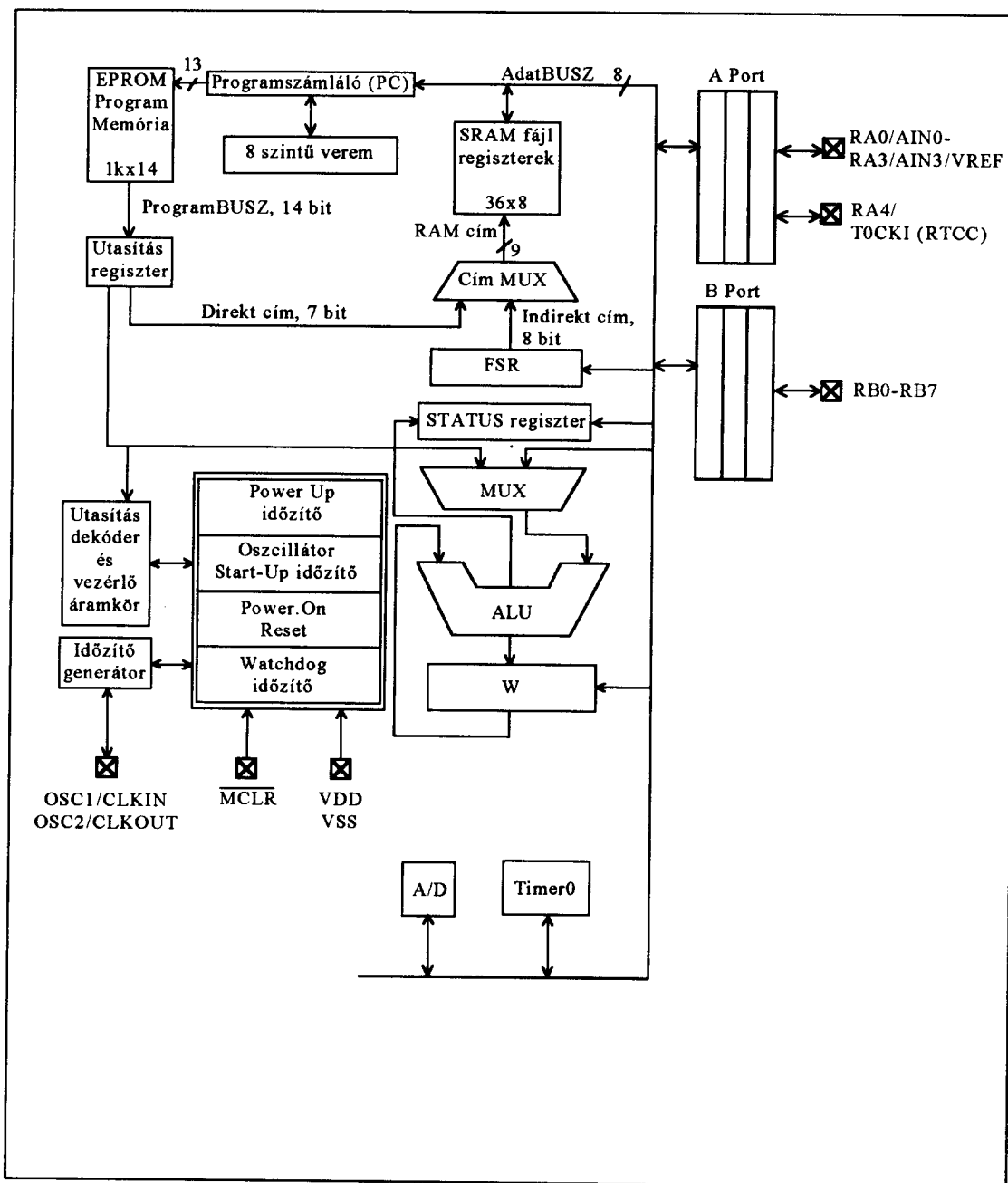
1. Hol találhatóak a konfigurációs biztosítékok?
2. Sorolja fel, hogy alapállapotban milyen hatásokat váltanak ki a konfigurációs biztosítékok.
3. Hányféle RESET állapot lehetséges?
4. Hogyan és mikor működik a POR egység?
5. Hogyan és mikor működik az OST? Mekkora a késleltetése?
6. Hogyan és mikor működik a PWRT? Mekkora a késleltetése?
7. Sorolja fel a megszakítás forrásokat.
8. Mi a GIE bit szerepe?
9. Melyik megszakítások ébreszthetnek SLEEP-ből?
10. Hogyan indul a működés, ha megszakítás ébresztett SLEEP-ből?
11. Hogyan indul a működés, ha a WDT ébresztett SLEEP-ből?
12. Hogyan alakulhat a WDT működési ideje?
13. Melyik utasítások törlik a WDT tartalmát?
14. Hogyan kerül a mikrovezérlő SLEEP állapotba?
15. Hányféle esete van a kódvédelemnek? Melyek azok, hol állíthatóak be?
16. Milyen tartalmat kapunk védett rekesz olvasásakor?
17. Programozható-e a védett területen lévő memória rekesz?

3.2. További PIC16Cxx mikrovezérlők

A továbbiakban néhány gyakori PIC típusról szeretnénk még képet adni. Elsősorban ábrákat fogunk megadni, amelyek segítségével a mikrovezérlő fő jellemzői megismerhetőek. A részletek általában megegyeznek a 16C74 megfelelő részegységével, ha mégis van eltérés, arról szólunk majd.

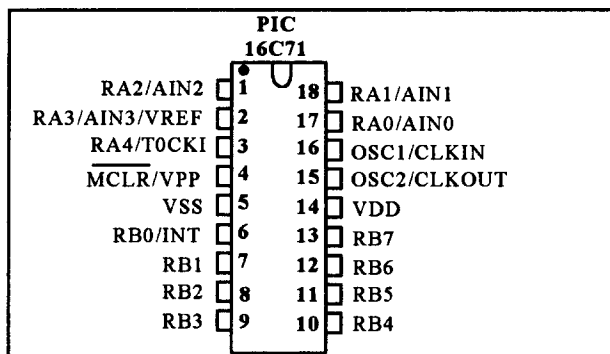
3.2.1. A PIC16C71 mikrovezérlő

Egyszerű felépítésű, kisméretű, de az A/D modulrt tartalmazó mikrovezérlő. A belső felépítése a 3.71. ábrán található meg.



3.71. ábra A PIC16C71 belső felépítése

A lábkiosztást (DIL tokozás esetére) a 3.72. ábra mutatja be.



3.72. ábra A PIC16C71 lábkiosztása

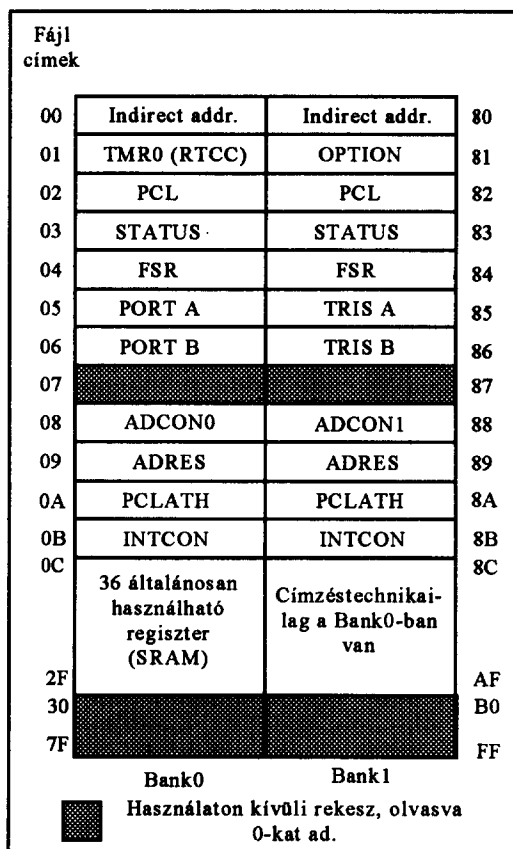
A csatlakozó pontok szerepét a 3.41. táblázat foglalja össze.

3.41. táblázat A 16C71 kivezetései funkciói

Megnevezés	DIP SOIC	Típus	Puffer	Leírás
OSC1/CLKIN	16	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	15	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR /VPP	4	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0/AN0	17	I/O	TTL	A PORT, kétirányú I/O Port 0 analóg bemenet 1 analóg bemenet 2 analóg bemenet 3 analóg bemenet/VREF TMR0 órabemenete, kimenetként OC
RA1/AN1	18	I/O	TTL	
RA2/AN2	1	I/O	TTL	
RA3/AN3/VREF	2	I/O	TTL	
RA4/T0CKI	3	I/O	ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem írható elő minden bemeneten. Külső megszakítás (ekkor ST bemenet) Megszakítás változásra Megszakítás változásra Megszakítás változásra/soros program. óra (ekkor ST) Megszakítás változásra/soros program. adat (ekkor ST)
RB0/INT	6	I/O	TTL/ST	
RB1	7	I/O	TTL	
RB2	8	I/O	TTL	
RB3	9	I/O	TTL	
RB4	10	I/O	TTL	
RB5	11	I/O	TTL	
RB6	12	I/O	TTL/ST	
RB7	13	I/O	TTL/ST	
VSS	5	P	-	GND
VDD	14	P	-	Pozitív tápfeszültség

I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás,
TTL=TTL bemenet, ST=Schmitt-trigger bemenet

A regiszter fájl terület a 3.73. ábrán látható, a fájl regisztereket a 3.42. táblázat tartalmazza.

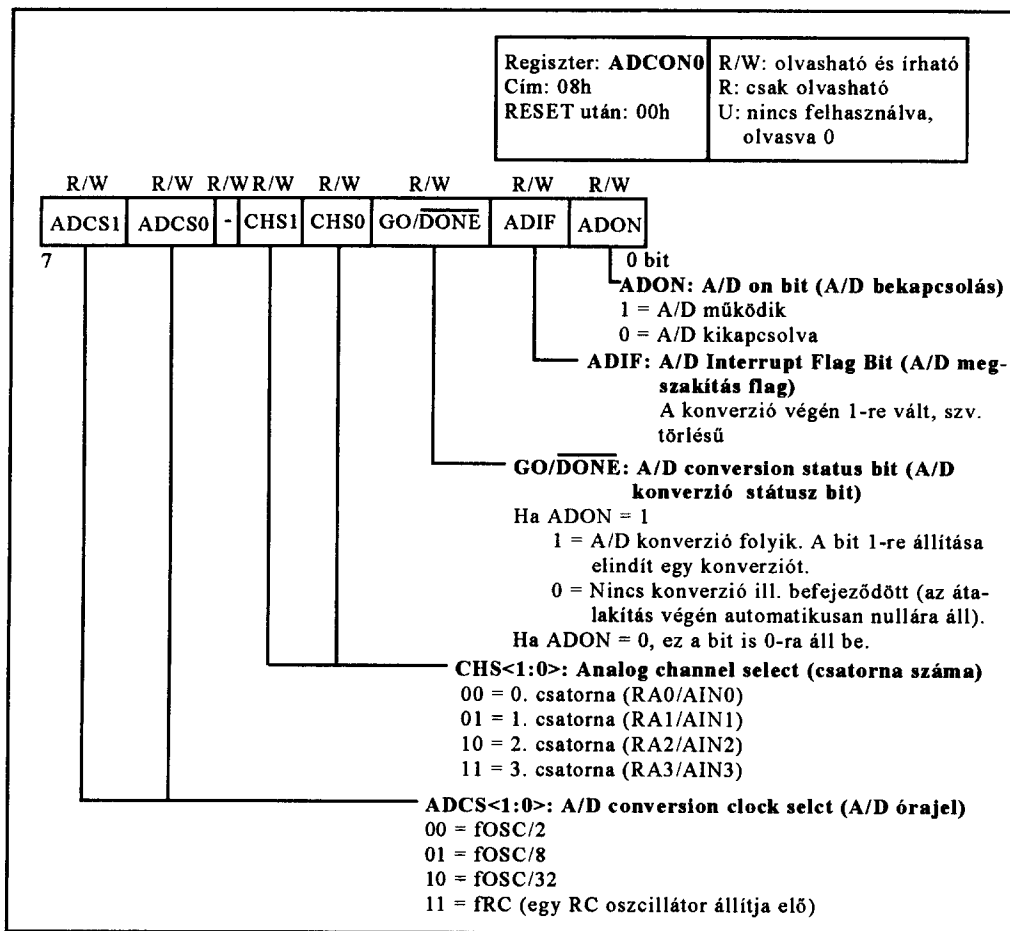


3.73. ábra A 16C71 fájl regiszterei

3.42. táblázat A PIC16C71 speciális funkciójú regiszterei

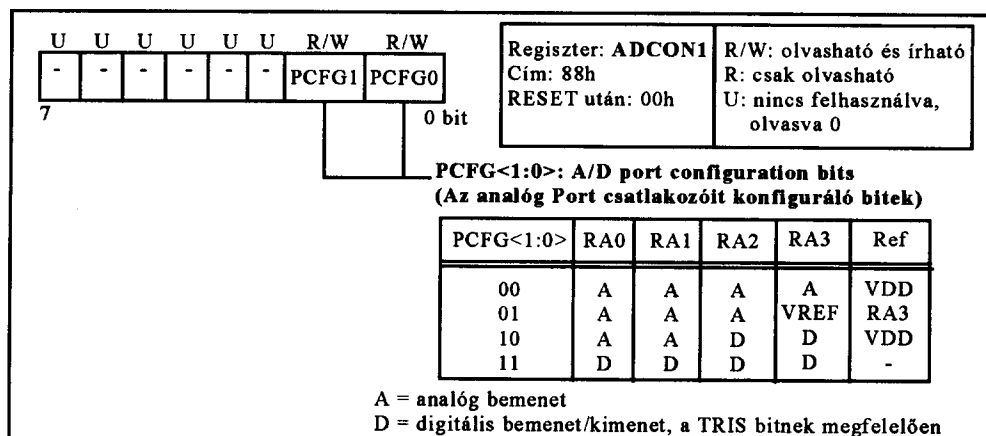
Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a rekesszel címezve, címként az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó byte-ja							
03*	STATUS	RP0	RP1	RP0	TO	PD	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pointere							
05	PORTA	A Port adat latch (íraskor), az A Port csatlakozó láb jelszintje (olvasáskor)							
06	PORTB	B Port adat latch (íraskor), a B Port csatlakozó láb jelszintje (olvasáskor)							
07									
08	ADCON0	ADCS1	ADCS0	-	CHS1	CHS0	GO / DONE	ADIF	ADON
09	ADRES	A/D eredmény regiszter							
0A*	PCLATH	A programszámláló (PC) felső öt bitje puffereit regisztere							
0B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
1 Bank									
80*	INDF	Ezzel a rekesszel címezve az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION	RBPU	INTEDG	TOCS	TOSE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó byte-ja							
83*	STATUS	RP0	RP1	RP0	TO	PD	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pointere							
85	TRISA	A Port adatrány regisztere							
86	TRISB	B Port adatrány regisztere							
87									
88	ADCON1	-	-	-	-	-	-	PCFG1	PCFG0
89	ADRES	A/D eredmény regiszter							
8A*	PCLATH	A programszámláló (PC) felső öt bitje puffereit regisztere							
8B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

Az ADCON0 regiszterben (3.74. ábra) eltér a 16V74-től a 16C71, részben azért, mert a 16C71-ben csak 4 analóg bemeneti csatorna van, másrészt a '71-ben itt helyezték el az A/D megszakítás flag bitet (ADIF).



3.74. ábra Az ADCON0 regiszter (16C71)

Az analóg bemeneteket kezelő ADCON1 regiszter is egyszerűbb (3.75. ábra).



3.75. ábra Az ADCON1 regiszter (16C71)

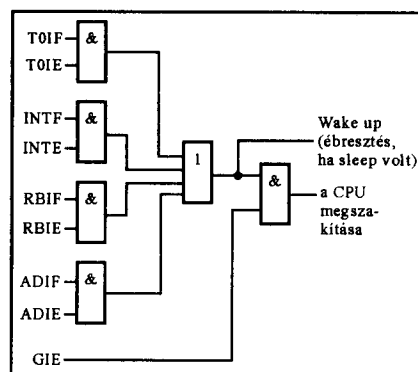
A megszakítási rendszer is egyszerűbb, mint a 16C74 mikrovezérlő esetében (3.76. ábra).

Az ADIF engedélyező bitje az INTCON regiszterben van (ADIE az INTCON<6> biten). A Tad minimális értéke a 16C71 esetében 2 μ s.

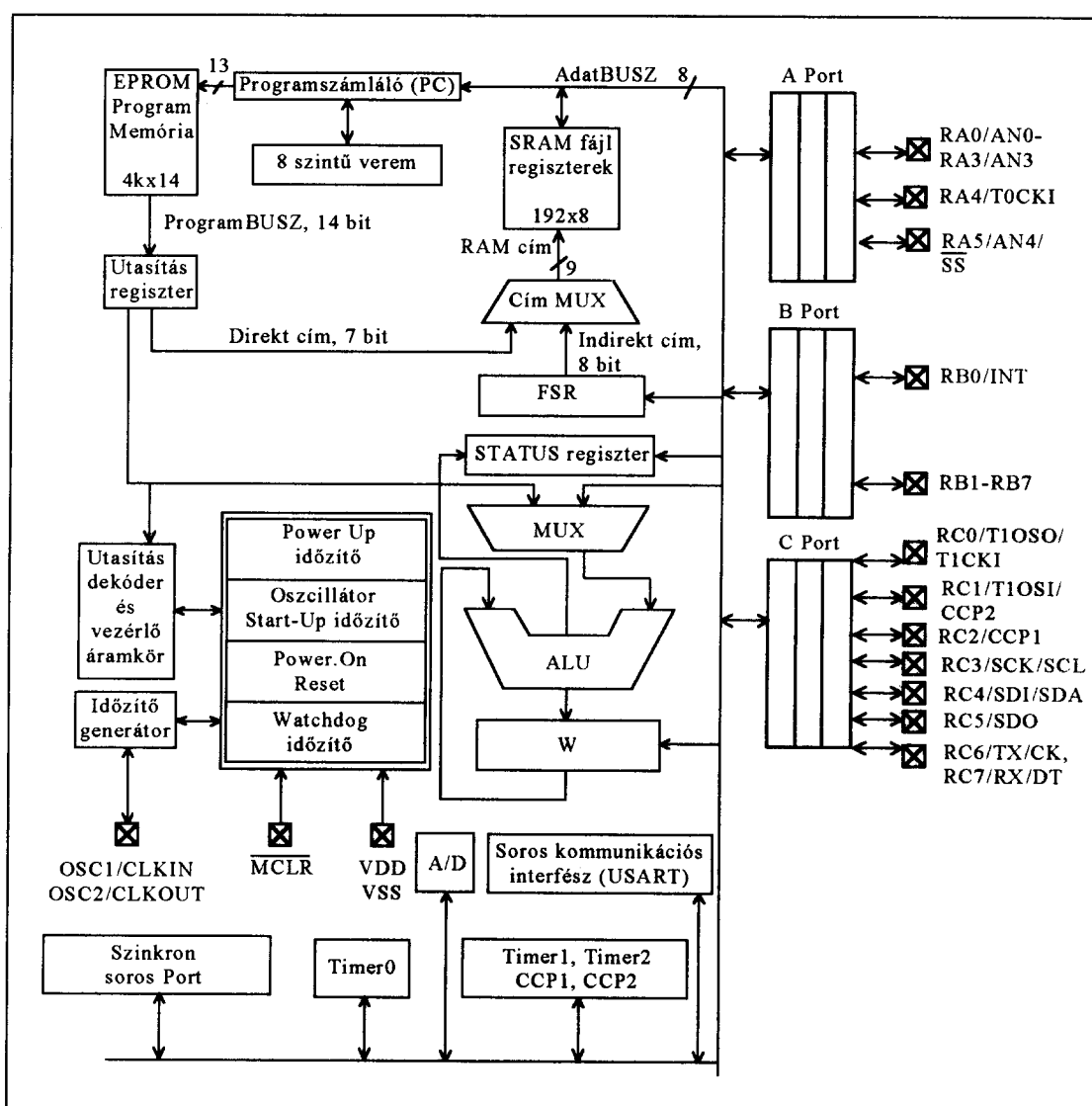
3.2.2. A PIC16C73 mikrovezérlő

A 16C73 is egyszerűbb kialakítású, mint a 16C74 - két Port hiányzik belőle, a D és az E Port, s ezek után természetesen hiányzik az E Port alternatív funkciója, a párhuzamos Slave Port is. Így viszont ez a csip is elhelyezhető egy kisebb méretű, 28 kivezetéses tokban.

A 16C73 belső felépítését a 3.77. ábra mutatja be.

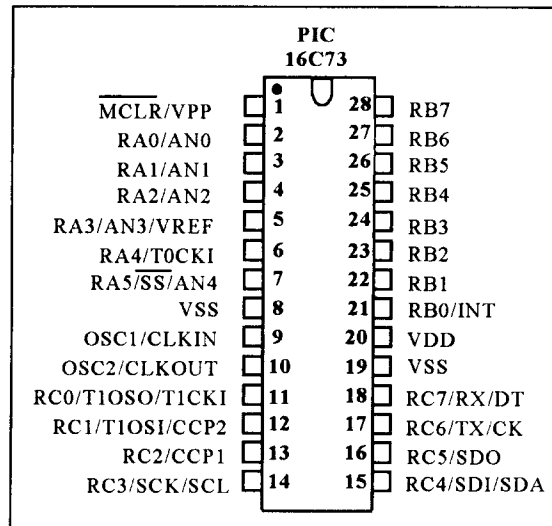


3.76. ábra *A 16C71 megsza-
kítás logikája*



3.77. ábra A PIC16C73 belső felépítése

A mikrovezérlő lábkiosztását a **3.78. ábrán** láthatjuk (94. oldal).



3.78. ábra A 16C73 lábkiosztása

A csatlakozó pontok szerepét a 3.43. táblázatban találjuk meg.

3.43. táblázat A 16C73 csatlakozó pontjai szerepe

Megnevezés	DIP SOIC	Típus	Puffer	Leírás
OSC1/CLKIN	9	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	10	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR /VPP	1	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0/AN0	2	I/O	TTL	A PORT, kétirányú I/O Port 0 analóg bemenet 1 analóg bemenet 2 analóg bemenet 3 analóg bemenet/VREF TMR0 órabemenete, kimenetként OC szinkron soros Portnál slave választó/4 analóg bemenet
RA1/AN1	3	I/O	TTL	
RA2/AN2	4	I/O	TTL	
RA3/AN3/VREF	5	I/O	TTL	
RA4/T0CKI	6	I/O	ST	
RA5/AN4/SS	7	I/O	TTL	
RB0/INT	21	I/O	TTL/ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem irható elő minden bemeneten. Külső megszakítás (ekkor ST bemenet) Megszakítás változásra Megszakítás változásra Megszakítás változásra/soros program. óra (ekkor ST) Megszakítás változásra/soros program. adat (ekkor ST)
RB1	22	I/O	TTL	
RB2	23	I/O	TTL	
RB3	24	I/O	TTL	
RB4	25	I/O	TTL	
RB5	26	I/O	TTL	
RB6	27	I/O	TTL/ST	
RB7	28	I/O	TTL/ST	
RC0/T1OSO/T1CKI	11	I/O	ST	C Port, kétirányú I/O Port Timer1 oszcillátor kimenet/Timer1 óra bemenet Timer1 oszc. bemenet/Capture2 bemenet, Compare2 kime- net/PWM2 kimenet Capture1 bemenet/Compare1 kimenet/PWM1 kimenet Szinkron soros óra (SPI, IIC üzemmódhoz) Data In (SPI)/adat I/O (IIC) Data Out (SPI) aszinkron soros adás/szinkron óra (SCI) aszinkron soros vevő/szinkron adat (SCI)
RC1/T1OSI/CCP2	12	I/O	ST	
RC2/CCP1	13	I/O	ST	
RC3/SCK/SCL	14	I/O	ST	
RC4/SDI/SDA	15	I/O	ST	
RC5/SDO	16	I/O	ST	
RC6/TX/CK	17	I/O	ST	
RC7/RX/DT	18	I/O	ST	
VSS	8,19	P	-	
VDD	20	P	-	

I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás,
TTL=TTL bemenet, ST=Schmitt-trigger bemenet

A regiszter fájl kialakítása a 3.79. ábrán látható.

Fájl címek			
00	Indirect addr.	Indirect addr.	80
01	TMR0	OPTION	81
02	PCL	PCL	82
03	STATUS	STATUS	83
04	FSR	FSR	84
05	PORT A	TRIS A	85
06	PORT B	TRIS B	86
07	PORT C	TRIS C	87
08			88
09			89
0A	PCLATH	PCLATH	8A
0B	INTCON	INTCON	8B
0C	PIR1	PIE1	8C
0D	PIR2	PIE2	8D
0E	TMR1L	PCON	8E
0F	TMR1H		8F
10	T1CON		90
11	TMR2		91
12	T2CON	PR2	92
13	SSPBUF	SSPADD	93
14	SSPCON	SSPSTAT	94
15	CCPR1L		95
16	CCPR1H		96
17	CCP1CON		97
18	RCSTA	TXSTA	98
19	TXREG	SPBRG	99
1A	RCREG		9A
1B	CCPR2L		9B
1C	CCPR2H		9C
1D	CCP2CON		9D
1E	ADRES		9E
1F	ADCON0	ADCON1	9F
20	Általános célú regiszter	Általános célú regiszter	A0
7F			FF

0 Bank 1 Bank

 Használaton kívüli rekesz, olvasva 0-kat ad.

3.79. ábra A 16C73 regiszter fájl

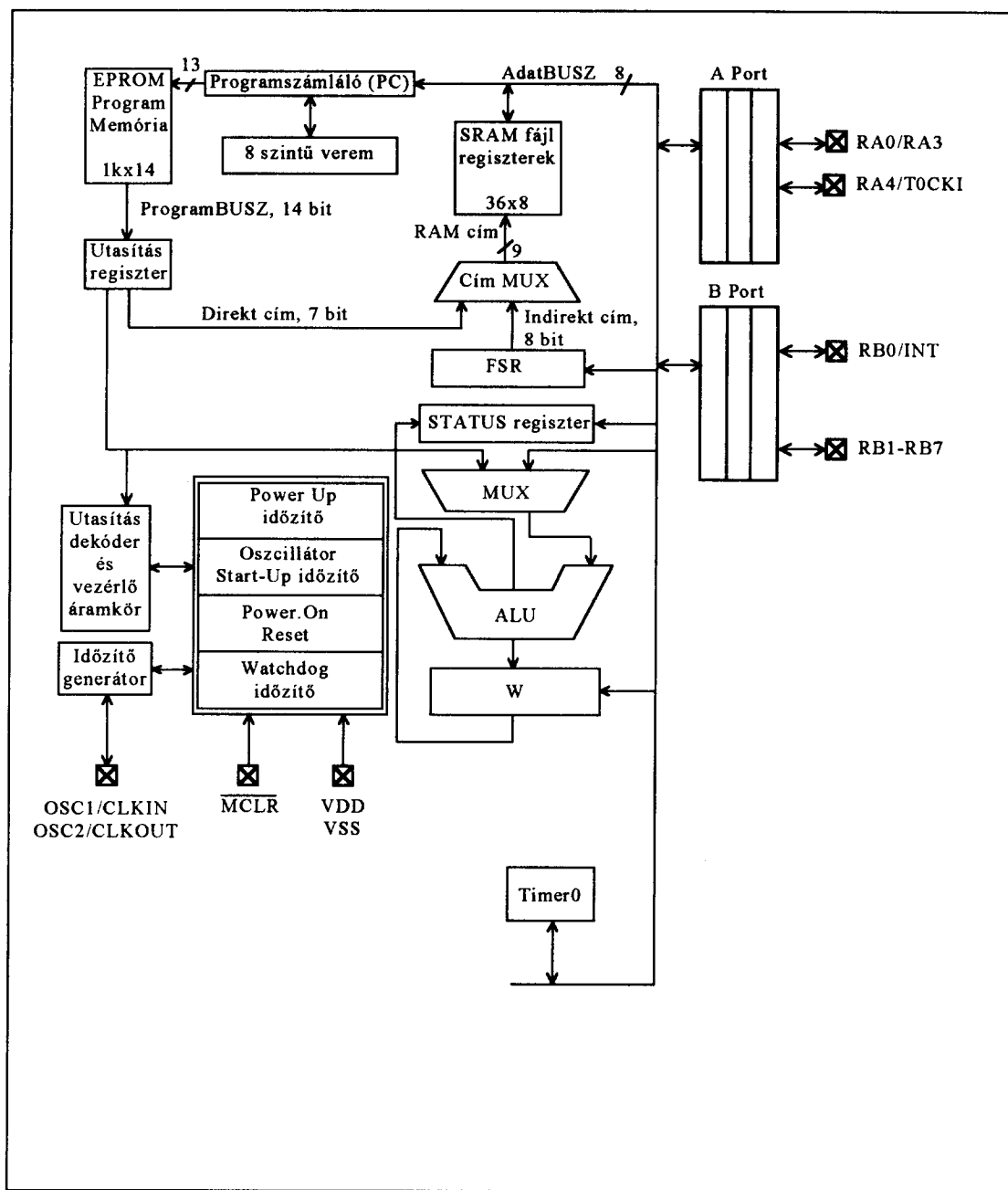
A speciális funkciójú regisztereket a 3.44. táblázat sorolja fel. A 16C73 megszakítási rendszere meg-
egyezik a 16C74 megszakítási hierarchiájával, mindössze annyi az eltérés, hogy a 16C73 esetében hi-
ányzik a párhuzamos Slave Port (azaz a mikroprocesszor Port) megszakítása, a PSPIF.

3.44. táblázat A 16C73 speciális funkciójú regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a rekesssel címezve, címként az FSR tartalma használdók fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó bájtja							
03*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pointer							
05	PORTA	A Port adat latch (írásakor), az A Port csatlakozó láb jelszintje (olvasáskor)							
06	PORTB	B Port adat latch (írásakor), a B Port csatlakozó láb jelszintje (olvasáskor)							
07	PORTC	C Port adat latch (írásakor), a C Port csatlakozó láb jelszintje (olvasáskor)							
08									
09									
0A*	PCLATH	A programszámláló (PC) felső öt bitje puffereelt regisztere							
0B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
0C	PIR1		ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
0D	PIR2	-	-	-	-	-	-	-	CCP2IF
0E	TMR1L	Timer1 alsó bájt							
0F	TMR1H	Timer1 felső bájt							
10	T1CON	-	-	T1CKPS1	T1CKPS0	T1OSCEN	T1INSYNC	TMR1CS	TMR1ON
11	TMR2	Timer2							
12	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
13	SSPBUF	Szinkron soros Port vevő puffer / adó regiszter							
14	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
15	CCPR1L	Capture/Compare/Duty Cycle Regiszter (LSB)							
16	CCPR1H	Capture/Compare/Duty Cycle Regiszter (MSB)							
17	CCP1CON	-	-	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0
18	RCSTA	SPEN	RC8/9	SREN	CREN	-	FERR	OERR	RCD8
19	TXREG	SCI adó adat regiszter							
1A	RCREG	SCI vevő adat regiszter							
1B	CCPR2L	Capture/Compare/Duty Cycle Regiszter 2 (LSB)							
1C	CCPR2H	Capture/Compare/Duty Cycle Regiszter 2 (MSB)							
1D	CCP2CON	-	-	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0
1E	ADRES	A/D eredmény regiszter							
1F	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO / DONE	-	ADON
1 Bank									
80*	INDF	Ezzel a rekesssel címezve az FSR tartalma használdók fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION	RBPV	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó bájtja							
83*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pointer							
85	TRISA	A Port adatirány regisztere							
86	TRISB	B Port adatirány regisztere							
87	TRISC	C Port adatirány regisztere							
88									
89									
8A*	PCLATH	A programszámláló (PC) felső öt bitje puffereelt regisztere							
8B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
8C	PIE1	PSPIE	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
8D	PIE2	-	-	-	-	-	-	-	CCP2IE
8E	PCON	-	-	-	-	-	-	POR	-
8F									
90									
91									
92	PR2	Timer2 Period regiszter							
93	SSPADD	Szinkron soros Port (IIC üzemmód) cím regiszter							
94	SSPSTAT	-	-	D/A	P	S	R/W	UA	BF
95									
96									
97									
98	TXSTA	CSRC	TX8/9	TZEN	SYNC	-	BRGH	TRMT	TXD8
99	SPBRG	Baud rate generátor							
9A									
9B									
9C									
9D									
9E									
9F	ADCON1	-	-	-	-	-	PCFG2	PCFG1	PCFG0

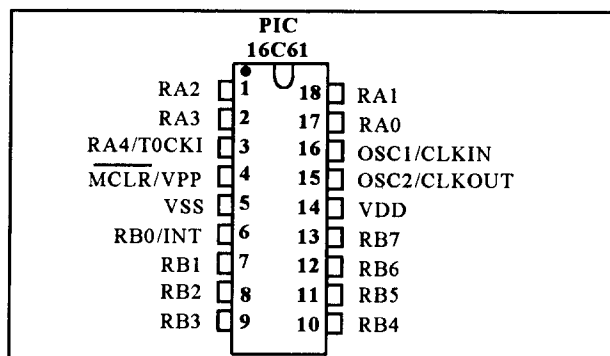
3.2.3. A PIC 16C61 mikrovezérlő

A PIC16C61 mikrovezérlőt a legegyszerűbben úgy lehet jellemezni, hogy ez egy olyan 16C71, amelyből hiányzik az A/D konverter. Belső felépítése a **3.80. ábrán** látható.



3.80. ábra A PIC16C61 belső felépítése

A lábkiosztást a **3.81. ábrán** találjuk meg, a 98. oldalon.



3.81. ábra A 16C61 lábkiosztása

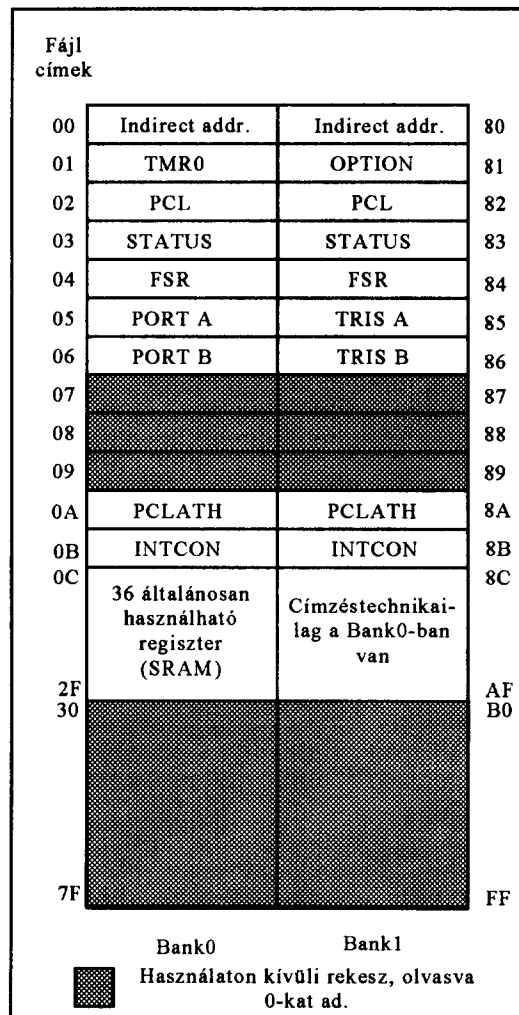
A csatlakozó pontok szerepe a 3.45. táblázatból olvasható ki.

3.45. táblázat A 16C61 csatlakozó pontjai szerepe

Megnevezés	DIP	SOIC	SSOP	Típus	Puffer	Leírás
OSC1/CLKIN	16	16	18	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	15	15	17	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR/VPP	4	4	4	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0	17	17	19	I/O	TTL	A PORT, kétirányú I/O Port
RA1	18	18	20	I/O	TTL	
RA2	1	1	1	I/O	TTL	
RA3	2	2	2	I/O	TTL	
RA4/T0CKI	3	3	3	I/O	ST	
						TMR0 órabemenete, kimenetként OC
RB0/INT	6	6	7	I/O	TTL/ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem írható elő minden bemeneten.
RB1	7	7	8	I/O	TTL	
RB2	8	8	9	I/O	TTL	Külső megszakítás (ekkor ST bemenet)
RB3	9	9	10	I/O	TTL	
RB4	10	10	11	I/O	TTL	
RB5	11	11	12	I/O	TTL	
RB6	12	12	13	I/O	TTL/ST	
RB7	13	13	14	I/O	TTL/ST	Megszakítás változásra Megszakítás változásra Megszakítás változásra/soros program. óra (ekkor ST) Megszakítás változásra/soros program. adat (ekkor ST)
VSS	5	5	5,6	P	-	GND
VDD	14	14	15,16	P	-	Pozitív tápfeszültség

I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás, TTL=TTL bemenet, ST=Schmitt-trigger bemenet

A regiszter fájl elrendezése és elemkészlete a 3.82. ábrán szerepel.



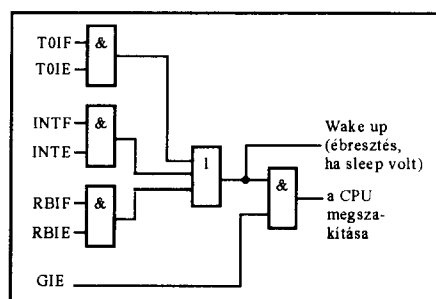
3.82. ábra *A 16C61 regiszter fájl*

A speciális funkciójú regisztereket a **3.46. táblázat** foglalja össze.

3.46. táblázat A 16C61 speciális funkciójú regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a relesszel címezve, címként az FSR tartalma használdók fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó bájtja							
03*	STATUS	IRP	RPI	RP0	$\overline{TO}$	$\overline{PD}$	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pontere							
05	PORTA	A Port adat latch (írásakor), az A Port csatlakozó láb jelszintje (olvasáskor)							
06	PORTB	B Port adat latch (írásakor), a B Port csatlakozó láb jelszintje (olvasáskor)							
0A*	PCLATH	A programszámláló (PC) felső öt bitje puffereit regisztere							
0B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
1 Bank									
80*	INDF	Ezzel a relesszel címezve az FSR tartalma használdók fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION		INTEDG	RTS	RTE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó bájtja							
83*	STATUS	IRP	RPI	RP0	$\overline{TO}$	$\overline{PD}$	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pontere							
85	TRISA	A Port adatrány regisztere							
86	TRISB	B Port adatrány regisztere							
8A*	PCLATH	A programszámláló (PC) felső öt bitje puffereit regisztere							
8B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

A megszakítási logika a 3.83. ábra szerint alakul.

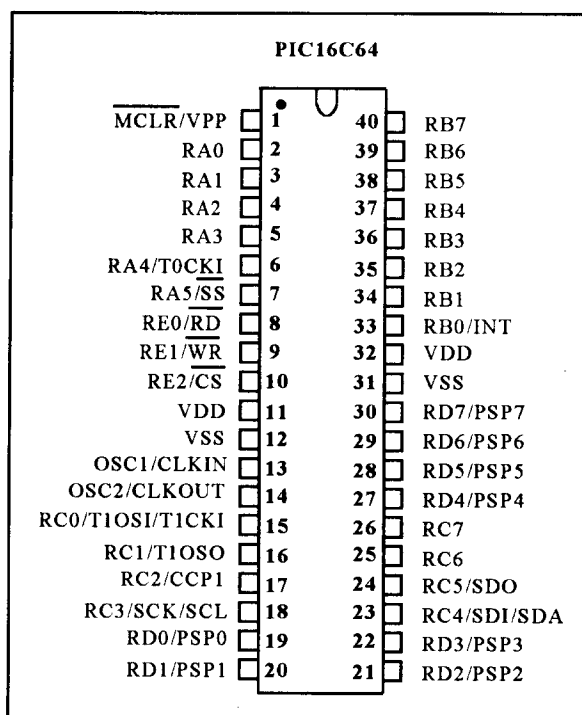


3.83. ábra A 16C61 megszakítás rendszere

3.2.4. A PIC16C64 mikrovezérlő

A PIC16C64 mikrovezérlő majdnem minden elemet tartalmaz, amit a 16C74. Hiányzik belőle az A/D konverter, a CCP2 modul és a soros kommunikációs Port. A belső felépítését a 3.84. ábra mutatja be (101. oldal). A 40 kivezetésű DIL tokozású változat lábkiosztása a 3.85. ábrán látható.

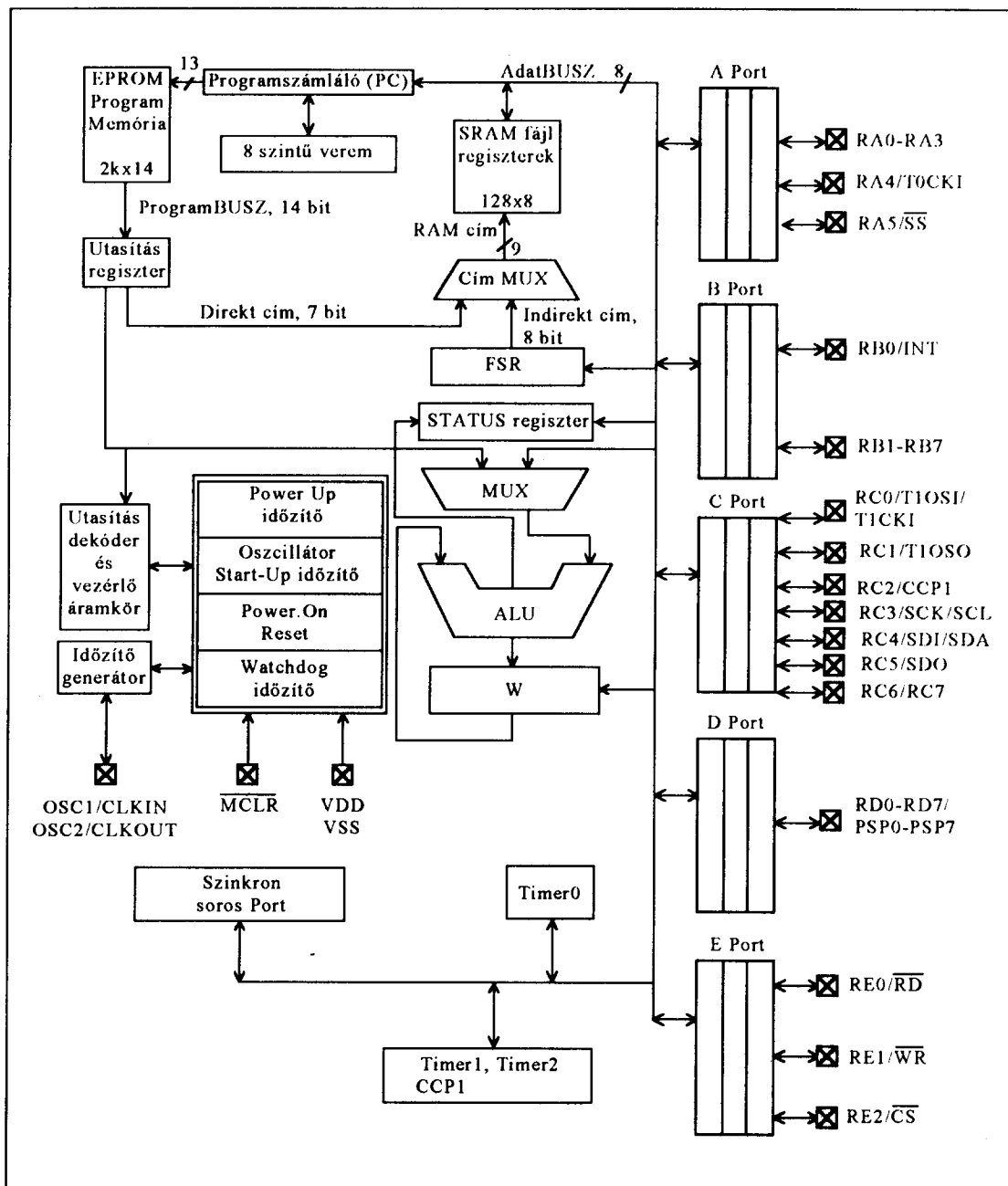
A kivezetések szerepét a 3.47. táblázat ismerteti (102. oldal).



3.85. ábra A PIC16C64 lábkiosztása

Egy lényeges eltérésre külön is fel szeretnénk hívni a figyelmet. A PIC16C családban minden mikrovezérlőnél, amelyikben megtalálható a Timer1, egységesen használják a C0 és a C1 Port pontok alternatív funkcióit - kivéve a 16C64-et! Ennél a mikrovezérlőnél, ahogyan ezt a lábkiosztás rajzán is és a kivezetések funkcióit ismertető táblázatban is ellenőrizni lehet, az alternatív funkciók így jelennek meg:

RC0/T1OSI/T1CKI
RC1/T1OSO.



3.84. ábra A PIC16C64 belső felépítése

3.47. táblázat A PIC16C64 csatlakozó pontjai szerepe

Megnevezés	DIP	PLCC	PQFP	Típus	Puffer	Leírás
OSC1/CLKIN	13	14	30	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	14	15	31	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR /VPP	1	2	18	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0	2	3	19	I/O	TTL	A PORT, kétirányú I/O Port
RA1	3	4	20	I/O	TTL	
RA2	4	5	21	I/O	TTL	
RA3	5	6	22	I/O	TTL	
RA4/T0CKI	6	7	23	I/O	ST	
RA5/ $\overline{SS}$	7	8	24	I/O	TTL	TMR0 órabemenete, kimenetként OC szinkron soros Portnál slave választó/4 analóg bemenet
RB0/INT	33	36	8	I/O	TTL/ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem írható elő minden bemeneten. Külső megszakítás (ekkor ST bemenet)
RB1	34	37	9	I/O	TTL	
RB2	35	38	10	I/O	TTL	
RB3	36	39	11	I/O	TTL	
RB4	37	41	14	I/O	TTL	
RB5	38	42	15	I/O	TTL	
RB6	39	43	16	I/O	TTL/ST	
RB7	40	44	17	I/O	TTL/ST	
RC0/T1OSI/T1CKI	15	16	32	I/O	ST	C Port, kétirányú I/O Port Timer1 oszcillátor bemenet/Timer1 óra bemenet Timer1 oszc. kimenet Capture1 bemenet/Compare1 kimenet/PWM1 kimenet Szinkron soros óra (SPI, IIC üzemmódhoz) Data In (SPI)/adat I/O (IIC) Data Out (SPI)
RC1/T1OSO	16	18	35	I/O	ST	
RC2/CCP1	17	19	36	I/O	ST	
RC3/SCK/SCL	18	20	37	I/O	ST	
RC4/SDI/SDA	23	25	42	I/O	ST	
RC5/SDO	24	26	43	I/O	ST	
RC6	25	27	44	I/O	ST	
RC7	26	29	1	I/O	ST	
RD0/PSP0	19	21	38	I/O	ST/TTL	D Port, kétirányú I/O Port vagy párhuzamos slave port, mikroprocesszor BUSZ-hoz (ekkor TTL)
RD1/PSP1	20	22	39	I/O	ST/TTL	
RD2/PSP2	21	23	40	I/O	ST/TTL	
RD3/PSP3	22	24	41	I/O	ST/TTL	
RD4/PSP4	27	30	2	I/O	ST/TTL	
RD5/PSP5	28	31	3	I/O	ST/TTL	
RD6/PSP6	29	32	4	I/O	ST/TTL	
RD7/PSP7	30	33	5	I/O	ST/TTL	
RE0/ $\overline{RD}$	8	9	25	I/O	ST/TTL	Kétirányú I/O pontok TTL írásvez. párhuzamos slave Porthoz
RE1/ $\overline{WR}$	9	10	26	I/O	ST/TTL	
RE2/ $\overline{CS}$	10	11	27	I/O	ST/TTL	
VSS	12, 31	13,34	6,29	P	-	GND
VDD	11, 32	12,35	7,28	P	-	Pozitív tápfeszültség
NC	-	1,17, 28,40	12,13 33,34	-	-	Nincs belső csatlakozás, használatkor maradjanak bekötetlenek

I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás, TTL=TTL bemenet, ST=Schmitt-trigger bemenet

A regiszter fájl elemeit a 3.86. ábra mutatja be.

Fájl címek			
00	Indirect addr.	Indirect addr.	80
01	TMR0	OPTION	81
02	PCL	PCL	82
03	STATUS	STATUS	83
04	FSR	FSR	84
05	PORT A	TRIS A	85
06	PORT B	TRIS B	86
07	PORT C	TRIS C	87
08	PORT D	TRIS D	88
09	PORT E	TRIS E	89
0A	PCLATH	PCLATH	8A
0B	INTCON	INTCON	8B
0C	PIR1	PIE1	8C
0D			8D
0E	TMR1L	PCON	8E
0F	TMR1H		8F
10	T1CON		90
11	TMR2		91
12	T2CON	PR2	92
13	SSPBUF	SSPADD	93
14	SSPCON	SSPSTAT	94
15	CCPR1L		95
16	CCPR1H		96
17	CCP1CON		97
18			98
19			99
1A			9A
1B			9B
1C			9C
1D			9D
1E			9E
1F			9F
20	Általános célú regiszter	Általános célú regiszter	A0
			BF
			C0
7F			FF

0 Bank 1 Bank

 Használaton kívüli rekesz, olvasva 0-kat ad.

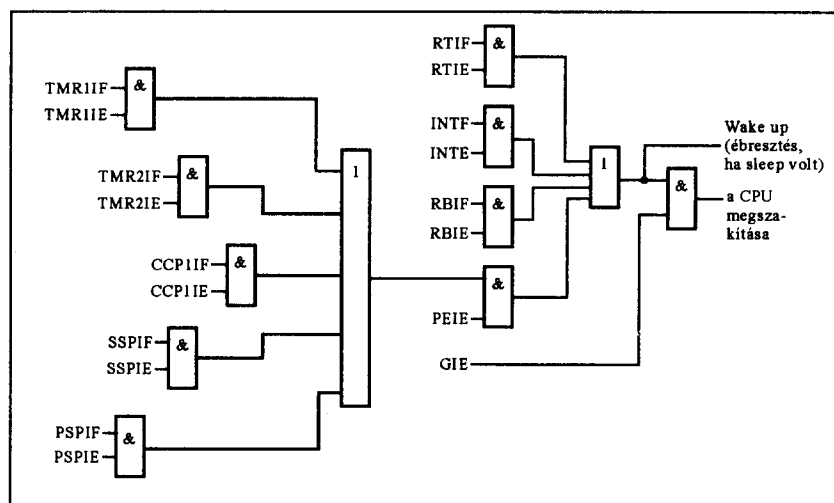
3.86. ábra Regiszter fájl (16C64)

A speciális funkciójú regisztereket a **3.48. táblázat** foglalja össze.

3.48. táblázat A 16C64 speciális funkciójú regiszterei

Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a rekesszel címezve, címként az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó bájtja							
03*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pontere							
05	PORTA	A Port adat latch (írásakor), az A Port csatlakozó láb jelszintje (olvasáskor)							
06	PORTB	B Port adat latch (írásakor), a B Port csatlakozó láb jelszintje (olvasáskor)							
07	PORTC	C Port adat latch (írásakor), a C Port csatlakozó láb jelszintje (olvasáskor)							
08	PORTD	D Port adat latch (írásakor), a D Port csatlakozó láb jelszintje (olvasáskor)							
09	PORTE	E Port adat latch (írásakor), az E Port csatlakozó láb jelszintje (olvasáskor)							
0A*	PCLATI	A programszámláló (PC) felső öt bitje puffertelt regisztere							
0B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
0C	PIR1	PSPIF	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
0D									
0E	TMR1L	Timer1 alsó bájt							
0F	TMR1H	Timer1 felső bájt							
10	T1CON	-	-	T1CKPS1	T1CKPS0	T1OSCEN	T1HNSYN	TMR1CS	TMR1ON
11	TMR2	Timer2							
12	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
13	SSPBUF	Szinkron soros Port vevő puffér / adó regiszter							
14	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
15	CCP1L	Capture/Compare/Duty Cycle Regiszter (LSB)							
16	CCP1H	Capture/Compare/Duty Cycle Regiszter (MSB)							
17	CCP1CON	-	-	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0
1 Bank									
80*	INDF	Ezzel a rekesszel címezve az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION	RBPU	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó bájtja							
83*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pontere							
85	TRISA	A Port adatirány regisztere							
86	TRISB	B Port adatirány regisztere							
87	TRISC	C Port adatirány regisztere							
88	TRISD	D Port adatirány regisztere							
89	TRISE	IBF	OBF	IBOV	PSPMODE	-	TRISE2	TRISE1	TRISE0
8A*	PCLATH	A programszámláló (PC) felső öt bitje puffertelt regisztere							
8B*	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
8C	PIE1	PSPIE	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
8D									
8E	PCON	-	-	-	-	-	-	POR	-
8F									
90									
91									
92	PR2	Timer2 Period regiszter							
93	SSPAD	Szinkron soros Port (IIC üzemmód) cím regiszter							
94	SSPSTAT	-	-	D/A	P	S	R/W	UA	BF

A megszakítási logika hasonló felépítésű, mint a 16C74-ben, csak a hiányzó modulok megszakításai is elmaradnak, azaz nincs CCP2IF, ADIF, TXIF, RCIF. A megszakítási rendszert a **3.87. ábra** mutatja be.



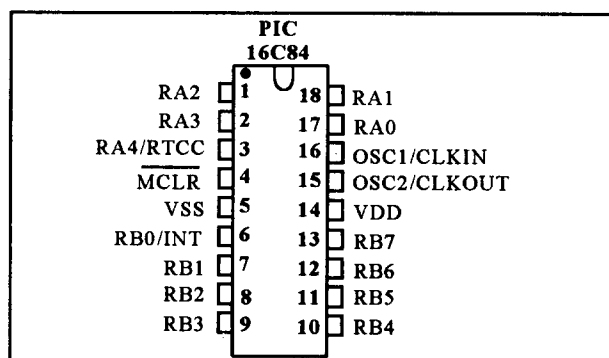
3.87. ábra A 16C64 megszakítási logikája

3.2.5. A PIC 16C84 mikrovezérlő

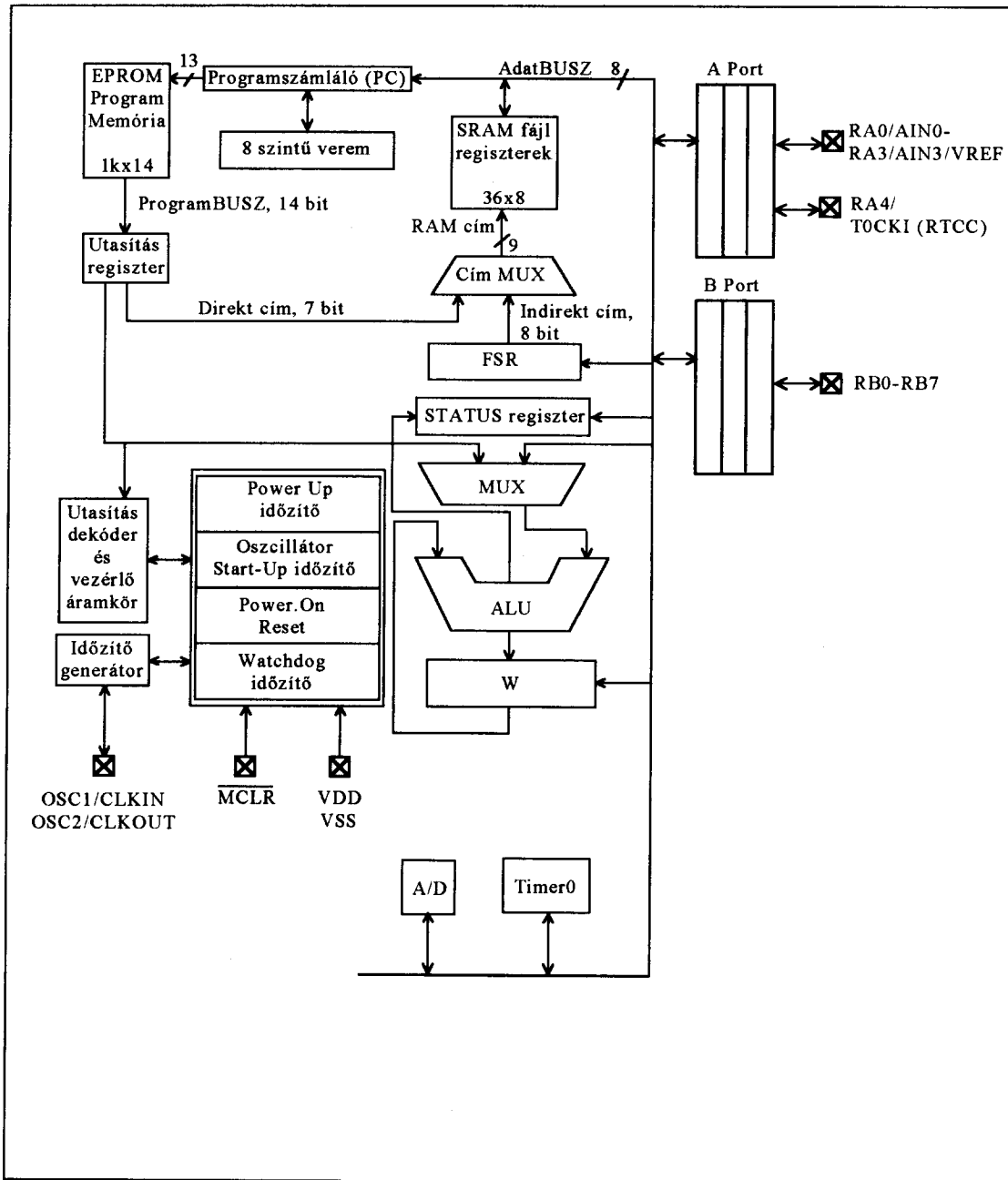
A 16C84 felépítése leginkább a 16C61-re emlékeztet, de rendkívüli tulajdonsága is van: a belső programtárolója EEPROM és egy külön EEPROM adattároló területet is tartalmaz. Az EEPROM villamos jellel törölhető, átprogramozható ROM - így ezt a mikrovezérlőt már nem kell ablakos tokozással gyártani. Ugyanaz az IC típus szolgál programfejlesztésre is, mint a végleges készülékbe való beépítésre. A 16C84 belső felépítését a 3.88. ábra mutatja be (106. oldal).

A programtár újszerűsége a programfejlesztéskor nem okoz változást, mivel a beégetés és a törlés a fejlesztő rendszer feladata. Az EEPROM adattár kezelésére egy új fájl regiszter szolgál, az EECON1. Szerepel még az EEPROM modullal kapcsolatban az EECON2 regiszter is, de ez fizikailag nem létező regiszter.

A mikrovezérlő 18 kivezetésű kis tokban is elfér, a lábkiosztása a 3.89. ábrán látható.



3.89. ábra A 16C84 lábkiosztása



3.88. ábra A PIC16C84 belső felépítése

A kivezetések szerepét a 3.49. táblázatban találjuk meg.

3.49. táblázat *A 16C84 csatlakozó pontjai szerepe*

Megnevezés	DIP SOIC	Típus	Puffer	Leírás
OSC1/CLKIN	16	I	CMOS	Kvarc bemenet/külső óra bemenet
OSC2/CLKOUT	15	O	-	Kvarc kimenet/RC módban CLKOUT
MCLR /VPP	4	I/P	ST	Master clear (Reset) bemenet/programozó feszültség bemenet
RA0	17	I/O	TTL	A PORT, kétirányú I/O Port
RA1	18	I/O	TTL	
RA2	1	I/O	TTL	
RA3	2	I/O	TTL	
RA4/T0CKI	3	I/O	ST	
				TMR0 órabemenete, kimenetként OC
RB0/INT	6	I/O	TTL/ST	B Port, kétirányú I/O Port, szoftverrel gyenge felhúzó elem írható elő minden bemeneten. Külső megszakítás (ekkor ST bemenet)
RB1	7	I/O	TTL	
RB2	8	I/O	TTL	
RB3	9	I/O	TTL	
RB4	10	I/O	TTL	
RB5	11	I/O	TTL	
RB6	12	I/O	TTL/ST	
RB7	13	I/O	TTL/ST	
VSS	5	P	-	GND
VDD	14	P	-	Pozitív tápfeszültség

I=bemenet, O=kimenet, I/O=bemenet/kimenet, P=tápfeszültség csatlakozás, TTL=TTL bemenet, ST=Schmitt-trigger bemenet

Fájl címek	Bank0	Bank1	
00	Indirect addr.	Indirect addr.	80
01	TMR0 (RTCC)	OPTION	81
02	PCL	PCL	82
03	STATUS	STATUS	83
04	FSR	FSR	84
05	PORT A	TRIS A	85
06	PORT B	TRIS B	86
07			87
08	EEDATA	EECON1	88
09	EEADR	EECON2	89
0A	PCLATH	PCLATH	8A
0B	INTCON	INTCON	8B
0C			8C
	36 általánosan használható regiszter (SRAM)	Címzésttechnikai- lag a Bank0-ban van	
2F 30			AF B0
7F			FF

Bank0 Bank1

 Használaton kívüli rekesz, olvasva 0-kat ad.

A regiszter fájl a **3.90. ábrán** látható.

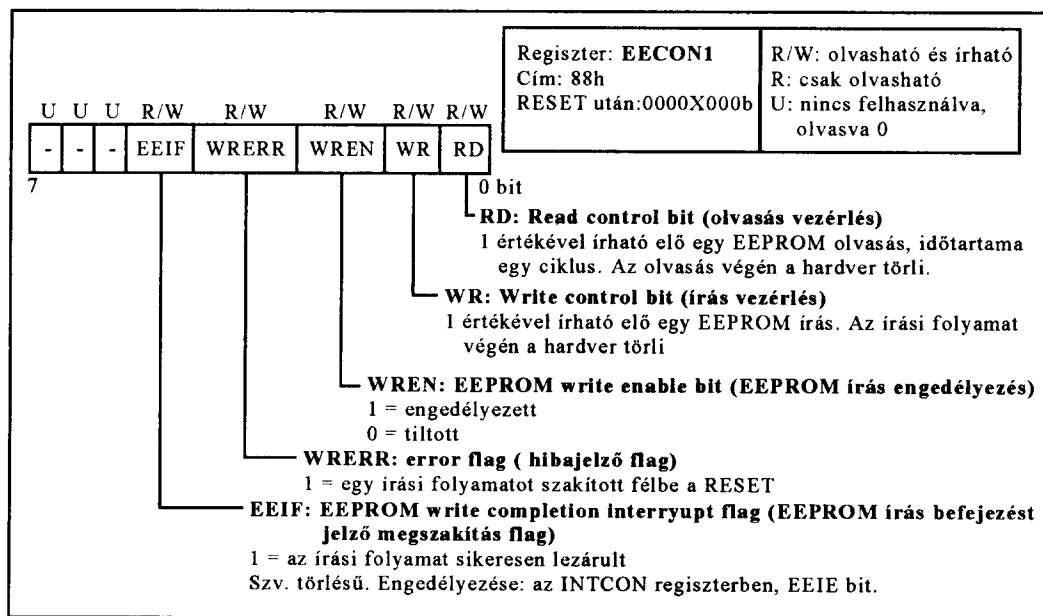
A speciális funkciójú regisztereket a **3.50. táblázat** tartalmazza (108. oldal).

3.90. ábra Regiszter fájl (16C84)

3.50. táblázat A 16C84 speciális funkciójú regiszterei

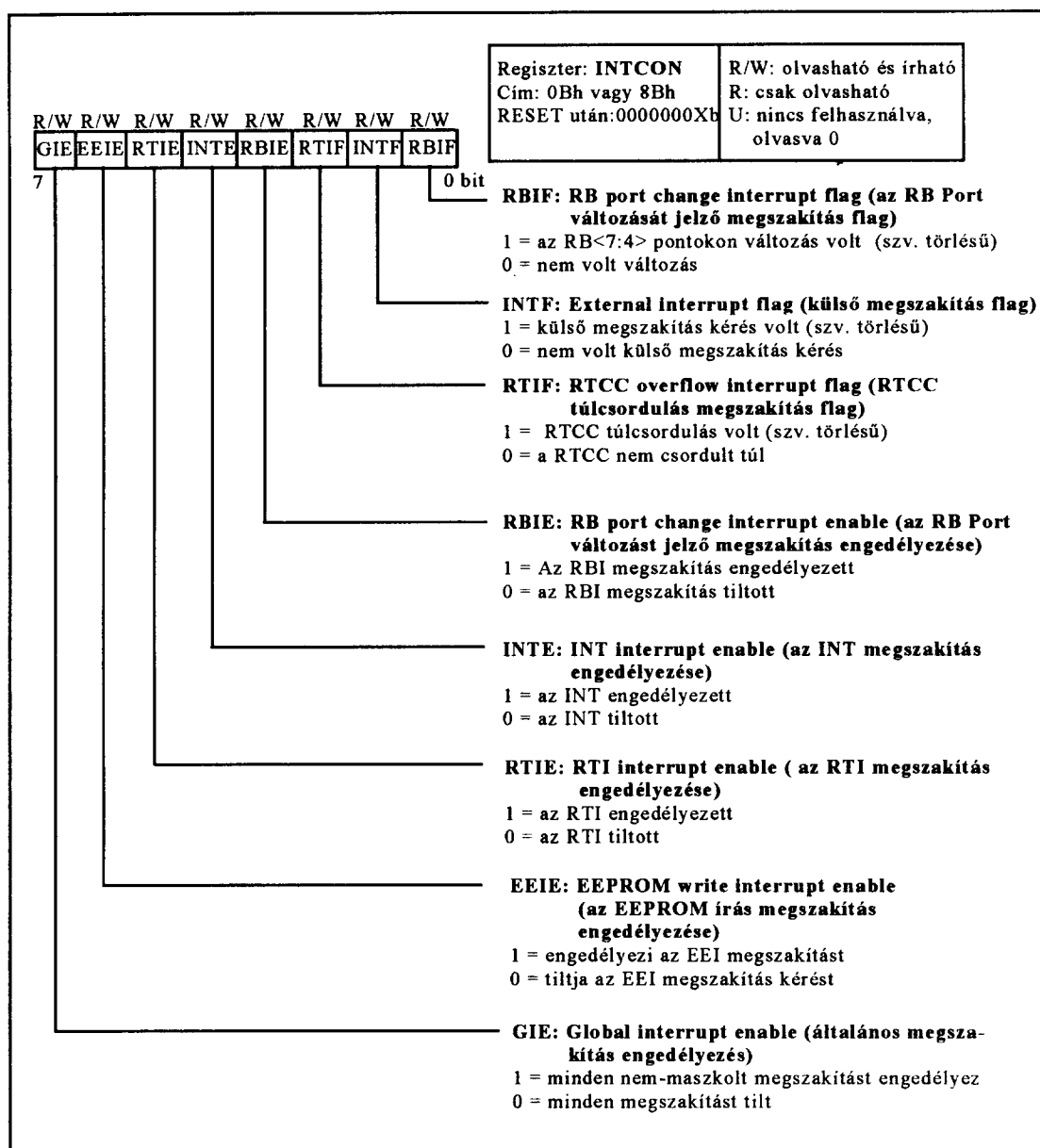
Cím	Név	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
0 Bank									
00*	INDF	Ezzel a renesszel címezve, címként az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
01	TMR0	Timer0							
02*	PCL	A programszámláló (PC) alsó bájtja							
03*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
04*	FSR	Az adatmemória indirekt címző pointer							
05	PORTA	A Port adat latch (íráskor), az A Port csatlakozó láb jelsintje (olvasáskor)							
06	PORTB	B Port adat latch (íráskor), a B Port csatlakozó láb jelsintje (olvasáskor)							
07									
08	EEDATA	EEPROM adat regiszter							
09	EEADR	EEPROM cím regiszter							
0A*	PCLATH	A programszámláló (PC) felső öt bitje puffertelt regisztere							
0B*	INTCON	GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF
1 Bank									
80*	INDF	Ezzel a renesszel címezve az FSR tartalma használódik fel az adatmemóriánál (fizikailag nem létezik)							
81	OPTION	RBP	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0
82*	PCL	A programszámláló (PC) alsó bájtja							
83*	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
84*	FSR	Az adatmemória indirekt címző pointer							
85	TRISA	A Port adatirány regisztere							
86	TRISB	B Port adatirány regisztere							
87									
88	EECON1	-	-	-	EEIF	WRERR	WREN	WR	RD
89	EECON2	EEPROM vezérlő regiszter (fizikailag nincs kialakítva)							
8A*	PCLATH	A programszámláló (PC) felső öt bitje puffertelt regisztere							
8B*	INTCON	GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

Az EECON1 regisztert a 3.91. ábrán láthatjuk.



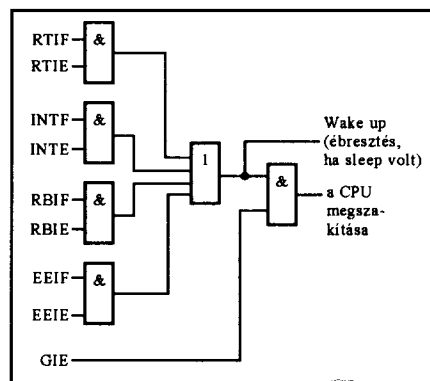
3.91. ábra Az EECON1 regiszter (16C84)

Az EEPROM adattár megszakítását engedélyező bit az INTCON regiszterbe került (3.92. ábra).



3.92. ábra Az INTCON regiszter (16C84)

A megszakítási logikát a 3.93. ábra mutatja be.



3.93. ábra Az INTCON regiszter (16C84)

Az EEPROM adatmemória 64 x 8 kapacitású (00h ... 3Fh), programból elérhető, írható és olvasható. Nincs a fájl regiszter memóriához kapcsolva, speciális regisztereken át lehet elérni. A fájl regiszterek között már megismertük ezeket a regisztereket:

az EEPROM adatregisztert: EEDATA (08h)

az EEPROM címregisztert: EEADR (09h).

A vezérlést két további regiszter útján lehet megszervezni, ezek a következők:

EECON1 (88h), valódi regiszter (ld. 3.91. ábra)

EECON2 (89h), fizikailag nincs megvalósítva.

RESET folyamat után az EEDATA és az EEADR tartalma véletlenszerű, az EECON1 ilyenkor 0000x000b értékű.

Ha egy EEPROM rekeszt írunk, ez a folyamat automatikusan törli az előző tartalmát. Az EEPROM írási ciklusa kb. 10 ms időt igényel, ezt egy független belső időzítő egység kezeli. Az írás előkészítéseként a címet be kell helyezni az EEADR, az adatot az EEDATA regiszterbe. A WREN bitnek 1 értékűnek kell lennie, s az írás megkezdéséhez 1-re kell váltani a WR bitet (ezt a hardver fogja törölni az írási folyamat után). Ezt azonban egy sajátos szoftver részletnek kell megelőznie, az alábbi módon (és itt derül majd ki az EECON2 regiszter feladata is):

MOVLW	55h	
MOVWF	EECON2	
MOVLW	AAh	
MOVWF	EECON2	
BSF	EECON1, WR	;a WR bit 1-re vált, az írási folyamat ;megkezdődik

A CPU-nak ezután van szüksége a 10 ms írási időre. Ez alatt megszakítást sem fogadhat (le kell tiltani). Az írási folyamat végét a WR bit 0-ra váltása és az EEPROM megszakítás flag bit (EEIF) 1-re váltása is jelzi, ez utóbbit szoftverrel kell törölni. Az írási ciklus végét szoftver hurokkal vagy megszakítással lehet érzékelni.

Olvasáskor az EEADR regiszterbe kerül a cím, majd az RD bitet 1-re kell változtatni (a törlését a CPU automatikusan elvégzi az olvasási folyamat végén). A legközelebbi ciklusban a kiolvasott adat már rendelkezésre is áll az EEDATA regiszterben.

4. A BASIC STAMP számítógépek

A Microchip PIC mikrovezérlőkhöz a Parallax cég folyamatosan gyártja a fejlesztő rendszereket. Amikor megjelentek a 16C5x családrész újabb és újabb elemei, majd a 16C6x, 16C7x, 16C8x mikrovezérlők újabb típusai - rövid idő elteltével már vásárolható volt a Parallax fejlesztő rendszere az új elemekhez is.

A parallax azonban még nagyobb kihívásnak értékelte a PIC mikrovezérlőket és megalkotta velük a világ egyik legkisebb számítógép sorozatát. Az első BASIC STAMP egy 29 x 48 mm méretű nyomtatott huzalozású kis panelen készült el, de még ennek a kis panelnek is kb. a fele egy furatgalvanizált forrasztási helyekkel sűrűn telepakolt terület, ahol a felhasználó saját áramköri részleteit felépítheti (protoboard area). A BASIC STAMP elsősorban vezérlési feladatok ellátására készült. Egy tartós kivitelű 9V-os rádióteleppel évekig működőképes (a 9V-os telep csatlakozói is rajta vannak a BASIC STAMP paneljén).

A BASIC STAMP 16C56 mikrovezérlővel és egy 256 szavas EEPROM memóriával (93LC56) készült, a panelen elhelyezték az 5 V-os feszültségstabilizátort és a mikrovezérlő oscillátorát hangoló kerámia rezonátort is. A működtető programot az EEPROM tárolja. Ez a működtető program alapvetően BASIC jellegű, és IBM PC kompatibilis számítógépen lehet megírni. A BASIC programot a BASIC STAMP fejlesztő rendszer erőteljesen tömöríti (tokenizálja), s ez a tokenizált változat kerül a BASIC STAMP programtárolójába. A 16C56 mikrovezérlő belső programtárolójában az a program helyezkedik el, amelyik a tokenizált BASIC utasításokat tudja értelmezni.

A BASIC STAMP nem rendelkezett csatlakozó szerelvényvel, a hozzávezetésekét rá kellett forrasztani a panel megfelelő pontjaira. Ma ez a mikroszámítógép a BS-I típusjelet viseli, mert időközben megjelent a nagyobb teljesítményű BS-II is.

Később a BS-I is új formában készült már, protoboard terület nélkül, s a panel egyik szélén csatlakozók sorakoztak, ezt a változatot már foglalatba is be lehetett helyezni, illetve nyomtatott áramkörbe is be lehetett ültetni.

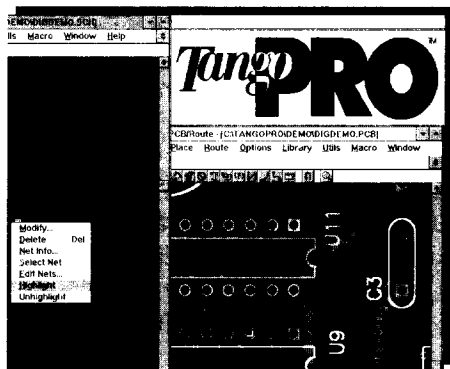
A BASIC STAMP ismertetésére, az utasításkészlet leírására sajnos ebben a jegyzetben területi okok miatt nincs lehetőség. Mégis érdekes lehet néhány különleges képesség megemlítése. A BASIC STAMP programnyelvének alapja a BASIC nyelv, így a BASIC alaputasításokat természetesen tartalmazza a BS programnyelv is. A kiegészítő utasítások eredményeképpen képes egyetlen utasítással soros aszinkron kimenőjelet generálni, ilyen jelet fogadni; potenciométer állást leolvasni; PWM kimenőjelet előállítani; impulzusszélességet mérni; hang jelet kiküldeni. Mindez azt jelenti, hogy a BASIC STAMP egy ideális eszköz vezérlési feladatokat teljesítő beépített számítógép megvalósítására. Néhány külső alkatrész felhasználásával képes különböző érzékelők jeleinek fogadására, billentyűzet kezelésre, különféle beavatkozók, végrehajtók működtetésére.

Az IBM PC-n egy egyszerű, de kényelmes BASIC nyelvű fejlesztői környezetet alakított ki a PARALLAX. A fejlesztés közben a BASIC STAMP és a PC egy kábelén keresztül összeköthető, így a megírt program közvetlenül betölthető a BASIC STAMP program memóriájába és azonnal futtatható is. Ha a BASIC STAMP és a PC közötti kábeles kapcsolat jelen van, a BASIC STAMP a futó programról képes információkat visszaküldeni a PC-be (DEBUG lehetőség).

A BS-II egy szabványos, 24 kivezetésű IC toknak megfelelő csatlakozó elrendezésű egység. A beépített mikrovezérlő már nagyobb képességű (16C57), a programtároló EEPROM is nagyobb (2 K szó). Az I/O pontok száma a BS-I-nél 8, a BS-II-nél 16. Az utasításkészlet is bővült, a BS-II képes szinuszhullám előállítására, telefon hívóhangok kezelésére, frekvencia mérésére, erősáramú hálózaton vezérlőjelek továbbítására.

A BASIC STAMP számítógépekkel azért is érdemes közelebbről megismerkedni, mert egy nagyszemű alkalmazási példaként is szolgálnak a PIC mikrovezérlőkhöz. Az óragerátor frekvencia meghatározó eleme itt kerámia rezonátor, a külső RESET áramkör gyakorlatilag hiányzik. Külön érdekesség, hogy a BASIC STAMP soros adatelérésű EEPROM memóriájának tápfeszültség ellátását a PIC mikrovezérlő egyik PORT kimenetének jelével oldották meg.

EGYETLEN HELYEN A MEGOLDÁS



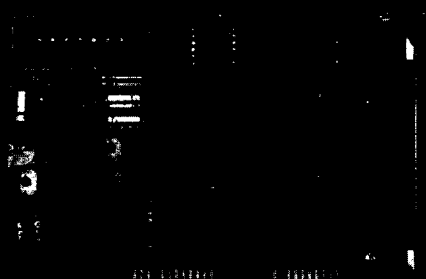
Tango
nyomtatott-
áramkörtervező
rendszerek
egyszerűbb és
összetettebb
kiépítésben,
igényekhez
mértén és
szabottan

ICAP/4
analóg és digitális
áramkörsimulátor
magyar nyelvű könyvvvel

egyszerűsített és teljes
tudású változatai
kezdőtől a pro-
fesszionális
szintig



PB-10 a gazdaságos programozó
8/16 bites EPROM-hoz és Intel mikrovezérlőkhöz



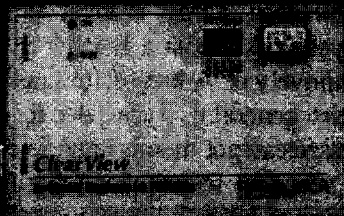
Basic Stamp I és II
PBASIC nyelven programozható
bélyegszámítógépek különféle
alkalmazásokra a hobbi szinttől az
ipari vezérlőként való felhasználásig

In-circuit emulátorok
(PIC16/17, 875x, MCS251 MC68xx, MC68xxx)
Parallax, Microchip, Metalink, Pentica Systems
EPROM Emulátorok 27256-274000

TechTools

Fordító programok, Debuggerek,
Parallax, Microchip, ByteCraft, KEIL, Hiware
Fuzzy logic fejlesztő

Microchip, Inform, Hiware



A MICROCHIP
PIC16CXX és PIC17CXX
mikrovezérlő családja
CMOS RISC felépítéssel,
nagy sebességgel és
kis fogyasztással,
változatos memória és
integrált periféria-
választékkal rendelkezik

Soros EEPROM memóriák 8 Kbájtig

EMP-20 univerzális programozó
szinte minden eszközhöz.

Mikrokontroller
EEPROM
EPROM,
PLD,
FPGA,
stb.



HUMANsoft Kft

1149 Budapest, Angol u. 77

E-mail: info@HUMANsoft.hu

Tel: 251 0288/119 Fax: 163 7889

Tartalomjegyzék

Bevezetés	3. o.
1. A PIC16C mikrovezérlő család felépítése, általános tulajdonságai	5. o.
1.1. A PIC mikrovezérlők oszcillátor típusai	5. o.
1.2. A PIC mikrovezérlők átfedésez utasítás-végrehajtása	7. o.
1.3. A PIC mikrovezérlők utasításkészlete	8. o.
1.4. Bekapcsolási RESET megoldások	16. o.
2. A PIC16C5x mikrovezérlő csoport	
2.1. A PIC16C5x elemek általános leírása, belső felépítésük	19. o.
2.2. A PIC16C5x mikrovezérlő típusok	21. o.
2.3. A belső működések regiszterei	22. o.
2.4. Az I/O működések	25. o.
2.5. Általános célú regiszterek	26. o.
2.6. Speciális regiszterek	26. o.
2.7. A RESET folyamat	26. o.
2.8. Az előosztó és működési módjai	27. o.
2.9. A Watchdog-Timer (WDT)	28. o.
2.10. Az oszcillátor start-up működés	28. o.
2.11. Kis fogyasztású üzemmód (Sleep)	29. o.
2.12. Konfigurációs biztosítékok	30. o.
3. A PIC16Cxx mikrovezérlők	31. o.
3.1. A PIC16C74 mikrovezérlő	33. o.
3.1.1. A PIC16C74 felépítése, fontosabb jellegzetességei	33. o.
3.1.2. A 16C74 memória szervezése	36. o.
3.1.3. Az I/O Portok	46. o.
3.1.4. Az időzítő modulok	50. o.
3.1.4.1. A TMR0 modul	51. o.

3.1.4.2. A Timer1 modul	51. o.
3.1.4.3. A Timer2 modul	54. o.
3.1.5. A CCP modulok	55. o.
3.1.5.1. A Capture (kiolvasás) működés	56. o.
3.1.5.2. A komparálás (Compare) működés	56. o.
3.1.5.3. A PWM működés	57. o.
3.1.6. A szinkron soros Port	59. o.
3.1.6.1. Az SPI üzemmód	59. o.
3.1.6.2. Az I ² C üzemmód	63. o.
3.1.7. A soros kommunikációs interfész (SCI) modul	68. o.
3.1.7.1. Az SCI aszinkron működése	71. o.
3.1.7.2. Az SCI szinkron működése	74. o.
3.1.8. Az analóg/digitál konverziós modul	76. o.
3.1.9. A 16C74 speciális tulajdonságai	81. o.
3.2. További PIC16Cxx mikrovezérlők	89. o.
3.2.1. A PIC16C71 mikrovezérlő	89. o.
3.2.2. A PIC16C73 mikrovezérlő	93. o.
3.2.3. A PIC16C61 mikrovezérlő	97. o.
3.2.4. A PIC16C64 mikrovezérlő	100.o.
3.2.5. A PIC16C84 mikrovezérlő	105.o.
4. A BASIC STAMP számítógépek	111.o.

BASIC STAMP® MODULOK

Sok új utasítással a Stamp II

A számítóbélyegek sok alkalmazásra tökéletesen alkalmasak, kezdve a modellvasúttaktól egészen az ipari érzékelők felügyeletéig. 8 vagy 16 ki/bemeneti lábbal rendelkeznek, amelyek különböző digitális és analóg célokra használhatók. Az egyszerűség kedvéért BASIC nyelven programozhatók. A speciális "PBASIC" programozási nyelvükben az ismerős utasítások mellett, mint a GOTO, FOR...NEXT, és IF...THEN, található soros ki/bemeneti, nyomógomb kezelő, DTMF hanggeneráló stb. utasítás is.

A BASIC Stamp fejlesztő csomag mindent tartalmaz, amire szüksége van a számítóbélyegek programozásához a PC-jét használva. A csomag magában foglalja a programszerkesztő szoftvert, programozó kábeleket, kézikönyveket, alkalmazási példákat és ingyenes műszaki tanácsadást. A csomag 14600Ft-ért kapható. A számítóbélyegeket és a hordozópaneleket külön kell megvásárolni.

BASIC Stamp II modul (BS2-IC)

16 általános célú ki/bemeneti láb
2048 bájt program terület (kb. 600 utasítás)
20 MHz órajel (38400 baud soros sebesség, stb)
7200Ft, 9900Ft opcionális hordozópanellel



* Csak Stamp II-nél használható

COUNT *

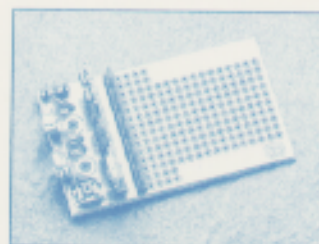
Ciklusokat számol egy adott ki/bemeneti lábon. Leegyszerűsíti a fordulatszám-mérő kezelést vagy más számláló-, vezérlőprogram írását.

SHIFTOUT *

Két ki/bemeneti láb felhasználásával vezérel soros-párhuzamos regisztereket. Több eszköz vezérlésére használható nem túl sok Stamp ki/bemeneti láb felhasználásával.

Basic Stamp 1 modul (BS1-IC)

8 általános célú ki/bemeneti láb
256 bájt program terület (kb. 100 utasítás)
4 MHz órajel (2400 baud soros sebesség, stb)
4600Ft, 6600Ft opcionális hordozópanellel



A hordozópanelek a fejlesztési területen kívül elemcsatlakozót, reset gombot és programozó csatlakozót tartalmaznak. A Stamp modulok enélkül is használhatók, de a hordozópanelek megkönnyítik a tervezést és tesztelést.



DTMFOUT *

DTMF hangot generál csak egy ki/bemeneti láb, két kondenzátor és egy hangszóró felhasználásával. A másik ehhez kapcsolódó utasítás a FREQOUT, amelyik egy vagy két hangot generál egyidejűleg. Mindkét hang programozható 1 Hz-es lépésekben 0-tól 32767Hz-ig.



XOUT *

Szabványos X-10 teljesítményvezérlő jeleket generál két láb és egy olcsó TTL-AC konverter felhasználásával. A jeleket egy kis modul veszi, ami vezérelhet világítást vagy háztartási készülékeket. A vevőmodul külön megvásárolható.

74HC595

OUT1	1	16	VCC
OUT2	2	15	OUT6
OUT3	3	14	SEN
OUT4	4	13	OE
OUT5	5	12	CLK
OUT6	6	11	SCLK
OUT7	7	10	CLR
QND	8	9	OUT7

Megjelent a Basic Stamp PBASIC parancsait megvalósító gépkódú program gyűjteménye



A könyv a programlemez mellékletével ideális segítséget nyújt a PIC mikrovezérlők gépkódú programozásához a bélyegszámítógépet használók részére.

4000,-Ft



1 000000 002119

GAMF

Madarász PIC 16C mikrovez.